



Jakarta Server Pages and Servlets

Jakarta Server Pages and Servlets

© 2024 Aptech Limited

All rights reserved.

No part of this book may be reproduced or copied in any form or by any means – graphic, electronic, or mechanical, including photocopying, recording, taping, or storing in information retrieval system or sent or transferred without the prior written permission of copyright owner Aptech Limited.

All trademarks acknowledged.

APTECH LIMITED

Contact E-mail: ov-support@onlinevarsity.com

Edition 1 – 2024



Onlinevarsity

A man in a light blue striped shirt and dark tie is sitting on a paved road, leaning back on his legs with a laptop on his lap. He is looking off to the side with a thoughtful expression, his hand near his chin. In the foreground, another laptop is open on a patch of green grass. The background shows a clear sky and some distant structures.

**LEARN
ANYWHERE
ANYTIME**

Preface

This book introduces the Jakarta Enterprise Edition (EE) 10 platform. Jakarta Server Pages (JSP) is a powerful tool for creating interactive and data-driven Web pages within Jakarta EE applications. By embedding Java code directly into HTML, JSP allows for dynamic content generation. When a user requests a JSP page, the server processes these embedded elements to produce the final output. Servlets, which are Java programs on the Web server, handle client requests and perform necessary server-side operations. Together, JSP and servlets streamline Web development by integrating HTML with Java, enabling the creation of robust and dynamic Web applications.

The Jakarta EE 10 platform supports Servlet 6.0 and Jakarta Server Pages 3.1. You can use these versions to create servlets and server components for your Website. You will also learn to secure your Web applications using annotations.

This book is the result of a concentrated effort of the Design Team, which is continuously striving to bring you the best and the latest in Information Technology. The process of design has been a part of the ISO 9001 certification for Aptech-IT Division, Education Support Services. As part of Aptech's quality drive, this team does intensive research and curriculum enrichment to keep it in line with industry trends.

We will be glad to receive your suggestions.

Design Team



**MANY
COURSES
ONE
PLATFORM**



Onlinevarsity App

for **Android** devices

Download from **Google Play Store**

Table of Contents

Sessions

Session 1 – Introduction to Web Applications

Session 2 – Introduction to Jakarta EE

Session 3 – Developing and Deploying a Web Application

Session 4 – Jakarta Servlets

Session 5 – Session Tracking

Session 6 – Jakarta Server Pages

Session 7 – Filters and Annotations

Session 8 – Database Access and Event Handling

Session 9 – Asynchronous Servlet Communication

Session 10 – JSP Implicit Objects and Standard Actions

Session 11 – Model-View-Controller Architecture

Session 12 – JSP Expression Language

Session 13 – Standard Tag Library and Internationalization

Session 14 – Securing Java Web Applications

Session 15 – Building an Application Using JSP and Servlet

Appendix

Introduction to Web Applications

Welcome to the Session, **Introduction to Web Applications**.

This session presents an overview of Web applications. It explains the Web application architecture and the components of the Web application. The session explains the request-response mechanism offered by HTTP and the advantages of HTTP/3 protocol in accessing the Web resources from the Web servers. Further, the session explains the role of Java in Web application development and its components, such as Servlet and Jakarta Server Pages (JSP) used for extending the functionality of the Web server. Finally, it explains the sequence of events from request to response in a Java Web application and the services provided by a Web container.

In this session, you will learn to:

- Explain Web applications and their advantages
- Describe the architecture and components of a Web application
- Describe the role of HTTP protocol and its methods used for accessing Web pages
- Describe the HTTP/3 protocol and its benefits over its predecessors
- Explain different types of components used in developing a Java Web application
- List the advantages and disadvantages of Servlets
- Describe the sequence of events from request to response in a Java Web application
- Explain the services provided by a Web container

1.1 Introduction

An application is a collection of programs designed to perform a particular task. Different types of applications are designed for different purposes. Based on the environment in which an application is run and its accessibility by the users, they are classified as follows:

- **Desktop Application**

A desktop application runs only on a local machine and can neither be viewed nor be controlled by other users. For example, Microsoft Word. Figure 1.1 depicts the desktop application.

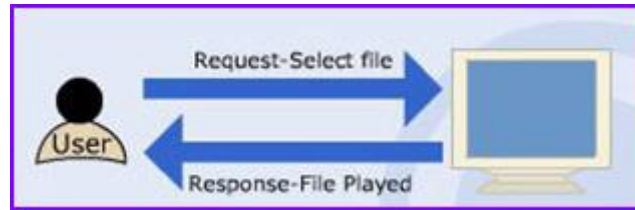


Figure 1.1: Desktop Application

- **Networking Application**

A network application runs in a Local Area Network (LAN), Metropolitan Area Network (MAN), or World Area Network (WAN) and can be viewed and controlled by users in that particular network only. For example, Novell Netware. Figure 1.2 depicts the networking application.

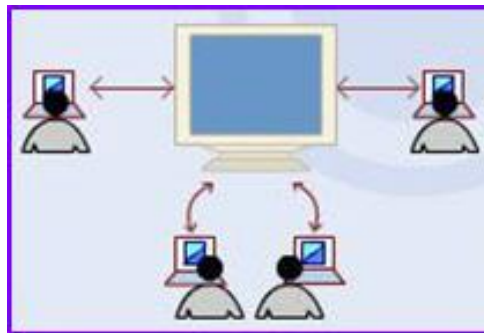


Figure 1.2: Networking Application

- **Web Application**

A Web application runs at a remote location. It can be viewed and controlled by all the users having administrative or equivalent privileges, throughout the globe, at any instance of time. For example, Internet mail services, such as www.yahoo.com or www.rediff.com.

Figure 1.3 depicts the Web application.

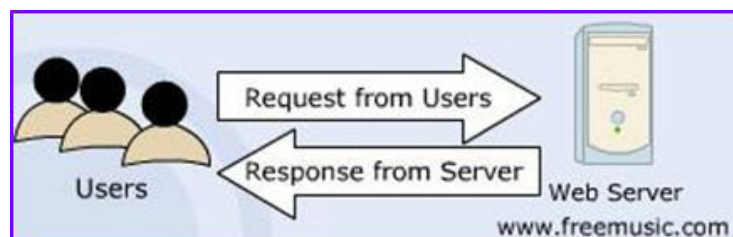


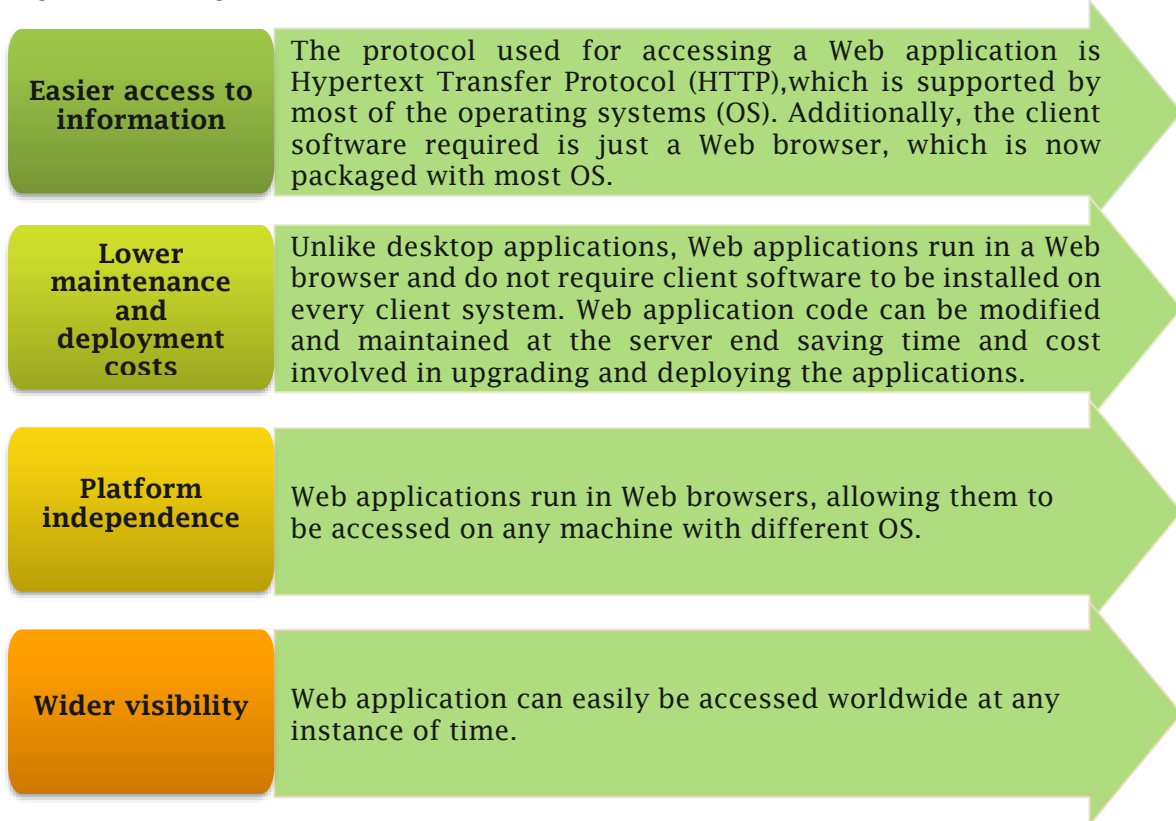
Figure 1.3: Web Application

1.2 Web Applications

A Web application is a software application that runs on a Web server. A Website that a user visits is an application running on a Web server at a remote location and executed in a Web browser.

A Web application consists of Web pages that can be static or dynamic. The static

Web pages are created using Web technologies, such as HyperText Markup Language (HTML), Cascading Style Sheet (CSS), and JavaScript. However, to achieve dynamic functionalities on Web pages, server-side programs are used. The dynamic Web pages provide interactivity with users. For example, an online shopping Web application allows users to select and deselect the items on the Web page. Advantages of Web applications over desktop applications include:



1.2.1 Web Application Architecture

Web application architecture refers to the application logic distribution between a client and a server. Different types of Web application architecture can be classified as follows:

- **Single-Page Applications (SPA):** This refers to an architecture where there is a dynamic rewriting of the current page. It does not load new pages from a server when data is required. Some examples of SPA are Gmail, Facebook, and Google Maps. These applications offer a good user experience and are fast as they use fewer server resources. In SPA, a request is sent to the server as HTML only for the first time. It uses Java Script Object Notation (JSON) for subsequent requests.

Figure 1.4 displays the SPA architecture.

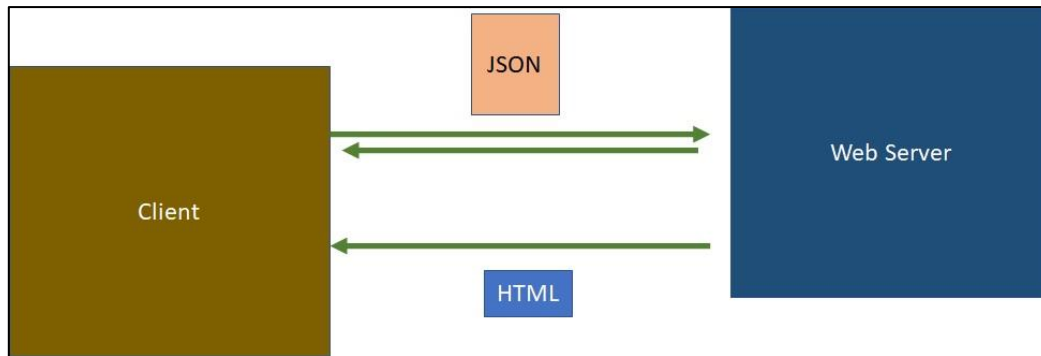


Figure 1.4: SPA Architecture

- **Serverless Architecture:** This refers to an architecture where applications are built and run without the requirement to manage infrastructure. In this type, server management is done by third-party services. One of the examples of this architecture is Amazon Web Services (AWS). The server-side code is managed by AWS while developers are involved in only client-side coding. This enhances the productivity of a developer. It reduces cost as a third party manages server-side infrastructure.
- **Microservices:** This refers to micro and lightweight services that are used to execute a single functionality. An application is divided into small service components as shown in Figure 1.5. These service components are managed independently and are loosely coupled. There is a small development team for each service that manages the service. This allows the updating of a particular service without the necessity to update or rebuild the entire application.

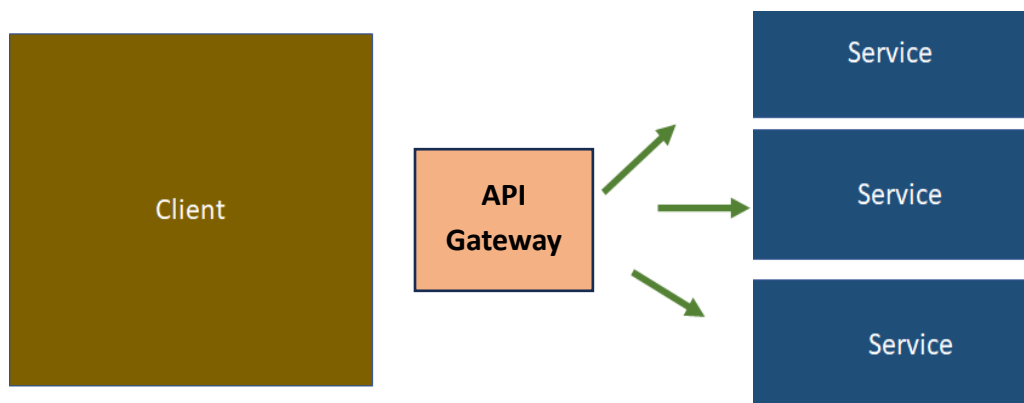


Figure 1.5: Microservices Architecture

- **Progressive Web Applications (PWAs):** The architecture of PWAs is about creating a user experience that is fast, reliable, and engaging, regardless of the platform or network conditions. They are built using standard Web technologies but offer a user experience comparable to native apps. PWAs run smoothly on any device and browser, easily adapting across platforms. They are discoverable via simple Uniform Resource Locators (URLs), eliminating the

necessity for application store downloads. They excel in poor network conditions, seamlessly transitioning to offline mode. Leading companies such as Uber, AliExpress, and Starbucks leverage PWAs for enhanced user experiences and wider audience reach. Introduced by Google in 2015, PWAs revolutionized Web application development, offering rich functionality, reliability, and easy installation.

1.2.2 Web Application Components

Components of a Web application can be categorized as structural components and UI/UX components.

- **Structural Components:** Two major structural components of a Web application are client and server components.
 - **Client Components or the front end** - Client components of a Web application refer to components developed for user interaction. These components are a part of the browser and do not require any changes as per the device or the OS. These are developed in CSS, HTML, and JS.
 - **Server Components or the back end** – Server components comprise the Web server used for the application and also include the database. Various frameworks used for the development of server components are PHP, JAVA, Python, Node.js, .NET, and Ruby on Rails.
- **UI/UX Components:** UI refers to the user interface design and UX refers to the user experience with the application. The interface of a Web application is built using UI/UX components. These components are responsible for the display part of the application and enhance the user experience. This includes visual components such as dashboards, activity logs, notifications, layouts, statistics, and so on.

1.3 Web Application Technologies

The most common technologies for communication on the Web are HTML and HTTP.

1.3.1 HTML

HTML is a presentation language that enables Web designers to create visually attractive Web pages. These pages are stored on the Web. The users accessing the Web pages on the World Wide Web (WWW) are referred to as Web clients.

HTML allows Web designers to add images, create forms, and embed media objects on the Web pages. All these are referred to as Web resources that are stored on the Web server along with HTML pages.

Figure 1.6 shows the Web page designed using HTML.

```
<html>
<body>
    <h1> Sample HTML Page </h1>
    <!-- Displaying an image -->
    An image:
    </p>

    <p>
        This Web page is created in <b> HTML </b> language which has following
        features: </P>
    <!-- Displaying a list -->
    <ol>
        <li> Stands for Hyper Text Markup Language </li>
        <li> Is a markup language </li>
        <li> Contain HTML tags and plain text </li>
        <li> Also called as web pages </li>
    </ol>
</body>
</html>
```

Figure 1.6: Web Page

The Web pages are transferred between the Web client and Web server through HTTP protocol.

1.3.2 HTTP Protocol

The requests and responses sent to a Web application from one computer to another computer are sent using HTTP.

An HTTP client, such as a Web browser, opens a connection and sends a request message to an HTTP server asking for a resource. The server, in turn, returns a response message with the requested resource. Once the resource is delivered, the server closes the connection.

HTTP does not store any connection information about the page and hence, it is referred to as a stateless protocol.

Figure 1.7 depicts HTTP request and response to a Web application.

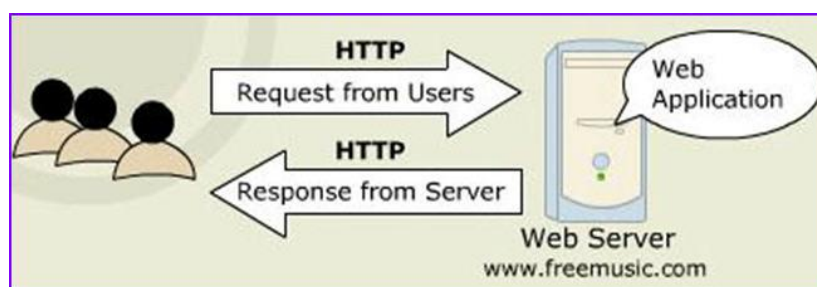


Figure 1.7: HTTP Request and Response Cycle

- **HTTP Request**

The client sends the request message as a continuous flow of text data. Figure 1.8 depicts the request message structure.

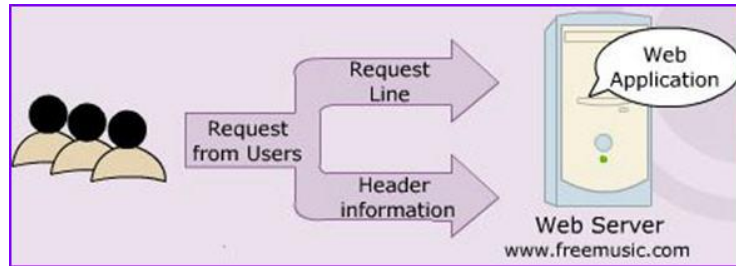


Figure 1.8: Request Message Structure

Elements in the request message are as follows:

➤ **Request line**

This element contains the HTTP method, the address of the Web page or document referred to as the URL and the HTTP version it is using to send the request message.

Figure 1.9 shows the request line of the request message.

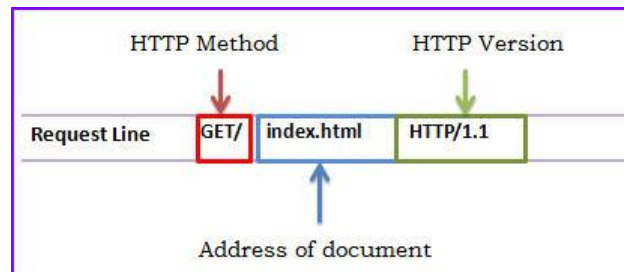


Figure 1.9: Request Line Structure

➤ **Header information**

This element specifies the User-Agent along with the Accept header. The header element of the User-Agent indicates the browser used by the client. The Accept header element provides information on what media types the client can accept.

After the header, the client sends a blank line indicating the end of request message.

Figure 1.10 shows the sample header information sent in the request message.

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:69.0)
Accept: application/json, text/javascript, */*; q=0.01
```

Figure 1.10: Header Information

• **HTTP Response**

The Web application processes the request sent by a client and generates a response message for the requesting client.

Figure 1.11 depicts the response message structure.

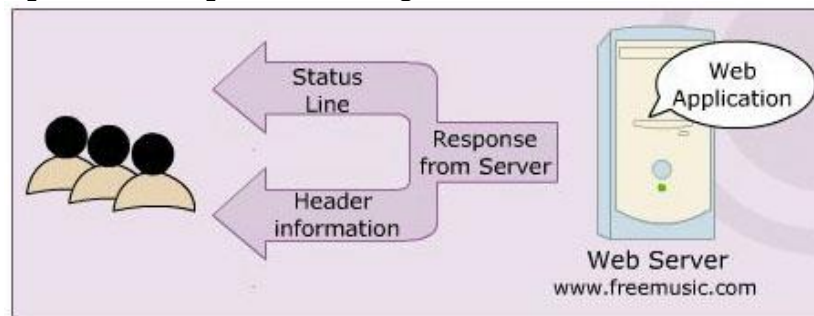


Figure 1.11: Response Message Structure

Elements in the response message are as follows:

- **Status line**
This element indicates the status of the requested process.
- **Header information**
The header information contains information such as server, last modified date and time, content-length, and content type. The server header specifies the software being used on the server. The last-modified header indicates the last modified date and time of the requested file. Content-length specifies the size of the file in bytes. Content-type header specifies the type of document.

Figure 1.12 shows the sample message format sent in the response with the HTML content from the Web server.

```

HTTP/1.1 200 OK
Server: APACHE
Content-Type: text/html
Content-length: 170
Last Modified: Mon, 17 Jun 2014 10:14:49 GMT

<html>
<body>

  <h1> Sample HTML Page </h1>

  <!-- Displaying an image -->
  An image:
  </p>

  <p>
    This Web page is created in <b> HTML </b> language which has following
    features: </P>

  <!-- Displaying a list -->
  <ol>
    <li> Stands for Hyper Text Markup Language </li>
    <li> Is a markup language </li>
    <li> Contain HTML tags and plain text </li>
    <li> Also called as web pages </li>
  </ol>

  </body>
</html>

```

← HTTP Response

Figure 1.12: Response Message Format

1.3.3 HTTP Methods

Web applications allow users to enter information using forms and send it to the server for processing. The data entered into the fields in a form is sent to the Web server through a URL. The data is processed on the Web Server and the appropriate response is generated which is sent back to the user.

For example, on a banking site, the user enters a username and password. The information is sent to the Web server through the request parameters and after validating the data received with the database, the account information is sent as a response. The server treats the data entered in each field as a request parameter. The server can extract the values of each request parameter in the request for processing.

Figure 1.13 shows an example of request parameters.



Figure 1.13: Example of Request Parameters

The HTTP request messages use the given HTTP methods for transmitting request data over the Web. The HTTP methods specified in the request-line element of the request indicate the type of operation to be performed on the resource.

The most commonly used HTTP methods are as follows:

- **GET**

The GET method informs the server to retrieve information from the request URL. The information is passed as a sequence of characters that are appended at the end of the request URL. This forms a query string. For example, a search engine such as www.google.com uses the GET method to retrieve search results for search strings entered by the user.

Here is an example of a query string created when transmitting user data within the request URL.

<http://www.abcBank.co.uk/search?name=john&pass=j7652>

The length of the query string is restricted to 240 to 255 characters depending on the server. Thus, this method cannot be used to send large amounts of data from the forms. Also, the data sent on the URL is visible; hence, this method is not reliable for sending important information such as a credit card number or password.

- **POST**

The POST method is used when sending information, such as credit card numbers or information to be saved in the database. Data sent using POST is in encrypted format and not visible to the client and there is no limit on the amount of data being sent. The data is passed to the server in the body of the HTTP message. Thus, the request cannot be bookmarked or emailed. Figure 1.14 shows the request structure with HTTP POST method.

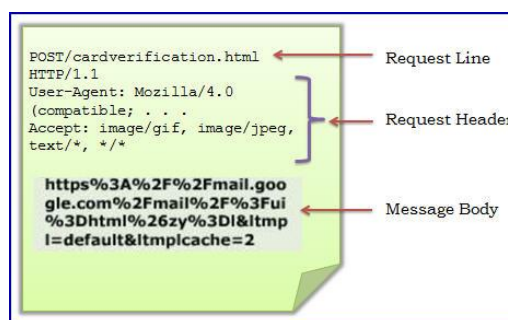


Figure 1.14: POST Method

- **HEAD**

The HEAD method is functionally same as GET, except that the client uses HEAD method to receive only the headers of the response and not the message body. This method is advantageous when the user wants to check the characteristics of a resource without actually downloading the complete resource.

- **PUT**

The PUT method is used to request the server to store the data enclosed in the HTTP message body at a location provided in the request URL.

- **DELETE**

The DELETE method is used to request the server to delete file at a location specified in the request URL.

- **OPTIONS**

The OPTIONS method can be used to query the server on the methods supported for a particular resource available on the server.

- **TRACE**

The TRACE method is used for debugging and testing the request sent to the server. It requires the user to echo the sent request back. This method is useful when the request sent to the server reaches through the proxies.

- **PATCH**

The PATCH method is used to request modification of a file based on the partial update defined in the enclosed representation within the request. This method is useful for conserving bandwidth by updating only a portion of a file or document without transferring it entirely.

1.3.4 HTTP/3 Protocol

HTTP/3 is the newest version of the HTTP protocol and is faster and more efficient than the previous version, HTTP/2, HTTP/1.1, or HTTP. It is binary, not textual, and fully enables multiplexing. This protocol introduces Quick UDP Internet Connections (QUIC) as a substitute for the conventional Transmission Control Protocol (TCP) at the transport layer, to enhance connection speeds and integrate encryption directly into the protocol for secure communication. It is faster, simpler, and more robust than its predecessors due to certain features such as:

- **Faster Connections:** HTTP/3 deploys the QUIC protocol, which is founded on User Datagram Protocol (UDP), offering greater efficiency, particularly when users transition between different network types such as Wi-Fi and mobile data.
- **Resuming Visits Instantly:** HTTP/3 allows for immediate connection resumption with servers that have been previously accessed, expediting reconnecting processes.
- **Stronger Security:** HTTP/3 automatically encrypts data, making it harder for hackers to steal information.
- **Reduced Lag:** By dealing with data loss better, HTTP/3 minimizes delays (latency) users might experience while browsing.
- **Smoother Network Switching:** With QUIC's support for connection migration, changes in a user's IP address, such as when switching from cellular data to Wi-Fi, do not disrupt the ongoing connection.

1.4 Java in the Web

The potential of Java as a server-side development platform extends the power of the Web in creating Web applications. However, the first release of Java did not support the APIs for developing Web applications but was intended for use in embedded devices. Also, during that time, even the Web pages contain static content.

Soon with the development of Java Applets that support the execution of Java code within the browsers, Java emerged as a powerful platform for developing Web components that can be accessed on various platforms, enabling dynamic functionality on Web pages.

Web components are functional software running on the Web server, which handles the incoming HTTP request and provides responses. Java Web applications are composed of several key components, which are as follows:

- **Servlet:** Servlets are Java classes that dynamically process HTTP requests and construct responses.
- **JSP:** A JSP page is a text document that creates dynamic Web content, usually to display content in the Web browser.

Servlet, a key component in Web development, emerged as a pivotal element in Java Web technology. However, before Servlets, there were technologies such as Common Gateway Interface (CGI), Active Server Pages (ASP), and so on, which were practiced for generating dynamic content for the pages.

1.4.1 Prevalent Server-Side Technologies

Currently, there are various server-side technologies, which execute various programs on the Web server.

These are as follows:

- Proprietary Web server API (ISAPI)
- Server-side JavaScript
- Java Servlets
- Jakarta Server Pages (JSP)

➤ **Proprietary Web Server APIs (ISAPI, NSAPI)**

Microsoft and Netscape, each developed their own APIs that allow developers to write server applications very quickly. The server application can be used as a shared library. Several libraries are loaded into the same process area with the Web server. These libraries can service multiple requests without creating a new process. These can either be loaded when they are required. Once any of these have been idle for a set amount of time, they are unloaded by the Web server from the memory.

➤ **Server-side JavaScript**

The server-side uses JavaScript as its scripting language. It is an extension of the core JavaScript language with features such as database access, session management, and inter-operability with server-side Java classes. It works with Node.js, an open-source server environment.

➤ **Servlet - Solution to CGI Problems**

Servlet are Java codes that are used to add dynamic content to a Web server. The processing of Servlet is similar to the CGI program, as it also responds to HTTP

requests and generates dynamic responses. However, it is different in its execution.

There is only a single instance of Servlet created on the Web server. To service multiple client's requests, the Web server creates multiple threads for the same Servlet instance. Each thread handles request received from the client and generates response that is sent to the Web engine, which in turn sends the response to the client.

Figure 1.15 shows the working of the Servlet.

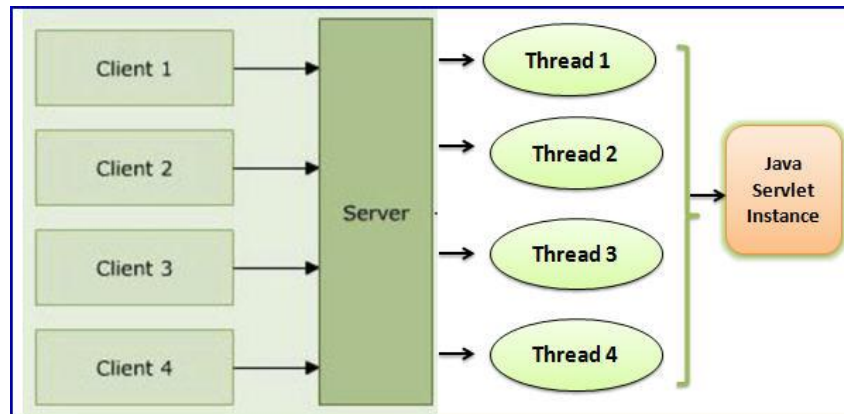


Figure 1.15: Servlets

The initialization code in Servlets is executed only once at the beginning of the program and is refreshed automatically on receiving separate requests. Whereas in CGI, separate instances of the program are being invoked each time a process is executed.

The multi-threading capability of servlets aids in managing individual requests by assigning resources to distinct threads. This enhances performance, thus improving the efficiency of servlets.

➤ Jakarta Server Pages (JSP)

Jakarta Server Pages, which were formerly called Java Server Pages, resemble HTML pages embedded with Java code, simplifying the task for Web designers. The JSP specification enables developers to utilize HTTP and XML markup, custom processing tags, expression language and embedded Java code for processing and displaying dynamic Web content. While servlets are used in managing backend processing and business logic, JSPs are tailored for developing the user interface and front-end elements. Utilizing both technologies within a single application can maximize their collective advantages. A JSP page is essentially an HTML page with optional tags and/or embedded code, denoted by a .jsp page suffix.

1.4.2 Merits of Servlet

Servlets are written using Java, which makes extensive use of Java API.

The merits of servlets are described as follows:

Enhanced efficiency	Servlet is required to be executed only once at the beginning with the initialization code. Thereafter, it gets auto-refreshed each time a request is sent for execution.
Ease of use	Servlets do not have too many complexities. It is <u>simply basic</u> Java along with HTML implemented within the Java code, which increases the ease of use.
Powerful	The usage of Java APIs makes the Servlets a powerful language for server-side scripting.
Portable	Java is platform-independent and since Servlet uses Java code for writing scripts, it can be executed on any platform.
Safe and cheap	Usage of Java codes provides high security of data to be sent and received thereby maintaining safety.

1.4.3 Demerits of Servlet

Servlets, although widely used in Web development due to their Java-based nature and extensive Java API support, also have certain drawbacks, as follows:

- **Complicated Development Process**
Creating servlets can be slow and challenging, requiring careful attention to detail and time-consuming debugging.
- **Tight HTML-Java Connection**
Close integration of HTML and Java code within servlets often presents difficulties in maintenance. The reason for this is that servlets commonly mix presentation elements with business logic, complicating the process of implementing changes.
- **Extensive Knowledge Required**
Building servlets requires a deep understanding of Java, making it challenging for non-Java developers to engage.

➤ **Lack of Modern Features**

Servlets do not include typical modern Web development features such as templating, validation, security, and dependency injection.

➤ **Dependency on Java Runtime**

The reliance on Java Runtime Environment (JRE) for servlet execution restricts deployment options, limiting flexibility in server configurations.

1.5 Web Application Workflow

The sequence of events from request to response in a Java Web application involves several steps that facilitate communication between the client and the server. Each step plays a distinct role in ensuring the proper handling and processing of the client's request and the subsequent delivery of the response. Figure 1.16 illustrates the sequence of events.

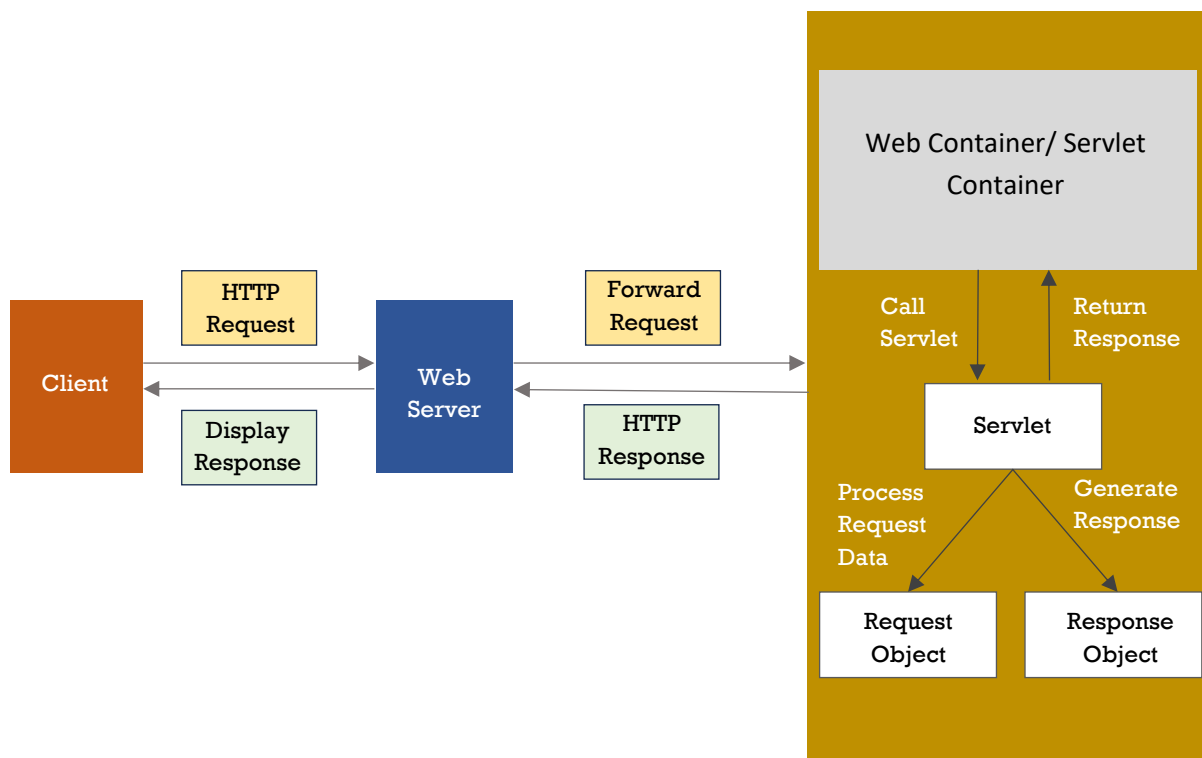


Figure 1.16: Web Application Workflow

1. The client initiates an HTTP request to the Web server, specifying the resource required.
2. Upon receiving the request, the Web server forwards it to the Web container for processing. The Web container may operate within the same process as the host Web server or run in a separate process on the same machine or even on a different machine altogether from the Web server.

3. The Web container, configured with servlet mappings, determines the appropriate servlet to handle the request.
4. Upon identification, the Web container invokes the corresponding servlet, passing along the request and response objects.
5. The servlet extracts relevant information from the request object, such as user data and parameters.
6. Utilizing the programmed logic, the servlet processes the request to generate a suitable response.
7. Using the response object, the servlet constructs the response data.
8. The servlet then, sends the response back to the servlet container for further processing.
9. The servlet container ensures proper handling of the response and manages its transmission.
10. After processing, the Web container forwards the HTTP response to the Web server.
11. Finally, the Web server delivers the response to the client, which renders it in the browser.

1.5.1 Capabilities of a Web Container

A Web container also referred to as a Servlet container, is a vital part of Web servers. It manages servlets, maps URLs to specific servlets, and ensures access rights. It handles requests for servlets, JSP files, and other server-side code files. The container provides a runtime environment for Web components, including security and deployment services. It creates servlet instances, manages request and response objects, and handles loading and unloading processes. Servlet containers play a key role in delivering dynamic Web content efficiently.

- **Servlet Container Lifecycle**

The lifecycle of a servlet container consists of three main phases, which are as follows:

- **Initialization Phase**

During the initialization phase, the servlet container loads the servlet either at server startup or upon receiving the first request. It then, instantiates the servlet and invokes its `init` method. This method is called only once in the servlet's lifecycle and is responsible for initializing the servlet. It receives a `ServletConfig` object containing initialization parameters from the Web application.

➤ **Request Handling Phase**

In the request handling phase, the container creates `HttpServletRequest` and `HttpServletResponse` objects and then, calls the `service` method of the servlet. Each request is processed in a separate thread to enable concurrent execution. Within the `service` method, the servlet accesses request information using the `HttpServletRequest` object. It then, calls the appropriate method (`doGet`, `doPost`, and so on) based on the HTTP request type and formulates a response using the `HttpServletResponse` object.

➤ **Destruction Phase**

The destruction phase occurs when the servlet is no longer required by the server. The servlet container invokes the `destroy` method to perform cleanup before unloading the servlet. This method is called just once in the servlet's lifecycle. It releases any held resources and prepares the servlet for garbage collection.

Figure 1.17 illustrates the phases of a servlet container's lifecycle.

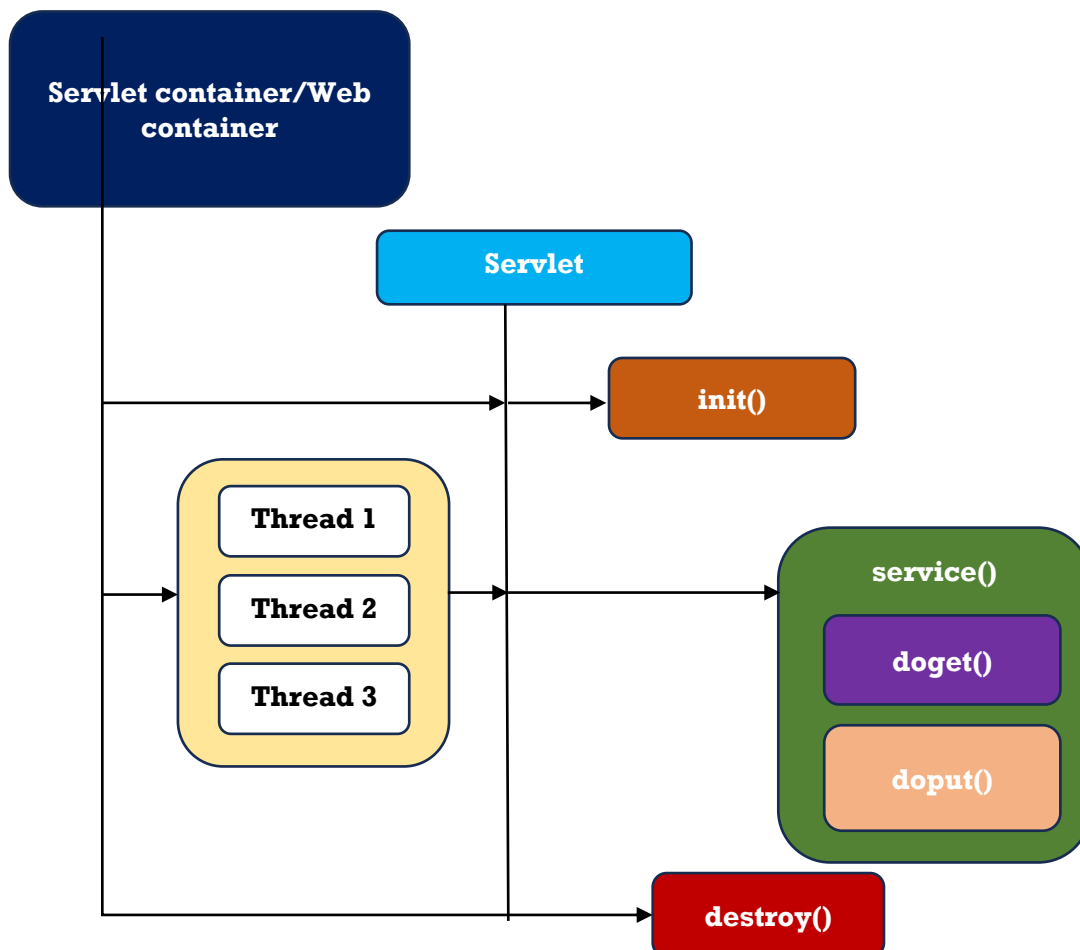


Figure 1.17: Servlet Container Lifecycle

Summary

- An application is a collection of programs designed to perform a particular task. Different types of applications are designed for different purposes.
- A Web application is a software application that runs on a Web server.
- Web application architecture refers to the application logic distribution between a client and a server.
- The primary technologies utilized for communication on the Web include HTML and HTTP. HTML serves as a presentation language, allowing Web designers to craft visually appealing Web pages.
- The requests and responses sent to a Web application from one computer to another computer are sent using HTTP.
- The HTTP request messages use the HTTP methods for transmitting request data over the Web.
- Web applications are packaged in .war files, which allow the content of the Web application to be accessed from the application's context root.
- Servlets, while Java-friendly, pose challenges in development and maintenance due to their complexity and reliance on Java runtime.
- Web container manages servlets, maps URLs to specific servlets, ensures access rights, and provides a runtime environment for Web components, facilitating efficient delivery of dynamic Web content.

Check Your Progress

1. Identify the features of a Web client-server model.

(A)	The architecture of Web client-server model can be one-tier, two-tier, three-tier, or N-tier.
(B)	Web applications have higher maintenance cost.
(C)	Code related to presentation, business, and data access logic are all clubbed together in the one-tier architecture of Web application.
(D)	Web-applications are platform independent.

A.	A, C	C.	B, C
B.	B, D	D.	A, D

2. Which of the following statements about Request and Response message structure are true?

(A)	Request message structure consists of only header information.
(B)	Header information contains information such as server, last modified date, content-length, and content type.
(C)	Request line returns the User-Agent along with the status code header.
(D)	Response message structure consists of status line and header information.

A.	A, C	C.	A, D
B.	B, D	D.	C, D

3. Identify the accurate characteristic of the HTTP request GET methods.

(A)	The GET method is utilized to retrieve information over the Web.
(B)	Information sent via the POST method is not encrypted.
(C)	The POST method serves for obtaining responses over the Web.
(D)	The query string length in the GET method is constrained to 240 to 255 characters.

4. Which of the following options is a limitation that is overcome by a Web server?

(A)	Username and password are retained by the browser.
(B)	Each time user is prompted for username and password for accessing subsequent pages.
(C)	Dynamic Web content can be managed from the client end.
(D)	It is feasible to maintain static Web content from the customer's end.

5. Which of the options are disadvantages of using CGI?

(A)	Reduced Efficiency
(B)	Ease of use
(C)	Reloading PERL interpreter
(D)	Safe and cheap
(E)	The single threading property of servlets helps in handling separate requests

A.	A, B	C.	A, C
B.	B, C	D.	B, D

6. Identify the merits of servlets.

(A)	Servlets are easy to use and enhance efficiency due to their auto-refresh feature.
(B)	Servlets are portable and can run on any platform due to their Java-based nature.
(C)	Servlets are powerful due to their use of Java APIs, contrary to being less efficient than other scripting languages.
(D)	Servlets are safe and cost-effective for development, ensuring data security.
(E)	Initialization code in servlets is executed only once at the beginning, not every time the program runs, thus improving efficiency.

A.	A, C	C.	A, B, C
B.	B, D	D.	A, B, D

7. What is the primary responsibility of the `init` method in the servlet lifecycle?

(A)	Handling client requests
(B)	Initializing the servlet
(C)	Generating dynamic Web content
(D)	Destroying servlet objects

Answers

1.	A
2.	B
3.	D
4.	C
5.	C
6.	D
7.	B

Try It Yourself

1. Identify and classify three applications based on their environment and accessibility one desktop application, one networking application, and one Web application. Provide examples for each category.
2. Compose a sample HTTP response message format containing HTML content that a Web server might send to a client.
3. Create a list of five commonly used HTTP methods and provide a scenario for each method where it would be appropriate to use.
4. Outline the benefits of using the HTTP/3 protocol for improving the performance of Web applications.

Introduction to Jakarta EE

Welcome to the Session, **Introduction to Jakarta EE**.

Web applications that have the potential to significantly improve organizational effectiveness, reduce inefficiency, and enhance the quality and security of data are becoming more acceptable in business organizations today. Jakarta EE, an open-source framework, is used for developing Java-based Web applications, currently referred to as cloud-native Java applications. Jakarta EE provides components to encapsulate business logic in addition to a range of APIs and tools specifically designed to meet the requirements of enterprise-level Web development. Its stability and maturity make it a trusted choice for developers globally.

This session discusses the key specifications of the Jakarta EE platform, its architecture, profiles, application model, components, advantages, and disadvantages. In addition, it explains the features, capabilities, and installation of Eclipse IDE. The session ends with an explanation of the procedure for installing Jakarta EE components.

In this session, you will learn to:

- Explain the Jakarta EE platform
- Describe Jakarta EE architecture and profiles
- Describe Jakarta EE application model and its components
- Identify the microservices and cloud-native principles of Jakarta EE
- List the advantages and disadvantages of Jakarta EE
- Explain the key Jakarta EE specifications
- Explain the installation of JDK and the process of setting up the system variables
- Identify the features and capabilities of Eclipse IDE
- Explain the installation of Eclipse IDE

2.1 Background of Jakarta EE

The Jakarta EE platform was previously known as Java EE, that is, Java Enterprise Edition. It was developed originally by Sun Microsystems and managed by Oracle Corporation. The Java Community Process or JCP was instrumental in maintaining the platform. In 2017, Jakarta EE was taken over by the Eclipse Foundation. The two management entities, Oracle and Eclipse started disagreeing over a few major matters. Trademarks such as 'javax' and 'java' could no longer be used. Hence, the platform was renamed as Jakarta.

2.1.1 Current Status of Jakarta

Jakarta EE applications are commonly developed in Java. The Jakarta EE suite offers a comprehensive set of tools that simplify the development process for enterprise-level Java applications. Enterprise applications meet the stringent requirements for security, scalability, and reliability demanded by large corporations. These applications may encompass batch processing, Web applications, Web services, and so on.

Developers can choose the specific services they require, which can be used independently or alongside other frameworks and libraries. Thus, by handling a lot of the underlying infrastructure, Jakarta EE frees up developers to focus on creating the distinctive features and logic that drive their application's functionality. This leads to more efficient development cycles and potentially higher-quality software. Additionally, as Jakarta EE comprises open-source standards supported by a variety of vendors, developers are not tied to any single vendor. Each standard is thoughtfully developed, reflecting contemporary trends and practices.

Some of the important highlights of Jakarta EE include:

It has a better alignment with Java Standard Edition (SE) to ensure a more seamless development experience.

It uses the JESP or Jakarta EE Specification Process in order to provide guaranteed documentation of specification in the form of JavaDocs.

It provides pre-defined Application Programming Interfaces (APIs) for each specification.

Documents and a Technology Compliance Kit (TCK) are included with every specification. These two include details on the tests that must be carried out to verify that the specifications are being followed in a Web application.

2.1.2 Objectives of Jakarta EE

The Jakarta EE platform uses a simple programming model. XML deployment descriptors are optional. Instead, it allows developers to embed configuration information directly into Java source files using annotations. The Jakarta EE server will then, automatically configure the components at the time of deployment and execution. The goal is to enable developers to define the configuration details near the relevant code elements. Additionally, Jakarta follows the Contexts and Dependency Injection (CDI) technique, where objects will not create their own dependencies or resources they require, such as other objects or services.

Instead, these dependencies are 'injected' into the object by an external container. The goals of adopting the CDI technique are as follows:

Cleaner code

By separating the creation of resources from the business logic component, DI makes sure that the code becomes easier to read and manage, and less prone to errors.

Increased adaptability

DI makes it easier to modify dependency implementations without affecting the dependent components. This flexibility comes in particularly useful when the application must adjust to new standards or advances in technology.

Improved reusability

Components designed with DI in mind are more reusable because they do not have hard-coded dependencies. This means they can be used in different contexts or applications with ease.

Streamlined testing

Testing is much simpler since, DI enables developers to inject mock dependency implementations. This results in more focused and reliable unit tests by enabling more efficient control over the testing environment.

Benefits of Using Jakarta EE:

Jakarta EE makes development and implementation of Java Enterprise applications easier and quicker.

Using the APIs and services provided by the Jakarta EE platform, developers can reduce the duration and complexity of app development while at the same time improving the performance of their applications.

2.1.3 Latest Version of Jakarta EE

The latest version of Jakarta EE is Jakarta EE 10. Jakarta EE 10 brings new capabilities for creating streamlined, modern, and efficient cloud-native applications in Java.

2.2 Jakarta EE Architecture

Jakarta EE architecture enables the building of multi-tier applications through its containers, components, and services that are built on top of Java Standard Edition (SE). Typically, a multi-tier application is made of three layers comprising the Presentation Tier, Application or Business Tier, and the Data Tier.

Figure 2.1 shows the architecture of a multi-tier application with three layers.

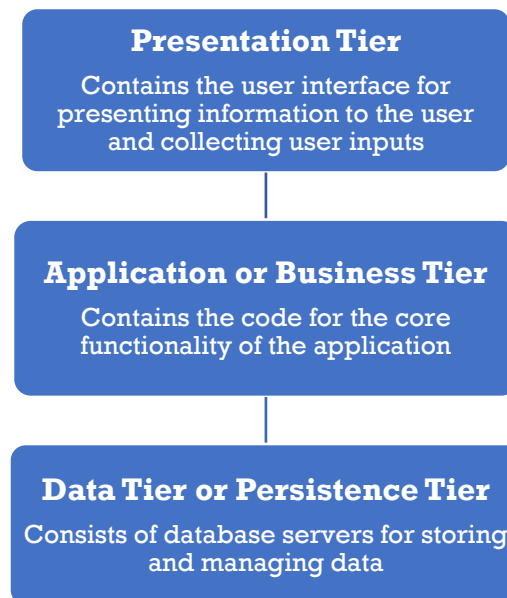


Figure 2.1: Multi-Tier Application Architecture

Jakarta EE architecture provides a standard platform or containers for each tier of a multi-tier application. A container is a runtime environment that houses different Jakarta EE components and creates the environment for them to run within it by making use of the services provided by the Jakarta EE server. The three containers of Jakarta EE are as follows:

- **Application Client Container:** It acts as the interface between various client-side Java applications such as applets and the Jakarta EE server-side application such as Jakarta Servlets, Jakarta Server Pages (JSP), and Enterprise Java Beans (EJB). It resides on the client machine.
- **Web Container:** It contains components such as Jakarta Servlets, JSP, and Jakarta Faces, which are responsible for interacting with the user interface.
- **Business Container:** It contains EJB and CDI. The EJB component manages the execution of enterprise beans, which contain the business logic. CDI manages the lifecycle of beans and provides dependency injection services to create loosely coupled components.

The three containers within Jakarta EE interact with the Enterprise Information System (EIS) to perform operations related to data persistence, transactions, and more. ESI are large-scale, legacy systems that store and manage enterprise data.

Figure 2.2 depicts the architecture of Jakarta EE.

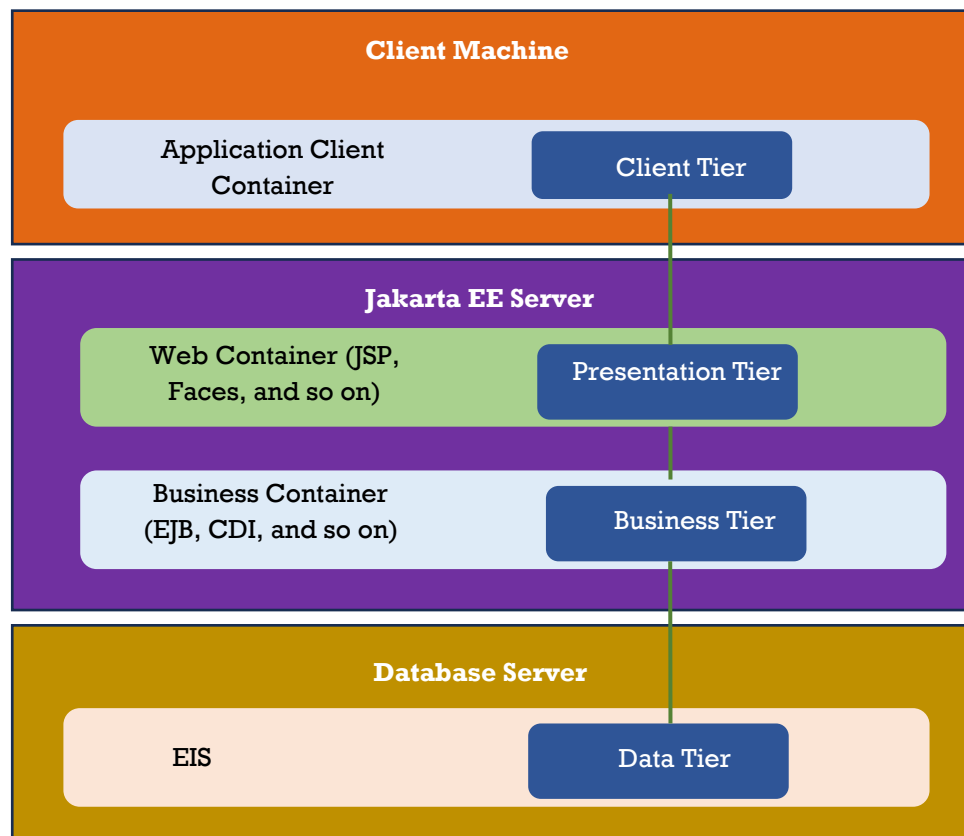


Figure 2.2: Jakarta EE Architecture

The Jakarta EE server also provides different services such as:

- Deployment, management, and execution APIs to support the components
- Naming and directory services APIs (to locate components and resources)
- Transaction management services APIs (to ensure read/write operation to or from a database or other systems)
- Security services APIs (for authentication and authorization)
- Persistence services APIs (to enable object-relational mapping)
- Messaging services APIs (for point-to-point and publish-subscribe messaging)

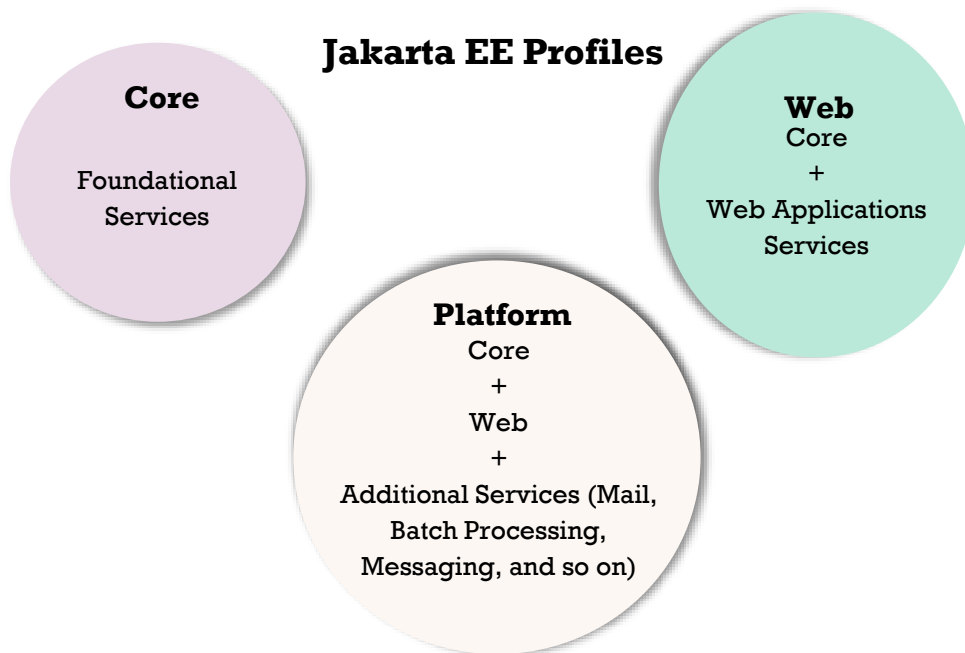
The Jakarta containers allow the application components within them to access the services provided by the server and communicate with each other.

2.3 Jakarta EE Profiles

Jakarta EE is structured into three distinct profiles to cater to different developer communities and application types.

The three profiles are as follows:

- **Core Profile:** This foundational profile encompasses critical APIs such as CDI and Java API for RESTful Web Services (JAX-RS), along with fundamental resource management capabilities. Use this profile for building non-Web applications and when only the core functionalities are sufficient.
- **Web Profile:** Expanding on the Core Profile, this profile incorporates further APIs tailored for Web development such as Servlets, JSPs, and more. This profile is ideally suited for developing Web applications.
- **Full Platform:** This comprehensive profile integrates all features from the Core and Web Profiles, plus any additional APIs included within the Jakarta EE platform. To utilize the complete range of Jakarta EE features, opt for the Full Platform.



2.4 Jakarta EE Application Model and Its Components

Jakarta EE architecture follows a multi-component model, offering an alternative to the traditional single-unit application framework. Using the multi-component design approach, developers can build enterprise applications by assembling various self-contained components. Each component in this model represents a functional unit that can interact with other components through well-defined interfaces and services provided by the Jakarta EE platform.

Some of the important features of the Jakarta EE application model are as follows:

- **Assembled Components**
Applications are crafted by integrating components that adhere to various Jakarta EE specifications. These specifications outline APIs for a range of functionalities, including dependency injection, Web development, and security measures.

- **Defined Profiles**

The three categories of Jakarta profiles encompass a curated collection of specifications tailored for particular types of development projects.

- **CDI**

CDI is a fundamental principle in Jakarta EE applications, facilitating the management of inter-component dependencies. This fosters a design that is loosely coupled and more straightforward to test.

- **Alignment with Microservices**

The architecture of Jakarta EE 10 is conducive to microservices. The minimalistic design of the Core Profile is particularly well-suited for creating and deploying self-reliant, modular microservices.

- **Annotations**

Jakarta EE employs annotations to streamline configuration processes and minimize repetitive code. By annotating classes and methods, developers can specify operations such as dependency injection, interceptors, and Web services.

2.5 Microservices and Cloud-Native Principles of Jakarta EE

Most organizations nowadays are either opting for the microservices architecture or Cloud to conduct their operations. Jakarta EE blends with both these easily and helps build applications that are scalable, easily tested and gel well with the ever-changing requirements.

2.5.1 Microservices Architecture

Jakarta EE offers a well-suited foundation for developing and deploying microservices applications. Its core design principles and features directly address the requirements of distributed, loosely coupled, and independently deployable services.

Here is how its design and features empower microservice development:

- **Modular Architecture:** Jakarta EE employs a modular architecture, allowing developers to select and utilize only the necessary components. This minimizes overall application footprint and complexity, aligning with the philosophy of microservices, which promotes building small, focused services.
- **Lightweight Runtime:** The Core Profile within Jakarta EE provides a lightweight runtime environment. This characteristic is crucial for microservices, where resource efficiency is paramount. The lightweight runtime facilitates rapid startup, shutdown, and scaling of microservices as required.
- **Dependency Injection with CDI:** Jakarta EE facilitates dependency management between application components through CDI. This enables independent development and testing of microservices, fostering clean separation of functional components.

- **Restful Web Services Support:** Jakarta EE fosters the creation of RESTful Web services through JAX-RS. This aligns well with the microservices architecture, where services often communicate via HTTP using REST principles.
- **Containerization Compatibility:** Microservices are often deployed within containers, which provide isolated execution environments. Jakarta EE applications are well-suited for containerization, as they can be packaged with only the essential runtime dependencies.

2.5.2 Cloud-Native Principles

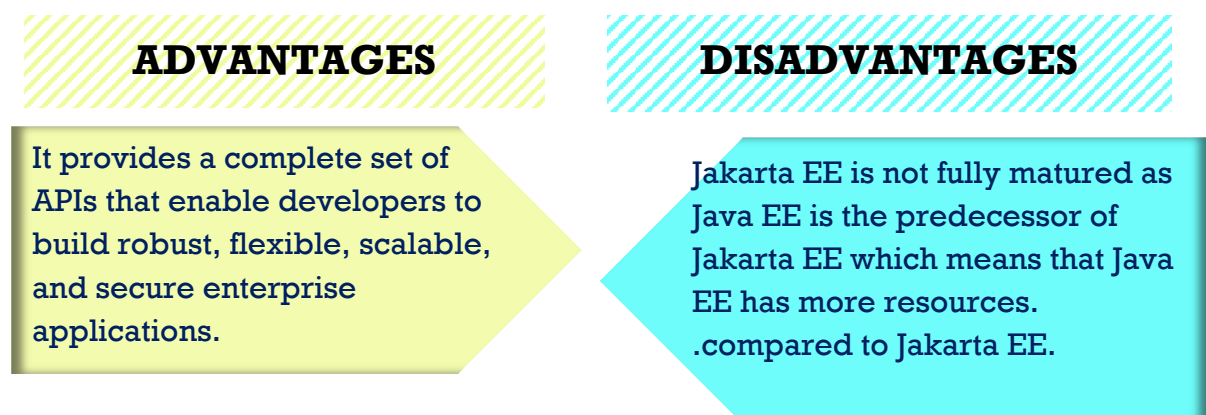
Jakarta EE applications are deployable to different cloud platforms. Hence, they are not fixated with any specific cloud provider. This makes the applications flexible enough to make vendors wade through various platforms. It also helps in scaling the applications in accordance with a variety of cloud features.

Additionally, Jakarta EE offers containerized packaging of applications that makes them deployable across different cloud platforms. Each individual service can be scaled easily if required due to these containers. With extensions such as MicroProfile, Jakarta EE is able to develop cloud-native microservices with customized APIs and fault tolerance features.

Jakarta EE also aligns well with DevOps, Continuous Integration (CI) and Continuous Delivery/Deployment (CD) approaches as it seamlessly integrates with these methodologies along with other technologies such as Kubernetes, Docker, and so on. Thus, its developers are empowered to leverage cloud agnostic applications that are resilient, agile, and efficient.

2.6 Advantages and Disadvantages of Jakarta EE

Similar to other technological advancements, Jakarta EE too has some advantages and disadvantages.



Standard solutions are provided for common challenges faced by enterprise such as security infringement, transaction mismanagements and lost control over concurrency.

Some application tools and servers do not support the functionalities of Jakarta EE completely.

Due to its lightweight Core Profile, it blends well with microservices as well as cloud-native environments.

For smaller applications, Jakarta EE proves to be complex and heavyweight as it has a comprehensive set of APIs.

It allows for platform-independent applications which makes them portable and deployable across different environments and avoids making codebase changes.

It has more infrastructure expenses due to high concurrency and more memory consumption.

There is reduced footprints in microservices architecture and minimal level of complexity due to containerization.

Initiation of a new project might require extra migration efforts and steep learning curve for new developers using the platform.

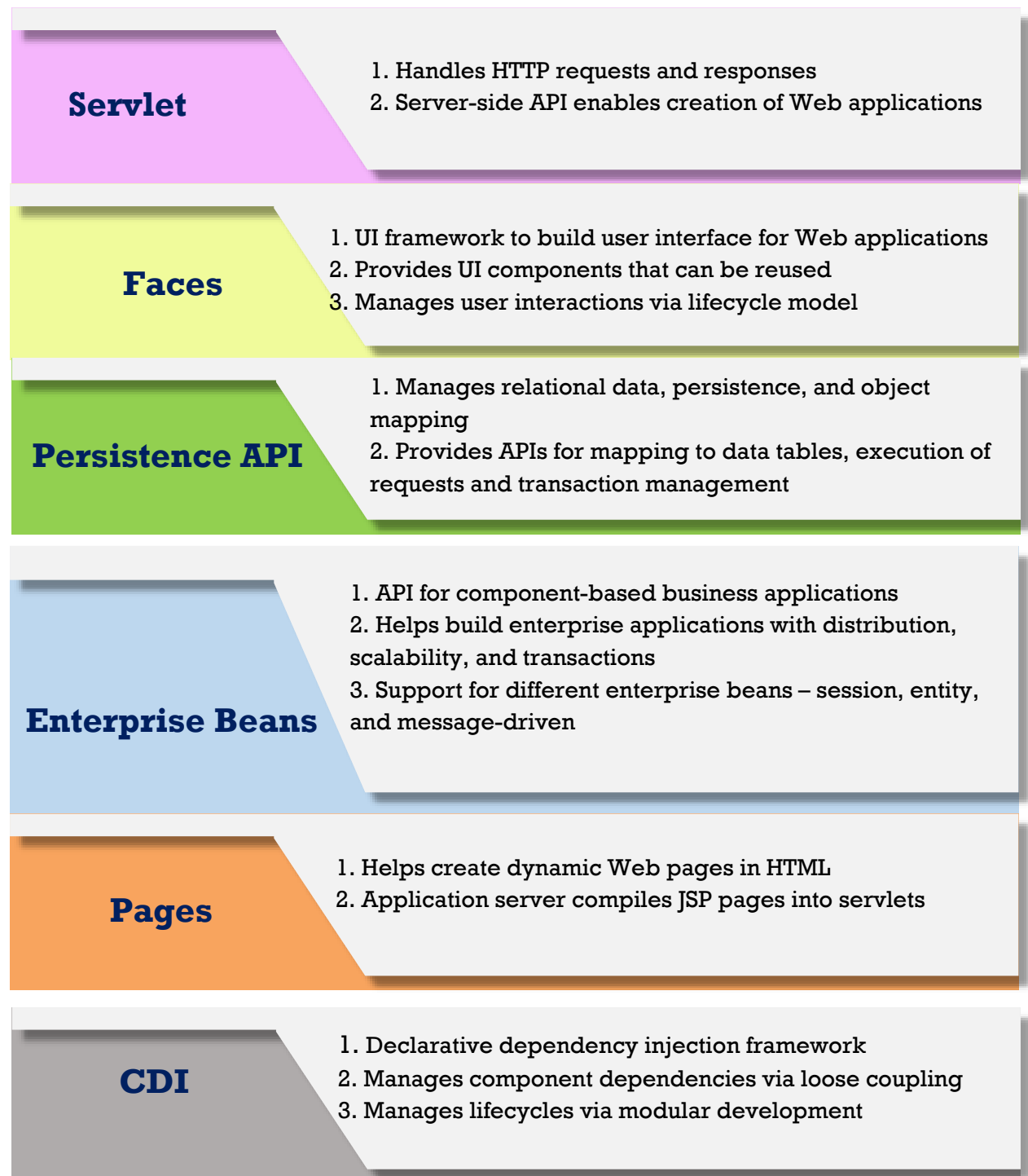
New annotations are introduced promoting open-source collaboration, reducing boilerplate code and simplifying development.

The Jakarta EE community has faced significant fragmentation due to transition from Java EE to Jakarta EE leading to incompatibility, competition. and confusion.

2.7 Key Jakarta EE Specifications

Jakarta EE has several specifications encompassing different aspects of enterprise application development and management. These specifications ensure

robustness, scalability, security, and standardized foundation to the platform and makes it compatible for different services such as concurrency, transactional management, messaging, and security.



Message Service

1. Messaging applications can be built using API in Java
2. Receive and send asynchronous messages
3. Integration with other messaging systems (example: Apache)

RESTful Web Services

1. Enables RESTful Web services development
2. Provides APIs and annotations to handle HTTP requests
3. Defines resource classes and marshals and unmarshalls data

WebSocket

1. Provides API for communication channels via TCP connection
2. Developers can use Jakarta to build WebSocket endpoints

Authorization

1. API to authenticate custom mechanisms in Jakarta EE containers
2. Integrates external systems to customize the process of authentication

2.8 JDK Installation

As Jakarta EE is built on top of Java SE it extends Java SE with additional specifications for enterprise features. Therefore, having the JDK installed is a prerequisite for setting up a Jakarta EE development environment. The JDK provides the necessary tools and environment for compiling and running Java applications, which is essential for developing Jakarta EE applications. The JDK includes the Java Runtime Environment (JRE), as well as an array of development tools such as the Java compiler (javac), Java Virtual Machine (JVM), and others.

Note: Ensure to download the version of the JDK that is compatible with the version of Jakarta EE planned to use for developing the projects. For example, Jakarta EE 10 is compatible with Java SE 11, 17, and 21, so download either of these versions accordingly.

Installing the Java Development Kit (JDK) is a straightforward process. Here are the general steps to install JDK:

1. Visit the Oracle official Website's downloads page <https://www.oracle.com/in/java/technologies/downloads/>.
2. On the Web page, select **JDK21**. Navigate to **Windows** tab. Then, click the link for **x64 MSI installer** as shown in Figure 2.3.

JDK 22JDK 21JDK 17GraalVM for JDK 22GraalVM for JDK 21GraalVM for JDK 17

JDK Development Kit 21.0.3 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

LinuxmacOSWindows

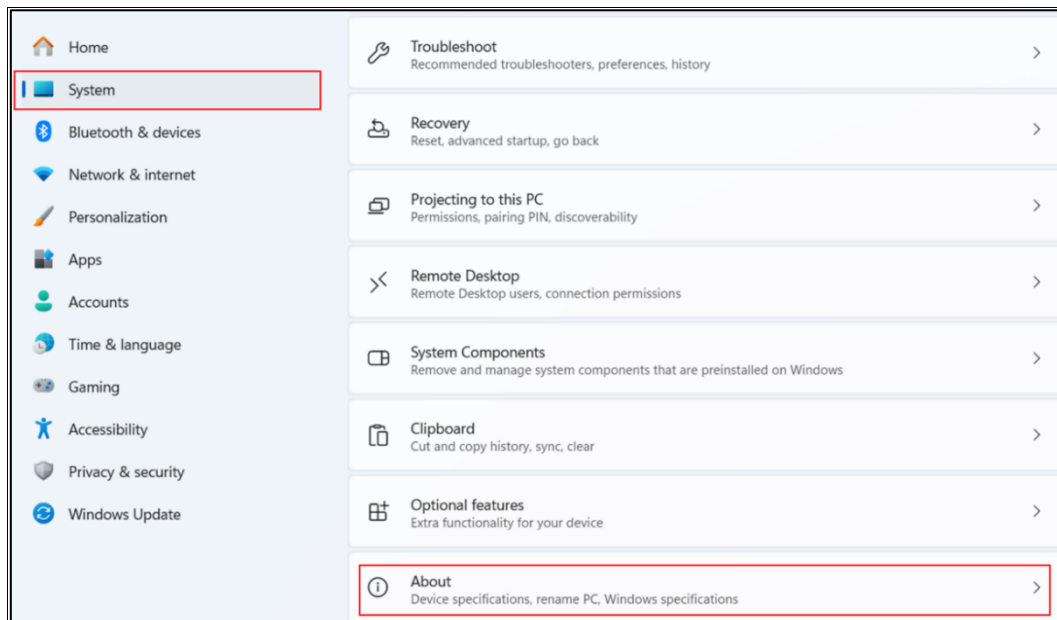
Product/file description	File size	Download
x64 Compressed Archive	185.78 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	164.16 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	162.91 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

Figure 2.3: Settings Window

After the MSI installer package is downloaded, double-click **jdk-21_windows-x64_bin** to start the installation process. Then, follow the instructions provided by the installer.

After the installation is complete, set the environment variable to point to the JDK installation directory. The steps to set the environment variables are as follows:

1. To add the Path variable for the JDK installation, open the **Windows Settings** app in Windows 10 or 11.
2. From the left pane of the **Settings** window, click **System**.
3. In the right pane, scroll down and click **About** as shown in Figure 2.4.

**Figure 2.4: Settings Window**

4. On the **About** page, click **Advanced system settings** as shown in Figure 2.5.

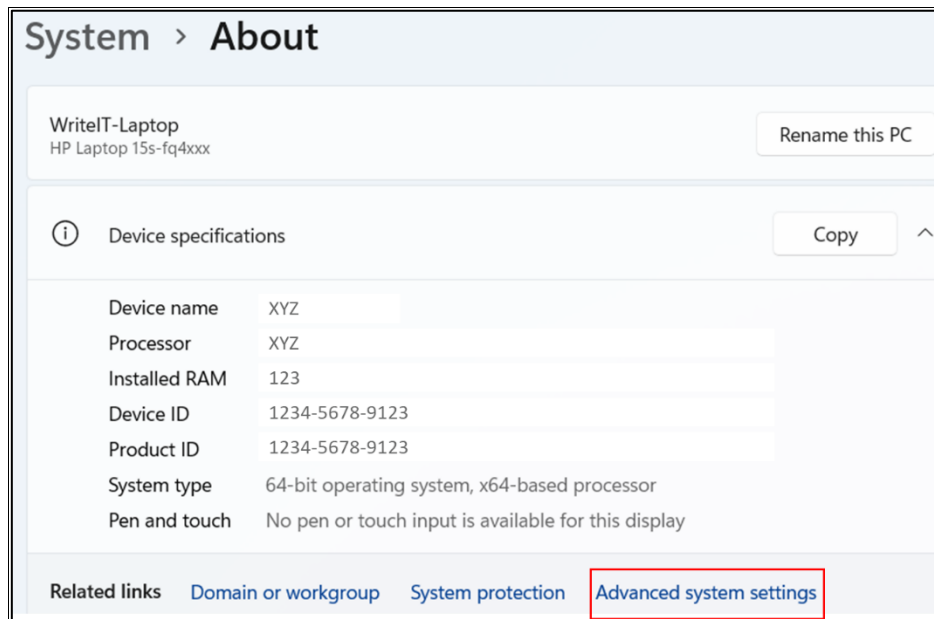


Figure 2.5: About Page

5. In the System Properties dialog box, click Environment Variables as shown in Figure 2.6.

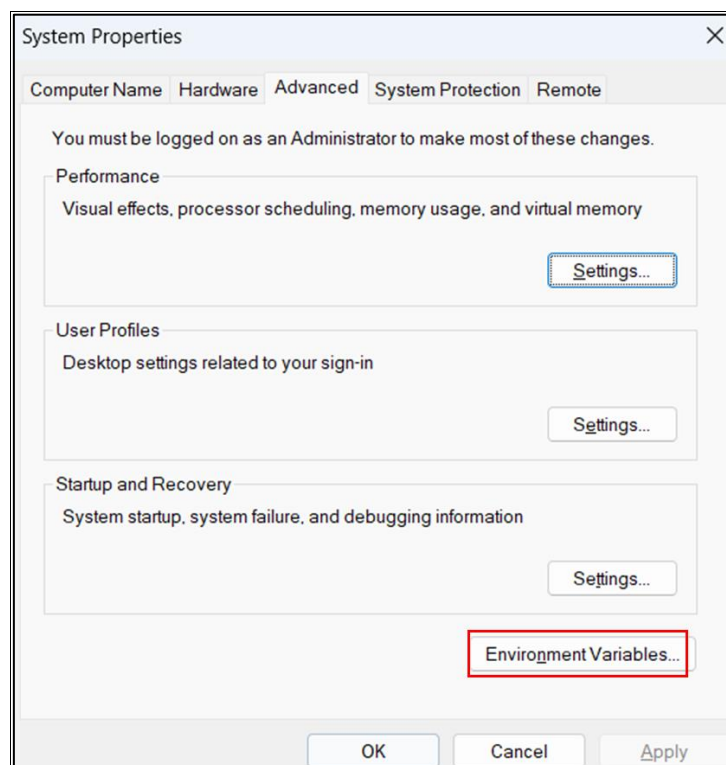


Figure 2.6: System Properties Dialog Box

The **Environment Variables** dialog box is displayed. Alternative approach to launch this dialog box is to open the Control Panel and then, search for **Environment Variables**.

6. In the **Environment Variables** dialog box, under **User variables**, select **Path**, and click **Edit** as shown in Figure 2.7.

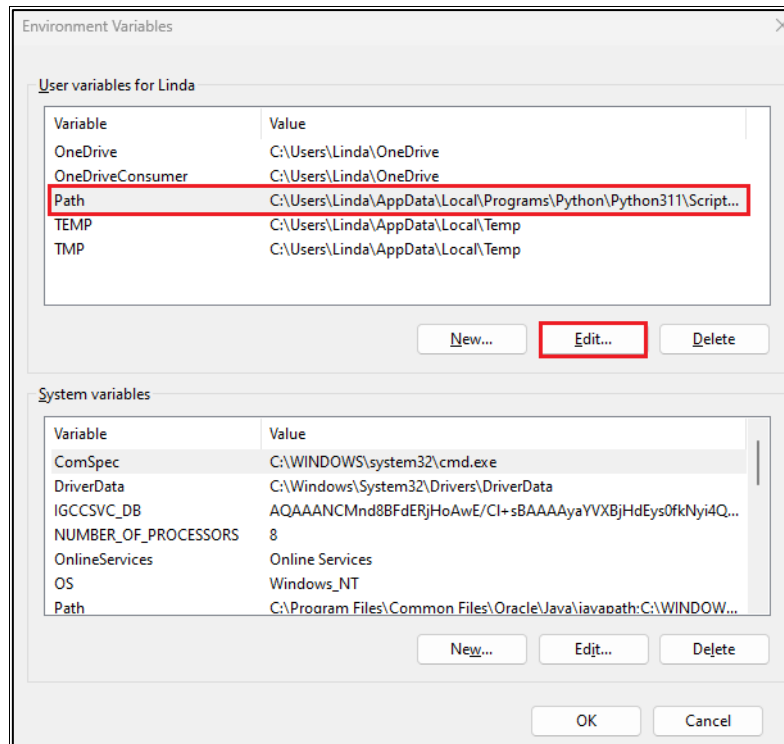


Figure 2.7: Environment Variables Dialog Box – User Variables

7. In the **Edit environment variable** dialog box, click **New**, enter the new path as **C:\Program Files\Java\jdk-21**, and click **OK** as shown in Figure 2.8.

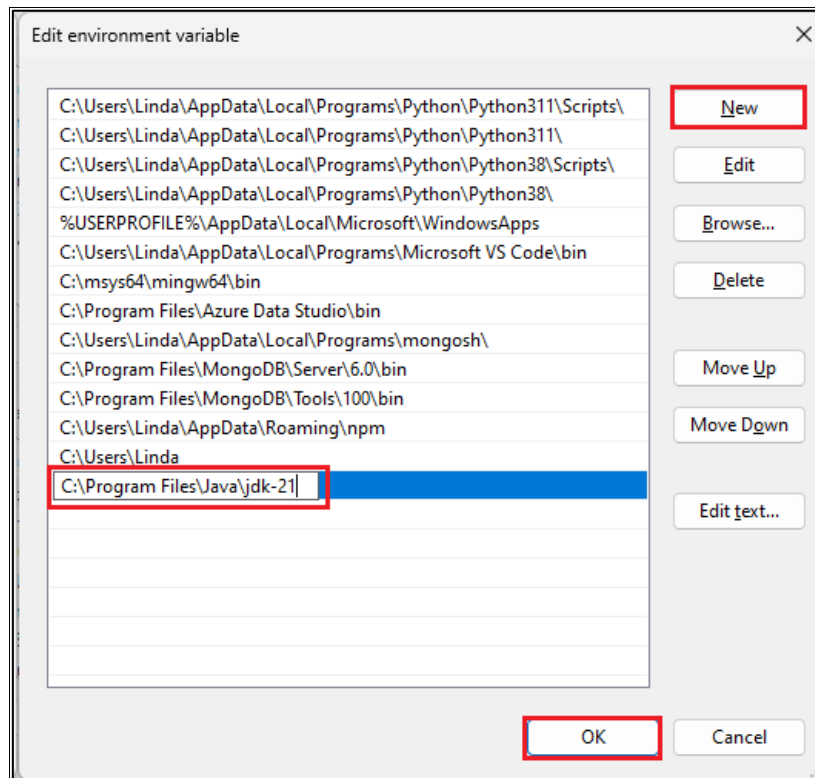


Figure 2.8: Edit Environment Variable Dialog Box – User Variables

8. In the **Environment Variables** dialog box, under **System variables**, select **Path**, and click **Edit** as shown in Figure 2.9.

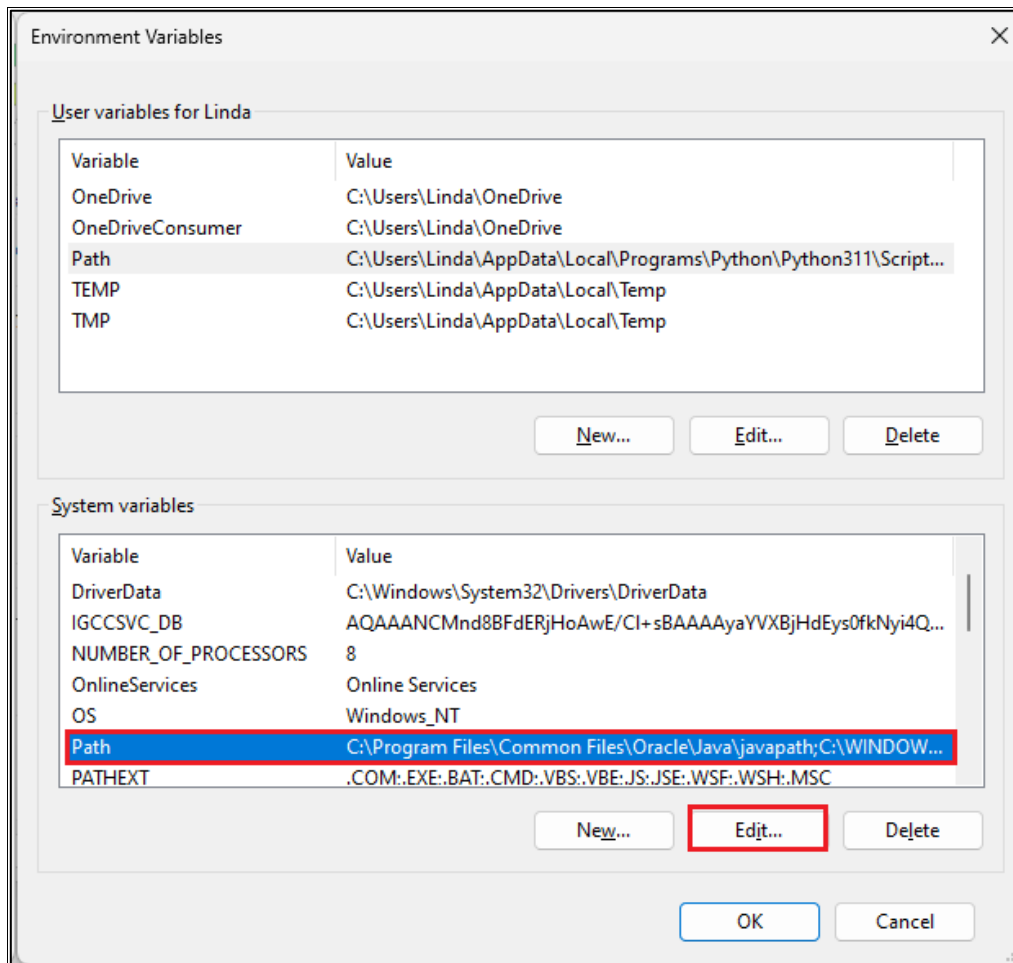


Figure 2.9: Environment Variables Dialog Box – System Variables

9. In the **Edit environment variable** dialog box, click **New**, enter the new path as **C:\Program Files\Java\jdk-21** and click **OK** as shown in Figure 2.10.

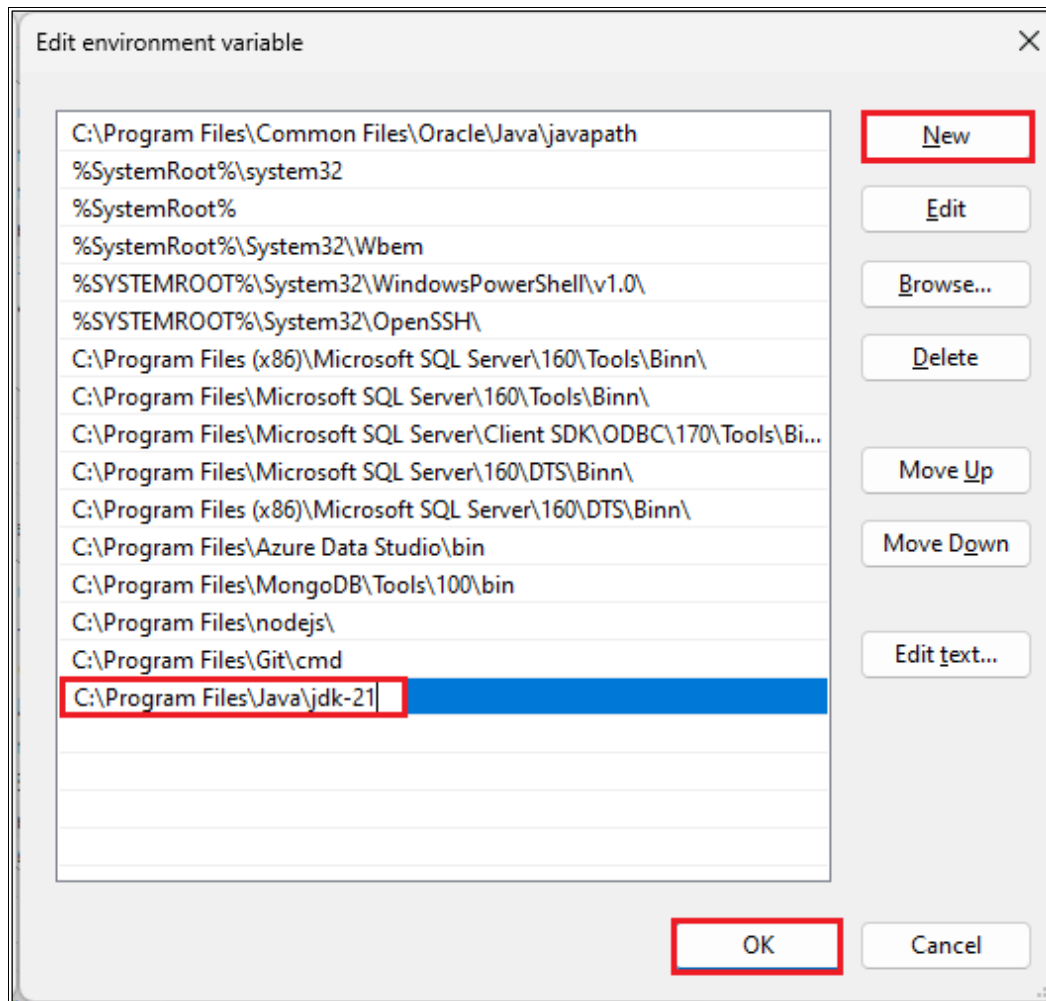


Figure 2.10: Edit Environment Variable Dialog Box – System Variables

10. To set the JAVA_HOME variable, in the **Environment Variables** dialog box, under **System variables**, click **New**.
11. In the **New System Variable** dialog box, enter the **Variable name** as **JAVA_HOME** and the **Variable value** as **C:\Program Files\Java\jdk-21** and then, click **OK**, as shown in Figure 2.11.

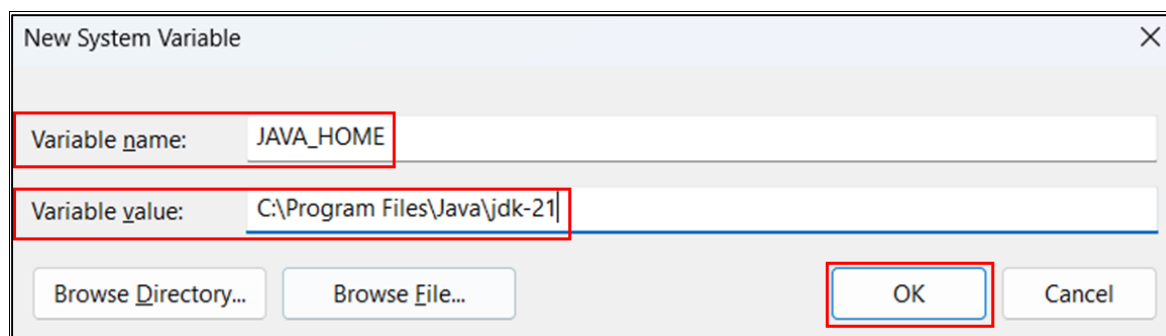


Figure 2.11: Creating the JAVA_HOME Variable

The JAVA_HOME variable is listed under System variables as shown in Figure 2.12.

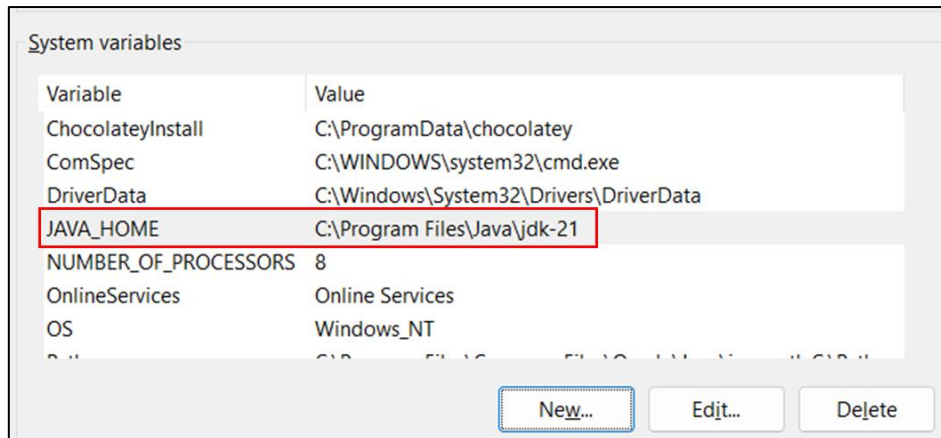


Figure 2.12: JAVA_HOME Variable

12. To add the JAVA_HOME variable to the path, In the **Environment Variables** dialog box, under **System variables**, select **Path**, and click **Edit**
13. In the **Edit environment variable** dialog box, click **New**, enter the new path as **%JAVA_HOME%\bin**, and click **OK**.

Close all open windows. To verify the Java installation, open a Command Prompt window and type the command:

```
java -version
```

The version of Java installed on the system is displayed as shown in Figure 2.13.

```
C:\>java -version
java version "21.0.2" 2024-01-16 LTS
Java(TM) SE Runtime Environment (build 21.0.2+13-LTS-58)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.2+13-LTS-58, mixed mode, sharing)
```

Figure 2.13: Java Version

2.9 Features and Capabilities of Eclipse IDE

IBM originally developed Eclipse with other companies in 2001. It was supposed to be an open-source project initially and has become one of the most accepted Integrated Development Environments ever since.

2.9.1 What is Eclipse IDE?

Eclipse IDE is a platform where developers can create different applications using programming languages such as Java, Ruby, C and C++, Python, PHP, and so on. Although Java is primarily preferred in Eclipse IDE, other programming languages are also supported well. The platform has many features and plug-ins that provide advanced application development.

Some of the major reasons for the popularity of Eclipse IDE are as follows:

- Supports several programming languages
- Has advanced features for Java-based applications
- Simple user interface and installation process
- Updated version available every year
- Provides extensive plug-ins

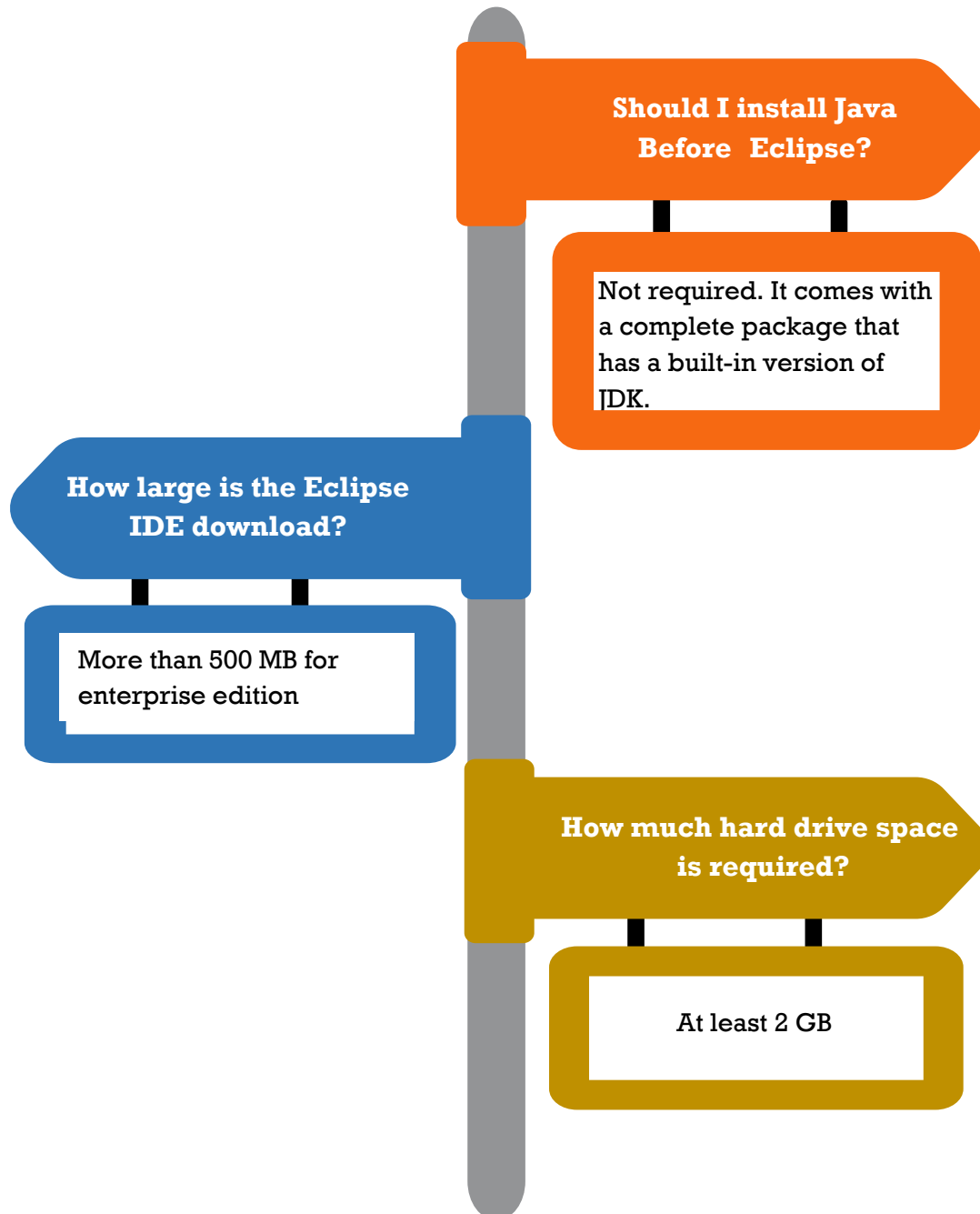
2.9.2 Features of Eclipse IDE

Some of the key features of Eclipse IDE include:

- **Code Competition:** Eclipse IDE identifies the keywords of all the programming languages that it supports using plugins. It has a very powerful code editor that is capable of understanding the functions and keywords, syntax, code templates and code completion and tools of refactoring. This identification helps greatly in development of applications using this software.
- **Single Workspace:** A single workspace that holds all the source files and bundles up all the projects makes user functionality easier. The workspace concept allows users to manage multiple projects within a single environment, making it easier to switch between them and maintain a consistent set of configurations.
- **Customized Perspective:** Users can navigate and modify content within the IDE using editors and views. Perspectives in Eclipse are used to provide different views for various data groups, and they are essential for task-oriented interaction with resources. Some common perspectives in Eclipse are debugging, plug-in development, and JEE.
- **Software Update:** The update system of Eclipse IDE software is very simple. Plug-ins are available that help the developer hop on to the advanced version of the applications.
- **Limited Dependencies:** The Eclipse platform has a dialog box which the user can use to select any software or add-ons that they require for application development. Thus, there is no dependence on any external tools.
- **Powerful Tools:** Software tools such as Java, JPA, JSF, Spring, and so on are supported by the Eclipse platform. Enterprise-based applications, Web service development, XML file editing and other actions can be easily performed using these tools.
- **Testing and Plug-ins:** Eclipse IDE provides easy debugging process to detect bugs and threats. Application testing can be done using debugging tools locally and remotely.
- **Platform Support:** Windows, MacOS, and Linux are some of the operating systems that support Eclipse. It has enhanced user experience across multiple platforms.

2.10 Installation of Eclipse IDE

Installation of Eclipse IDE is a very simple process. However, even before you consider installing it, some questions might bother you. Here are a few FAQs that will make it easier for you to prepare yourself for the installation process.



Steps to install Eclipse IDE are as follows:

1. Open the browser and go to <https://www.eclipse.org/downloads/>.
2. Select the **Download x86_64** button, as shown in Figure 2.14.

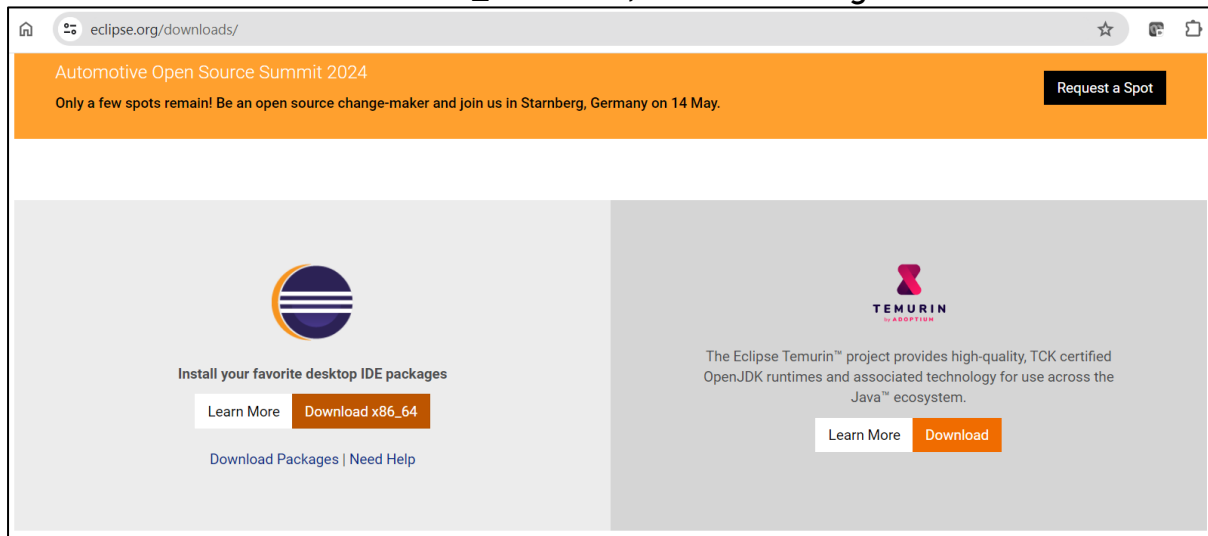


Figure 2.14: Eclipse Download x86_64 Selected

3. Click the **Download** button to start downloading the Eclipse Installer, as shown in Figure 2.15.

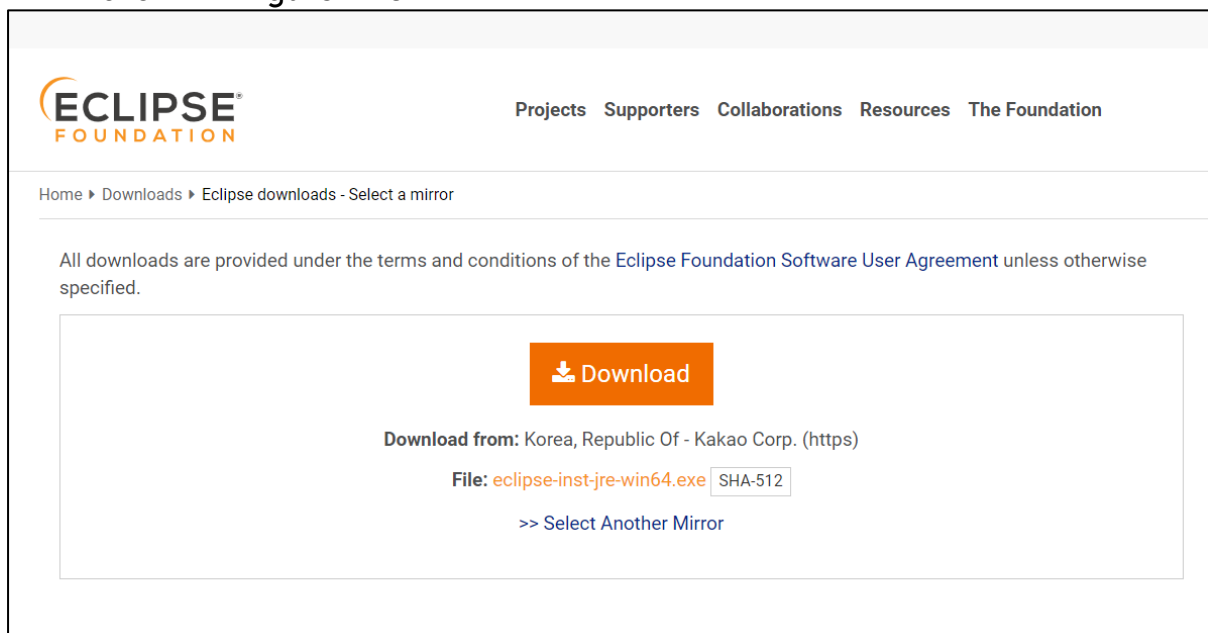


Figure 2.15: Download Button

4. Once the download is complete, double-click the downloaded file **eclipse-inst-jre-win64.exe**.
5. In the installer, select **Eclipse IDE for Enterprise Java and Web**

Development, as shown in Figure 2.16.



Figure 2.16: Eclipse Installer

6. Select the JDK installation directory. Then, click **Install** to proceed with the installation, as shown in Figure 2.17.



Figure 2.17: Eclipse IDE for Enterprise Java and Web Developers Installation Page

After installation is complete, launch Eclipse and start creating new projects.

Summary

- The Jakarta EE platform previously known as Java EE, was developed originally by Sun Microsystems and managed by Oracle Corporation.
- JESP or Jakarta EE Specification Process is used to create and maintain the Jakarta EE platform.
- Jakarta EE has made the development and implementation of Java Enterprise applications easier and quicker due to the use of APIs or Application Programming Interfaces.
- The Jakarta EE profile has three categories: Core Profile, Web Profile, and Platform Profile.
- Jakarta EE has a multi-component model as against a single-block application model.
- Jakarta EE blends easily microservices architecture and Cloud Native.
- Jakarta EE has several specifications encompassing different aspects of enterprise application development and management.
- Eclipse IDE is a platform where developers can create different applications using programming languages such as Java, Ruby, C and C++, Python, PHP and so on.

Check Your Progress

1. The Jakarta EE platform was previously known as:
 - a. Jakarta EF
 - b. Jakarta EE 8
 - c. Java EE
 - d. JavaPages
2. Where is the business logic present in the Jakarta EE application?
 - a. Web Tier
 - b. Presentation Tier
 - c. Business Tier
 - d. Integration Tier
3. Which of these services is supported by Platform Profile?
 - a. Messaging
 - b. RESTful Web Services
 - c. Relational Data Persistence
 - d. Dependency Injection
4. Why has the Jakarta EE community faced fragmentation?
 - a. Due to its complexity and non-functionality
 - b. Due to the takeover by Eclipse IDE
 - c. Due to lack of interest among developers
 - d. Due to transition from Java EE to Jakarta EE
5. Which language is primarily preferred in Eclipse IDE?
 - a. Python
 - b. Java
 - c. C++
 - d. Ruby

Answers

1	c
2	c
3	a
4	d
5	b

Try It Yourself

1. Install Java SE 21 on your system. After installation, set the `JAVA_HOME` environment variable to point to the JDK installation directory. Verify the installation.
2. Download Eclipse IDE for Enterprise Java and Web Development and configure it to point to the JDK installation directory.
3. Open Eclipse and create a simple JSP application to display the message: "Hello World!".
4. Download Eclipse IDE on a device that has less than 2 GB hard drive. Check and note what operational issues are faced when the device has such limited memory.

Developing and Deploying a Web Application

Welcome to the Session, **Developing and Deploying a Web Application**.

This session describes the Apache Tomcat server and its benefits. This session explains the directory structure and lifecycle of a Web application. The session also describes how to configure Apache Tomcat server and create a Web application project in the Eclipse (IDE). In addition, the session describes the directory structure in Eclipse. Further, the session elaborates on the deployment of a Web application using Eclipse and Tomcat.

In this session, you will learn to:

- Explain Apache Tomcat server
- Describe the lifecycle and directory structure of a Web application
- Describe the process of Web application development with Eclipse IDE
- Describe directory structure in Eclipse
- Explain how to package a Web application
- Describe the process to build and deploy a simple Web application using Eclipse and Apache Tomcat server

3.1 Overview of Apache Tomcat Server

Apache Tomcat, also known as Tomcat Server, is a Web server used by Web developers to construct and administer dynamic Websites and applications. Apache Tomcat is free and open-source. It is a servlet container for Java-based programming applications, such as Java/Jakarta servlets, Java/Jakarta Server Pages (JSP), Java Expression Language, Java WebSocket, and so on. It is a powerful server used by many organizations.

Static Websites built using HTML require an HTTP server to host those Websites. On the other hand, dynamic Websites require an HTTP server and a container to run the Servlets; combining the HTTP server and the Servlet container results in the creation of a server such as Apache Tomcat.

Apache Tomcat is responsible for processing the Servlets and JSPs. It internally transforms JSPs into Servlets and renders JSPs. Apache Tomcat is responsible for monitoring all incoming requests made by clients and loading the appropriate servlet classes in order to fulfill those requests. It executes the servlets and finally, unloads the servlet class.



3.1.1 Benefits of Apache Tomcat

Apache Tomcat offers multiple benefits to the users. Some of the significant benefits include the following:

Platform Independent

Users can deploy it across different platforms, such as Windows, Mac OS, and Linux.

Open-source Environment

Anyone can use the core file to develop personal and commercial software applications. Downloading and installing the software is also free.

Lightweight

Uses less memory and resources, allowing the program to function smoothly on a wide range of computers without requiring special system resources.

Well-Documented

Learning is easy for developers. Various Websites also offer free Tomcat tutorials and Web resources, including installation instructions, startup settings, Server setup, and platform support.

Highly Secure

Offers more protection against potential attackers. Tomcat supports Secure Socket Layer (SSL); therefore, it can be configured with an SSL certificate to safeguard sensitive data.

3.1.2 Installing Apache Tomcat

Apache Tomcat has different versions available. These versions are based on the specifications of Servlets or JSP. Apache Tomcat 10.x should be preferred as it is a recent version.

Following steps show the installation process of Apache Tomcat 10.1.20:

1. Navigate to the Website **tomcat.apache.org** and click **Tomcat 10** in the left panel as shown in Figure 3.1.

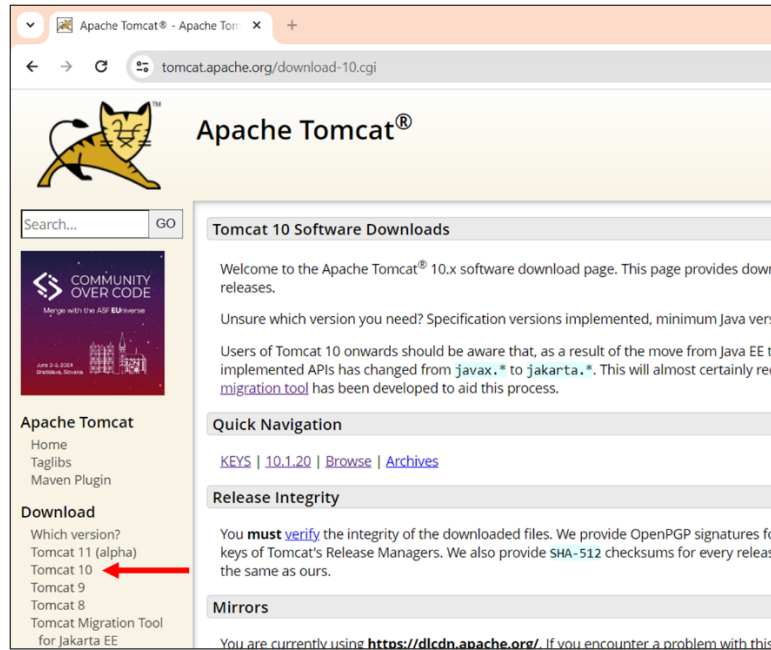


Figure 3.1: Apache Tomcat

- On the <https://tomcat.apache.org/download-10.cgi> page that is displayed, scroll down to view binary distributions. Choose the appropriate configuration, 32/64-bit Windows, as shown in Figure 3.2. Select and download the **32-bit/64-bit Windows Service Installer** file to initiate the installation. Using Tomcat Windows Installer instead of ZIP immediately starts the background process.

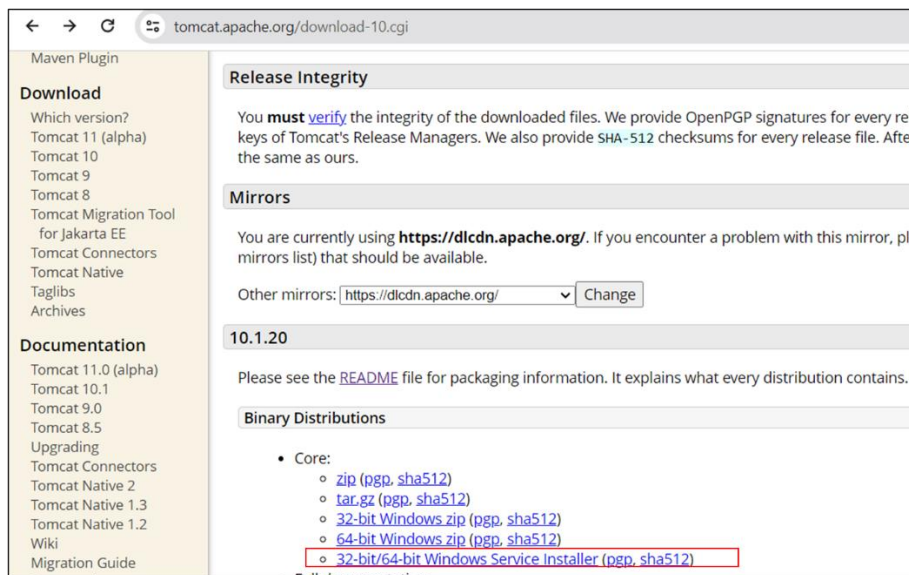


Figure 3.2: Selecting the Installer

- Navigate to the folder where the installer is downloaded and double-click the downloaded file to initiate the installation.
- In the Apache Tomcat Setup dialog box, click **Next** to continue the installation as shown in Figure 3.3.

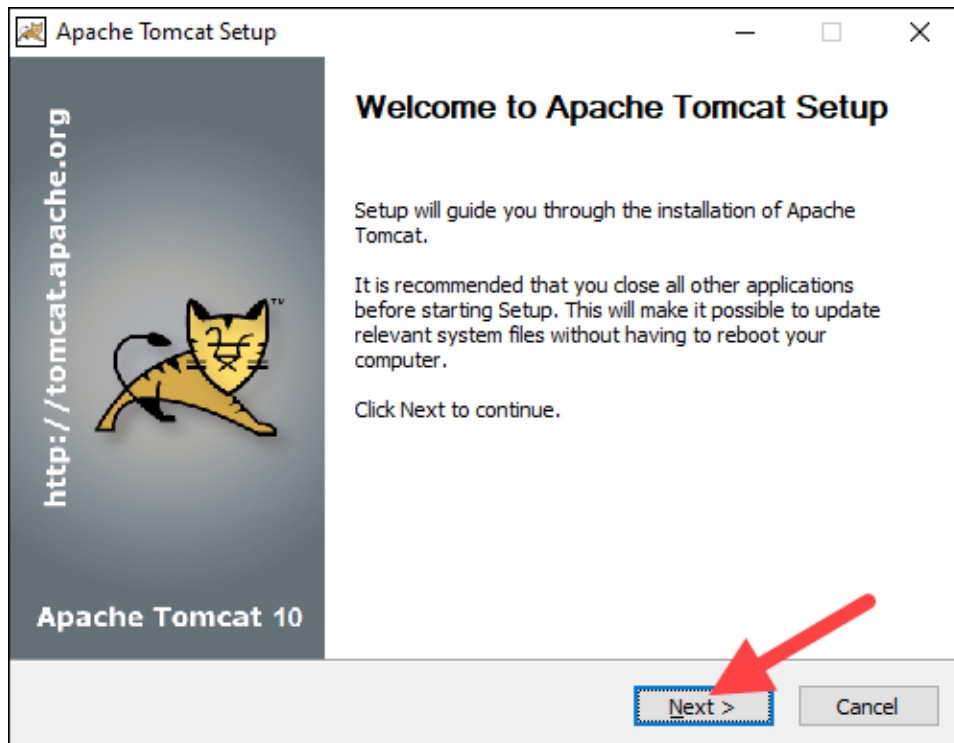


Figure 3.3: Apache Tomcat Setup

5. Read the License Agreement and then, click **I Agree** to continue as shown in Figure 3.4.

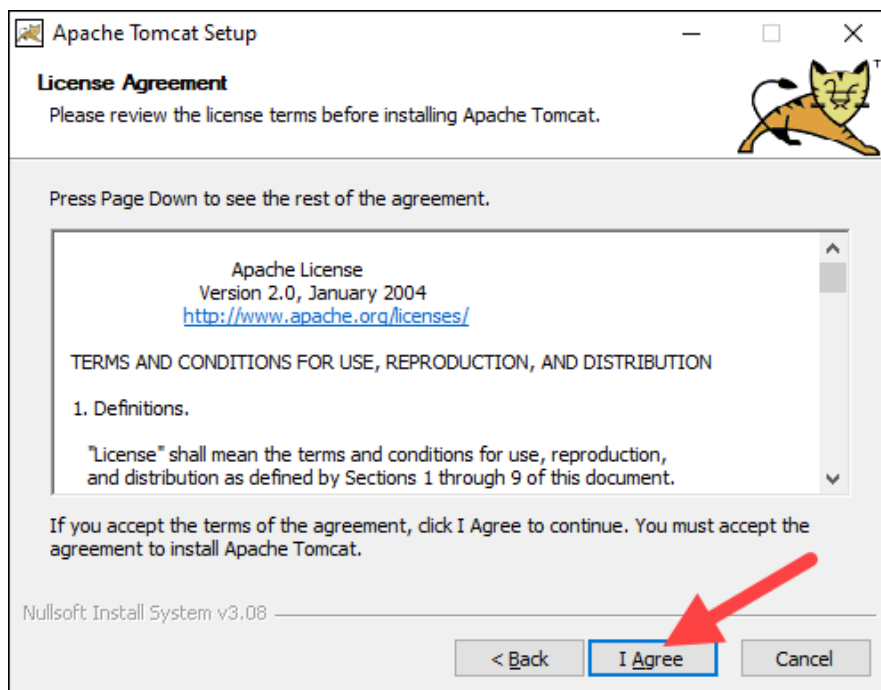


Figure 3.4: License Agreement

6. Select the components to be installed as shown in Figure 3.5 and click **Next** to continue.

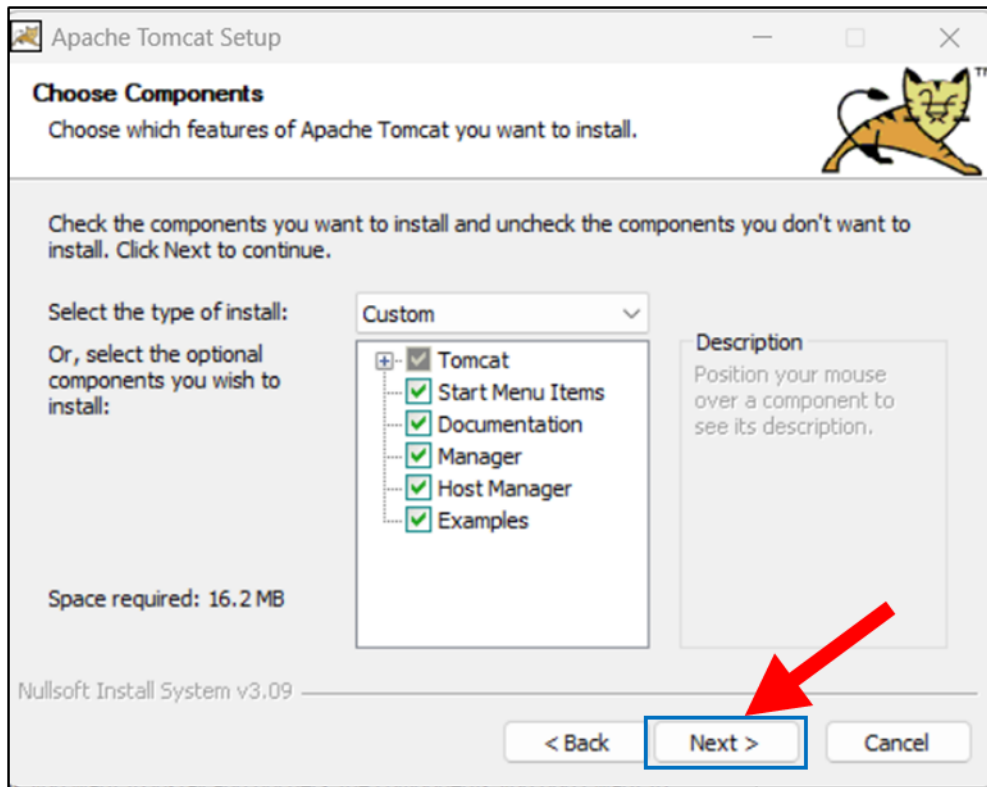


Figure 3.5: Choose Components

7. Type the **User Name** and the **Password** in the respective boxes and then, click **Next** as shown in Figure 3.6.

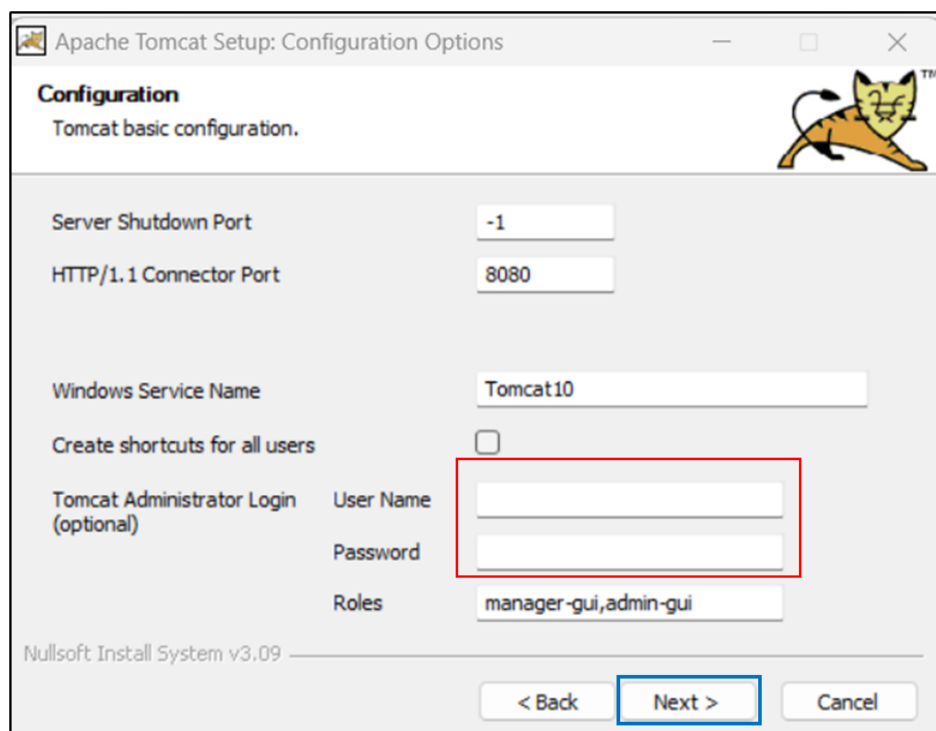


Figure 3.6: Configuration Options

8. Select the correct path of JDK installation as shown in Figure 3.7 and click **Next**.

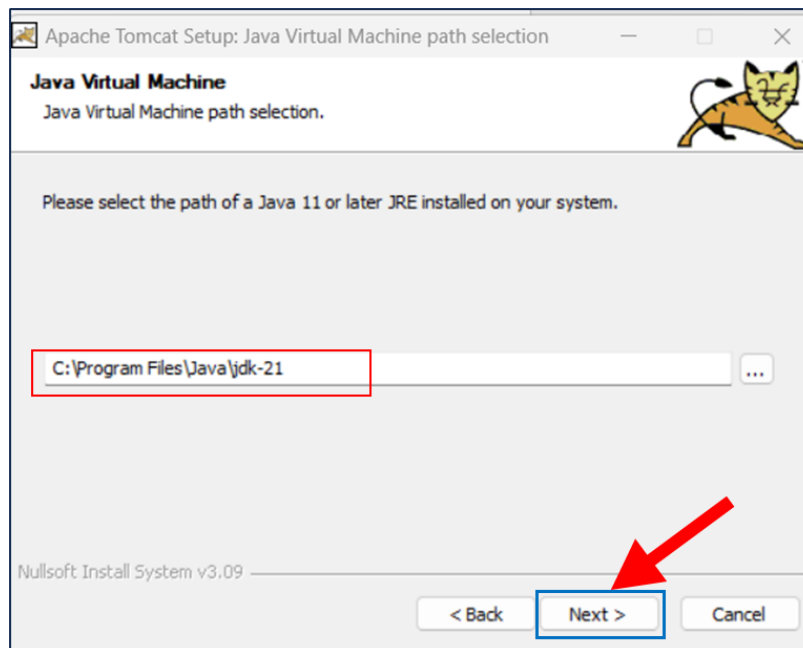


Figure 3.7: Java Virtual Machine Path Selection

9. Ensure that the correct folder is selected in the Destination Folder field as shown in Figure 3.8 and click **Install** to continue the installation.

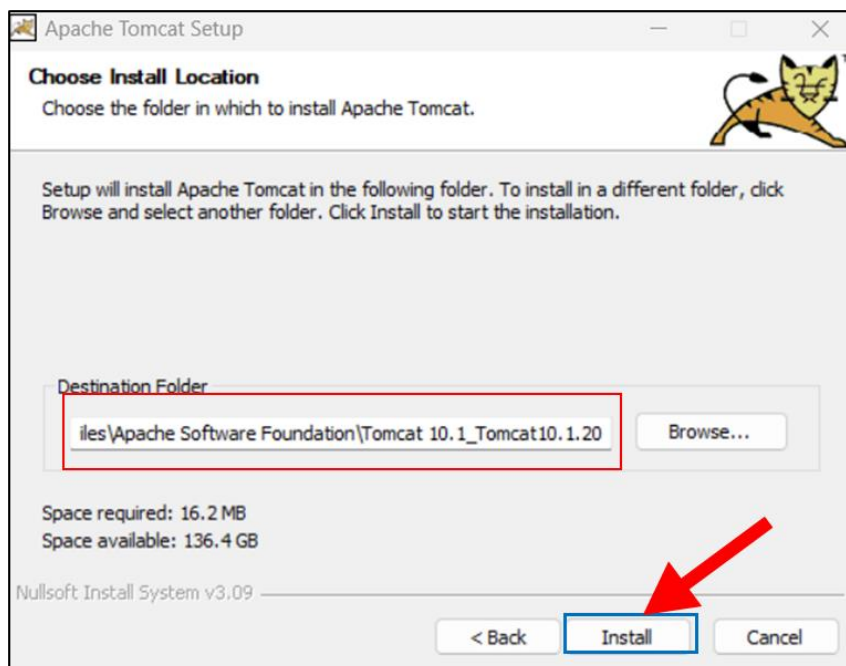


Figure 3.8: Choose Install Location

10. Click **Finish** to complete the installation as shown in Figure 3.9. A message prompting the user to restart the machine is displayed.

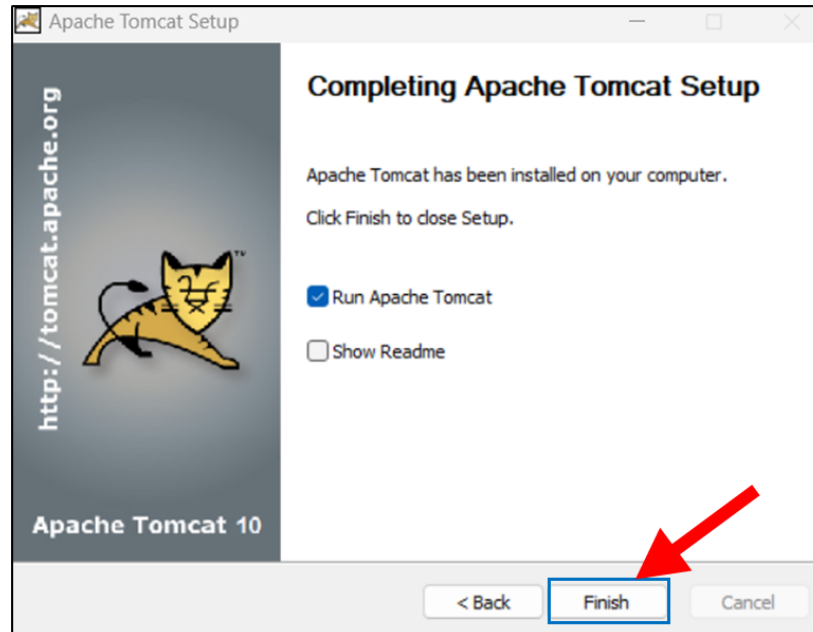


Figure 3.9: Completing Apache Tomcat Setup

11. Finally, check the browser and type **localhost:8080** into the address bar, then press Enter. The **installed successfully** page should appear as shown in Figure 3.10.

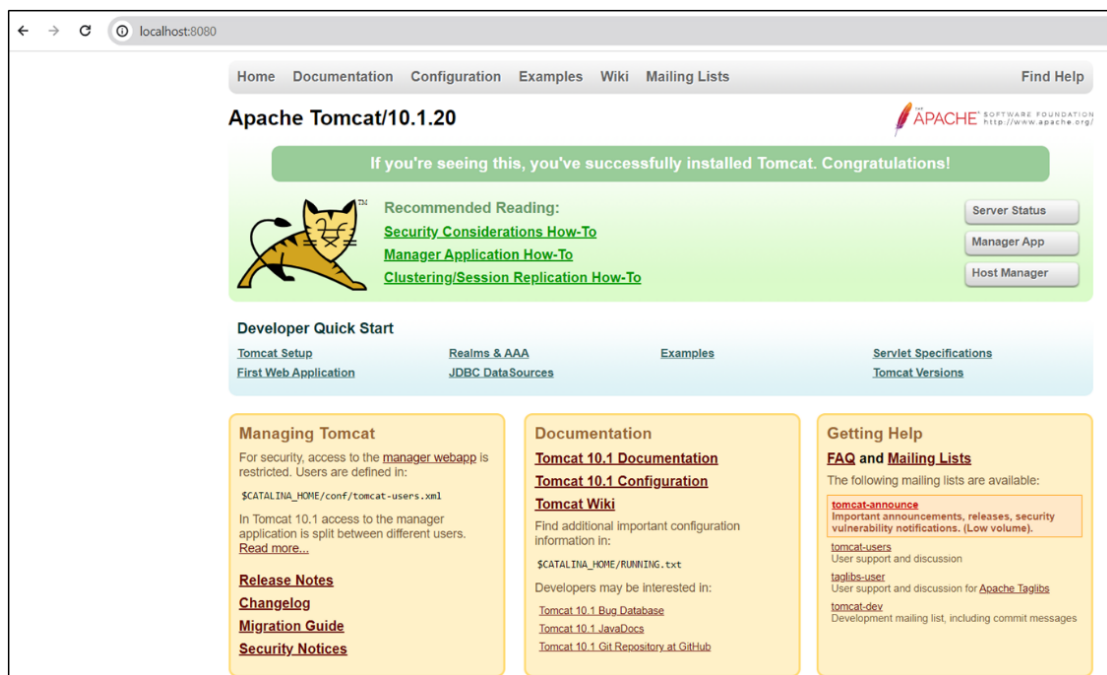


Figure 3.10: Successful Installation Page

After installing Tomcat, users may start and stop the server by clicking the Tomcat icon.

3.2 Lifecycle and Directory Structure of Web Applications

A Jakarta EE Web application may comprise servlets, JSP pages, images, HTML files, Jakarta Beans, and Java classes.

If a Web application is to be ported to some other server or system, a developer has to copy or move all these files to the new system. However, an easy alternative is to package all the files associated with Web application into a single Web Archive (war) file and deploy this war file on the Web server.

Java Web applications are deployed on a Java-enabled Web server such as Tomcat. The Web container within the Web server is responsible for handling the Web components present in the Web application.

3.2.1 Web Application Lifecycle

In an earlier session, Web application workflow was described. Let us now discuss the development lifecycle of a Web application. Web applications are required to create a dynamic application using different Web components such as servlets, JSP, HTML pages, images, and JavaScript helper classes and libraries. The process or lifecycle for creating, deploying, and executing a Web application can be summarized as follows:

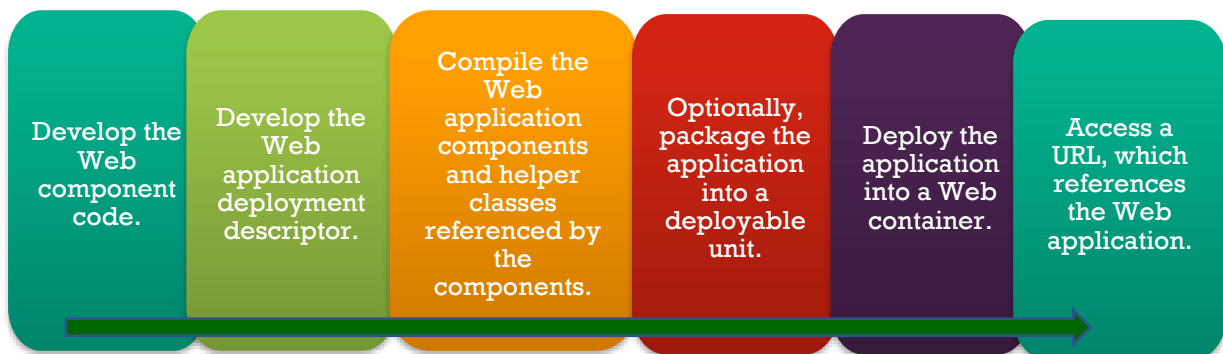


Figure 3.11 describes the Web application lifecycle.

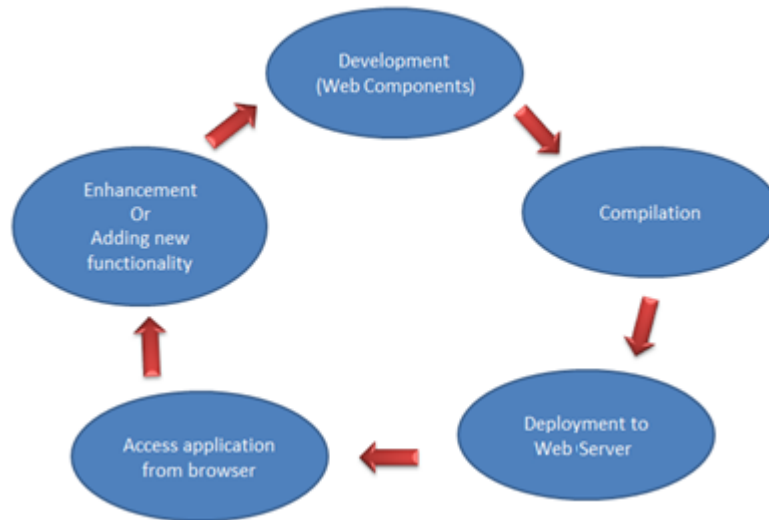


Figure 3.11: Web Application Lifecycle

3.2.2 Web Application Context Root

The Web container supports multiple Web applications, so each application is identified or assigned with a context root. The context root or context path of the Web application is the base URL used for locating Web pages and Web components such as servlet and JSP within the application.

For example, consider a `.war` file named **WebApp** contains a file named `index.html` in a **page** folder. When the war file is deployed on the Java Web server, then you require the URL to access the pages from the application. Suppose the application is deployed on the local Web server, then the URL to access the `index.html` page will be `http://localhost:8080/WebApp/page/index.html`. Thus, the context root of the deployed application is **/WebApp**.

3.2.3 Web Application Directory Structure

Contents of the Web application are accessed from the application's context root. The context root of the Web application contains two main components in the directory structure. These are described as follows:

Static files	WEB-INF
All the HTML pages and images comprise the static files. Images can be collectively stored in an images directory. For example, to access a file <code>index.html</code> in the book folder on the Website <code>www.corejava.com</code> , the URL would be <code>http://www.corejava.com/book/index.html</code> .	This directory exists within the context root. However, this directory cannot be referenced. For example, referencing <code>www.corejava.com/book/WEB-INF</code> is not possible.

Figure 3.12 shows the directory structure of the `.war` file.

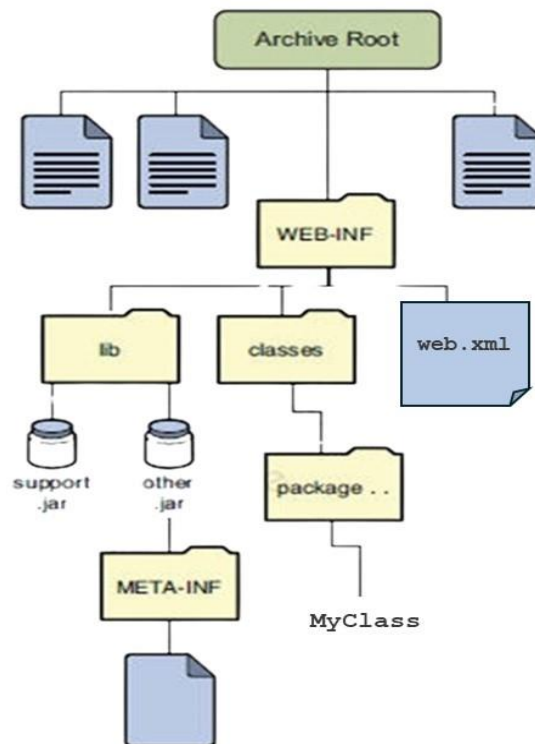


Figure 3.12: Directory Structure

The **WEB-INF** directory consists of following contents:

classes directory

servlet classes, JavaBean classes, and any other classes required by the Web applications are stored in this directory. servlets and JavaBean classes that are part of a package are placed to match their package names. For example, if a servlet or JavaBean class is part of a package such as `www.bookstore.com`, then the corresponding class file will be placed in `classes/com/bookstore` directory.

web.xml

This file is called the deployment descriptor of the Web application. It is basically an XML structured file which provides configuration information for the components of the Web application. The configuration information includes information, such as the default pages to show, mapping servlets with their URLs, and so on.

lib directory

It contains .java archive files required for Web application such as database drivers. In other words, the library JAR files used by the application are stored within the **WEB-INF/lib** directory.

tags directory

This directory contains tag files, which provide implementation for custom tags. Tag files have `.tag` extension.

Tag Library Descriptor files

These files provide information about the attributes of a custom tag and the class to be invoked when the tag comes across in a JSP page. These files have `.tld` extension. However, tag library descriptor files are required only if tag files must be distributed as a separate package or if the custom tags are implemented using Java.

3.2.4 Deployment Descriptor

A deployment descriptor is a configuration file that describes how the Web application should be deployed. It is written using XML with name `web.xml` and placed under the **WEB-INF** folder, that is, `WEB-INF/web.xml`.

When the Web server receives a request for the application, it uses the deployment descriptor to map the URL of the request to the servlet that handles the request. For example, to map a URL request to a servlet, you must do the following settings in the `web.xml` file:

1. Declare the servlet with the `<servlet>` element.
2. Define a mapping from a URL path to a servlet declaration with the `<servlet-mapping>` element.

The `<servlet>` element declares the servlet, including a name used to refer to the servlet by other elements in deployment descriptor and the servlet class. The name for each servlet must be unique across the deployment descriptor.

The `<servlet-mapping>` element specifies a URL pattern and the name of a declared servlet (declared in `<servlet>` element) to use for requests whose URL matches the pattern.

Code Snippet 1 shows the configuration setting for a `GameServlet` class created in the Web application named **ServletDemo**.

Code Snippet 1: `web.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
.
.
.
<servlet>
  <servlet-name>GameServlet</servlet-name>
  <servlet-class>com.GameServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>GameServlet</servlet-name>
  <url-pattern>/trygame</url-pattern>
</servlet-mapping>
</web-app>
```

As shown in the given Code Snippet, the URL pattern to access the `GameServlet` class will be `http://localhost:8080/ServletDemo/trygame`.

Figure 3.13 shows the mapping of Servlet with the URL through which it is accessible to the application.

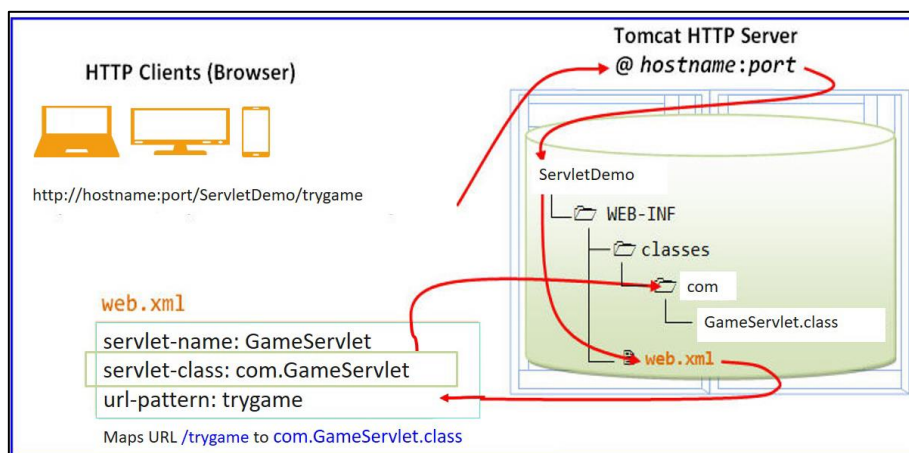


Figure 3.13: Servlet Mapping in web.xml

From Java EE 6 onwards, the `web.xml` file is optional, as the configuration information can be applied with the annotations. Annotations are meta information embedded in the Java source files. The meta information creates the dynamic configuration setting for the deployed component during runtime.

From Servlet 3.0, a new feature called **Web fragments** is introduced. Web fragment is the mechanism to include deployment descriptor information which is specific to a library. This helps to segregate the unrelated information from the main deployment information.

The developer might have created functionality for the application separately in some other Jakarta EE project and import its .jar in WEB-INF\lib as plug-in within the Web application.

The content of web.xml and web-fragment.xml are almost the same, but the difference in its location, we put web.xml in WEB-INF\web.xml and web-fragment.xml in META-INF\web-fragment.xml.

Code Snippet 2 displays a web.xml file.

Code Snippet 2:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:web="http://java.sun.com/xml/ns/javaee/Web-app_2_5.xsd"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/Web-app_2_5.xsd"
id="WebApp_ID" version="2.5">
<servlet>
    <servlet-name>Queue</servlet-name>
    <jsp-file>/jsp/FirstJSP.jsp</jsp-file>
</servlet>
<servlet-mapping>
    <servlet-name>Queue</servlet-name>
    <url-pattern>/FirstJSP.jsp</url-pattern>
</servlet-mapping>
</web-app>
```

3.3 Web Application Development in Eclipse

Eclipse is a popular option for Java application development. It offers an IDE for application development allowing the users to do all coding tasks inside a single application. It is also open-source providing developers free access to its source code. Eclipse 2021 or higher version is recommended for Jakarta EE 10 application development.

3.3.1 Configuring Tomcat in Eclipse

Configuring Tomcat in Eclipse is a best practice that aligns with efficient Web application development. It simplifies the development and testing process by allowing developers to manage the Tomcat server directly within the IDE. Additionally, it saves time by automating the build and deployment process, which can be particularly beneficial when making frequent changes to the application.

To demonstrate the process of configuring Tomcat in Eclipse:

1. Open **Eclipse** → **Windows** → **Show View** → **Servers** as shown in Figure 3.14.

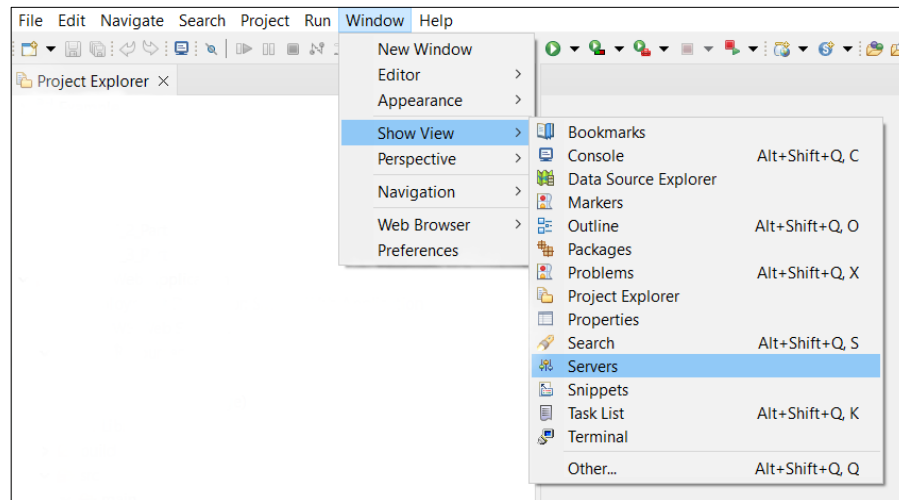


Figure 3.14: Eclipse Window

2. Click the link as shown in Figure 3.15.



Figure 3.15: Servers

3. **Define a New Server** page appears. Now, double-click Apache as shown in Figure 3.16.

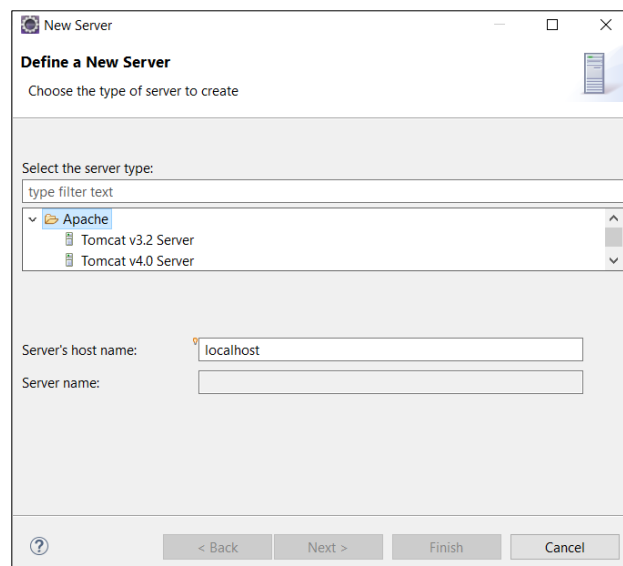


Figure 3.16: Define a New Server Page

4. Scroll down and select **Tomcat v10.1 Server** and click **Finish** as shown in Figure 3.17.

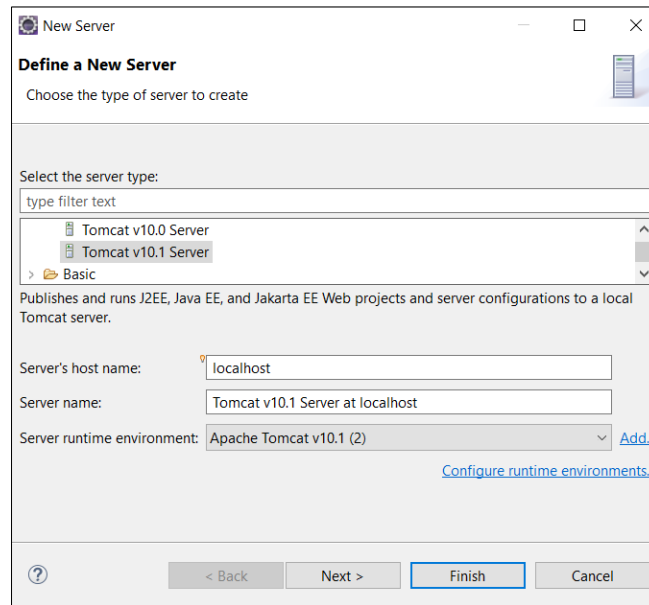


Figure 3.17: Selecting Tomcat Under Define a New Server Page

Tomcat server is successfully created.

5. Now, double-click the Tomcat server link as shown in Figure 3.18.



Figure 3.18: Servers Tab Showing Tomcat Server

6. Change the **Tomcat admin port** and **HTTP/1.1** port and then, save the changes and close the page.

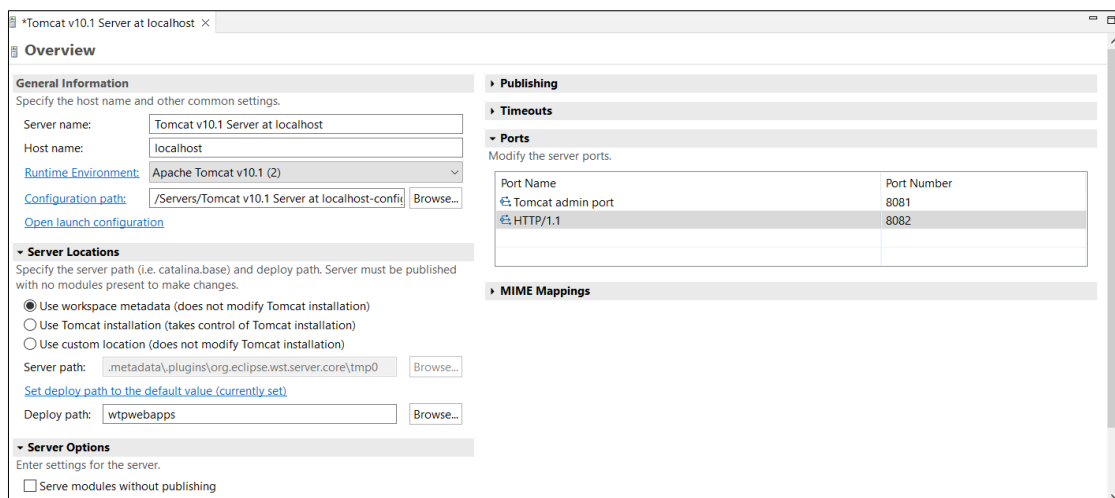


Figure 3.19: Tomcat Server Page at Localhost

7. Tomcat server is added successfully.

3.3.2 Creating Web Projects in Eclipse

Creating a new Web project in Eclipse is a straightforward process. Following steps demonstrate the process:

1. Open Eclipse and select **File** → **New Project** → **Dynamic Web Project** and then, click **Next** as shown in Figure 3.20.

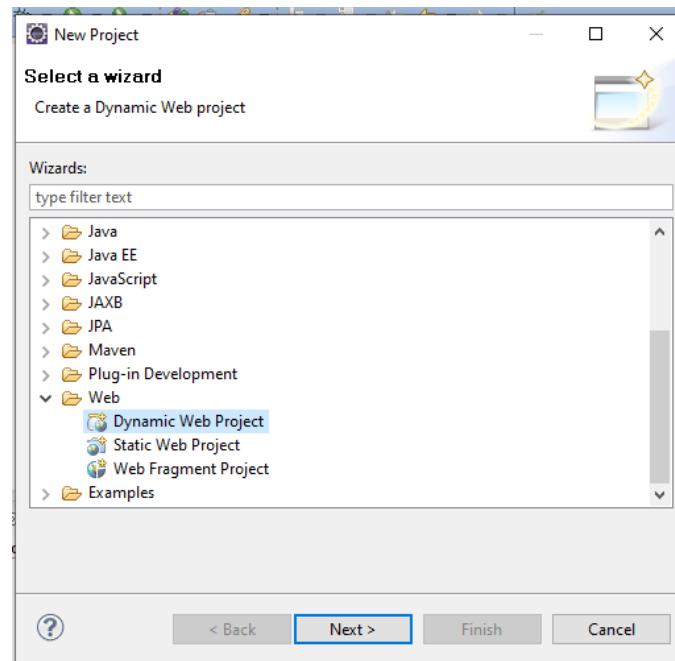


Figure 3.20: New Project

2. In the **New Dynamic Web Project** wizard, type the Project name and click **Finish** as shown in Figure 3.21.

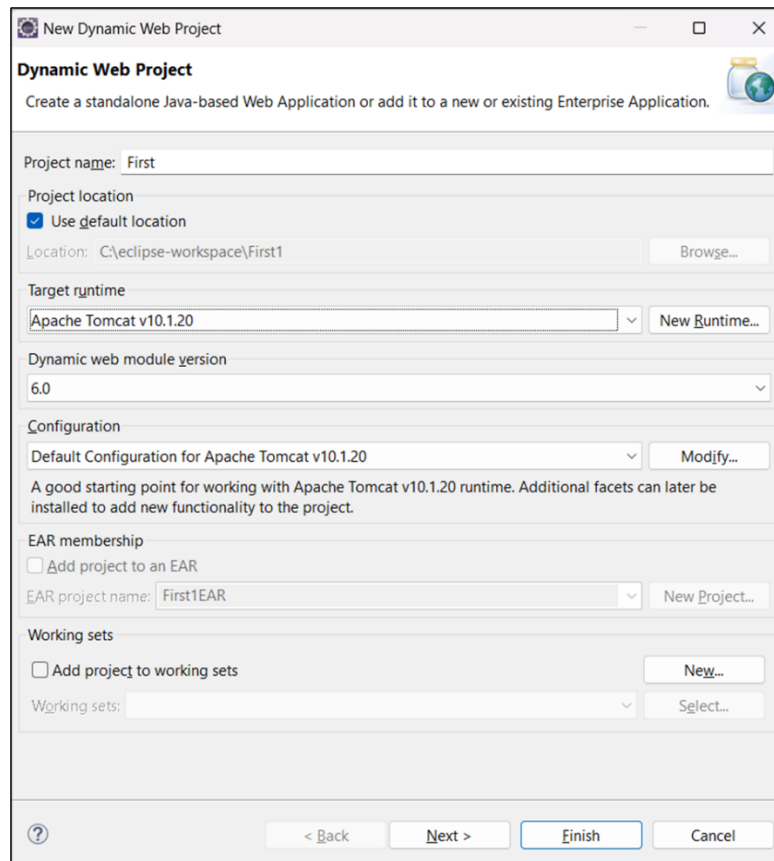


Figure 3.21: New Dynamic Web Project Wizard

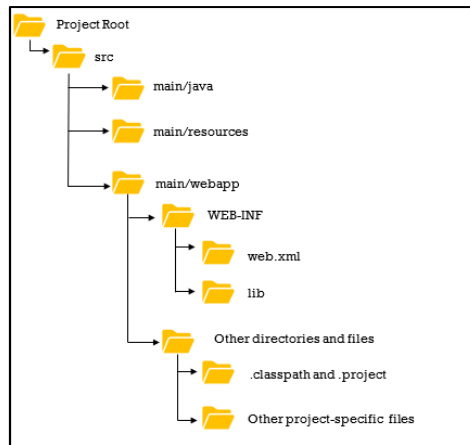
Following this step, developers can create servlet, JSP, and other files as part of the application. The **Run** menu option or toolbar options can be used by developers to execute their applications.

Building (compiling) the application happens by default in the Eclipse IDE. You can set various Preferences such as code editor fonts, server runtime environments, and more using **Window → Preferences**.

3.3.3 Directory Structure in Eclipse

The Eclipse Web application project directory is structured in a standardized way to ensure organization and ease of management.

The details of the essential directories are as shown in Figure 3.22.

**Figure 3.22: Directory Structure**

Project Root	The primary directory containing all the Web application's files and subdirectories.
src	This directory is the repository for the application's source code. It usually includes a 'main' subdirectory, which is subdivided into: main/java, main/resources, and main/webapp
main/java	This subdirectory is dedicated to the Java source code files to systematize the required Java classes.
main/resources	This subdirectory is designed for assets utilized by the application, such as images, property files, or configuration files.
main/webapp	This directory houses all the dynamic content that will be deployed to the Web server, such as: WEB-INF, Other directories and files.
WEB-INF	A crucial subdirectory that houses deployment descriptor files such as web.xml and lib folder.
web.xml	It serves as a central configuration file for the Web application. This file defines settings related to servlet configurations, filter configurations, welcome file lists, and other such parameters that dictate how the Web application behaves.

lib	This directory is for storing external libraries (JAR files) that the Web application project relies on.
Other directories and files	This is where developers can store the JSP pages, HTML files, Servlets, and other Web content, either directly under 'webapp' or within further organized subdirectories for improved structure (such as separate directories for CSS, JavaScript, or images).
.classpath and .project	These files are specific to Eclipse and set up the project's settings and Eclipse's interaction with the project.
Other project-specific files	Depending on the project's requirements, developers may include additional files at the project root, such as build scripts or other configuration files.

Figure 3.23 shows a sample directory structure of a Web application project in Eclipse.

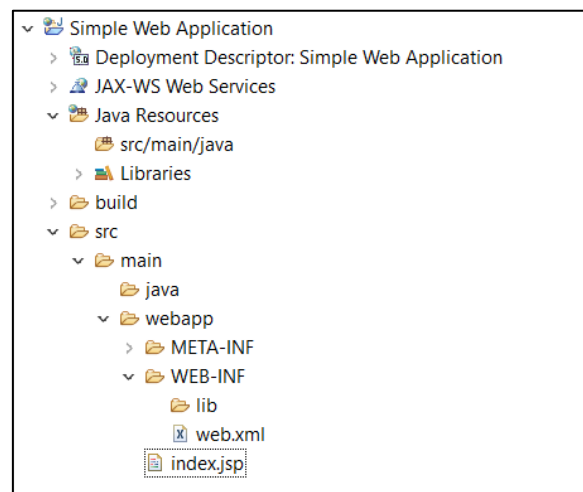


Figure 3.23: Web Application Directory Structure in Eclipse

3.4 Packaging a Web Application

Once an application is created, all its files have to be packaged together for the application to work. The Web container relies on a specified directory structure of the package file to ensure that the Web application works successfully. As a result, Web apps are stored in a Web Archive (WAR) distinguishing them from other types of Java programs. As a WAR file contains all the contents of a Web application, it speeds up the process of delivering huge files across the network.

WAR file is created using the jar utility in the JDK development environment. To create a WAR file, following command is used:

```
jar -cvf projectname.war *
```

where,

- c switch is used to create a file
- v provides the verbose output
- f is used to indicate the archive filename

The asterisk * is used to include all the files of the directory including the subdirectory.

3.4.1 Deploying the WAR File

There are two deployment methods for the WAR file:

- Using the console panel on the Server.
- By placing the WAR file manually in a specified Server folder. To manually deploy the WAR file, navigate to the Web apps directory of Apache Tomcat and paste the WAR file in that location.

To extract the WAR file, use the -x switch of the JDK jar tool.

The syntax of the command to extract the WAR file is as follows:

```
jar -xvf projectname.war
```

where,

- x switch is used to extract the file
- v provides the verbose output
- f is used to indicate the archive filename

3.4.2 web.xml Deployment Descriptor Elements

Deployment descriptor elements are in the web.xml file. The root element is the <web-app>. The <web-app> element in turn contains following elements:

context-param	This is an optional element and is used to declare the servlet context initialization parameters for a Web application.
session-config	The current session attributes of an application are defined by this element.
Servlet element	The servlet element contains a servlet's declarative data. It defines the canonical name of the servlet, which is used elsewhere in the deployment description to reference the servlet definition.

welcome-file-list

This element specifies the list of welcome files. If no file is specified in the browser, the file in <welcome-file-list> is called.
 Inside this element, users may specify an optional <welcome-file> element if required. <welcome-file>-This optional sub-element of the <welcome-file-list> contains the file name to use as a default welcome file, such as index.html.

taglib

This is an optional element. It declares that the application uses a tag library and describes that JSP tag library, it is also an optional element.

error-page

This is an optional element and connects the error code or exception type to the location of the Website.

3.5 Build and Deploy a Web Application Using Eclipse and Tomcat

Create a simple Web application that displays the message ‘**Welcome to the JSP Project!**’. The steps involved are as follows:

1. Create the Web application project.
2. Add the .jsp file.
3. Run the Web application.

3.5.1 Creating the Web Application Project

Creating a new Web application project in Eclipse IDE involves the steps as follows:

1. Open Eclipse.
2. To create a new Web application project. In the **File** menu, click **New** → **Dynamic Web Project** as shown in Figure 3.24.

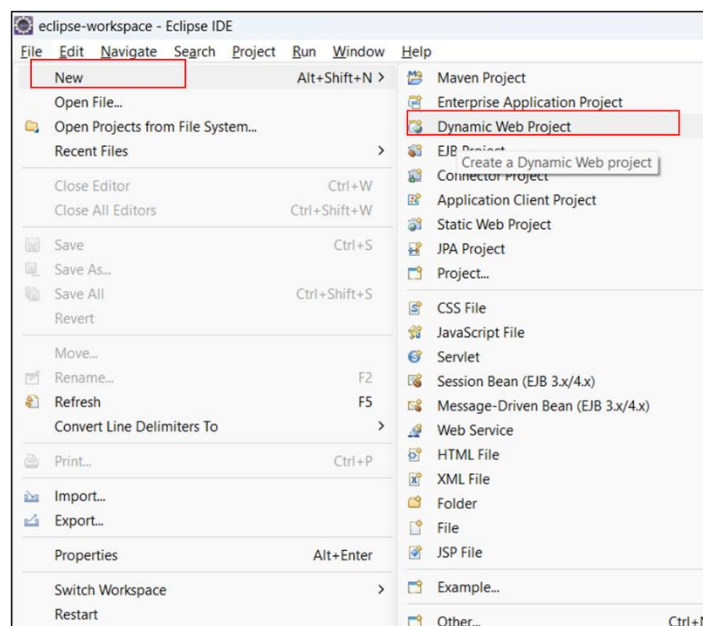


Figure 3.24: Creating a Dynamic Web Project

3. The **New Dynamic Web Project** wizard opens. On the **Dynamic Web Project** page, enter the name for the project. For example, in the **Project name** box, enter **Simple Web Application**, and click **Next** as shown in Figure 3.25.

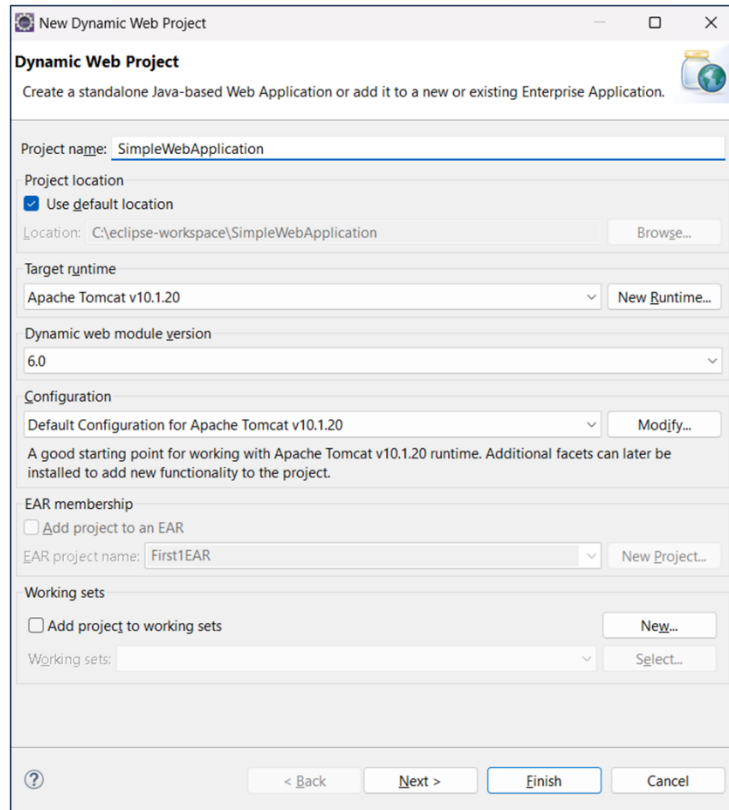


Figure 3.25: New Dynamic Web Project

4. On the **Configure project for building a Java application** page, click **Next** as shown in Figure 3.26.

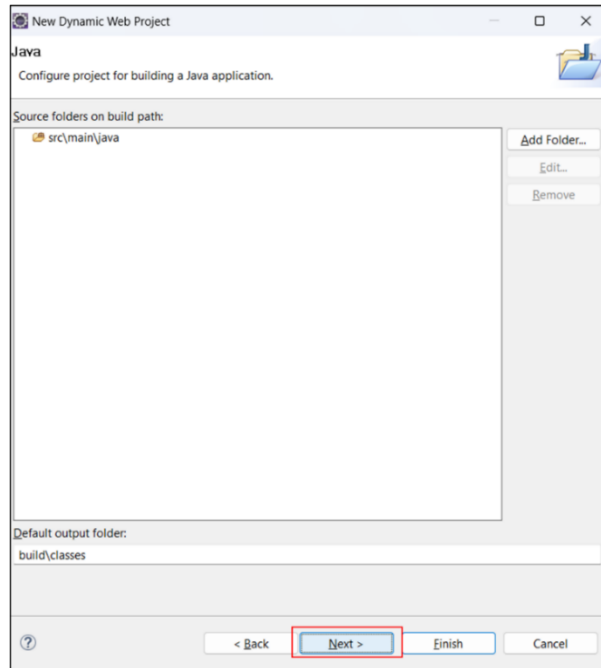


Figure 3.26: Source Folder

5. On the **Web Module** page, select the **Generate web.xml deployment descriptor** check box and click **Finish** as shown in Figure 3.27.

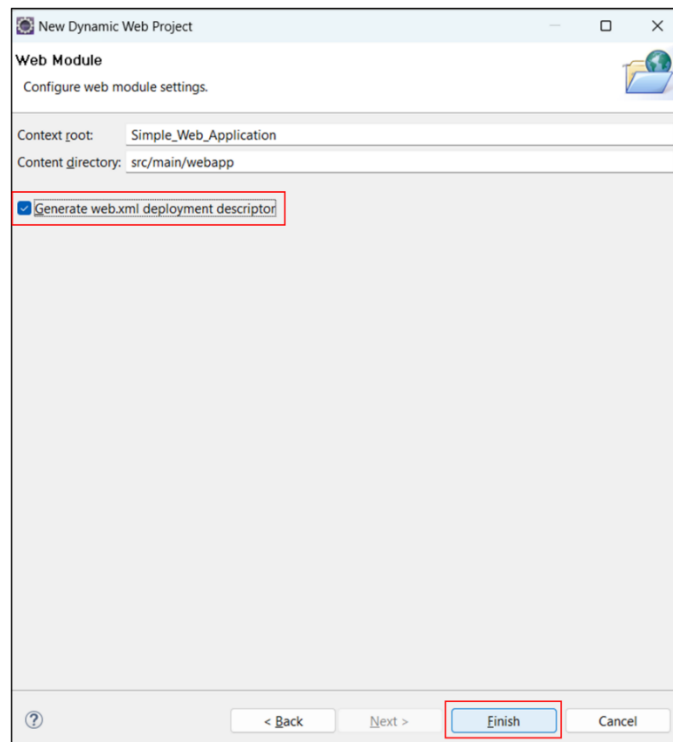


Figure 3.27: Web Module

The **Simple Web Application** project will be successfully created and the project folder will be listed in the Project Explorer workspace as shown in Figure 3.28.



Figure 3.28: Simple Web Application Project Folder Structure

3.5.2 Adding the JSP File

To create a new JSP file in the Web application project, the steps are as follows:

1. Under **Simple Web Application**, select **src** → **main**. Right-click the **webapp** directory and from the context menu, select **New** and click **Other** as shown in Figure 3.29.

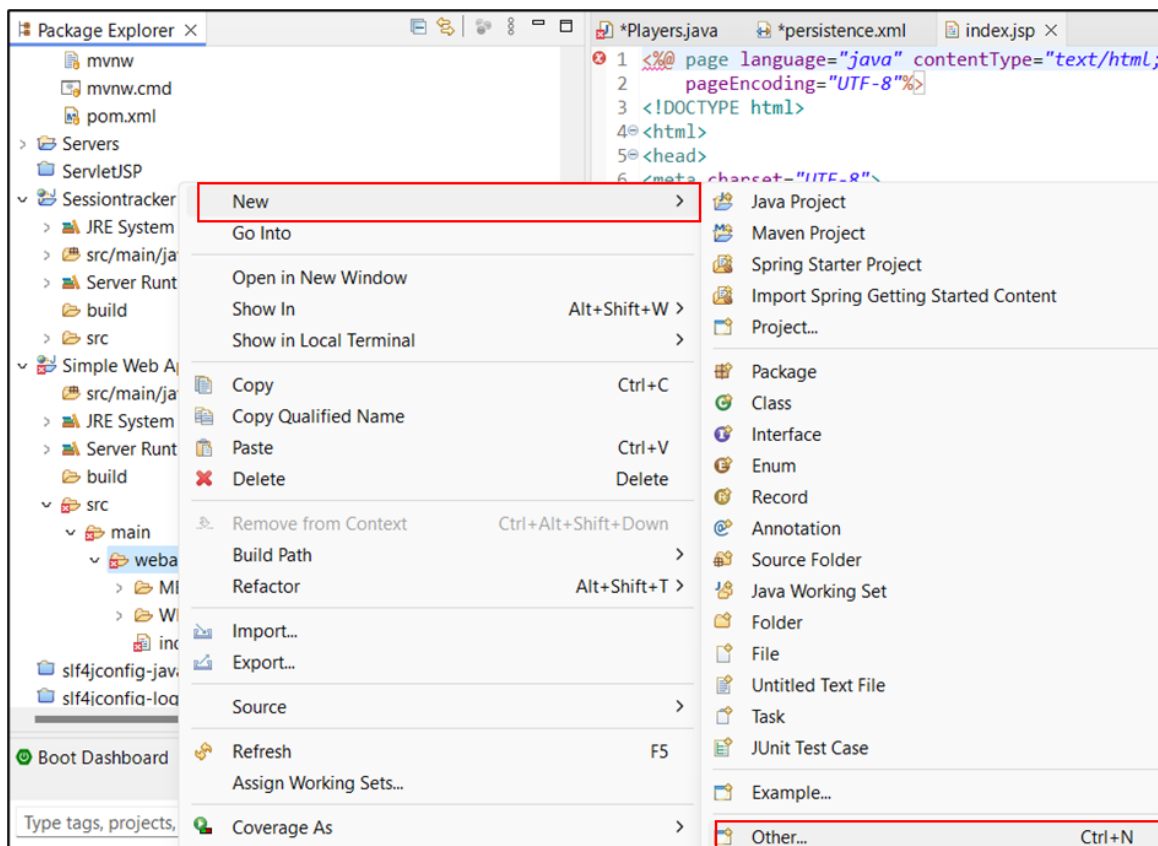


Figure 3.29: Create a New JSP File

2. In **Select a wizard** screen, select **JSP File** and click **Next** as shown in Figure 3.30.

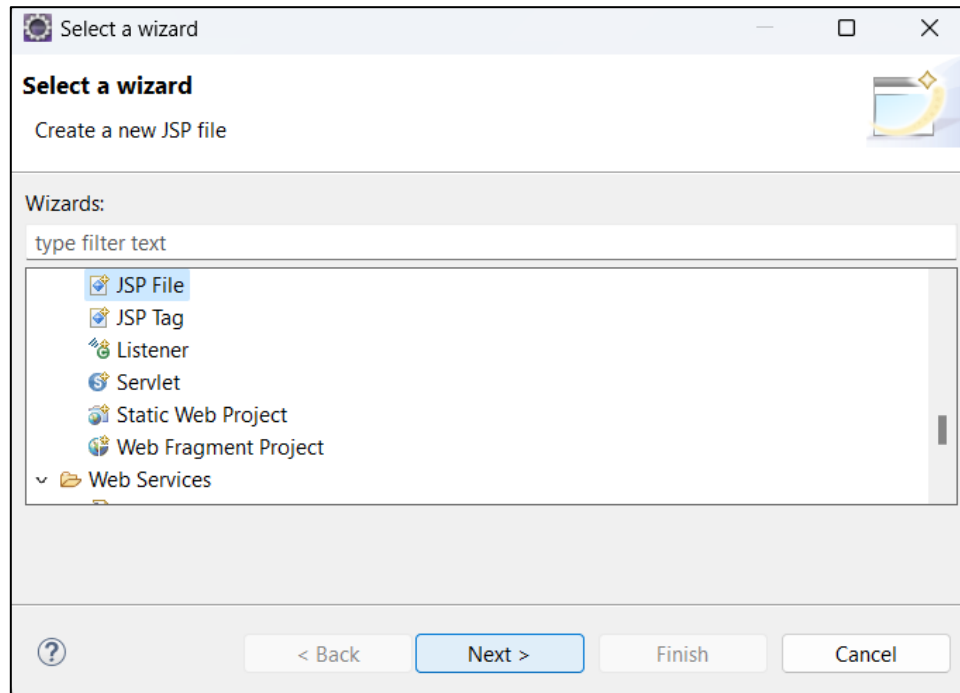


Figure 3.30: Select JSP File

3. In the **New JSP File** wizard, on the **JSP** page, enter the name for the JSP file. For example, in the **File name** box, enter **index.jsp**. Then, click **Next** as shown in Figure 3.31.

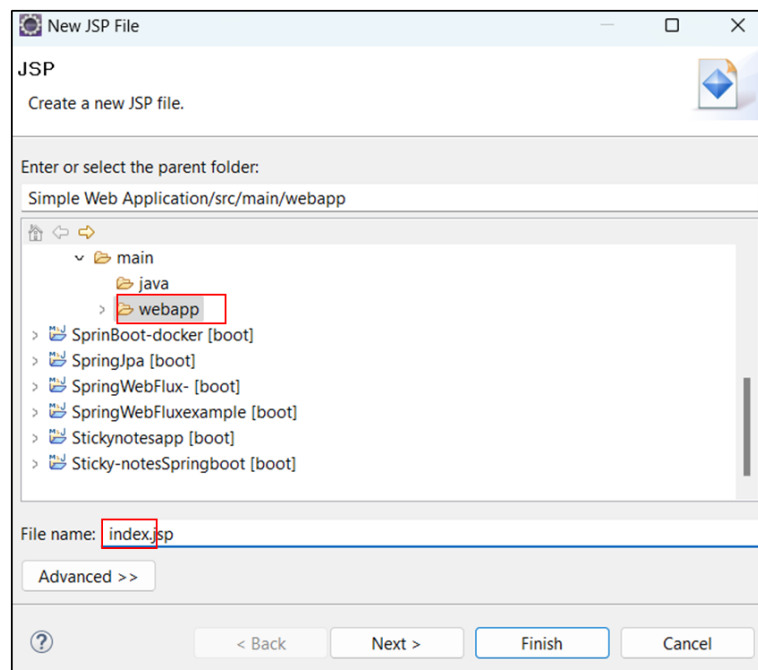


Figure 3.31: Enter JSP File Name

4. On the **Select the JSP Template** page, keep the default selection and click **Finish** as shown in Figure 3.32.

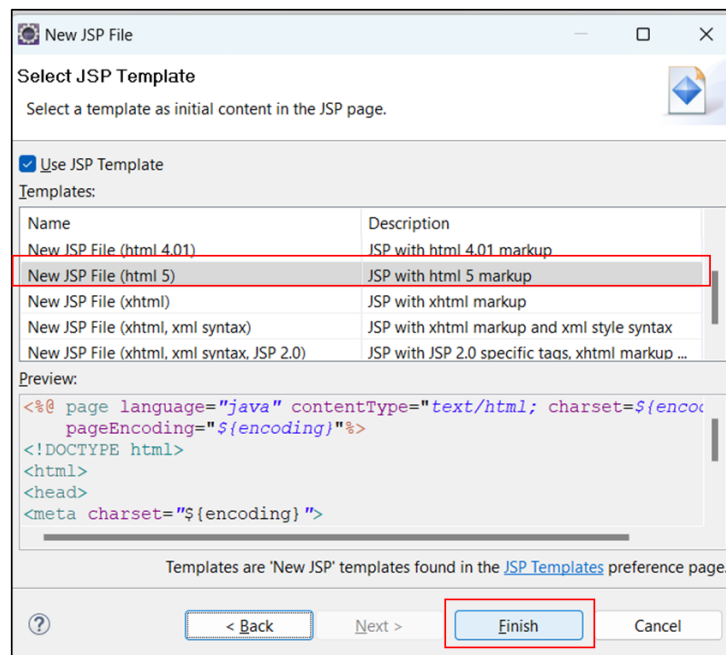


Figure 3.32: Select JSP Template

The newly created **index.jsp** file opens.

5. Add the code given in Code Snippet 3.

Code Snippet 3:

```
<%@ page language="java" contentType="text/html;
charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Welcome</title>
</head>
<body>
<h1>Welcome to the JSP Project! </h1>
</body>
</html>
```

3.5.3 Running the Web Application

The Tomcat server must be running to deploy and run the project in the Tomcat server.

The steps to execute a Web application are as follows:

1. Right-click the **Simple Web Application** project folder in the Project Explorer, select **Run As** and click **Run on Server** as shown in Figure 3.33.

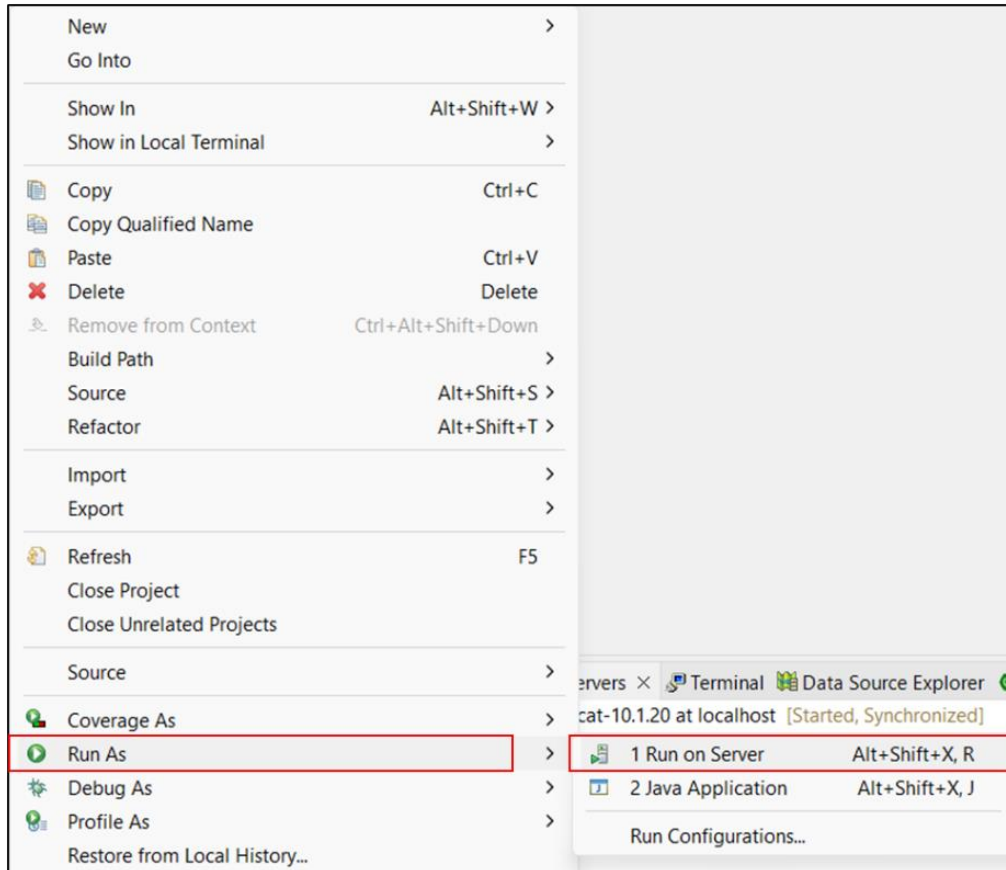


Figure 3.33: Run on Server

2. In the **Run on Server** wizard, select the Tomcat server as shown in Figure 3.34. Then, click **Next**.

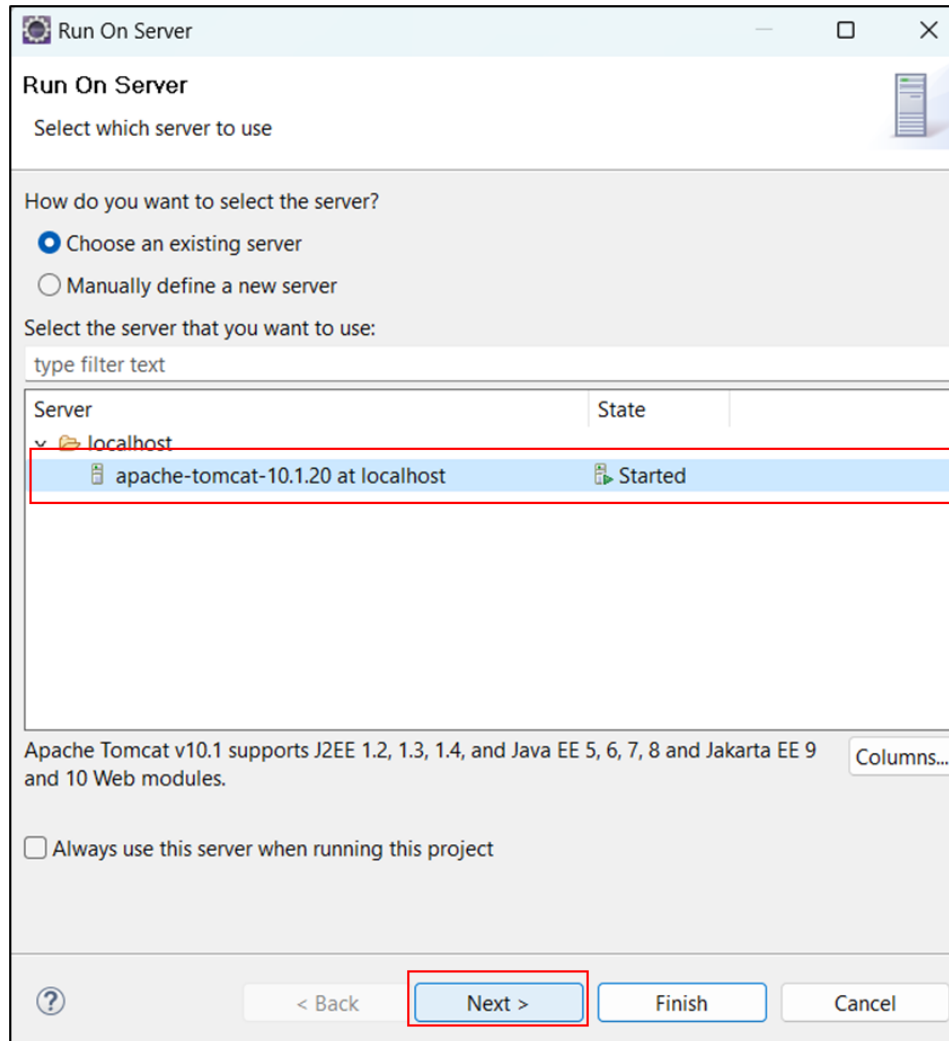


Figure 3.34: Select Tomcat Server

3. On the **Add and Remove** page, add the **Simple Web Application** to be configured on the server and click **Finish** as shown in Figure 3.35.

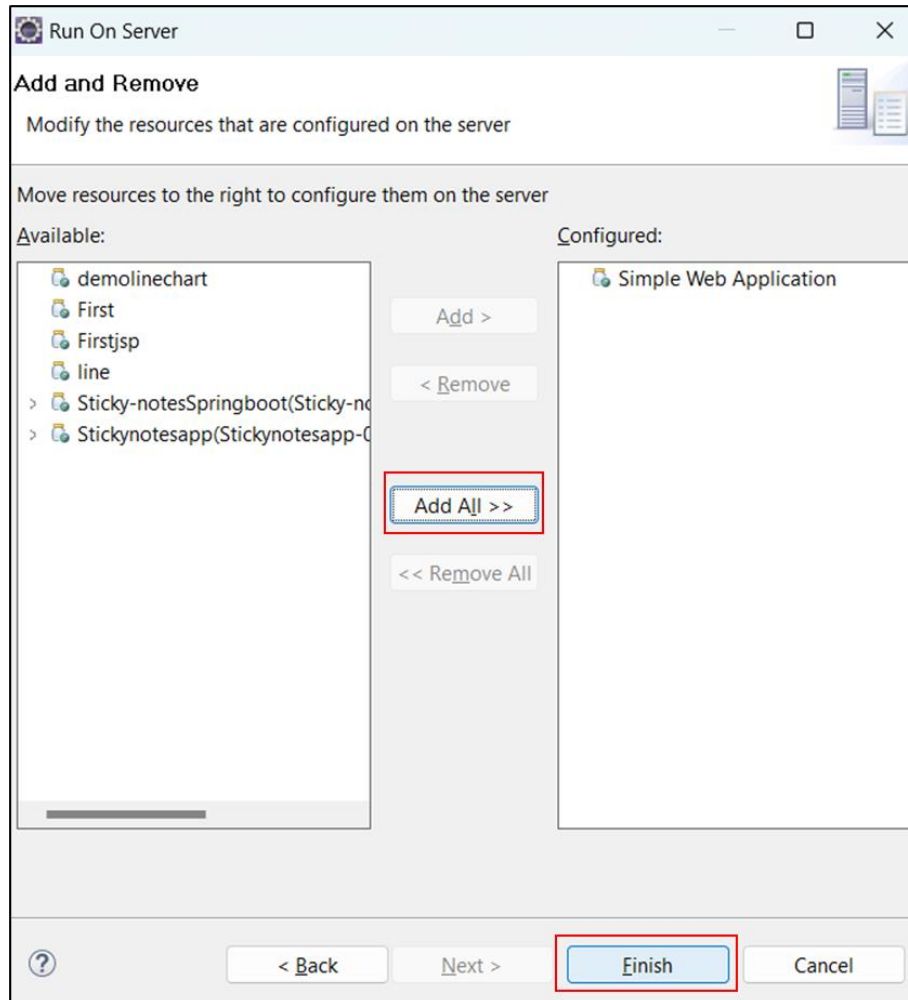


Figure 3.35: Add the Web Application

The code executes and the output is displayed in the internal Web browser as shown in Figure 3.36.



Figure 3.36: Output on Internal Web Browser

It is possible to change the browser on which the output must be displayed. To do this, perform these steps:

4. Select **Window** → **Web Browser** → **Chrome** as shown in Figure 3.37.

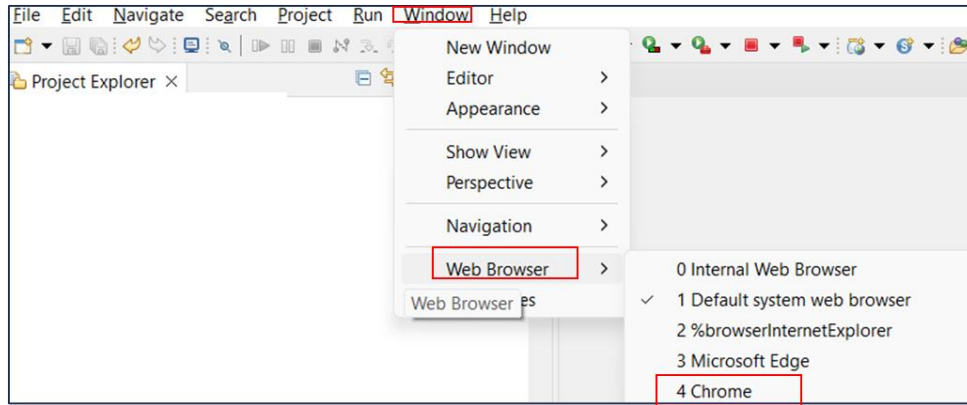


Figure 3.37: Web Browser

5. Run the code again and follow the same procedure to get an output from Chrome, as shown in Figure 3.38.



Figure 3.38: Output on Chrome

Summary

- Apache Tomcat is a free and open-source Web server and servlet container for Java-based programming applications such as Java servlets, Java Server Pages, Java Expression Language, Java WebSocket, and others.
- Apache Tomcat is responsible for monitoring all incoming requests made by clients and loading the appropriate servlet classes in order to fulfill those requests.
- The lifecycle of a Web application is the process of building, deploying, and running it.
- All resources such as, classes, servlets, static files, and so on are stored as a directory structure for better organization.
- In the directory structure, the WEB-INF contains `web.xml` and `Weblogic.xml`. These are deployment descriptors.
- Web apps are stored in a Web Archive (WAR) distinguishing them from other types of Java programs.
- A WAR file contains all the contents of a Web application, it speeds up the process of delivering huge files across the network.
- The Eclipse Web application project directory is structured in a standardized way to ensure organization and ease of management.

Check Your Progress

1. Which of the following statements are true about Tomcat?

(A)	It is platform specific
(B)	It is well documented
(C)	It does not support SSL
(D)	None of these

2. Identify the incorrect statement from the following:

(A)	A Web application is accessible to all users connected to the Web.
(B)	Container is a program, which manages the execution of a JSP program or servlets.
(C)	The Web container takes request from a servlet and passes it to a Web server for processing.
(D)	Performance of a servlet is independent of the efficiency of the container.

3. Which of the following statements about deployment descriptor are true?

(A)	Instructs a deployment tool to create specific configuration requirements for deployment of a servlet.
(B)	Written using HTML syntax.
(C)	Placed under the <code>WEB-INF</code> directory.
(D)	Stored in <code>web.xml</code> file.

4. Which of the following is not a step in the lifecycle for creating, deploying, and executing a Web application?

- Develop the Web component code.
- Develop the Web application deployment descriptor.
- Compile the Web application components and helper classes referenced by the components.
- Zip the file and upload it to Google Drive.
- Deploy the application into a Web container.
- Access a URL, which references the Web application.

(A)	a, c
(B)	b, c
(C)	d
(D)	e, f

5. Which of the following folders in Eclipse for JSP projects typically contains the deployment descriptor file?

(A)	main/java
(B)	WEB-INF
(C)	main/resources
(D)	lib

Answers

1.	A
2.	B
3.	D
4.	C
5.	B

Try It Yourself

1. Install Jakarta EE 10 and Eclipse IDE on your laptop or computer.
2. Install Tomcat Server version 10.1.20 on your laptop or computer.
3. Develop a Web application using JSP in Eclipse, to display the months of the year sequentially and test it on the Tomcat server to ensure proper functionality.

Jakarta Servlets

Welcome to the Session, **Jakarta Servlets**.

The session explains about Jakarta Servlets that process Web client requests and generate response. It describes the Jakarta Servlet API, its architecture, and the lifecycle of a Servlet. Further, the session explains request and response objects passed to the Servlet. The session introduces `RequestDispatcher` interface that helps in Servlet communication. It also explains how to work with the Servlet initialization parameters. Finally, the session concludes with an explanation of how to handle errors in the Servlet and also, file handling with Jakarta Servlet technology.

In this session, you will learn to:

- Explain the Jakarta Servlet API
- Explain the servlet architecture and lifecycle of Servlet
- Describe the servlet mapping specifications
- Explain methods of `ServletRequest` and `HttpServletRequest` interfaces
- Explain methods of `ServletResponse` and `HttpServletResponse` interfaces
- Describe use of response headers
- Explain how to read text and binary data from a request
- Explain `ServletConfig` and `ServletContext` interfaces
- Explain Servlet context Attribute
- Explain redirection of client requests
- Explain `RequestDispatcher` interface
- Explain error handling in Servlet
- Explain uploading files with Jakarta Servlet technology

4.1 Introduction

A servlet is a Web component based on Jakarta EE technology. It is managed by a container and is responsible for generating dynamic content. It is platform-independent and is loaded dynamically and run by Jakarta technology-enabled servers.

A servlet container implements the request/response communication with the Web client. It decodes Multipurpose Internet Mail Extensions (MIME) requests and also formats the responses. Servlet containers are also responsible for managing the lifecycle of servlets. These are already built into or installed in a Web-enabled application server. All servlet containers support HTTP protocol. They may also

support HTTPS protocols. The container also controls the environment of the servlet.

Servlet can have different types of clients such as a Web client using HTTP protocol or a Web service endpoint that uses SOAP protocol.

4.2 Servlet API

All classes related to developing and managing Servlets are provided in following packages:

- `jakarta.servlet`
- `jakarta.servlet.http`

<code>jakarta.servlet</code>	<code>jakarta.servlet.http</code>
The <code>jakarta.servlet</code> package contains generic interfaces and classes that are implemented and extended by all Servlets. This package provides a framework for the Servlet operations. It includes an interface named <code>Servlet</code> which defines the lifecycle methods for a Servlet. Every Servlet must implement the <code>jakarta.servlet.Servlet</code> interface directly or indirectly (by extending the class, which already implements this interface).	The <code>jakarta.servlet.http</code> package contains classes and interfaces used for developing a Servlet that works with HTTP protocol.

Figure 4.1 displays the Servlet API hierarchy.

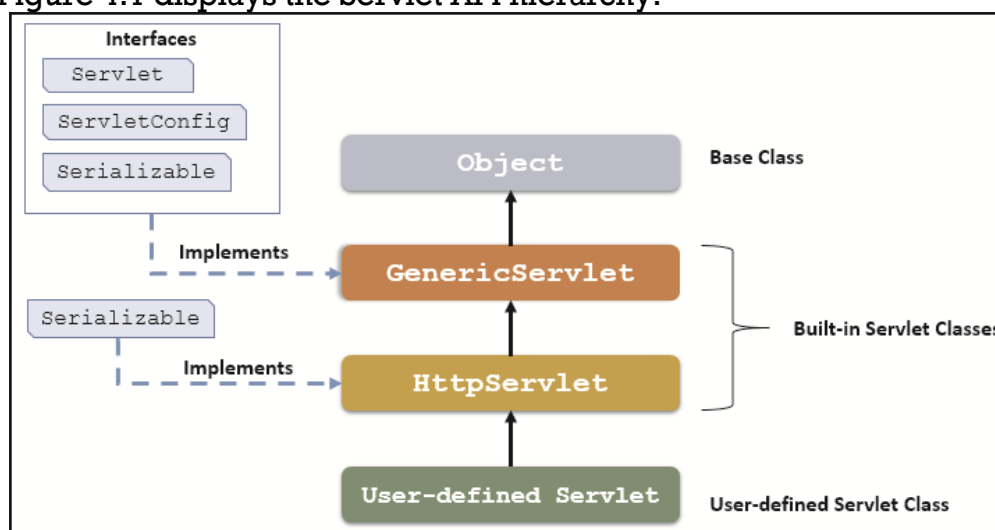


Figure 4.1: Servlet Hierarchy

The `jakarta.servlet` package contains an abstract class named `GenericServlet` which inherits from `Object` class and helps to design a protocol independent Servlet. You can inherit the `GenericServlet` class to design your own Servlet.

The `GenericServlet` implements three interfaces, which are as follows:

Servlet	This interface defines the lifecycle methods for a Servlet.
ServletConfig	This interface defines methods that are used by Servlet container to pass information to a Servlet instance during its initialization.
Serializable	This interface is defined in <code>java.io</code> package to serialize the state of an object.

Consider a situation where you must develop a login Servlet to validate the user information. Code Snippet 1 demonstrates how to create a protocol-independent login servlet.

Code Snippet 1:

```
public class LoginServlet extends GenericServlet{
    . . .
}
```

In the given code, the class named `LoginServlet` is a generic servlet.

To develop a Servlet which is accessible only through a Web browser, you can create an HTTP Servlet by extending `HttpServlet` class. The `HttpServlet` class is a subclass of `GenericServlet` and enables to create an HTTP-based Servlet as part of a Web application.

Code Snippet 2 shows the creation of login servlet to be accessed on the Web.

Code Snippet 2:

```
public class LoginServlet extends HttpServlet{
    . . .
}
```

In the given code, the `LoginServlet` is defined as HTTP servlet, so it will be accessed through HTTP protocol.

4.3 Servlet Lifecycle

The lifecycle methods define how the servlet is loaded, initialized, handles requests from clients, and is taken out of service.

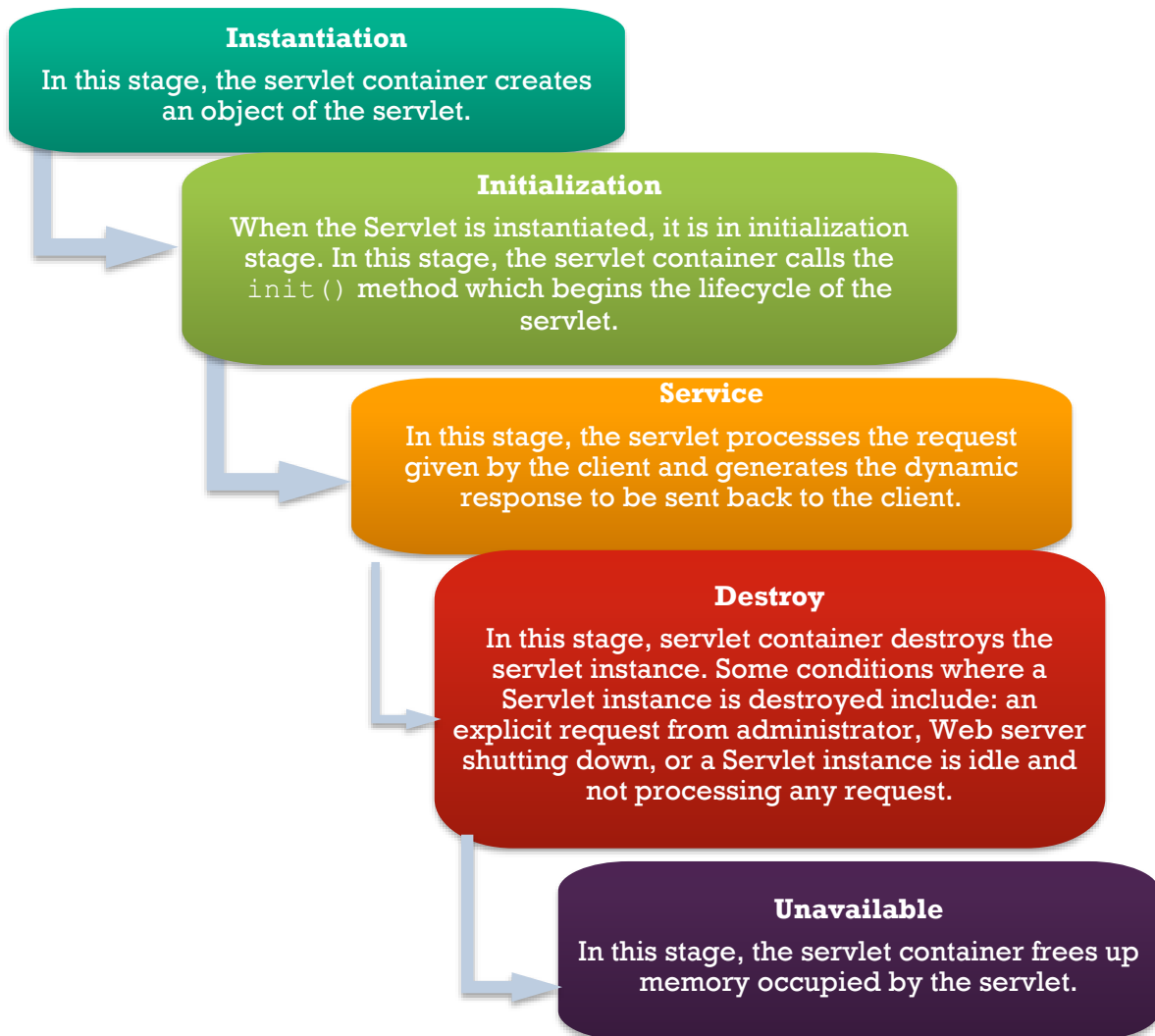


Figure 4.2 depicts the servlet lifecycle.

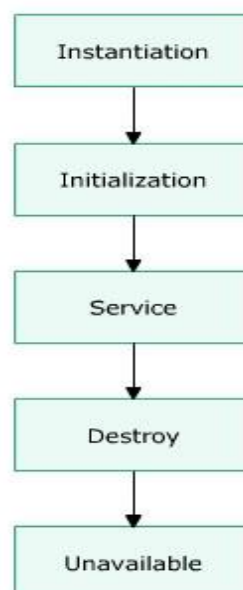
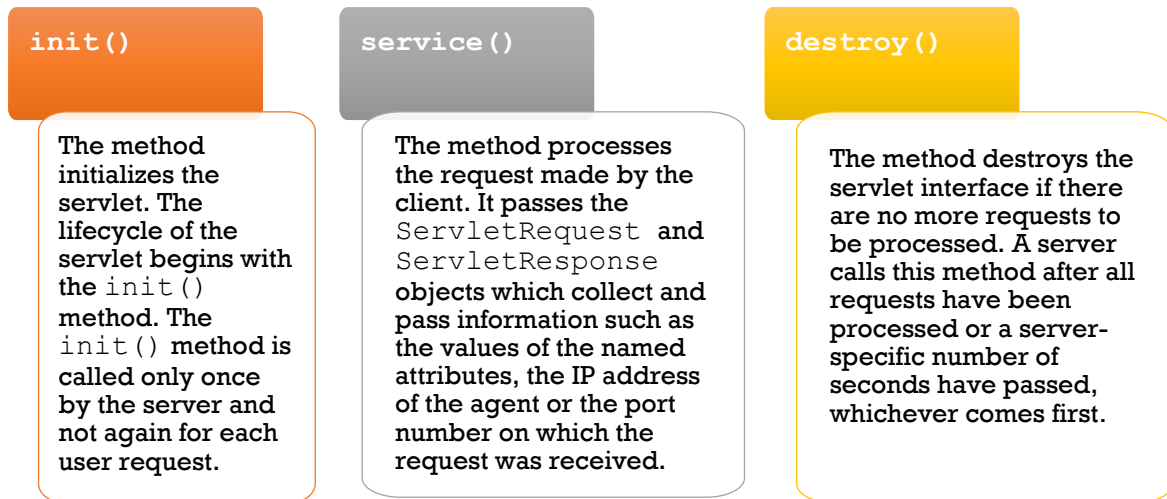


Figure 4.2: Servlet Life Cycle

4.3.1 Servlet Life Cycle Methods

The servlet lifecycle is defined by following methods:



4.3.2 Servlet Architecture

The Servlet container is responsible for handling Servlet execution, which is based on multithreaded model. This means that, when container receives a request for the same Servlet, it creates a new thread that handles execution of the Servlet and dies once the execution is completed.

Figure 4.3 shows handling of multiple clients request by a Servlet.

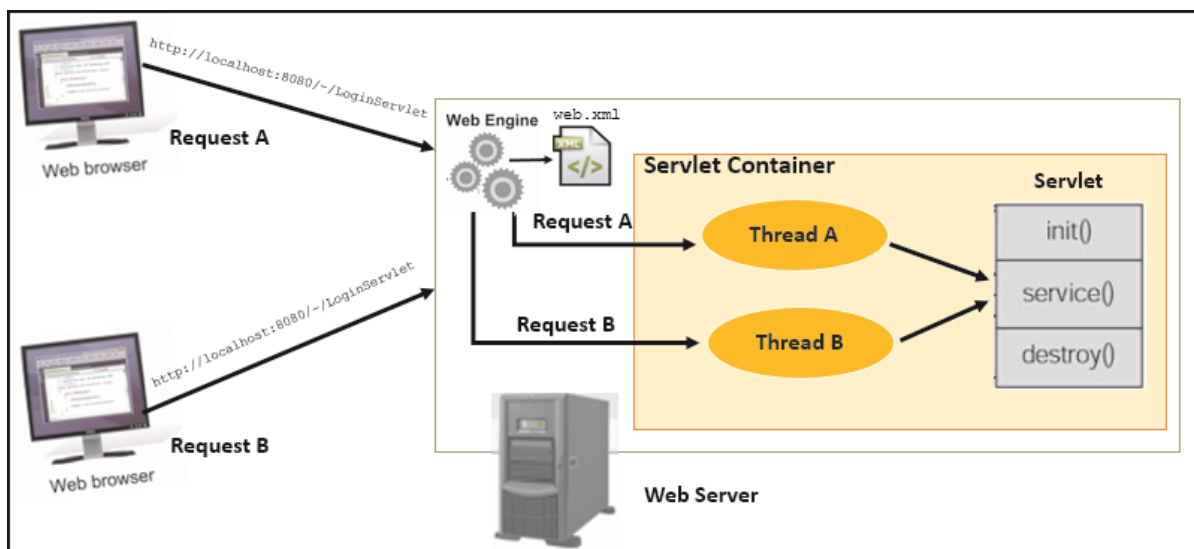


Figure 4.3: Servlet Multithreaded Model

As shown in Figure 4.3, the flow of information in a Servlet is as follows:

1. The Web browser sends an HTTP request to the Web server through a Uniform Resource Locator (URL). For example, a URL, `http://localhost:8080/.../LoginServlet` is sent to the Web server.
2. On receiving the request, the Web engine looks into the `web.xml` deployment descriptor file to match the URL with the registered Servlets.
3. Once the requested Servlet is found, the Web engine forwards it to the Servlet container, which is responsible for instantiating the Servlet instance.
4. The container first locates the servlet class, loads the servlet, and creates an instance of the servlet in the memory.
5. After the Servlet is loaded by the Servlet container, it is initialized completely and then, the client request is processed.

To service multiple client's request, the servlet container creates multiple threads for the same Servlet instance. Each instance handles request received from the client and generates response that is sent to the Web engine, which in turn, sends the response to the client.

4.3.3 HTTP Specific Request Handling Methods

The `HttpServlet` abstract subclass has additional methods beyond the basic Servlet interface that are automatically called by the service method in the `HttpServlet` class. These methods are as follows:



`doGet()` and `doPost()` methods are most used in application development while other methods are used by developers familiar with HTTP programming.

4.4 Handling Servlet Request

When a servlet is requested to handle a request, it requires specific information about the request so that it can process the request appropriately. The `ServletRequest` interface provides access to this specific information.

When a servlet handles a request, the server passes a request object, which implements the `ServletRequest` interface. With this object, the servlet can find out information about the actual request such as protocol, Uniform Resource Locator (URL) and type. It can access parts of the raw request such as headers and input stream. It can also retrieve client-specific request parameters such as data entered in online forms.

Figure 4.4 depicts a Servlet request.

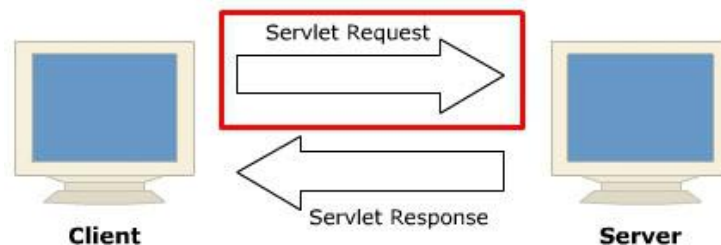


Figure 4.4: Servlet Request

4.4.1 ServletRequest Interface

The `ServletRequest` interface defines an object that makes client request information available to servlets. The `ServletRequest` object is passed as an argument to the `service()` method of the servlet.

Some of the methods defined in the `ServletRequest` interface are as follows:

- `public String getParameter(String s)`
Returns the value of a specified parameter that is sent along with the request information. For example, the statement, `String name1 = request.getParameter("name1");` retrieves the value from the parameter `name1` passed in the request.
- `public Object getAttribute(String name)`
The method retrieves the value of an attribute specified by name, that was set using `setAttribute()` method and returns null when no attribute with the specified name exists. For example, `Object obj1 = cnt1.getAttribute("obj1");`

- `public int getLength()`
Returns the length of content in bytes and returns -1 if the length is not known.

Code Snippet 3 shows the use of `getLength()` method.

Code Snippet 3:

```
// The length of the content is passed to len1 and compared
//with the size of file limit
if((len1 = request.getLength()) > fileLimit)
{
    System.out.println ("Name1 "+login +" Id1 " + id +
        "Expected Upload size is more than specified file
Limit");
}
```

- `public ServletInputStream getInputStream()` throws `IOException`

Returns the binary data of the body of request, requested by client and stores it in a `ServletInputStream` object. For example, to read the binary data, following statement can be used:

```
ServletInputStream inStream1 = request.getInputStream();
```

4.4.2 `HttpServletRequest` Interface

The `HttpServletRequest` defines an `HttpServletRequest` object, which is passed as an argument to `service()` method.

Methods defined by the `HttpServletRequest` interface are as follows:

- `public Cookie[] getCookies()`
Returns an array containing the entire `Cookie` objects and returns null if no cookies were found. For example, `Cookie[] cookies = request.getCookies();`
- `public String getHeader (String name)`
Returns the value of the specified request header as a `String` and returns null if the request did not include a header of the specified name. For example, `out.println(request.getHeader("host")+"<br>");` returns the value of the header `host`.
- `public String getMethod()`
Returns the name of the HTTP method used to make the request. For example, `GET`, `POST`, and `PUT`. For example, `out.println("<td align=center>" + request.getMethod() + "</td></tr>");`

- `public String getPathInfo()`
Returns the path information associated with a URL. For example, `String ses_id1 = req.getPathInfo();`
- `public String getAuthType()`
Returns the basic authentication scheme used to protect the servlet from unauthorized users. For example, `out.println(request.getAuthType());`
- `public String getPart(String name)`
Retrieves a specific part from a multipart request by its name, returning it as a `Part` object, or null if the part does not exist. For example, `Part filePart = request.getPart("file");`

4.4.3 Reading Parameters from Request

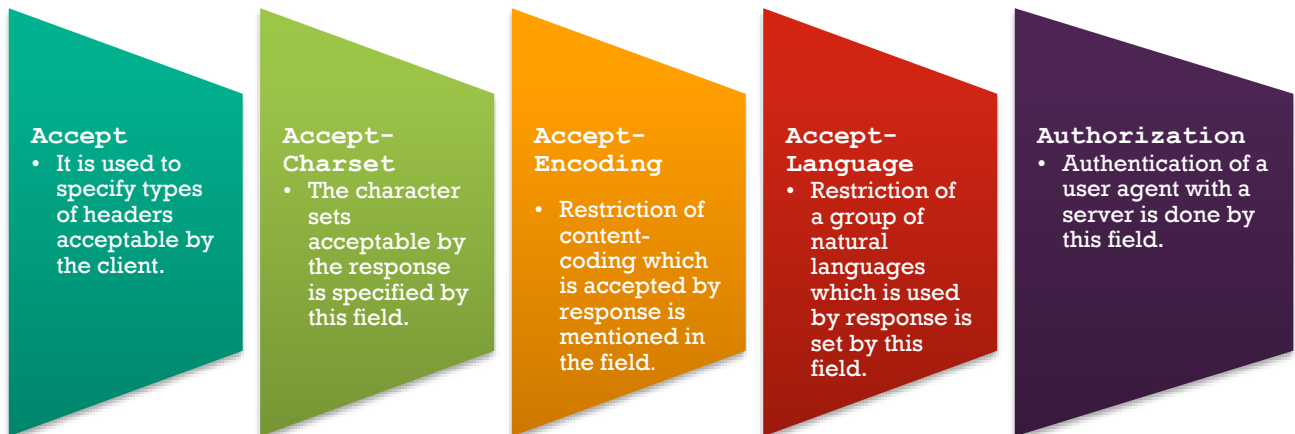
Request parameters are strings sent by the client to a servlet container as a part of its request. These parameters are stored as a set of name-value pairs. Following methods of the `ServletRequest` interface are available to access parameters:

- `public java.lang.String getParameter(java.lang.String name)`
Returns the value of a request parameter as a `String`, or null if the parameter does not exist. For HTTP servlets, parameters are contained in the query string or posted form data. For example, `String name = request.getParameter("names");`
- `public java.util.Enumeration getParameterNames()`
Returns an enumeration of string objects containing the names of the parameters of this request. If the request has no parameters, the method returns an empty enumeration. For example, `Enumeration Books = req.getParameterNames();`
- `public java.lang.String[] getParameterValues(java.lang.String name)`
Returns an array of string objects containing all of the parameter values or null if parameters do not exist. For example, `String[] value = request.getParameterValues("interests");`
- `public java.util.Map getParameterMap()`
Returns a `java.util.Map` of the parameter of the request, which contains names as keys and parameter values as map values.

4.4.4 Http Request Headers

The `request-header` fields allow the client to pass additional information about the request, and about the client itself, to the server.

Few of the request headers available include:



4.4.5 Reading Headers form Request

A servlet can access the headers of an HTTP request through following methods of the `HttpServletRequest` interface:

- `public String getHeader (String name)`
Returns the value of the specified request header as a `String`. For example,
`out.print (" <TD>" + request.getHeader(headerName));`
- `public java.util.Enumeration getHeaders(String name)`
Returns all the values of the specified request header as an `Enumeration` of `String` objects. For example, `Enumeration headers = request.getHeaders();`
- `public java.util.Enumeration getHeaderNames()`
Returns an enumeration of all the header names this request contains. If the request has no headers, this method returns an empty enumeration. For example, `Enumeration strHeaderName = request.getHeaderNames();`

Form data will be populated as parameters in following conditions:

- When the HTTP method is `POST`.
- When the request is an `HTTP` or `HTTPS` request.
- When the content type is `application/x-www-form-urlencoded`.
- When the servlet has made an initial call of any of the `getParameter` methods on the request object.

4.4.6 HTML Forms

HTML forms are used to collect user input data. These input data are stored as form fields and passed through browser URL for further processing in the Servlet. Several input types are provided by HTML to be used with `<form>` element. Some of them are as follows:

Text input	Drop-down list	Submit button
Allows the user to enter a single line of text.	Provides a list of options to the user to be selected.	Allows the user to send the data to the server for processing.

The HTML form has two major attributes:

- **Methods** – Defines the way of data transfer is done to the Web server. Methods such as `GET`, `POST`, and so on are used to forward the parameters with the HTTP request.
 - **GET** - With the `GET` method, all form data is encoded into the URL, appended to the action URL as query string parameters. The `GET` method is taken as the default if the method attribute is not specified in the HTML form.
 - **POST** - With the `POST` method, form data appears within the message body of the HTTP request.
- **Action** - Includes the servlet class name. For example, `<form method="POST" action="Login">`.

Code Snippet 4 shows an HTML page for accepting user choice.

Code Snippet 4: index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Test</title>
</head>
<body>
<form method="GET" action="LoginServlet">
  Username: <input type="text" id="username"
name="username"/>
  <br/>
  <select id="color" size="1" name="color">
    <option> light
    <option> amber
    <option> brown
    <option> dark
```

```

        </select>
        <input type="Submit" value="Submit" />
    </form>
</body>
</html>

```

Enctype

The `enctype` attribute specifies how the form-data is encoded when submitting it to the server. `enctype` attribute can be used when the method is **post**. The syntax is:

```
<form action="" method="" enctype="multipart/form-data">
```

Table 4.1 displays the value and description of the `enctype` attribute.

Value	Description
application/x-www-form-urlencoded	This is the default value. All characters are encoded. All special characters are converted to ASCII HEX and spaces are converted to "+" symbol
multipart/form-data	This value allows a user to upload a file through the form
text/plain	This value sends data without any encoding. This is not recommended

Table 4.1: Enctype Attribute

4.4.7 Handling Form Data Using Request Object

In a Servlet, `doGet()` and `doPost()` are methods that handle form data. The request object allows the servlet to obtain the user data from the form. To handle form data, call the data reading methods of the request object.

Code Snippet 5 shows the use of `getParameter()` method in handling data.

Code Snippet 5: LoginServlet.java

```

public void doGet(HttpServletRequest request,
    HttpServletResponse response) throws IOException,
    ServletException
{
    String name = request.getParameter("username");
    String selectedColorValue = request.getParameter("color");
    // data processing code here
}

```

The code invokes the `getParameter()` method to get value of the field name and color and store them as `String`. This code is incomplete as of now and hence will not display anything.

4.5 Handling Servlet Response

Handling Servlet response defines an object to help a Servlet in sending a response to the client using different interfaces.

4.5.1 ServletResponse Interface

Servlet response is the response sent by the servlet to the client. The `ServletResponse` and `HttpServletResponse` interfaces include all the methods required to create and manipulate the output of a Servlet.

Figure 4.5 depicts servlet response.

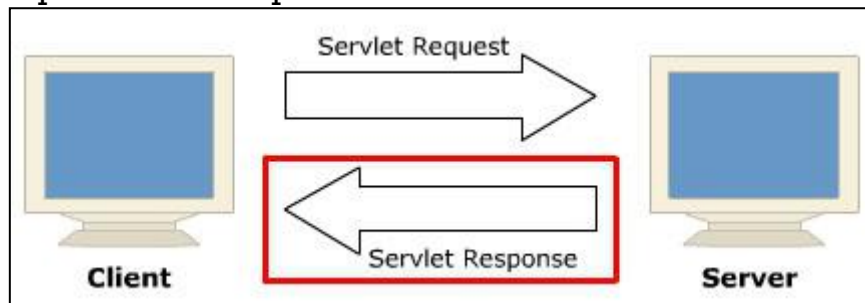


Figure 4.5: Servlet Response

The `ServletResponse` interface defines methods that allows to retrieve an output stream to send data to the client, decide on the content type, and so on. The `ServletResponse` interface defines an object, which is used to provide response to a client. This object is passed as an argument to `service()` method of a servlet.

Methods defined by the `ServletResponse` interface are as follows:

- **`public java.lang.String getContentType()`**
Returns the MIME type of the request body or null if the type is not known. For example,

```
out.println ("Message content  
type:"+msg.getContentType()+"<br />");
```
- **`public PrintWriter getWriter() throws IOException`**
Returns an object of `PrintWriter` class that sends character text to the client. For example, `PrintWriter out1= response.getWriter();`

```
out1.println("Some text and HTML");
```
- **`public ServletOutputStream getOutputStream() throws IOException`**

Uses `ServletOutputStream` object to write response as binary data to the client. For example, `ServletOutputStream out2 = response.getOutputStream(); out2.write(twoBytearray);`

- **public void setContentType(java.lang.String str)**
Used to set the format in which the data is sent to the client, either in normal text format or HTML format.
For example, `response.setContentType("text/html");`

A complete example is shown now. Create a Dynamic Web Project in Eclipse. Add the code shown in Code Snippet 6 to a file named `LoginServlet.java`.

Code Snippet 6: LoginServlet.java

```
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    public void doGet(HttpServletRequest request,
        HttpServletResponse response) throws IOException,
        ServletException {
        String name = request.getParameter("username");
        String selectedColorValue =
            request.getParameter("color");
        // data processing code here
        PrintWriter out1= response.getWriter();
        out1.println(name + ", you selected " +
            selectedColorValue);
    }
}
```

Add the code given in Code Snippet 7 to `web.xml`. Note that `web.xml` is to be present under **WEB-INF** folder. The HTML file named `index.html` whose code was given in Code Snippet 4 should be placed under **webapps** folder in the project structure.

Code Snippet 7: web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
```

```

id="WebApp_ID" version="3.1">
  <display-name>code4-5</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class> LoginServlet </servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>LoginServlet </servlet-name>
    <url-pattern>/LoginServlet </url-pattern>
  </servlet-mapping>
</web-app>

```

This file informs the application about the Servlet and how it is going to be called.

Right-click the application and select **Run As → Run on Server**. The output will be displayed as shown in Figures 4.6 and 4.7 respectively.

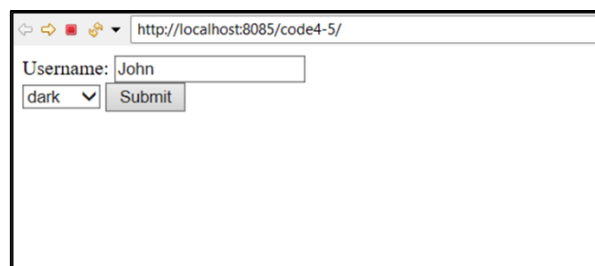


Figure 4.6: Initial Output of the Example



Figure 4.7: Processed Output of the Example

4.5.2 HttpServletResponse Interface

The `HttpServletResponse` interface extends `ServletResponse` interface and provides information to an `HttpServlet`. It defines an `HttpServlet` object, which is passed as an argument to the `service()` method of a servlet.

Using the `HttpServletResponse` interface, you can set HTTP response header, set the content type of the response, acquire a text stream for the response, acquire a binary stream for the response, redirect an HTTP request to another URL, or add cookies to the response. Figure 4.8 depicts HTTP servlet response.

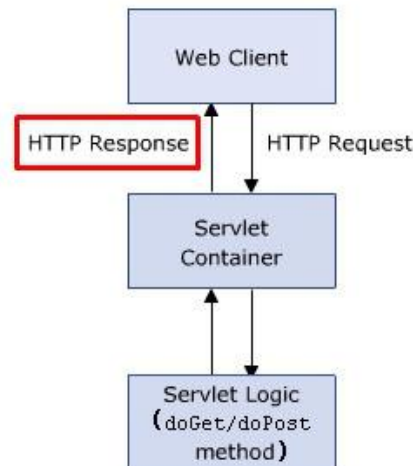


Figure 4.8: HTTP Servlet Response

4.5.3 Methods in HttpServletResponse Interface

Methods defined by the `HttpServletResponse` interface are as follows:

- **`public void addCookie(Cookie cookie)`**
Adds the specified cookie to the response, sent to the client. Code Snippet 8 shows the use of `addCookie()` method.

Code Snippet 8:

```
// Adds a cookie named cookie2 with color red to existing
//cookies
Cookie cookie2 = new Cookie("color", "red");
response.addCookie(cookie2);
```

- **`public void addHeader(java.lang.String name, java.lang.String value)`**
Used to add name and value to the response header. For example, `response.addHeader("Refresh", "15");`
- **`public boolean containsHeader(String name)`**
Used to verify if the response header contains any values. It returns true if the response header has any values. It returns false otherwise. For example, `res.containsHeader ("Cache");`
- **`public void sendError(int sc) throws java.io.IOException`**
Sends an error response to the client using the specified status code and clearing the buffer. Status codes are used to indicate the reason, a request is not satisfied, or that a request has been redirected. For example,
`httpResp.sendError (HttpServletResponse.SC_FORBIDDEN, "Good Bye for now!");`

4.5.4 Response Headers

The response header is attached to the files being sent back to the client. It contains the date, size, and type of file that the server sends back to the client and also data about the server itself.

Response headers can be used to specify cookies to supply the modification date. It is also used to instruct the browser to reload the page. In addition, it specifies how big the file is, to determine how long the HTTP connection must be maintained.

4.5.5 Sending Headers

Methods of `HttpServletResponse` interface that are used to send header information to the client are as follows:

Adds a response header with the given name and value. For example,
`res.addHeader("Cache", 3.14);`

addHeader ()

Adds a response header with the given name and date value. For example,
`res.addDateHeader("Cache", 20-02-2022 )`

**addDateHeader
()**

Adds a response header with the given name and integer-value. For example,
`res.addIntHeader("Cache", 3)`

**addIntHeader
()**

Returns a boolean value to indicate if response header has already been set.

containsHeader ()

4.5.6 Sending Data from Servlet Using Response Object

Let us now create a servlet to send a jar file data using `response` object to the client. The response object should perform following steps:

- Inform the browser, the type of file/data that is getting transferred.
- Convert the object data to stream.

Code Snippet 9 shows how to send data using `response` object. Ensure that `wordlist.doc` is present under `WEB-INF` folder. Also, create appropriate `web.xml` before attempting to run this code.

Code Snippet 9:

```
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.InputStream;
```

```

import java.io.OutputStream;
@SuppressWarnings("serial")
public class CodeReturnJARfile extends HttpServlet {
public void doGet(HttpServletRequest request,
HttpServletRequest response)
throws IOException, ServletException {
// Get the InputStream of the file from the ServletContext
response.setContentType("application/text");
ServletContext ctx = getServletContext();
InputStream is = ctx.getResourceAsStream
("/WEB-INF/wordlist.doc");
// Check if the InputStream is not null before proceeding
if (is != null) {
// Set response content type to MS Word
response.setContentType("application/msword");
response.setHeader("Content-Disposition",
"attachment; filename=\"wordlist.docx\"");

// Get OutputStream to write the file content to
the response
OutputStream os = response.getOutputStream();
// Write file content to the response
byte[] bytes = new byte[1024];
int read;
while ((read = is.read(bytes)) != -1) {
os.write(bytes, 0, read);
}
// Close streams
os.flush();
os.close();
is.close();
} else {
// If InputStream is null, send an error response
response.sendError(HttpServletResponse.SC_NOT_FOUND,
"File not found");
}
}
}

```

In the given Code Snippet,

`response.setContentType("application/text");` tells the browser to recognize that the response object carries a text file. The statement, `OutputStream os = response.getOutputStream();` reads the bytes and writes the bytes to the output stream that is received from the response object.

The outcome of the code will be a newly generated file named `worldlist.doc` having the same contents as `wordlist.doc`. This file will be saved under Downloads folder.

4.6 Reading Binary Data

Some Web applications such as e-mail must send files such as documents and images, from one system to another as attachment. In this case, data transmitted from a client to a Web application will be of binary type. In such cases, the user will specify the reference to the binary files in online form.

Multipart/form-data is an Internet media type that returns a set of values as the result of a user filling out a form. These set of values are transmitted as binary data. Each request handled by a servlet has an input stream associated with it.

You use `getInputStream()` to retrieve the input stream as a `ServletInputStream` object. The syntax of the method `getInputStream()` is as follows:

Syntax:

```
public ServletInputStream ServletRequest.getInputStream()
throws IOException
```

4.6.1 Producing Text and Binary Data

Two methods belonging to `ServletResponse` interface for producing output streams are as follows:

```
➤ public ServletOutputStream getOutputStream() throws
IOException
```

The `getOutputStream()` returns a `ServletOutputStream`, which can be used for binary data.

```
➤ public java.io.PrintWriter getWriter() throws IOException
```

The `getWriter()` returns a `java.io.PrintWriter` object, which is used only for textual output. The `getWriter()` method examines the content-type to determine what character set to use, so `setContentType()` should be called, before `getWriter()`.

4.6.2 Sending Text and Binary Data

Whenever any file sent to the client has to be encoded, for example, images or text attachments, it is sent as binary data. `ServletOutputStream` provides an output stream for sending binary data to the client.

Methods supported by the `ServletOutputStream` class are as follows:

- `public void print(boolean b) throws java.io.IOException`
Writes a boolean value to the client with no Carriage Return Line Feed (CRLF) character at the end.
- `public void println(char c) throws java.io.IOException`
Writes a character value to the client, followed by a CRLF.

4.7 Redirecting Requests

Whenever the client makes a request, the request goes to the servlet container. The Servlet container decides whether the concerned servlet can handle the request or not. If the servlet cannot handle the request, it decides that the request can be handled by another servlet or a Jakarta Server Page (JSP) file.

Then, the servlet calls `sendRedirect()` method and sends response to the browser along with the status code. The browser makes a new request, with name of the servlet that can now handle the request and displays result on the browser.

This redirecting is done by employing following methods:

- `public void sendRedirect(java.lang.String location) throws java.io.IOException`

Sends a redirect response to the client using the specified redirect location URL.

For example,

```
response.sendRedirect(response.encodeRedirectURL(contextPath
+ "/maps")); ; redirects the request to the specified location.
```

- `public java.lang.String encodeRedirectURL(java.lang.String url)`

Encodes the specified URL for use in the `sendRedirect()` method or, if encoding is not required, returns the URL unchanged. All URLs sent with the `sendRedirect()` method should be run encoded using this method.

4.7.1 RequestDispatcher

The servlets sometimes must access network resources to satisfy client requests.

The resources can be another servlet, a JSP page, or a CGI script. A servlet runs in an environment called the servlet context, which describes various parameters associated with the servlet. A servlet belongs to only one servlet context.

A servlet cannot access resources such as static HTML pages, which are stored in local files using the `RequestDispatcher` objects. The local resource can be accessed using the method `getResource(String path)` of `ServletContext` object that returns a `URL` object for a resource specified by a local Uniform Resource Identifier (URI), for example, `/html`.

The method `ServletRequest.getRequestDispatcher(java.lang.String)` allows using relative path whereas, the method `ServletContext.getRequestDispatcher(java.lang.String)` allows only absolute path. The servlets belonging to same application can share data using following methods of `RequestDispatcher` interface:

➤ **`forward()` method**

The method is used to forward request from one servlet to another resource on the same server.

Syntax:

```
public void forward(ServletRequest request1, ServletResponse
responsel) throws ServletException, IOException
```

where,

- `request1` is the request made by the client to the servlet
- `responsel` is the response made by the servlet to the client

Code Snippet 10 retrieves the servlet context from its `RequestDispatcher` instance, and then, forwards control to the JSP page.

Code Snippet 10:

```
public class MyServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request,
HttpServletResponse response)
        throws ServletException, IOException {
        request.setAttribute("title", "Home Page");
        ServletContext servletContext = getServletContext();
        RequestDispatcher dispatcher =
servletContext.getRequestDispatcher("/cart.jsp");
        dispatcher.forward(request, response);
    }
}
```

➤ **`include()` method**

The method is used to include the contents of another servlet, JSP page, or an HTML file.

Syntax:

```
public void include(ServletRequest request1, ServletResponse
response1) throws ServletException, IOException
```

where,

- request1 is the object that contains client request.
- response1 is the object that contains servlet response.

Code Snippet 11 demonstrates the code fragment that includes the content from the specified path.

Code Snippet 11:

```
// Include static page or servlet/JSP page from the specified
//path
RequestDispatcher dispatcher =
getServletContext().getRequestDispatcher(path);
if (dispatcher == null)
    out.println(path + " not found");
else
    dispatcher.include(request, response);
```

4.8 Initializing Servlets

In a Web application, the database connection information, such as SQL username and password are not sent to the servlet every time a client makes a request. Hence, this information must be passed to the servlet before beginning the servlet lifecycle.

To pass the parameters from the client side to the servlets for executing the first time and retrieve the data required as specified by the user the servlet must be initialized. To set up an uninterrupted client-server communication at the beginning of a process initialization is done. After setting up the first connection, the process is carried out by auto refreshing the page, which is managed by the servlet itself.

4.8.1 init Parameters

To pass as an argument during initialization, the servlet container uses an object of `ServletConfig` interface. The `ServletConfig` interface provides various methods to configure a servlet, before processing the data requested by the client.

The Servlet has the direct access to Servlet initialization parameters using the `getInitParameter()` method. This method retrieves value of the initialization parameter. It returns null if the specified parameter does not exist.

Code Snippet 12 demonstrates the code that retrieves the parameters of the servlet.

Code Snippet 12:

```
//Retrieves the parameters user and password
super.init(config);
String user1 = getInitParameter("user");
String password1 = getInitParameter("password");
```

The same method is available in the ServletConfig interface to read the initialization parameters. Figure 4.9 depicts servlet initialization.

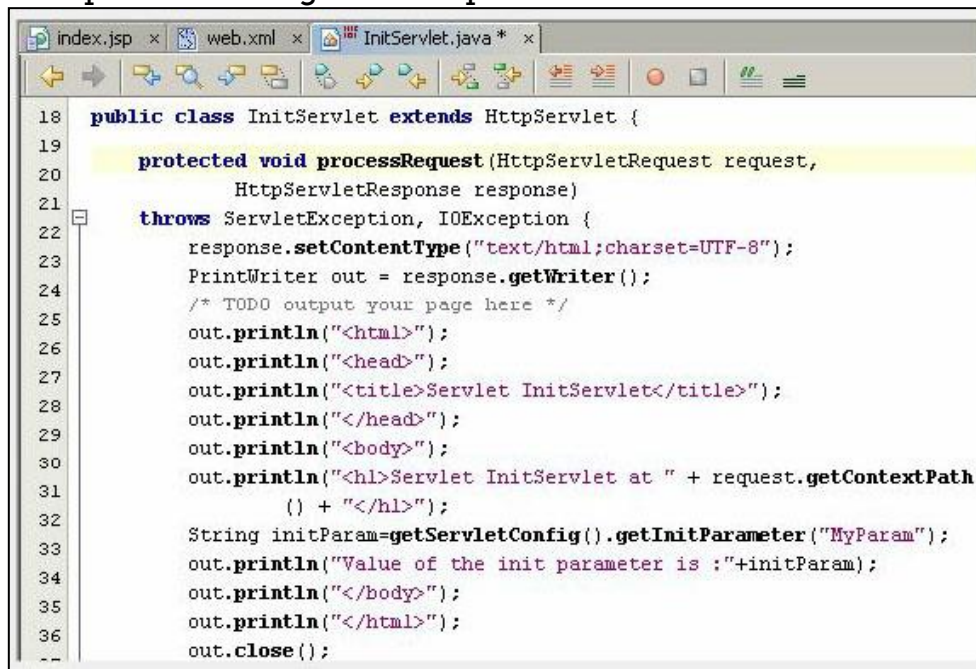


Figure 4.9: Servlet Initialization Using ServletConfig

4.8.2 Reading Parameters in init() Method

Parameters can be read in init() method.

➤ public void init() throws ServletException

Code Snippet 13 illustrates how a servlet reads initialization parameters.

Code Snippet 13:

```
int count;
public void init(ServletConfig config) throws
ServletException {
    super.init(config);
    String initial = config.getInitParameter("initial");
```

```

try {
    count = Integer.parseInt(initial);
} catch (NumberFormatException e) {
    count = 0;
}
}

```

4.8.3 ServletContext

The container first locates the servlet class, loads the servlet, and creates an instance of the servlet. It then, invokes `init()` method to initialize the servlet.

A `ServletConfig` interface object is passed as an argument to the `init()` method, which provides the servlet with access to `ServletContext` interface.

4.8.4 ServletContext Interface

The servlet context provides access to various resources and facilities, which are common to all servlets in the application. The servlet API is used to set the information common to all servlets in an application.

Servlets running in the same server sometimes share resources, such as JSP pages, files, and other servlets. This is required when several servlets are bound together to form a single application, such as a chat application, which creates single chat room for all users.

The servlet API also knows about resources to the servlets in the context. The attributes of `ServletContext` can be used to share information among a group of servlets.

4.8.5 ServletContext Methods

The `ServletContext` methods are used to get and set additional information about the servlet.

Table 4.2 lists some commonly used methods of `ServletContext` interface.

Method	Description
<code>public String getInitParameter(String name)</code>	Returns parameter value for the specified parameter name.
<code>public Enumeration getInitParameterNames()</code>	Returns names of the initialization parameters of the context.
<code>public void setAttribute(String name, Object object)</code>	Sets given object in the application scope.
<code>public Object getAttribute(String name)</code>	Returns attribute for the specified name.

Method	Description
public Enumeration getInitParameterNames()	Returns names of the context's initialization parameters as an Enumeration of String objects.
public void removeAttribute(String name)	Removes attribute with the given name from the servlet context.

Table 4.2: Commonly Used Methods of ServletContext

Code Snippet 14 shows the `web.xml` file for specifying the context parameters and servlet initialization parameters.

Code Snippet 14:

```
<web-app xmlns="http://java.sun.com/xml/ns/j2ee"
. . .
<context-param>
    <param-name>Global</param-name>
    <param-value>ValueToAll</param-value>
</context-param>
<!-- other stuff including servlet declarations -->
<servlet>
    <servlet-name> ContextParameters </servlet-name>
    <servlet-class> ContextParameters </servlet-class>
<init-param>
    <param-name>ServletSpecific</param-name>
    <param-value>ServletValue</param-value>
</init-param>
</servlet>
</web-app>
```

Code Snippet 15 demonstrates the code to read the context parameters from the `web.xml` file.

Code Snippet 15:

```
. . .
public class ContextParameters extends HttpServlet{
public void doGet(HttpServletRequest req,
HttpServletRequest res) throws ServletException,
IOException {
    res.setContentType("text/html");
    PrintWriter pw = res.getWriter();
    // Creating ServletContext object
    ServletContext context=getServletContext();
    //Getting value of initialization parameter and printing it
    String str = context.getInitParameter("Global");
    pw.println("driver name is="+ str);
}
```

```

        pw.close();
    }
}

```

4.9 Error Handling in Servlets

There are many situations where a Web server can give an error. A requested page may be moved from one location to another. The address may be wrongly typed. The requested page may be forbidden, maybe temporarily deleted or correct HTTP version might not have found. There are other situations where an error may be generated.

4.9.1 Status Codes

Errors may occur due to one reason or the other, but the status code for each type of error makes it easy to identify the error.

Table 4.3 lists some of the error codes along with their associated message and meaning.

Status Code	Associated Message	Meaning
301	Moved Permanently	Document is moved to a separate location as mentioned in the URL. The page is redirected to the mentioned URL, to find the document.
302	Found	Temporary replacement of file from one location to the other as specified.
400	Bad Request	The request placed is syntactically incorrect.
401	Unauthorized	Authorization is not given to access a password-protected page.
404	Not Found	Resource not found in the specified address.
408	Request Timeout	Time taken by client is very long to send the request (only available in HTTP 1.1).
500	Internal Server Error	Server is unable to locate the requested file. The servlet has been deleted or crashed or has been moved to a new location without informing.

Table 4.3: Status Codes

Figure 4.10 depicts an example of status code.



Figure 4.10: Status Code 404

4.9.2 Error Methods in HttpServlet Class

Errors during the execution of a Web application are reported using following methods:

➤ **sendError()**

This method checks for the status code and sends to the user the specified response message. After sending the error message, the buffer is cleared.

The `sendError()` method sends a predefined error message as a response for the file not found error.

Code Snippet 16 shows the use of `sendError()` method.

Code Snippet 16:

```
// Return the file file1
try {
    ServletUtils.returnFile(file1, out);
}
catch (FileNotFoundException err) {
    res.sendError(res.SC_NOT_FOUND);
}
```

➤ **setStatus()**

On receiving the `setStatus()` method, an error message is thrown or redirected to another default Web page.

Code Snippet 17 shows the use of `setStatus()` method.

Code Snippet 17:

```
/* If the client sent an If-Modified-Since header equal or
after the servlet's last modified time, send a short "Not
```

```

Modified" status code Round down to the nearest second since
client headers are in seconds */

if (servletLastMod == -1) {

    super.service(req, res);

}

else if ((servletLastMod / 1000 * 1000) <=

    req.getDateHeader("If-Modified-Since")) {

    res.setStatus(res.SC_NOT_MODIFIED);

}

```

4.9.3 Logging Errors

Servlets can store actions and errors through `log()` method of the `GenericServlet` class. The message along with the stack's trace is thrown to a servlet log. The `log()` method also assists in debugging by describing all the details about an error. The error can be viewed by using `log()` to record in a server.

- `log()`

The `log()` method can be used by passing either a single argument or two arguments as given here:

Code Snippet 18 shows the use of `log()` methods.

Code Snippet 18:

```

// Return the file
try {
    ServletUtils.returnFile(file, out);
} catch (FileNotFoundException e) {
    log("Could not find file: " + e.getMessage());
    res.sendError(res.SC_NOT_FOUND);
}
catch (IOException e) {
    log("Problem sending file", e);
    res.sendError(res.SC_INTERNAL_SERVER_ERROR);
}

```

The method writes an explanatory error message to the log and a stack trace for specified throwable object into the servlet log file.

4.9.4 Forwarding to Error Page

A `RequestDispatcher` object is used to redirect the client request to a Web page on receiving an error message on the server. The `RequestDispatcher` object is created by the servlet container.

Following methods create objects for the `RequestDispatcher` interface:

- **`ServletContext.getRequestDispatcher(String path)`**
This method takes a string argument as a path, which is within the scope of the `ServletContext`. The path, relative to root is passed as an argument. This path, in turn, is used to locate the servlet to which it is redirected.
- **`ServletContext.getNamedDispatcher(String name)`**
The method takes a string argument indicating the name of servlet to the `ServletContext`. It returns a `RequestDispatcher` object for the named servlet.
- **`ServletRequest.getRequestDispatcher(String path)`**
This method uses relative path which is the location of the file relative to current path. This is similar to the method in the same name as in the `ServletContext`. Transforming the current servlet path relative to a complete path.

Code Snippet 19 shows the dispatching of error object.

Code Snippet 19:

```
public class Dispatcher extends HttpServlet {
    public void doGet(HttpServletRequest req1,
        HttpServletResponse res1) throws ServletException,
        IOException {
        RequestDispatcher dispatcher =

        request.getRequestDispatcher("/error_page.jsp");
        if (dispatcher != null)
            dispatcher.forward(req1, res1);
        }
    }
```

In this code, the `Dispatcher` class requests for `error_page.jsp` file and wait for the response. If the value of `dispatcher` is not null, then the file is forwarded as response to the request received from the dispatcher class.

4.10 File Upload

When data is sent as form data or as multipart, files can be uploaded. This is provided by servlet container using following methods of `HttpServletRequest`:

- o `public Collection<Part> getParts()`
- o `public Part getPart(String name)`

`jakarta.servlet.http.Part` interface provides various methods to get size, header names, or content-type of an uploaded file. Developers can get the properties, such as `maxFileSize` of an uploaded file using annotation `@MultipartConfig`.

Code Snippets 20 and 21 show an example of using annotation with `getPart()` to upload a picture of maximum 10 MB in size. Ensure appropriate `web.xml` is present for the application.

Code Snippet 20: `index.jsp`

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <title>File Upload</title>
        <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
    </head>
    <body>
        <form method="POST" action="FileUploadServlet"
            enctype="multipart/form-data" >
            File to Upload: <br />
            <input type="file" name="file" id="file" /><br /><br />
            <input type="Submit" value="Upload" name="upload"
                id="upload" />
        </form>
    </body>
</html>
```

Code Snippet 21: `FileUploadServlet.java`

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.MultipartConfig;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
```

```

import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.Part;
@SuppressWarnings("serial")
@MultipartConfig( maxFileSize = 1024 * 1024 * 10)
public class FileUploadServlet extends HttpServlet {
    @Override
    public void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException{
        response.setContentType("text/html;charset=UTF-8");
        try{
            final Part filePart = request.getPart("file");
            String fileName = getFileName(filePart);
            fileName =fileName.substring(3);
            for (Part part : request.getParts()) {
                part.write("D:\\Images\\" + fileName);
            }
            response.getWriter().print("You have sucessfully
uploaded the file.");
        }catch(Exception f) {
            response.getWriter().print("You have exceeded file
size limit.");
        }
    }
    private String getFileName(final Part part) {
        for (String content : part.getHeader("content-
            disposition").split(";")) {
            if (content.trim().startsWith("filename")) {
                return content.substring(
                    content.indexOf('=') + 1
                ).trim().replace(
                    "\\\"", "\"");
            }
        }
        return null;
    }
}

```

The code will accept file name of an existing file from the user and upload it to a given path. However, if the file size exceeds 10 MB, a customized error would be displayed on the browser window. Refer to Figures 4.11 and 4.12.

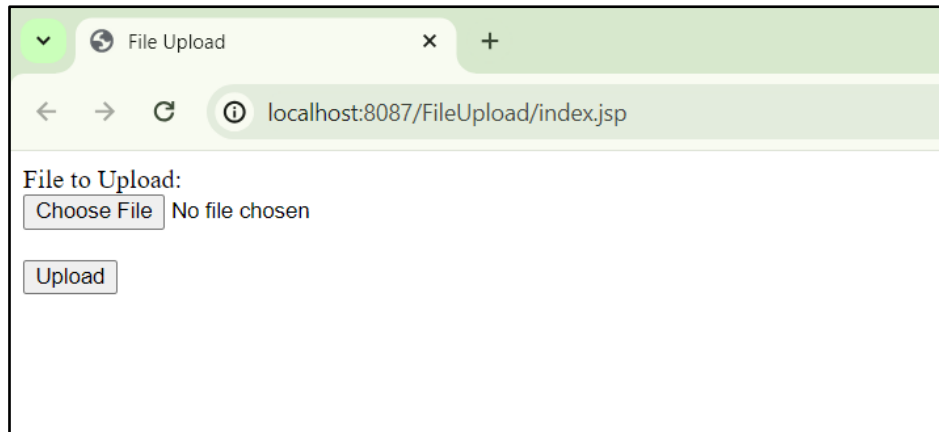


Figure 4.11: Output of File Upload

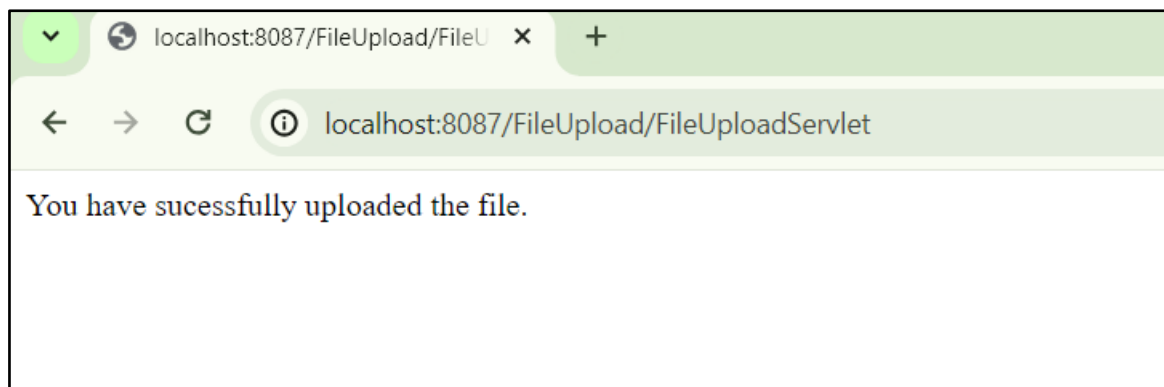


Figure 4.12: Output of File Upload with Success Message

Summary

- A servlet is a Web component based on Jakarta EE technology that is managed by a container and is responsible for generating dynamic content.
- The `init()`, `service()`, and `destroy()` methods are a servlet's lifecycle methods.
- A `GenericServlet` class defines a servlet that is not protocol dependent. To have better control over required servlets, `HttpServlet` is extended to `GenericServlet`.
- A servlet request contains the data to be passed between a client and the servlet. All requests implement the `ServletRequest` interface, which defines the methods for accessing the relevant information.
- A servlet response contains data to be passed between a server and the client. All responses implement the `ServletResponse` interface. This interface defines various methods to process response.
- You can use `getInputStream()` to retrieve the input stream as a `ServletInputStream` object. `ServletOutputStream` provides an output stream for sending binary data to the client.
- Resources are included to a servlet by forwarding request from one servlet to another by using the `forward` and `include` methods along with `RequestDispatcher` interface. Inter-servlet communication can be used by the servlets to gain access to other currently loaded servlets and perform some tasks on other servlets.
- Errors in Web applications with servlets are reported using `sendError` and `setStatus` methods. Errors are logged using the `log` method of `ServletContext` and forwarded to an error page using the `RequestDispatcher` interface.
- A `RequestDispatcher` object is used to redirect the client request to a Web page on receiving an error message on the server.

Check Your Progress

1. Identify the methods present in lifecycle of a servlet.

(A)	service()
(B)	destroy()
(C)	authorize()
(D)	init()

A.	B, C, D	C.	C, D
B.	A, B, D	D.	A, D

2. Which of the following request-header fields allow the client to pass additional information about the request?

(A)	Accept-Numeric
(B)	Accept-Encoding
(C)	Accept-Language
(D)	Accept-Charset
(E)	Accept

A.	A, B, C	C.	B, C, D, E
B.	B, C	D.	C, D, E

3. Match the methods of the `ServletResponse` interface with their corresponding description.

	Description		Method
(A)	Returns an object of <code>PrintWriter</code> class that sends character text to the client.	(1)	<code>setContentType()</code>
(B)	Uses <code>ServletOutputStream</code> object to write response as binary data to the client.	(2)	<code>getContentType()</code>
(C)	Returns the Multipurpose Internet Mail Extensions (MIME) type of the request.	(3)	<code>getWriter()</code>
(D)	Used to set the format in which the data is sent to the client.	(4)	<code>getOutputStream()</code>

A.	A-2, B-3, C-4, D-1	C.	A-4, B-1, C-2, D-3
B.	A-3, B-4, C-2, D-1	D.	A-1, B-2, C-3, D-4

4. Match the methods used to send text and binary data to a client with their corresponding description.

	Description		Methods
(A)	Writes a boolean value to the client	(1)	<code>getWriter()</code>
(B)	Writes a character value to the client	(2)	<code>getOutputStream()</code>
(C)	Method used to send binary data	(3)	<code>print(boolean b)</code>
(D)	Method used for textual output only	(4)	<code>println(char c)</code>

A.	A-2, B-4, C-1, D-3	C.	A-3, B-4, C-2, D-1
B.	A-4, B-1, C-3, D-2	D.	A-1, B-3, C-2, D-4

5. Which of the following methods is used for redirecting requests?

A.	<code>getHeader()</code>	C.	<code>sendRedirect()</code>
B.	<code>redirectRequest()</code>	D.	<code>RequestDispatcher()</code>

Answers

1.	B
2.	C
3.	B
4.	C
5.	C

Try It Yourself

1. Develop a Web application to display a list of users and their date of birth in the format (YYYY:MM:DD). Table 4.4 shows sample data for this example:

Name	Date of Birth
Adams	1990:08:15
Benjamin	1985:05:20
Isabella Lopez	1960:10:30
Lucian	2013:01:08
Sophia	1967:01:15

Table 4.4: Sample Data

Session Tracking

Welcome to the Session, **Session Tracking**.

This session explains the stateless nature of HTTP and HTTP/3 protocol and the necessity to track a session. It explains URL rewriting methods. It also explains the use of the `Cookie` class and its methods and describes how to use hidden form fields. It elaborates on storing and retrieving information in a user session. The session further details how to invalidate a user session using Eclipse and also, describes JSON Binding and JSON serialization.

In this session, you will learn to:

- Describe the stateless nature of HTTP and HTTP/3 protocols
- Explain the necessity of tracking client identity and state
- Explain the URL rewriting method for session tracking
- Explain how to use hidden form fields
- Explain the use of the `Cookie` class and its methods
- Explain how to store and retrieve information in a session
- Describe the use of the `HttpSession` interface and its methods
- Explain how to invalidate a session
- Explain invalidating a session with Eclipse
- Describe Java API with JSON binding
- Explain JSON serialization

5.1 Introduction

A protocol is a set of rules, which govern the syntax, semantics, and synchronization of communication.

When a protocol does not track the configuration setting, transaction, and information, then it is called a stateless protocol. In such a protocol, connections last for only one transaction.

The HTTP protocol is a stateless protocol that is based on a client-server model. An HTTP client, such as a Web browser, opens a connection and sends a request message to an HTTP server asking for a resource. The server returns a response message with the requested resource. Once the resource is delivered, the server closes the connection. Thus, no connection information is stored and hence HTTP is referred to as a stateless protocol.

HTTP3 is also stateless. It follows the same semantics as HTTP to be a generic stateless protocol.

The stateless nature of HTTP protocol has its own advantages and disadvantages.

Advantages	Disadvantages
The advantage of a stateless protocol is that hosts are not required to retain information about users between requests. This simplifies the server design because it does not require to dynamically allocate storage to manage conversations in progress or worry about freeing it when a client connection dies in mid-transaction.	A disadvantage of HTTP being stateless is that it may be necessary to include more information in each request and this extra information must be interpreted by the server each time. Another disadvantage is that the acknowledgement that the information has reached the client is absent. Hence, the loss of data, if any, is not known.

Figure 5.1 depicts the client-server model using HTTP protocol.

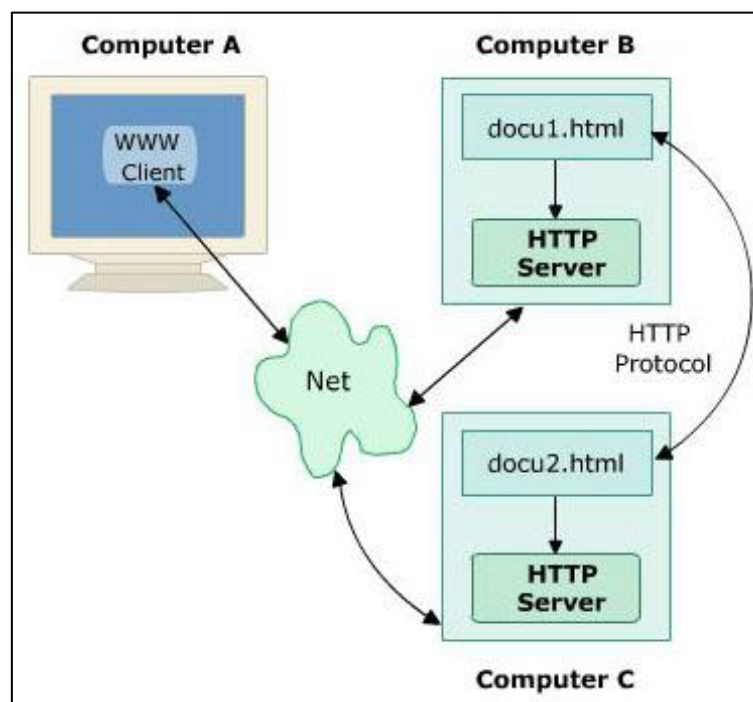


Figure 5.1: Client-Server Model Using HTTP Protocol

5.1.1 Session Tracking

Consider a scenario of an online shopping Website where items are displayed on different pages based on classifications, such as stationery, hardware, music, books, and so on. While shopping online, a customer may select items from various pages and put them in the shopping cart. However, due to HTTP and HTTP/3 being stateless protocols, when the customer clicks a new page, the information about the previously selected items is lost. As a result, the server may not keep track of the selected items on the pages.

To resolve this problem, it is necessary for Web applications to adopt a mechanism that can be used to keep track of successive requests made by the same user. The tracking mechanism allows the Web application to maintain information with the server as long as the customer does not log out from the Website. This tracking mechanism is known as session tracking.

The session tracking mechanism serves the purpose of tracking the identity of the client and other state information required throughout the session. Figure 5.2 depicts session tracking.

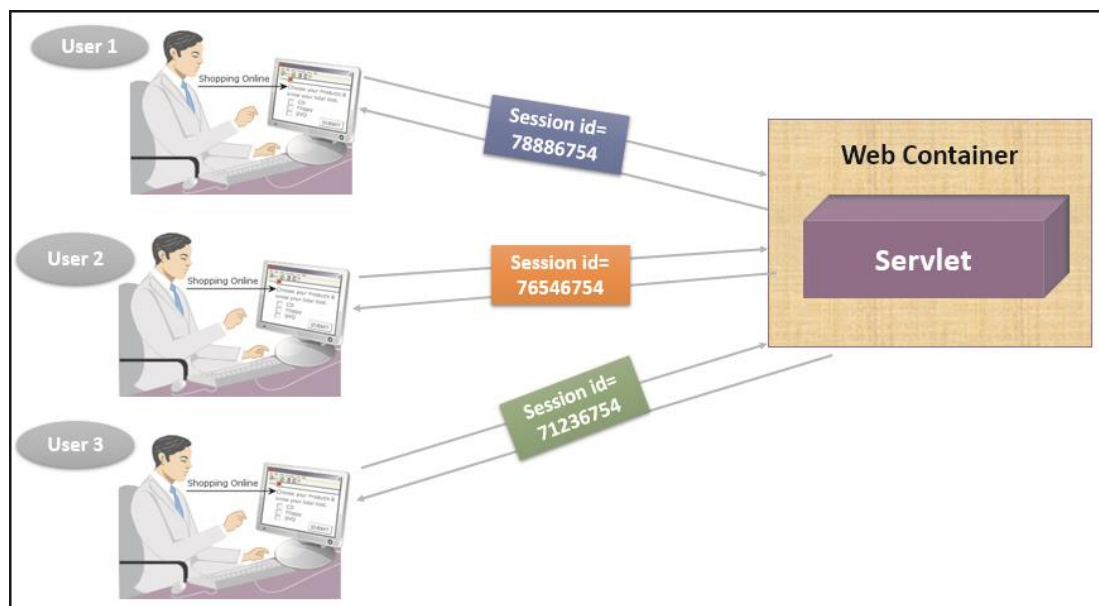


Figure 5.2: Session Tracking

5.1.2 Session Tracking Techniques

There are various techniques that can be used by a Web application to maintain a session that keeps track of data between multiple requests from a client.

Some of these techniques include:

URL rewriting	This technique is used to pass data from one page to another by appending a text string at the end of the request URL.
Hidden field	This technique is used to pass values from HTML form hidden fields to Web resources.
Cookie	This technique is used to store the information on the client machine in a text file.

These are the most common techniques used by Web applications for maintaining client data.

Apart from these, Jakarta Servlet API specification provides a session tracking mechanism through `jakarta.servlet.http.HttpSession` object. Servlet uses the object to store or retrieve information related to the user for maintaining the session on the server.

5.2 URL Rewriting

Uniform Resource Locator (URL) is the address of a resource located on the Web. The URL rewriting technique adds some extra data at the end of the URL to identify the session.

The extra information can be in the form of extra path information or added parameters. The user does not see extra information on the surface as such but when he/she clicks a link, the data from the page is appended after the '?' in the URL. Figure 5.3 shows the URL rewriting technique used for session management between the pages.

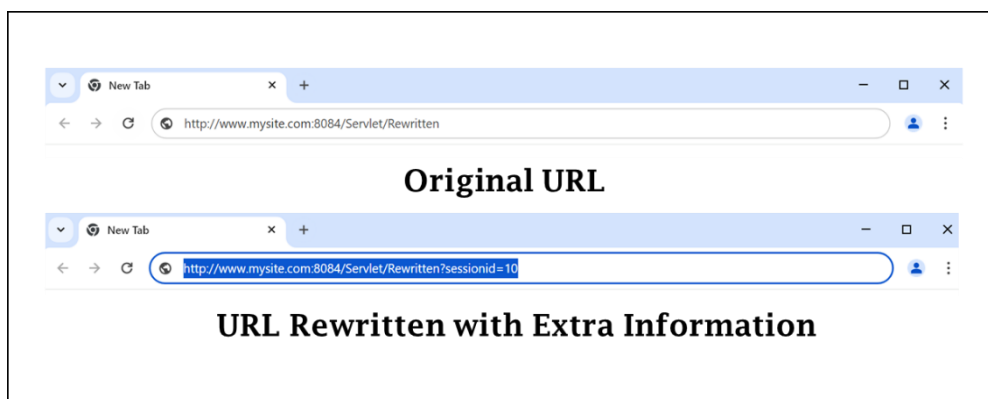


Figure 5.3: URL Rewriting

As shown in Figure 5.3, the original URL, `http://www.mysite.com:8084/Servlet/Rewritten` is appended with the

parameter `sessionId=10` at the end of the URL. The `sessionId` parameter is sent to the server as part of the client request and helps the server to identify the client.

The URL rewriting is the lowest priority technique used for session management and is used as an alternative for cookies. In case, the setting to receive cookies at the client side is disabled in the Web browser, URL rewriting is used.

5.2.1 Information in URL

A parameter or token is attached at the end of the query string in the request sent from the Web browser. The token consists of a name-value pair.

Figure 5.4 shows the token in the URL.



Figure 5.4: URL Rewriting Technique

Code Snippets 1 and 2 demonstrate the working of URL rewriting technique using two Servlets respectively.

Code Snippet 1: Servlet1.java

```
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.PrintWriter;
import java.io.IOException;

public class Servlet1 extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws
        ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        String sessionId = "Session1";
        PrintWriter pw = response.getWriter();
        pw.println("<html>");
        pw.println("<head></head>");
        pw.println("<body>");
        pw.println("Please click this link:<br>");
    }
}
```

```

        pw.println("<a href = /MyWebApp/Servlet2?sessionID=" +
sessionID + ">View Report</a><br>");
        pw.println("</body>");
        pw.println("</html>");
    }
}

```

Code Snippet 2: Servlet2.java

```

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.PrintWriter;
import java.io.IOException;
public class Servlet2 extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws
        ServletException, IOException {
        response.setContentType("text/html;charset = UTF-8");
        String sessionID =
            request.getParameter("sessionID");
        PrintWriter pw = response.getWriter();
        pw.println("<html>");
        pw.println("<head></head>");
        pw.println("<body>");
        pw.println("This is the Session ID of the last page:
<br>");
        pw.println("SessionID=" + sessionID + "<br>");
        pw.println("</body>");
        pw.println("</html>");
    }
}

```

In these codes, **Servlet1** generates a hyperlink element which, when clicked by the user, will send the request to **Servlet2**. A name-value parameter **sessionID=Session1** is appended with the URL link specified with the **<a>** element. **Servlet2** accesses the parameter using the **getParameter** method and displays its value on the page.

The **web.xml** for the project must contain code as shown in Code Snippet 3.

Code Snippet 3: web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee

```

```

https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
    <display-name>MyWebApp</display-name>
    <servlet>
<servlet-name>Servlet1</servlet-name>
<servlet-class>Servlet1</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>Servlet1</servlet-name>
<url-pattern>/Servlet1</url-pattern>
</servlet-mapping>
<servlet>
<servlet-name>Servlet2</servlet-name>
<servlet-class>Servlet2</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>Servlet2</servlet-name>
<url-pattern>/Servlet2</url-pattern>
</servlet-mapping>
</web-app>

```

Figure 5.5 shows the output of the project.

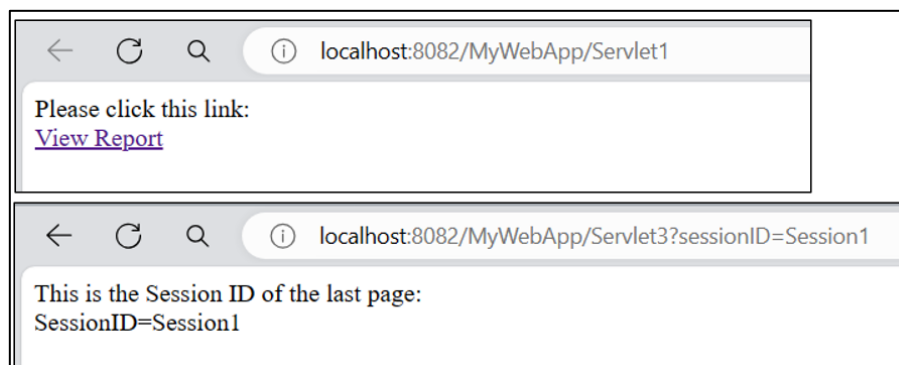


Figure 5.5: Output – URL Rewriting

5.2.2 Advantages and Disadvantages of URL Rewriting

Given are the advantages and disadvantages of URL rewriting:

- URL can be appended when sending the data from the form and it can be sent along with the dynamic content generated from a Servlet.
- URL is a preferred way to maintain the session when the browser does not support cookies or user disables the support for cookies.

Advantages

- The only way to send the URL is through hyperlinks on the Web page.
- Sometimes, the URLs are very long and cannot store that much information because of the URLs length limitation.
- URLs contain data information that is visible, therefore, it is not safe to be shared with others.

Disadvantages

5.3 Hidden Form Fields

Hidden form fields is another technique of session tracking. It is similar to URL rewriting, instead of appending values to the URL, hidden fields are placed within an HTML form. Hidden form fields are either a part of the static HTML form or a dynamic form generated through servlets. They can be used to hold any kind of data.

The hidden fields are not visible to the user and hence, are not interpreted by the browser. Each time the servlet receives the form data, it reads all the parameters passed to it from the form along with the values in the hidden fields received from the HTML page.

The advantage of this technique over URL rewriting is that one can pass much more information to the server and even character encoding is not necessary.

Syntax:

```
<INPUT TYPE="HIDDEN" NAME="..." VALUE="...">
```

Code Snippets 4 and 5 demonstrate the use of hidden field. Here, *job* is the hidden field name and *Developer* is the hidden field value.

Code Snippet 4: index.html

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8">
    <title>Insert title here</title>
</head>

<body>
    <form action="MyServlet" method="POST">
        <input type="hidden" name="job" value="Developer" />
Name:
        <input type="text" name="firstname" />
        <input type="submit" name="Submit" />
    </form>
</body>

</html>
```

Code Snippet 5: MyServlet.java

```
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```

import java.io.PrintWriter;
import java.io.IOException;

public class MyServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try {
            // Retrieves the name entered on the form
            String name =
request.getParameter("firstname");
            // Retrieves the value from the hidden field
            String job = request.getParameter("job");
            out.println("<h3> Welcome: " + name + "<br>");
            out.println("\n Your Job: " + job + "</h3>");
            out.close();
        } finally {
            out.close();
        }
    }
}

```

Figure 5.6 shows the output of Code Snippets 4 and 5.



Figure 5.6: Output - Hidden Form Field

5.3.1 Advantages and Disadvantages of Hidden Form Fields

Advantages of the hidden form field technique are as follows:

- Hidden form fields are supported in all browsers.
- Hidden form fields have no special server requirements from clients.
- Hidden form fields are not visible directly to the user but can be viewed using the **View page source** option from the browsers.
- Hidden form fields work with or without cookies, even if the cookies are disabled, they will not have any impact.

One of the disadvantages of hidden form fields is that they work only when the page receives a request through the submission of a form. This means if the target

page receives a request by clicking a hyperlink, it will not result in form submission.

5.4 Cookies

A cookie is a small piece of information sent by a server to the client's Web browser. The cookies are stored on the client machine and are read back by the server on receiving a request for the same page.

A cookie contains one or more name-value pairs, which are exchanged in request and response headers. The HTTP request header contains the request made by the client. It contains information such as method, URL path, and HTTP protocol version.

The HTTP response header contains the date, size, and type of the file that the server is sending back to the client. When the server sends a cookie, the client receives the cookie, saves it, and sends it back to the server each time the client accesses a page on that server.

Cookies are stored for a limited life span on the client machine and once the specified period is completed, they are automatically deleted.

The major drawback with cookies is that most browsers allow users to deactivate them (not accept them).

As the value of the cookie can uniquely identify a client, cookies are commonly used in session tracking.

Figure 5.7 depicts the concept of cookies.



Figure 5.7: Concept of Cookies

5.4.1 Cookie API

The servlet API provides `jakarta.servlet.http.Cookie` class for creating and working with cookies. These cookies are small amount of information sent by a servlet to the Web browser. The value of cookie is unique and can be used to identify a client thereby helping in session management.

Code Snippet 6 demonstrates how to create and add a cookie in the Servlet response.

Code Snippet 6:

```
//This snippet remembers an added item by adding to a cookie
public void doGet (HttpServletRequest request,
HttpServletRequest response)
    throws ServletException, IOException {
    //If the user wants to add an item in a cookie
    if (values != null) {
        ItemId = values[0];
        Cookie getItem = new Cookie("Buy", ItemId);
        getItem.setComment("User has indicated a desire "
+ "to buy this book from the bookstore.");
        response.addCookie(getItem);
    }
}
```

The `Cookie` class provides several methods, which help in cookie management.

These are as follows:

➤ **`public int getMaxAge()`**

This method returns an integer, which specifies the maximum age of the cookies in seconds.

Code Snippet 7 demonstrates how to get the cookie age.

Code Snippet 7:

```
/* This snippet checks whether the maximum age of the
cookie is greater than or equal to zero by retrieving the
max age of cookie. */
if (cookie.getMaxAge() >= 0) {
    if (version == 0) {
        buf.append(";Expires=");
        if (cookie.getMaxAge() == 0)
            DateTool.oldCookieFormat.format(new Date(10000),
            buf, new FieldPosition(0));
        . . .
    }
}
```

➤ **`public java.lang.String getValue()`**

This method returns the value of the cookie as a string.

- **public java.lang.String getPath()**
This method returns the path on the server to which the client returns the cookie. The cookie is available to all sub paths on the server. For example, this method returns a string specifying a path that contains a servlet name, /AptechDemo.
- **public java.lang.String getName()**
This method returns the name of the cookie. Once the cookie has been created its name cannot be changed. Code Snippet 8 demonstrates an example.

Code Snippet 8:

```
// Retrieves the name of the cookie and its value
for (int i = 0; i<cookies.length; i++) {
    String name = cookies[i].getName();
    String value = cookies[i].getValue();
    out.println("name = " + name +"; value = " + value);
}
```

- **public void setMaxAge(int expiry)**
This method sets the maximum age of the cookie in seconds. If the value is positive, then the cookie will expire after that many seconds which is specified by the expiry. For example, `demoCookie.setMaxAge(600);`.
- **public void setValue(java.lang.String newValue)**
This method assigns a new value to a cookie after the cookie is created. In case if binary value is used, base 64 can be used for encoding.

Code Snippet 9 demonstrates how to set the value of the cookie.

Code Snippet 9:

```
// Sets the value of the cookie
public void setValue(String value) {
    if (value == null || (value.equals(""))))
        throw new IllegalArgumentException("Invalid cookie
            value set in: " + getClass().getName());
    if (cookie != null)
        cookie.setValue(value);
}
```

- **public void setPath(String uri)**

This method sets the path for the cookie. The cookie is available to all pages specified in the directory and its subdirectories. Path of a cookie must include the servlet that sets the cookie.

Code Snippet 10 demonstrates how to set the path for the cookie.

Code Snippet 10:

```
// Sets the path for the cookie
cookie = new Cookie("sessionId", "erased");
cookie.setPath("/servlet/SessionCookie");
resp.setHeader("Set-Cookie", cookie.toString());
```

- **public Cookie[] getCookies()**
 Cookies can be read from a request by using the `HttpServletRequest.getCookies` method. The `getCookies` method returns an array containing all of the `Cookie` objects the client sends with the request.

Code Snippet 11 demonstrates how to read cookies received in the client request.

Code Snippet 11:

```
// Retrieves cookies from the request object
Cookie[] cookies = request.getCookies();
// Iterates through the array
for(int i=0; i<cookies.length; i++) {
    Cookie cookie = cookies[i];
    // Print cookie details
    out.println("Cookie Name: " + cookie.getName());
    out.println("Cookie Value: " + cookie.getValue());
}
```

- **void addCookie (Cookie cookie)**
 Cookies are sent by the servlet to the browser by using the `HttpServletResponse.addCookie(jakarta.servlet.http.Cookie)` method. This method adds a field to the HTTP response headers to send cookies to the browser, one at a time, and adds a specified cookie to the response. This method can be called multiple times to set more than one cookie. For example, `response.addCookie(new Cookie("cookieName", "cookieValue"));`

5.4.2 Securing Cookies

To secure the cookies from hackers on the Web, you can configure cookies with two security settings namely, `secure` and `HttpOnly`.

The `secure` flag informs the Web browser that cookies should be sent only on the SSL connection. This means any page of the Web application that is not secured will not be able to access cookies.

JavaScript can also be used to access the cookies from the machine. This means the cookies are available across scripts and may be accessed by hackers for manipulation.

Servlet 3.0 onwards provides an `HttpOnly` flag that informs the browser that the content of the cookie is not accessible within JavaScript. It is included in the HTTP response header and prevents cookies from certain kinds of attacks.

The security methods of the cookie class are as follows:

`public void
setSecure()`

This method informs the browser to send the cookie only through secured protocol, such as HTTPS.

`public void
setHttpOnly(boolean
lean)`

This method can be used to mark or unmark the cookie. If set as true, then cookie is marked as `HttpOnly` and are not exposed to client-side scripting code.

`public boolean
isHttpOnly()`

This method is used to check whether the cookie has been marked as `HttpOnly`.

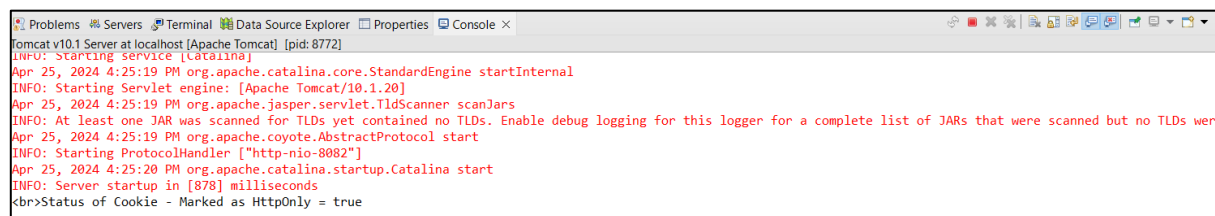
Code Snippet 12 shows how to set the cookies as `HttpOnly` and secure flag programmatically in Servlet.

Code Snippet 12:

```
protected void doGet(HttpServletRequest req,
    HttpServletResponse res)
    throws ServletException, IOException {
    . . .
    Cookie cookie = new Cookie("Color", "Cyan");
    Response.addCookie(cookie);
    cookie.setHttpOnly(true);
    cookie.setSecure(true);
    . . .
    boolean status = cookie.isHttpOnly();
    out.println("<br>Status of Cookie - Marked as HttpOnly = " +
    status);
}
```

Alternatively, you can provide the declaration settings in the `web.xml` configuration file.

Figure 5.8 shows the output of Code Snippet 12.



```
Problems Servers Terminal Data Source Explorer Properties Console x
Tomcat v10.1 Server at localhost [Apache Tomcat] [pid: 8772]
INFO: Starting service [Catalina]
Apr 25, 2024 4:25:19 PM org.apache.catalina.core.StandardEngine startInternal
INFO: Starting Servlet engine: [Apache Tomcat/10.1.20]
Apr 25, 2024 4:25:19 PM org.apache.jasper.servlet.TldScanner scanJars
INFO: At least one JAR was scanned for TLDs yet contained no TLDs. Enable debug logging for this logger for a complete list of JARs that were scanned but no TLDs were
Apr 25, 2024 4:25:19 PM org.apache.coyote.AbstractProtocol start
INFO: Starting ProtocolHandler ["http-nio-8082"]
Apr 25, 2024 4:25:20 PM org.apache.catalina.startup.Catalina start
INFO: Server startup in [878] milliseconds
<br>Status of Cookie - Marked as HttpOnly = true
```

Figure 5.8: Output - Securing Cookies

Code Snippet 13 shows the `web.xml` file with security setting for a cookie.

Code Snippet 13:

```
. . .
<session-config>
  <cookie-config>
    <http-only>true</http-only>
    <secure>true</secure>
  </cookie-config>
</session-config>
. . .
```

5.5 HttpSession

`HttpSession` helps in identifying a user in a multi-page request scenario and information about that user. The `HttpSession` interface is used to create a session between the client and server. The session is created between an HTTP client and an HTTP server by the servlet container using this interface.

When users make a request, the server assigns it a session object and a unique session ID. The session ID matches the user with the session object in subsequent requests. The session ID and the session object are then, passed along with the request to the server.

Methods of the `HttpSession` interface used to create a session include:

```
public Object
getAttribute(String
name)
```

This method returns the object that is bound with the specified name in the session. It returns null in case there is no object bound under the name.

```
public String getId()
```

This method returns a string containing the unique identifier assigned to this session. The servlet container assigns the identifier, and it is implementation independent.

```
public int
getMaxInactiveInterval(
)
```

This method returns the maximum time interval, in seconds, for which the servlet container will keep the session alive between the client accesses. Once, this interval is over session is invalidated by the servlet.

```
public ServletContext get
ServletContext()
```

This method returns the `ServletContext` object to which the current session belongs.

```
public void
invalidate()
```

This method invalidates the session and the objects bound to the session are unbounded. This method throws `IllegalStateException` if called on an already invalidated session.

```
public boolean isNew()
```

This method returns true if the client is unaware about the session or chooses not to be part of the session. For example, if only cookie-based session is used and the client had disabled the use of cookie, then each request would be considered as a session.

```
public void
setAttribute(String
name, Object value)
```

This method binds the object to the current session by using the specified name. In case of repetition of name of the object bound to the session, the object is replaced. This method throws `IllegalStateException` in case of being called on an invalidated session.

```
public void
removeValue(String
name)
```

This method removes the object bound with the specified name from the session. In case of absence of object bound to specified name, this method does nothing. This method throws `IllegalStateException` in case of being called on an invalidated session.

```
public void
setMaxInactiveInterval(
int interval)
```

This method specifies the time, in seconds, between the client requests before the servlet container invalidates the current session.

5.5.1 Storing Information in a Session

The data can be stored in an `HttpSession` object using the name-value pairs. The data which is stored is available throughout the current session. To store the data in a session, the method `setAttribute` is used.

The `setAttribute` method sets the value of the attribute which can be retrieved later.

Code Snippet 14 demonstrates how to create a new session object and set object in it.

Code Snippet 14:

```
public void doGet( HttpServletRequest request,
HttpServletResponse ) throws IOException, ServletException
{
    HttpSession httpSession = request.getSession(true);
    // Gets current session or create a new one if not exist
    HttpSession httpSession = request.getSession(true);
    // Set name and age attributes in the session (if
required)
    if (httpSession.isNew()) {
        httpSession.setMaxInactiveInterval(60);
        httpSession.setAttribute("name", "Jenny");
        httpSession.setAttribute("age", 20);
    }
}
```

In the code, `getSession()` returns the current session object associated with the request. If the session does not exist, then the boolean value `true` indicates to create a new session.

`isNew()` returns a boolean value indicating whether it is a new session or not. If it returns `true`, then the objects are bound to the new session through `setAttributes()`.

`setMaxInactiveInternal()` specifies the time between the requests from the client before servlet container invalidates the session.

5.5.2 Retrieving Information Stored in Session

The server, using the `HttpSession`, does session tracking. The previously stored value can be retrieved using the `getAttribute` method. Since, the return type is an object, typecasting of data associated with that attribute name in the session is done.

If the return value is `null`, it can be concluded that no such attribute exists. Therefore, it is required to check for `null` before calling the methods on the object.

Code Snippet 15 explains the procedure for storing name and age into a session and retrieving name and age from the session.

Code Snippet 15:

```
// Retrieve name and age from session (after potential
setting)
    String myText = (String)
httpSession.getAttribute("name");
    int myNumber = ((Integer)
httpSession.getAttribute("age")).intValue();
```

5.6 Invalidate a Session

Session invalidation means session destroying. Therefore, if a session is destroyed, it indicates that the server cannot identify the client who has visited in the previous session.

Sometimes an unauthorized user may hack into the Web application. To prevent the hacker from causing any harm, the `invalidate` method is called.

Calling `HttpSession.invalidate()` simply clears any object that is bound to it and marks it as invalid, so if you try to modify it afterward it will throw exceptions.

Once a session has been invalidated, the SessionID placed in a cookie on the client will be invalid too, and a new one will have to be created when a new session object is created. Hence, the new Session will have a new ID.

Code Snippet 16 demonstrates invalidating the session through Servlet code.

Code Snippet 16:

```
// Returns current session or a new session if it does not exist
HttpSession session = request.getSession (true);
// Checks the session
if (session.isNew() == false) {
// Invalidates the session if it is not a new session
session.invalidate();
// Creates a new session
session = request.getSession (true);
}
```

The code directs the session to invalidate itself if it is not created newly. To invalidate the session manually, the `invalidate` method is being called.

5.6.1 Session Timeout

Session timeout is necessary as a session utilizes the memory locations to store information and long periods of inactivity will occupy the memory unnecessarily.

After a certain period of inactivity, the session is destroyed to prevent the number of sessions from increasing infinitely. The data stored in the session disappears. Session timeout happens if the user remains inactive for a period greater than the set inactive period.

The session timeout period can be set either in the `web.xml` file or can be set by the method `setMaxInactiveInterval`. This method is used to specify the time between the requests from the client before the servlet container invalidates the session.

Syntax:

```
<session-config>
<session-timeout>N</session-timeout>
</session-config>
```

where, N in the fragment is the session timeout period.

The setting for session timeout should be written in the `web.xml` file.

5.6.2 Invalidating a Session on Logout Using Eclipse

To invalidate a session using an IDE such as Eclipse, an implicit object can be utilized to set the attribute of the session and invalidate it.

Note that this can be done using any other IDE as well.

Consider a project with three JSP files created as shown in Figure 5.9.

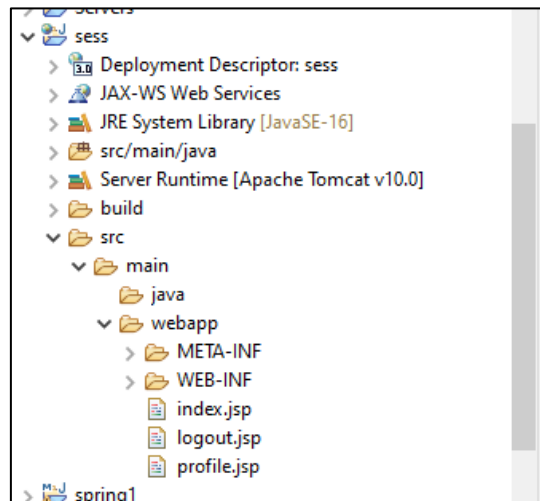


Figure 5.9: Example

Code Snippet 17 displays code for the login page with username and password fields in the file `index.jsp`.

Code Snippet 17:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>

<head>
    <meta charset="ISO-8859-1">
    <title>Login Page</title>
</head>

<body>
    <form method="post" action="profile.jsp">
        User Name
        <input type="text" name="un"> Password
        <input type="password" name="up">
        <input type="submit" value="login">
    </form>
</body>
</html>
```



```

    </form>
</body>

</html>

```

Figure 5.10 displays the output of `index.jsp`.

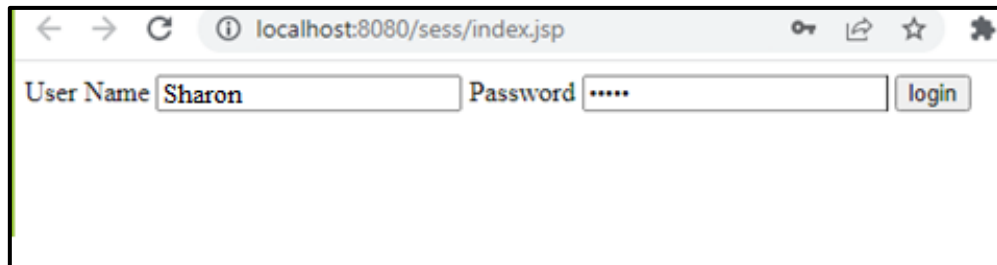


Figure 5.10: Output of `index.jsp`

Code Snippet 18 displays the `profile.jsp` code where the page title is given and `session.setAttribute` is used to bind the user to the session.

Code Snippet 18: `profile.jsp`

```

<%@ page language="java" contentType="text/html; charset=ISO-
8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Welcome</title>
</head>
<body>
<%
session.setAttribute("user", request.getParameter("un"));
%>
welcome <b> <%=request.getParameter("un") %>
<br>
<a href="logout.jsp">Logout</a>
</body>
</html>

```

Figure 5.11 displays the output generated after a user clicks login on `index.jsp` and `profile.jsp` is launched.



Figure 5.11: Outcome of profile.jsp

Next, in the `logout.jsp` file, `session.invalidate` method is used to invalidate a session after a user logs out as shown in Code Snippet 19.

Figure 5.12 depicts the output generated when `logout.jsp` is loaded.

Code Snippet 19: `logout.jsp`

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Logout</title>
</head>
<body>
<%
session.invalidate();
out.println("Session is not active.");
%>
<br>
You are successfully logged out.
</body>
</html>
```

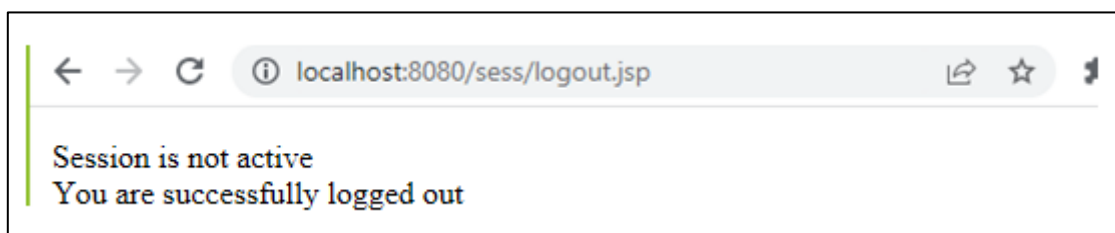


Figure 5.12: Outcome of logout.jsp

Thus, here without creating any Java files, the developer is able to call the `invalidate` method of the implicit session object and perform session invalidation.

5.7 JSON Binding

JavaScript Object Notation (JSON) is a format for data interchange. It is text-based and not dependent on any programming language making it easy to understand. It is based on two data structures, Objects and Arrays. JSON Objects are a collection of name-value pairs. JSON arrays are collections of values in an order.

For example, consider an object `Person` with `name`, `lastName`, and `age` fields. Consider an array of `Person` objects named `Personnel`. These can be constructed in JSON format as shown in Code Snippets 20 and 21.

Code Snippet 20:

```
{
  "name":"Dennis",
  "lastName" : "Ritchie",
  "age" : 25
}
```

Code Snippet 21: Personnel Array

```
{
  " Personnel ":
  [
    { " name ":"name1", "lastName":"lastName1","age":25},
    { " name ":"name2", "lastName":"lastName2","age":28},
    { " name ":"name3", "lastName":"lastName3","age":30}
  ]
}
```

JSON Binding (JSON-B) refers to a binding layer that converts Java objects to JSON messages and vice versa. It provides an algorithm to convert Java classes to JSON objects.

To create JSON object models, `JsonObjectBuilder` is used. It has methods to create a builder object such as `Json.createObjectBuilder().build()`. `JsonArrayBuilder` is used to create `JsonArray` models. `Json.createArrayBuilder().build()` is used to build an instance of `JsonArray`.

Table 5.1 lists the jar files that must be added to the Java Web application project to work with JSON data. These JAR files are integral to Java applications that require JSON processing and binding capabilities, providing standardized ways to handle JSON data efficiently.

JAR Files	Download Link	Description
jakarta.json-api-2.1.3.jar	https://mvnrepository.com/artifact/jakarta.json/jakarta.json-api/2.1.3	Part of the Jakarta JSON Processing API, which provides a Java-based framework for parsing, generating, transforming, and querying JSON documents in a standard and portable way
parsson-1.1.6.jar	https://mvnrepository.com/artifact/org.eclipse.parsson/parsson	An implementation of Jakarta JSON Processing API, specifically a provider for parsing and generating JSON documents within Java applications
jakarta.json.bind-api-3.0.1-javadoc.jar	https://central.sonatype.com/artifact/jakarta.json.bind/jakarta.json.bind-api/versions	Provides JavaDoc documentation for the Jakarta JSON acting as a standard binding layer for converting Java objects to and from JSON documents
json-20240303.jar	https://central.sonatype.com/artifact/jakarta.json.bind/jakarta.json.bind-api/versions	A Java library that implements JSON encoders and decoders and includes the capability to convert between JSON and XML, HTTP headers, Cookies, and Comma Delimited Lists (CDL)

Table 5.1: JSON JAR Files

To use the downloaded .jar files in Java Web applications, add the files to the Build Path of the Eclipse project containing the Servlet. Steps are as follows:

1. Copy **jakarta.json-api-2.1.3.jar**, **parsson-1.1.6.jar**, **jakarta.json.bind-api-3.0.1-javadoc.jar**, and **json-20240303.jar** files from the **Downloads** folder of your system.
2. Go to the required Web application folder. For example, **MyWebApp**. Then, navigate to **src** → **main** > **webapp** → **WEB-INF** → **lib** folder. Paste the jar files.
3. Navigate to **Eclipse**. In the **Project Explorer** workspace, right-click the **MyWebApp** project, select **Build Path**, and then, click **Configure Build Path** as shown in Figure 5.13.

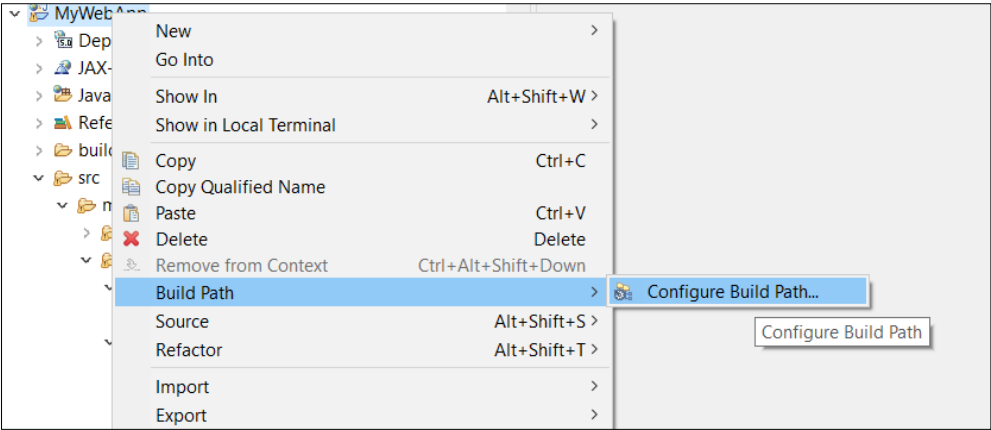


Figure 5.13: Configure Build Path

Properties for MyWebApp opens.

4. On the **Libraries** tab, select **ClassPath** and click **Add External Jars** as shown in Figure 5.14.

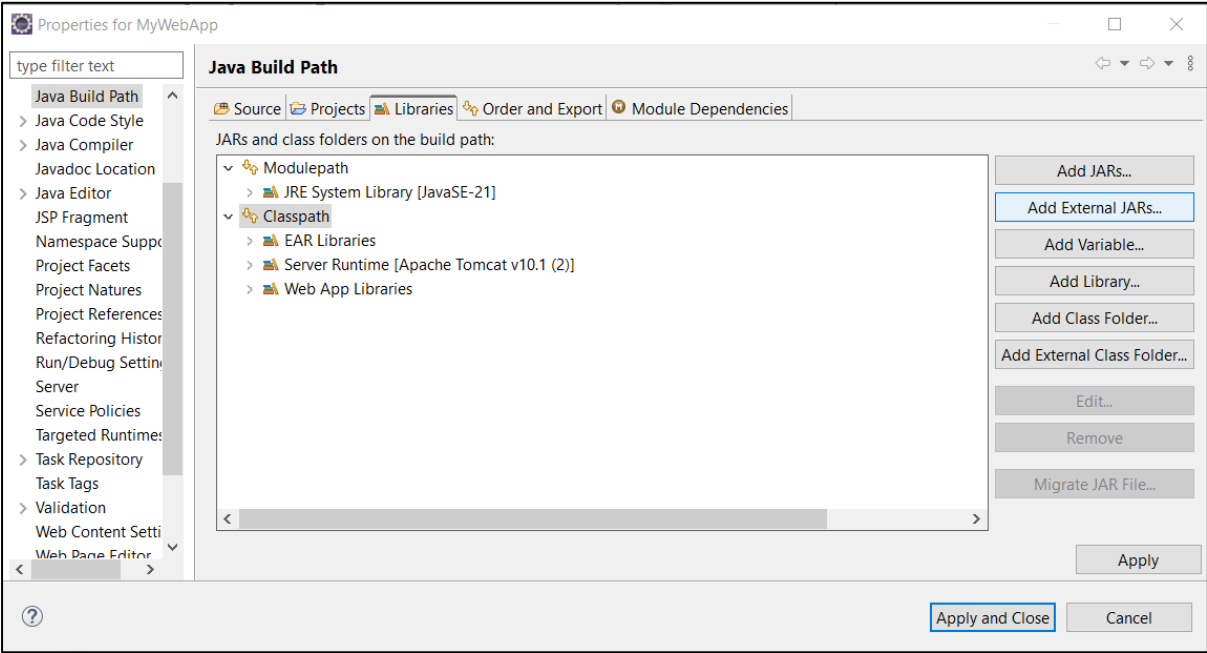


Figure 5.14: Add External JARs

JAR Selection dialog box opens.

5. Select **jakarta.json-api-2.1.3.jar**, **jakart-1.1.6.jar**, **jakarta.json.bind-api-3.0.1-javadoc.jar**, and **json-20240303.jar** files one by one and click **Open** as shown in Figure 5.15.

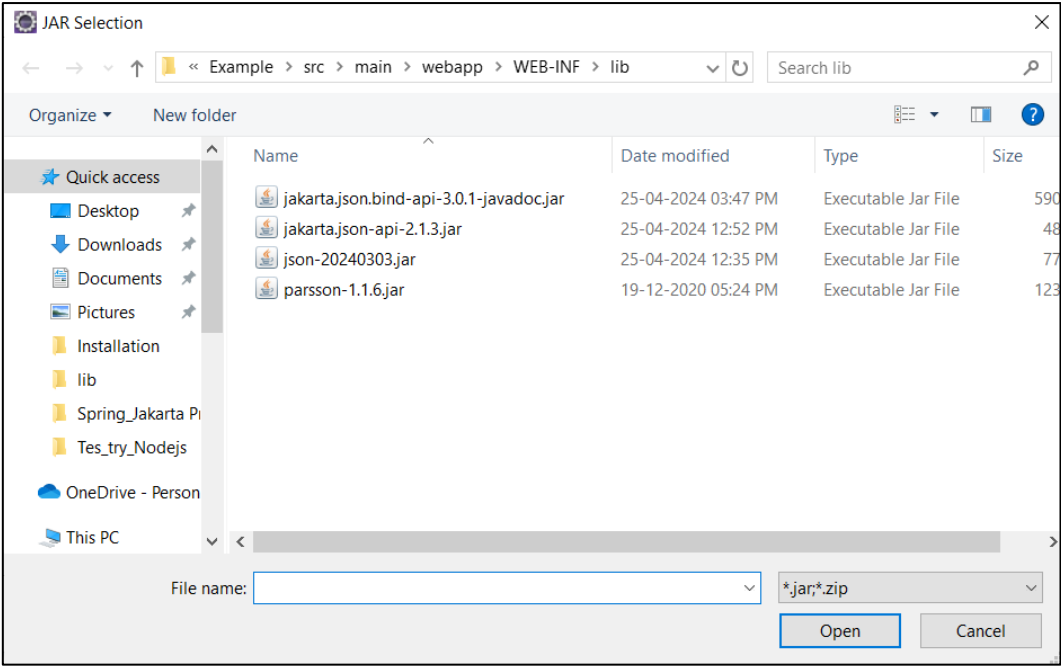


Figure 5.15: JAR Selection

6. Find the .jar files added to **Classpath**. Click **Apply**. Then, click **Apply and Close** as shown in Figure 5.16.

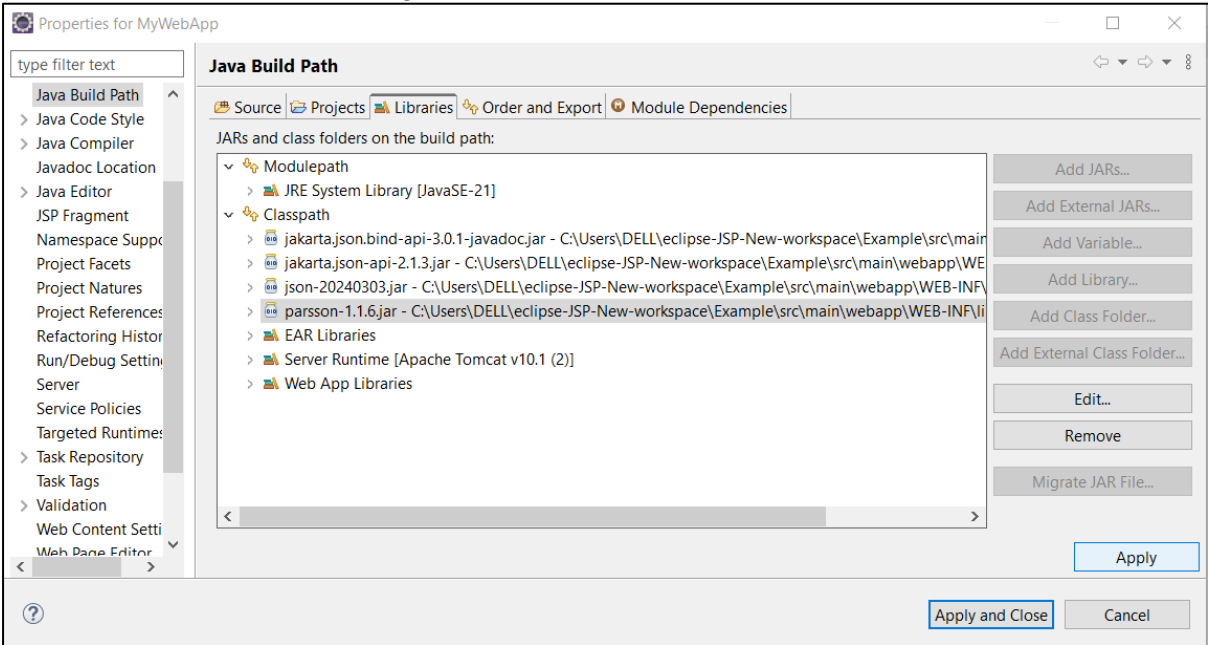


Figure 5.16: Classpath Added

The **lib** folder under **WEB-INF** has files as shown in Figure 5.17.

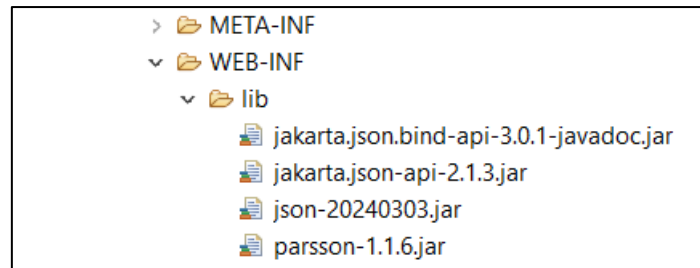


Figure 5.17: Jar Files Under Lib

Code Snippet 22 displays the creation of a JSON object `StudentObject`.

Code Snippet 22:

```
import jakarta.json.*;

public class JsonTest {
    public static void main(String[] args) {
        try {
            JsonObject studentObject =
            Json.createObjectBuilder()
                .add("stu_Name", "Den")
                .add("stu_Age", 25)
                .add("stu_Salary", 50000)
                .add("phoneNumber",

            Json.createArrayBuilder().add("1234567809")
                .add("5432123456").build())
                .build();
            System.out.println(studentObject);
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

The output of the code is shown in Figure 5.18.

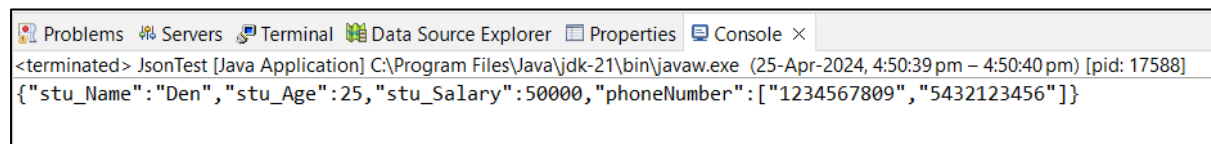


Figure 5.18: Output Showing JSON Data

Similar code can then, be used in a Servlet too. Code Snippet 23 shows the code for such an example.

Code Snippet 23:

```
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.PrintWriter;

import jakarta.json.*;

public class JsonDemoServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
        HttpServletResponse response) {
        try {
            response.setContentType("application/json");
            PrintWriter out = response.getWriter();
            JsonObject studentObject =
                Json.createObjectBuilder().add("stu_Name",
                    "Den").add("stu_Age", "25")
                    .add("stu_Salary", "50000")
                    .add("phoneNumber",
                        Json.createArrayBuilder().add("1234567809").add("5432123456")
                            .build()).build();
            System.out.println(studentObject);
            System.out.println(studentObject);
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

When the project is executed, the browser will generate the JSON data which can be downloaded to the local system.

5.7.1 JSON Serialization

JSON serialization refers to converting objects to JSON string. It encodes objects into a string. JSON deserialization is the inverse of serialization, and it decodes JSON strings back to objects.

Code Snippet 24 shows an example of JSON object being converted to string. Download the latest json.jar file from this path and place it under **lib** folder before creating the code so that there are no errors/issues on account of missing JSONObject.

<https://mvnrepository.com/artifact/org.json/json/20220320>

Code Snippet 24:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1" import="org.json.JSONObject"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Serialization</title>
</head>
<body>
    <%
        JSONObject user = new JSONObject();
        user.put("name", "Dennis");
        user.put("occupation", "Developer");
        user.put("siblings", Integer.valueOf(3));
        user.put("height", Double.valueOf(174.35));
        user.put("married", Boolean.TRUE);
        user.toString();
        System.out.println(user);
        out.println(user);
    %>
</body>
</html>
```

The output of the code is as shown in Figure 5.19.

```
{"siblings":3,"occupation":"Developer","name":"Dennis","married":true,"height":174.35}
```

Figure 5.19: Output of JSON Serialization

Summary

- Session tracking allows the server to keep track of successive requests made by the same client.
- Some of the session tracking techniques are namely, URL rewriting, hidden field, and cookie.
- Java Servlet specification provides a session tracking mechanism through `jakarta.servlet.http.HttpSession` object.
- The URL rewriting technique adds some extra data at the end of the URL to identify the session.
- Hidden form fields are used to pass data to the server-side resource invisibly from the user.
- A cookie is a small piece of information sent by a server to the client's Web browser. The cookies are stored on the client machine and are read back by the server on receiving a request for the same page.
- To secure the cookies from hackers on the Web, you can configure cookies with two security settings namely, `secure` and `HttpOnly`.
- The `HttpSession` interface is used to create a session between the client and server.
- JavaScript Object Notation (JSON) is a format for data interchange.
- JSON serialization refers to converting objects to JSON string. The process encodes objects into strings.

Check Your Progress

1. Which of the following statements describing the stateless nature of HTTP protocol and the necessity of tracking client identity and state are true?

(A)	When the configuration setting, transactions, and information are not tracked by the protocol, then it is called stateless protocol.
(B)	The HTTP protocol is based on a client-server model.
(C)	The HTTP protocol sends an acknowledgement that the information sent by the server has reached the client.
(D)	Session tracking allows the server to keep a track of successive requests made by the same client.
(E)	In HTTP protocol, the loss of data can be easily detected.

A.	B, C, D	C.	A, B, D
B.	A, B, C	D.	A, C, D

2. Which among the following statements relating to URL rewriting is incorrect?

(A)	URL is the address of a resource located on the Web.
(B)	The URL rewriting technique adds some extra data at the end of the URL to identify the session.
(C)	Generally, the extra information appended in the URL rewriting is the unique session ID and tracking can be done by retrieving this session ID.
(D)	Only information after ? is sent to the next page and information before ? is detached.

3. Which among the following methods returns a string containing the present value of the cookie?

(A)	<code>getMaxAge()</code>
(B)	<code>getValue()</code>
(C)	<code>setMaxAge()</code>
(D)	<code>setValue()</code>

4. Which among the following syntax does not belong to methods for reading cookies?

(A)	<code>public java.lang.int getValue()</code>
(B)	<code>public Cookie[] getCookies()</code>
(C)	<code>public java.lang.String getName()</code>
(D)	<code>public java.lang.String getValue()</code>

5. Which among the following methods is correct for sending cookie?

(A)	<code>HttpServletResponse.setCookie(jakarta.servlet.http.Cookie)</code>
(B)	<code>HttpServletResponse.addCookie(jakarta.servlet.http.Cookie)</code>
(C)	<code>HttpServletRequest.addCookie(jakarta.servlet.http.Cookie)</code>
(D)	<code>HttpServletResponse.addCookie(jakarta.servlet.http.addCookie)</code>

6. Which of the following descriptions about the methods of HttpSession interface are incorrect?

(A)	<code>getAttribute()</code> returns null in case there is no object bound under the name.
(B)	In the <code>getId</code> method, servlet container assigns the identifier, and it is implementation dependent.
(C)	The <code>getMaxInactiveInterval</code> method returns the maximum time interval, in seconds, for which the servlet container will keep the session inactive between the client accesses.
(D)	The <code>getServletContext</code> method returns the <code>ServletContext</code> to which the current session belongs.
(E)	The <code>invalidate</code> method invalidates the session and the objects bound to the session are unbounded.
(F)	The <code>isNew</code> method returns true if the client is aware of the session or chooses to be part of the session.
(G)	In the <code>setAttribute</code> method, in case of repetition of name of the object bound to the session, the object is renamed.
(H)	In the <code>removeAttribute</code> method, in case of the absence of an object bound to a specified name, this method throws <code>IllegalException</code> .

A.	B, C, D, E, F	C.	B, C, F, G, H
B.	A, B, C, D, E	D.	A, C, D, E, H

7. Which among the following statements related to retrieving information in a session is incorrect?

(A)	The previously stored value can be retrieved using <code>getAttribute()</code> .
(B)	While retrieving, the return type is a class.
(C)	Type casting of data associated with the attribute name in the session is possible.
(D)	It is required to check for null before calling the methods on the object.

Answers

1.	C
2.	D
3.	B
4.	A
5.	B
6.	C
7.	B

Try It Yourself

1. Create a simple Web application that collects Employee's Username, Password, and Date of Joining as input through an HTML form and processes it using a Java Servlet. The goal is to display a welcome message along with the employee's Username and Date of Joining entered in the form. Perform the given operations.

Username	Password	Date of Joining
Alex	21	10-03-2024

Table 5.2: Employee Details

- a) Create a new HTML file named Empdata.html and add a form that collects the employee's Username, Password, and Date of Joining.
- b) Create a new Java class named MyServlet.java.
- c) Implement the servlet to process form data and generate a response. Execute the code and enter the values as given in Table 5.2.

Jakarta Server Pages

Welcome to the Session, **Jakarta Server Pages**.

This session provides an overview of Jakarta Server Pages and explains its necessity and benefits. It also describes the lifecycle and architecture of JSP pages managed by the JSP container. The session lists different scripting and directive tags available on a JSP page. Furthermore, it explains the changes in JSP 3.1. Finally, it explains the procedure for integrating JSP and dispatching requests from servlet to JSP.

In this session, you will learn to:

- Define Jakarta Server Pages (JSP)
- Outline the differences between servlet and JSP
- Contrast Jakarta Server Pages against Java Server Pages
- Explain the necessity for JSP
- List the benefits of JSP
- Identify the situations where to use JSP and servlets
- Explain JSP architecture and lifecycle
- Explain various elements in JSP
- List and explain the use of directives in JSP
- Describe changes in JSP 3.1
- Explain procedure for integrating Jakarta servlet with JSP
- Describe dispatching request from servlet to JSP

6.1 Introduction

Jakarta Server Pages (JSP) is a collection of technologies. JSP enables the developers to create dynamic Web pages based on HTML, XML, SOAP, or other file types. Formerly, it was called Java Server Pages. JSP can be installed and executed on a compatible Web server with a servlet container such as Apache Tomcat or Jetty.

Consider a situation where a developer has to greet the user on his/her successful registration on the page. To accomplish this task, if the servlet technology is used, the `GreetingServlet` class is created. The `GreetingServlet` processes the request and generates a response to be sent to the client.

Code Snippet 1 shows the `GreetingServlet` class that would have been created years ago, when Java EE was being used.

Code Snippet 1:

```

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

@SuppressWarnings("serial")
@WebServlet("/student")
public class GreetingServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req,
        HttpServletResponse res) throws ServletException,
        IOException {
        // Set response content type
        res.setContentType("text/html");
        // Get PrintWriter to write HTML response
        PrintWriter out = res.getWriter();
        // Write HTML response
        out.println("<!DOCTYPE html>");
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Hello Student</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Hello Student</h1>");
        out.println("</body>");
        out.println("</html>");
    }
}

```

As seen in Code Snippet 1, to generate the output, the `GreetingServlet` code uses Java code that will display the string value in the output. Also, the code contains HTML elements embedded in the same servlet that take care of the presentation of the string on the Web page.

The HTML elements represent the template data displayed on the Web page. However, embedding HTML elements along with the Java code results in the development of rigid applications, which have a tight coupling between the presentation and application layer. Also, any modifications in the servlet result in page re-compilation, which means embedded HTML elements are also re-compiled.

Thus, to segregate the presentation text from the application, Sun Microsystems, which was maintaining Java back in those days (around 1998), laid a new specification referred to as Java Server Pages (JSP). In 2019, Java EE moved to the Eclipse Foundation and was renamed to Jakarta EE. As part of the change, as of today, **JSP when used with Jakarta EE platform, stands for Jakarta Server Pages**. The JSP specification is an extension of the existing servlet API and leverages all the features of servlets.

Figure 6.1 shows the output of Code Snippet 1.

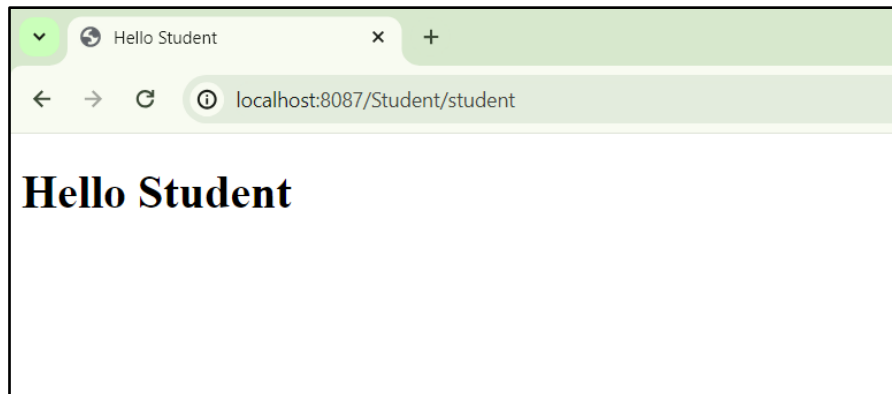


Figure 6.1: Output of GreetingServlet

Important differences between a servlet and JSP are as follows:

1. Modifying a JSP file is quicker than modifying a servlet file, which requires the server to be restarted, the file to be reloaded, and then recompiled.
2. JSP exclusively accepts requests from HTTP-based protocols, whereas servlets can accept requests from any protocols, including HTTP.
3. Servlets can manage massive data processing, whereas JSP cannot.
4. JSP programs take longer to execute than servlets because they have to be compiled and converted into servlet format.
5. JSP separates business logic and presentation logic. In servlets, both are implemented in a single servlet file.
6. JSP does not allow overriding the `service()` method, whereas servlets does.
7. JSP can only present the output, whereas servlets work as a controller in MVC architecture.

6.2 JSP Page

A JSP page is a text document containing static content, JSP tags, and Java code, which simplifies the generation of dynamic content. The static contents are expressed as HTML, XML, or XHTML markup tags and are used to generate the user interfaces on the page. The Java code and JSP elements are used to generate dynamic content on the Web page.

Thus, the segregation of HTML tags and JSP tags separate the presentation code from the application code.

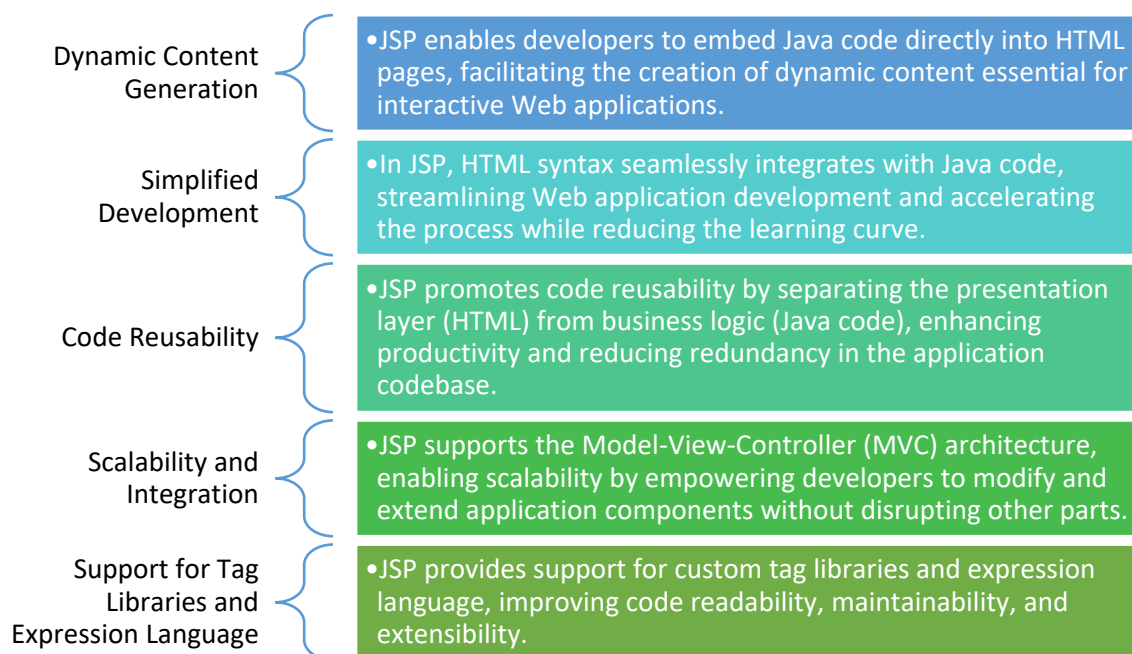
6.2.1 Contrast between Java Server Pages and Jakarta Server Pages

Java Server Pages and Jakarta Server Pages share similarities as tools for generating dynamic Web content, but they also exhibit distinct characteristics. Originally developed by Sun Microsystems, JSP has a legacy status, while Jakarta Server Pages emerged from the Eclipse Foundation with a more collaborative, open-source approach.

Due to its open-source nature and larger development team, Jakarta Server Pages may receive updates more swiftly than its predecessor. Both technologies are utilized for generating dynamic Web content, yet Jakarta JSP is specifically tailored for Jakarta EE technology stack, offering developers enhanced flexibility and efficiency in creating Web pages with dynamic content. Additionally, Jakarta Server Pages enables improved code encapsulation through mechanisms such as the JakartaBeans component architecture and tag libraries, fostering modularization and code reuse for heightened productivity and maintainability.

6.2.2 Necessity for JSP

Jakarta Server Pages is essential for modern Web development, offering a convenient way to generate dynamic Web content using Java. Here is why JSP is essential:



6.2.3 Application Layers

Figure 6.2 shows the application layers in a Web application.

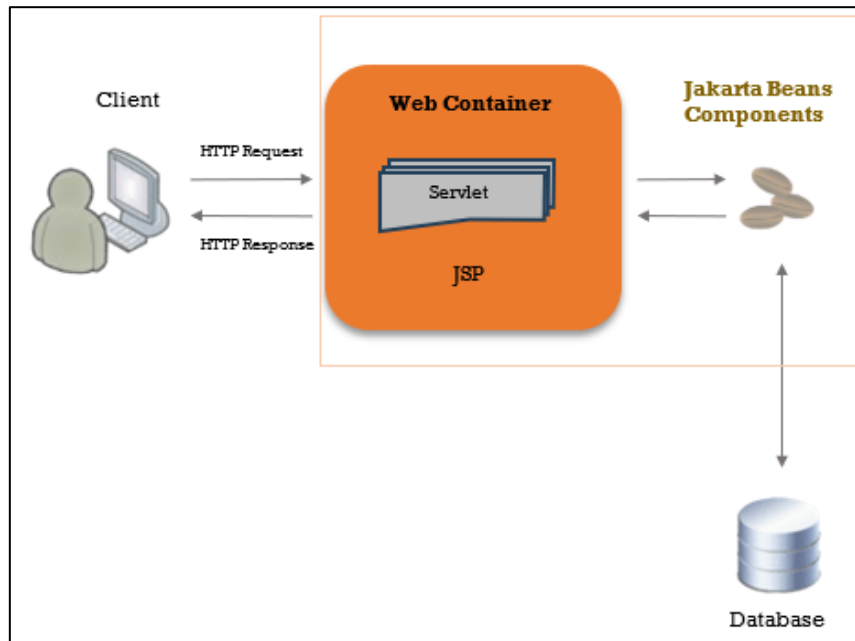
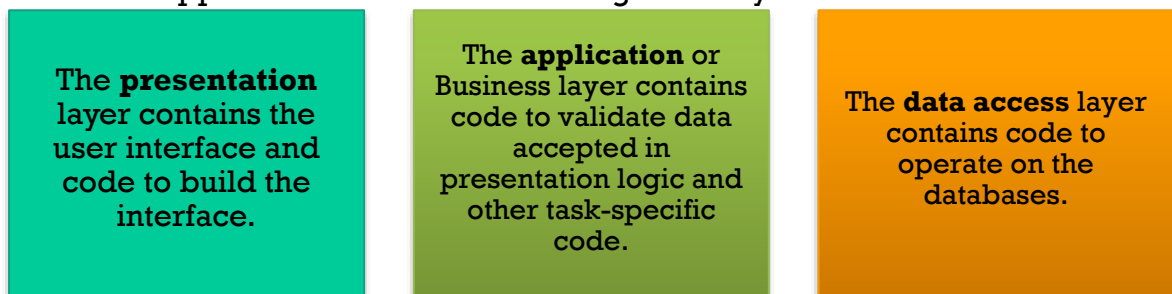


Figure 6.2: Web Application Layers

Most software applications consist of following three layers:



In a three-tier architecture, code related to each layer is separated from each other. JSP is used in presentation layer for presenting dynamic content on the Web page.

6.2.4 Benefits of JSP

JSP helps to access server data efficiently and with greater speed. Some of the advantages of using JSP are as follows:

Platform Independent

JSP is platform-independent, therefore, it helps in creation of dynamic Web pages that are not dependent on any specific platform. They have the benefit of being executed on any Web server. They can also be accessed from any Web browser.

Separation of Content Generation from Presentation	JSP separates content generation from the presentation by implementing presentation logic in a JSP page and content logic on server in a JakartaBean component.
Emphasis on Reusable Components	JSP pages use reusable components, such as JakartaBeans which can be used by multiple programs. JakartaBeans enables to perform complex functions in a JSP page. These beans can be shared and exchanged among the tiers of the Web application.
Simplified Page Development	JSP allows a Web developer with limited knowledge in scripting language to design a page. Programmers can also generate dynamic content using standard tags provided by JSP.
Access and Instantiate JakartaBean Components	JSP supports the use of JakartaBean components with JSP language elements. You can easily create and initialize beans and get and set their values.
Support for N-tier enterprise Application	Jakarta EE being an enterprise Java platform allows the development of complex N-tier applications. Jakarta Server Pages can be used for such applications.

6.2.5 Use of Servlet and JSP

JSP technology simplifies the process of creating pages by separating the Web presentation from Web content. In most applications, the response sent to the client is a combination of template data and dynamically generated data. In this situation, it is much easier to work with JSP pages than to do everything with servlets.

Servlets are well suited for handling binary data dynamically, for example, for uploading files or for creating dynamic images, since they do not have to contain any display logic.

6.3 JSP Architecture and Lifecycle

Every JSP page is translated into a servlet on a Web server before execution. The translated servlet is responsible for handling the request and generating dynamic responses.

6.3.1 Architecture

Figure 6.3 depicts architecture of a Web application with JSP and servlets.

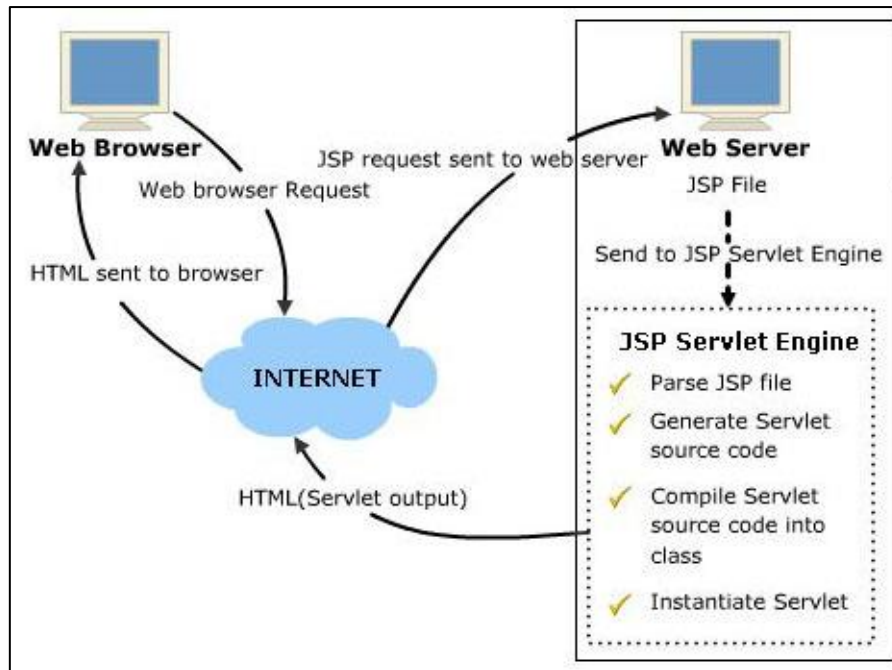


Figure 6.3: JSP Architecture

Whenever the Web browser sends a URL to the Web server requesting a JSP page, the Web engine passes the requested JSP file to the JSP container that parses the JSP file to generate a servlet source code. This servlet source code is compiled into a .class file and is instantiated by the lifecycle methods. The HTML output from the compiled servlet is sent through the Internet and results are displayed on the user's Web browser.

6.3.2 Lifecycle Phases of JSP

During the process of parsing and compiling a JSP page into the servlet, the JSP page is passed through two phases, which are described as follows:

➤ Translation Phase

Translation is the first phase of the JSP lifecycle. In this phase, a servlet code to implement JSP tags is automatically generated, compiled, and loaded into the servlet container. During translation, the JSP engine writes the entire static contents such as markup tags to the response stream in the servlet. Similarly, the JSP elements are converted into the equivalent Java code in the generated servlet code. Figure 6.4 depicts the translation of JSP page into the servlet class.

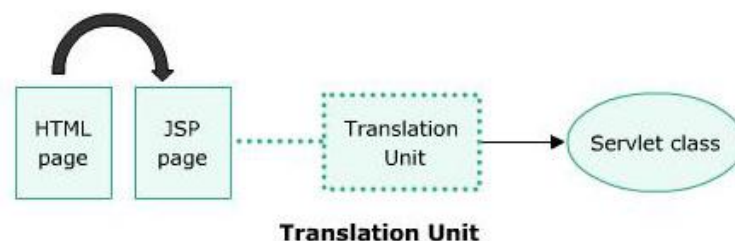


Figure 6.4: Translation Phase

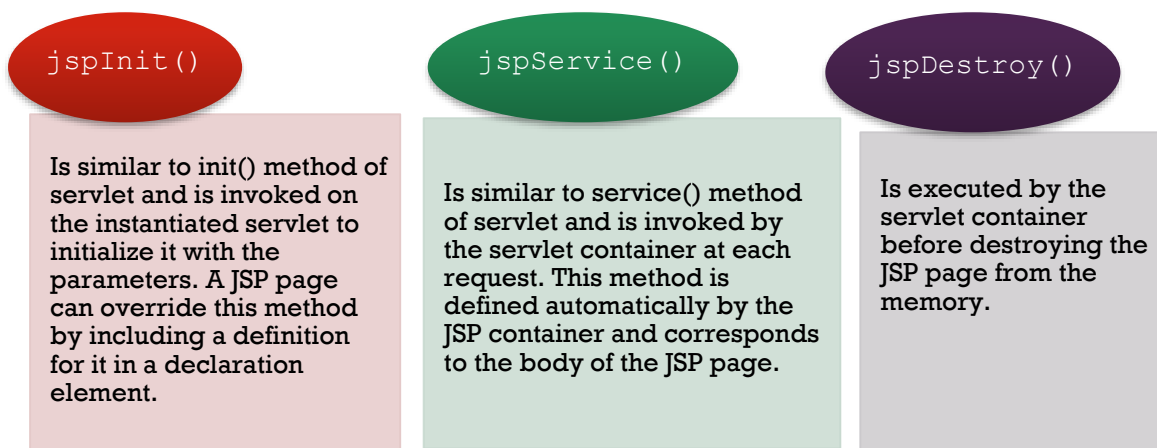
After translation, the servlet source code is compiled to `.class` file that is a compiled servlet class. This generated `.class` file is handed to the servlet container for managing the servlet lifecycle methods.

The process of translation is performed only for the first time invocation of the JSP page. For all subsequent requests, the compiled servlet is loaded into the servlet container for execution.

➤ Execution Phase

The execution of JSP page is similar to the servlet's execution. The generated Servlet class must implement the `jakarta.servlet.Servlet` interface defined in the JSP API. The `jakarta.servlet.Servlet` interface defines the lifecycle methods.

The lifecycle methods are executed by the servlet container on the loaded instance of the converted servlet. These methods are as follows:



6.4 Elements of a JSP Page

A JSP page contains HTML tags, which display static content on a Web page. It also contains JSP tags that are used to generate dynamic content. These tags are categorized into standard tags and custom tags.

The standard tags are used to process the request, include Java objects and directives, and perform actions on the JSP page. These tags include scripting tags, directive tags, and action tags. All the tags are written within angular brackets `<>`.

Custom tags perform operations, such as processing forms, accessing databases, and other enterprise services, such as e-mail and directories.

6.4.1 Scripting Tags

The JSP tags are used to embed Java code snippets in the JSP page for generating dynamic content on the Web page.

These tags are as follows:

➤ Expressions

JSP expressions can be used to display individual variables or the result of some calculation on a Web page.

A JSP expression contains a statement whose value will be evaluated and inserted into the generated Web page. At run time, JSP expression is evaluated, and the result is converted into a string. This evaluated string will be displayed on the Web page.

Syntax:

```
<%= Expression %>
```

where,

- `<%` = is the opening delimiter for the scriptlet.
- `Expression` indicates a valid Java statement.
- `%>` is the closing delimiter.

For example, the expression, `<%= Math.PI%>` looks similar to a markup is a Java expression that will insert the value of PI into the output.

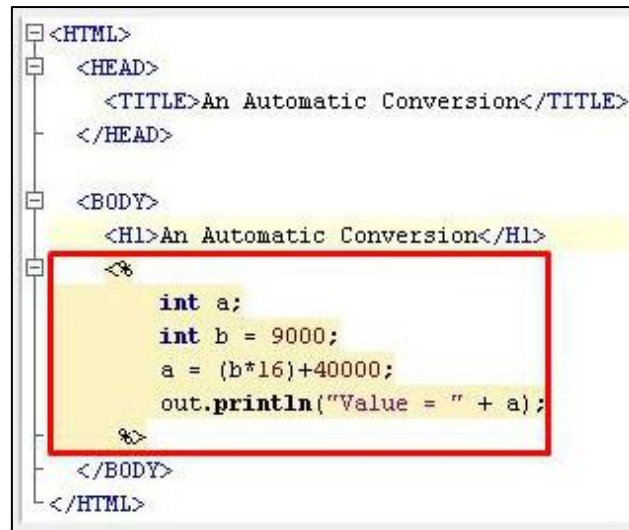
➤ Scriptlets

A JSP scriptlet is used to embed Java code within an HTML code. The Java code is inserted into the `jspService()` method of the servlet. The scriptlets are executed when the request of the client is being processed. The scriptlets are used to add complex data to an HTML form.

Syntax:

```
<%= code fragment %>
```

Figure 6.5 depicts the use of scriptlet in a JSP page.



```

<HTML>
<HEAD>
  <TITLE>An Automatic Conversion</TITLE>
</HEAD>
<BODY>
  <H1>An Automatic Conversion</H1>
  <%
    int a;
    int b = 9000;
    a = (b*16)+40000;
    out.println("Value = " + a);
  %>
</BODY>
</HTML>

```

Figure 6.5: Using Scriptlet in a JSP Page

➤ Declarations

JSP declarations allow the user to define variables and methods that are inserted into the servlet, outside the `service()` method.

You can declare variables within the scriptlet tag also, but that variable will be local to that scriptlet. However, variable declared using the declaration tag will be visible in the whole JSP page. Also, methods cannot be declared within the scriptlet tag.

A single declaration tag can be used to define multiple variables of same data type.

Syntax:

```
<%! declaration code %>
```

Code Snippet 2 demonstrates the use of declaration tag in the JSP page. In an earlier session, steps to create a JSP-based Web application was already covered. You may follow similar steps to create a new application to add and test this code given in Code Snippet 2.

Code Snippet 2:

```

<html>
...
<body>
  <h3>Area of a Rectangle</h3>
  <%! int length = 20, breadth = 30, area = 0; %>
  <%! private String units = "cm"; %>

  <% area = length * breadth; %>
  <p> The area of the Rectangle is:

```



```

        <%= area %> <%= units %>
    </p>
</body>
</html>

```

Figure 6.6 shows the output of Code Snippet 2.

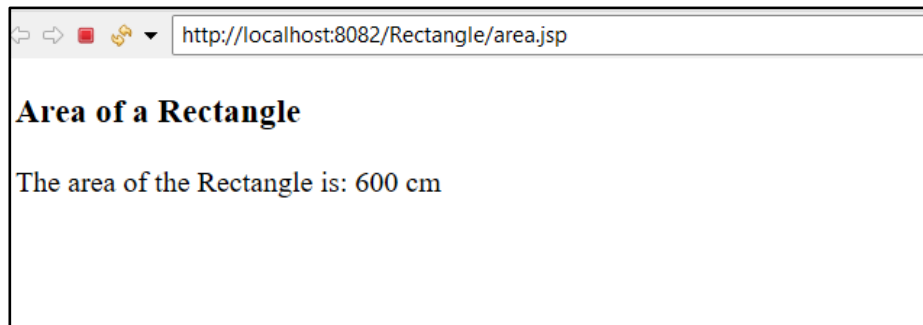


Figure 6.6: Output – Declaration Tag

➤ Comments

JSP comments are used for documenting JSP code that should not be visible to the client when viewing the source of the Web page. These comments are called server-side comments and are encoded within `<%--` and `--%>` symbol.

Syntax:

```
<%-- a JSP comment --%>
```

Example: `<%-- This is the JSP comment in a file --%>`

6.5 Directive Tags

JSP directives control the processing of the entire page. These directives identify the packages to be imported and the interfaces to be implemented. However, these directives do not produce any output. They inform the JSP engine about the actions to be performed on the JSP page.

The overall structure of the servlet class is affected by JSP directives.

Syntax:

```
<%@ directiveName attribute = "value"%>
```

where,

directiveName is the name of the directive.

attribute is attribute of directive.

value is the value of attribute specified.

You can declare these directives anywhere on the page. However, mostly they are declared at the top of the JSP page.

In JSP, there are three directives:

page	Include	taglib
The page directive provides instructions that control the structure of the page.	The include directive includes static contents on the current JSP page.	The taglib directive includes the custom library tags on the JSP page.

6.5.1 page Directive

The `page` directive provides instructions that control the structure of the page. It can be used to import classes from the packages, set the content type for the page, enable (or disable) page as error page, set the language on the page, and so on.

The `page` directive defines and manipulates a number of important attributes that affect the entire JSP page. A page directive is written at the beginning of a JSP page. A JSP page can contain any number of page directives. All directives in the page are processed together during translation and result is applied together to the JSP page. For example, `<%@ page import="java.util.*,java.io.*" %>` is used to import more than one package in the JSP page. If more than one package is imported, then separate the package names by commas.

Some of the attributes that can be defined for a page are as follows:

- **language**
The `language` attribute of the `page` directive is used to specify the language of the script. The default value of language in a JSP page is `Java`.
- **errorPage**
A path name to this JSP file sends exceptions. If the pathname begins with a `'/'`, the path is relative to the root directory and is resolved by the Web server. If not, the path name is relative to the current JSP file.
- **autoFlush**
The buffered output is flushed automatically when the buffer is full. If the `autoFlush` is set to `true`, which is default value, the buffer will be flushed. Otherwise, an exception will be thrown when the buffer overflows.
- **session**
The client must join an HTTP session in order to use the JSP page. If the value is `true`, the session object refers to the current or new session. The default value for `session` is `true`.

Syntax:

```
<%@ page attribute %>
```

where,

`page` indicates page directive.

attribute **specifies** attribute-value pair of page directive.

The valid parameters are as follows:

```
<%@ page
    language="java"
    extends="className"
    import="className{,+}"
    session="true|false"
    buffer="none|sizeInKB"
    autoFlush="true|false"
    isThreadSafe="true|false"
    info="text"
    errorPage="jspUrl"
    isErrorPage="true|false"
    contentType="mimeType{;charset=charset}"
%>
```

6.5.2 include Directive

The **include** directive is used to insert the content of another resource in a JSP page at run time. The contents are included physically during page translation. The resource or content can be a file in a text-based format, such as text, HTML, or JSP.

The remote resource should be given with the proper relative file path in the **include** directive.

Syntax:

```
<%@ include file = "Filename" %>
```

where,

include indicates the directive.

Filename specifies the name of a file to include along with relative path.

Figure 6.7 depicts the **include** directive.



```
<html>
<head><title>Test for include directive </title></head>
<body bgcolor="white">
<font color="red">
Today's date is :
<%@ include file="date.jsp" %>
</font>
</body>
</html>
```

Figure 6.7: include Directive

6.5.3 taglib Directive

The `taglib` directive allows the JSP page to create custom tags, which are defined by the user. Custom tags have access to all the objects that are available for a JSP page.

A tag library is a group of custom tags that extend the functionality of a JSP page one after the other. The `taglib` directive specifies name of the tag library, which contains compiled Java code for a tag to be used. Using this tag library, the JSP engine determines what action is to be taken when a particular tag appears in a JSP page.

Syntax:

```
<%@ taglib uri="tagLibraryURI" prefix="tagPrefix" %>
```

where,

`tagLibraryURI` specifies Uniform Resource Identifier (URI) that identifies the tag library descriptor.

`prefix` specifies tag name that is used to define a custom tag.

`tagPrefix` defines the prefix string in `prefix:tagname`.

Figure 6.8 depicts the `taglib` directive.



Figure 6.8: taglib Directive

6.6 JSP 3.1 New Features

JSP 3.1 was released as part of Jakarta EE 10. Major update was renaming of `javax.servlet.jsp` to `jakarta.servlet.jsp` package. To enable the Jakarta Server Pages 3.1 feature, add following element declaration inside the `featureManager` element of the `server.xml` file.

```
<feature>pages-3.1</feature>
```

`server.xml` could be located in Tomcat configuration at the path, `C:\Program Files (x86)\Apache Software Foundation\Apache Tomcat 10.\conf`

6.7 Integrating Jakarta Servlet with JSP

Integrating Jakarta Servlets with JSP involves a few steps. Here is a high-level overview of the process:

Create a Servlet

Write a servlet class that extends `HttpServlet`. In the servlet, override the `doGet` or `doPost` methods to handle HTTP GET or POST requests.

Create JSP Files

Create JSP files in the `WebContent` directory of the project. These files will contain HTML code mixed with JSP tags and directives.

Configure Web Descriptor

In the `web.xml` file, define the servlet and map it to a URL pattern. This informs the server which servlet to invoke based on the URL requested by the client.

Pass Data

Exchange data between servlet and JSP using request attributes, session attributes, or by encoding data in the URL.

Handle Form Submission

Handle form submissions by overriding the `doPost()` method in the servlet, retrieving form data, processing it, and generating a response.

Run and Test

Deploy your application to a Jakarta EE compatible server such as Tomcat and test it by accessing the mapped URL.

6.7.1 Dispatching Requests from Servlet to JSP

Dispatching a request from a servlet to a JSP is a common task in a Web application. This process allows a server to process user requests and then, forward the results to a JSP for presentation.

Steps involved are as follows:

- **Forward Requests:** Inside the servlet's `doGet` or `doPost` methods, use the `RequestDispatcher` to forward the request to the appropriate JSP file. This ensures a smooth transition from the servlet logic to the presentation layer in JSP.

Syntax:

```
RequestDispatcher dispatcher =
getServletContext().getRequestDispatcher("/WEB-
INF/myPage.jsp");
dispatcher.forward(request, response);
```

where,

- `RequestDispatcher` specifies the path to the JSP file to which the request will be forwarded.
 - `request` indicates the `HttpServletRequest` object containing the client's request.
 - `response` indicates the `HttpServletResponse` object for the servlet's response.
- **Pass Data to JSP:** Pass data from the servlet to the JSP using the request object. The `setAttribute()` method in the servlet before forwarding the request. This enables the JSP to access and utilize the data during the rendering process.

Syntax:

```
request.setAttribute("myAttribute", "Hello, JSP!");
```

where,

`setAttribute()` passes data between servlets and JSP.

- **Access Response in JSP:** Once data has been passed from the servlet to the JSP, it can be accessed within the JSP tags using the `getAttribute()` method. This enables the response data to smoothly integrate into the presentation layer.

Syntax:

```
<p><%= request.getAttribute("myAttribute") %></p>
```

where,

`getAttribute()` retrieves the value of the attribute set in the servlet.

6.7.2 Deploying and Running a Simple Servlet-JSP Application

To create a Servlet-JSP application, perform these steps:

1. Open Eclipse.
2. To create a new project, in the Eclipse window, click **File → New → Dynamic Web Project**, as shown in Figure 6.9.

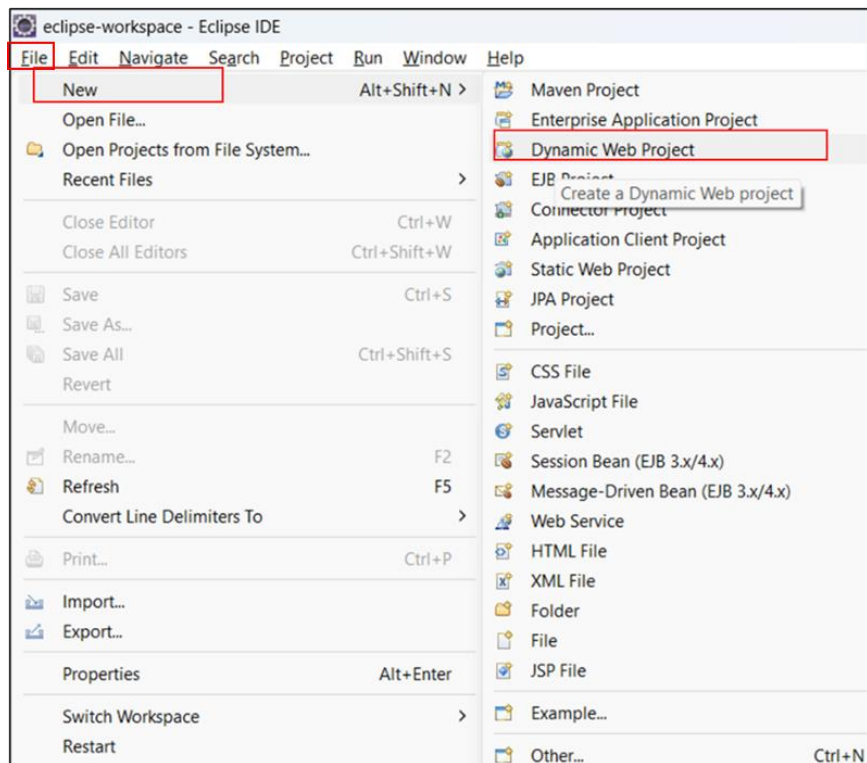


Figure 6.9: Creating a Dynamic Web Project

3. On the **Dynamic Web Project** page, enter the Project name as **ServletJSP** and click **Next** as shown in Figure 6.10.

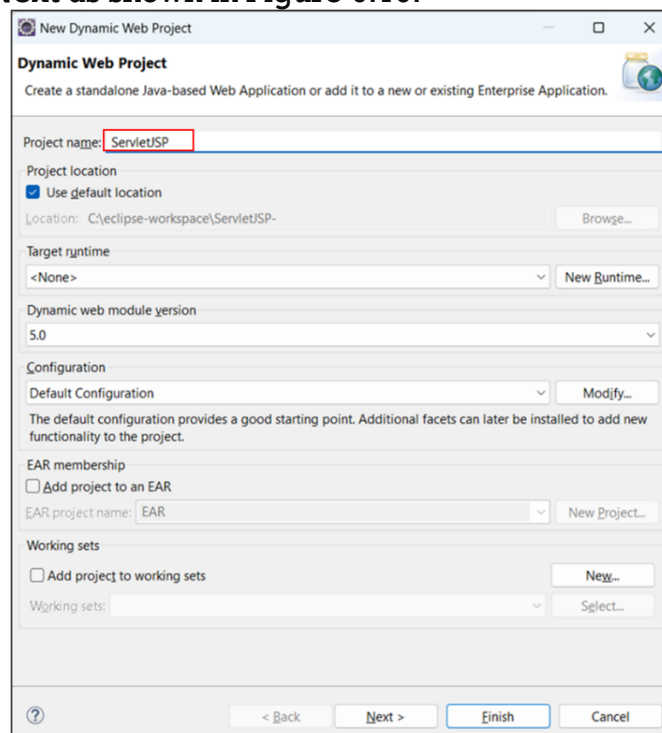


Figure 6.10: New Dynamic Web Project

4. On the **Java** page, verify the source folders and then, click **Next** as shown in Figure 6.11.

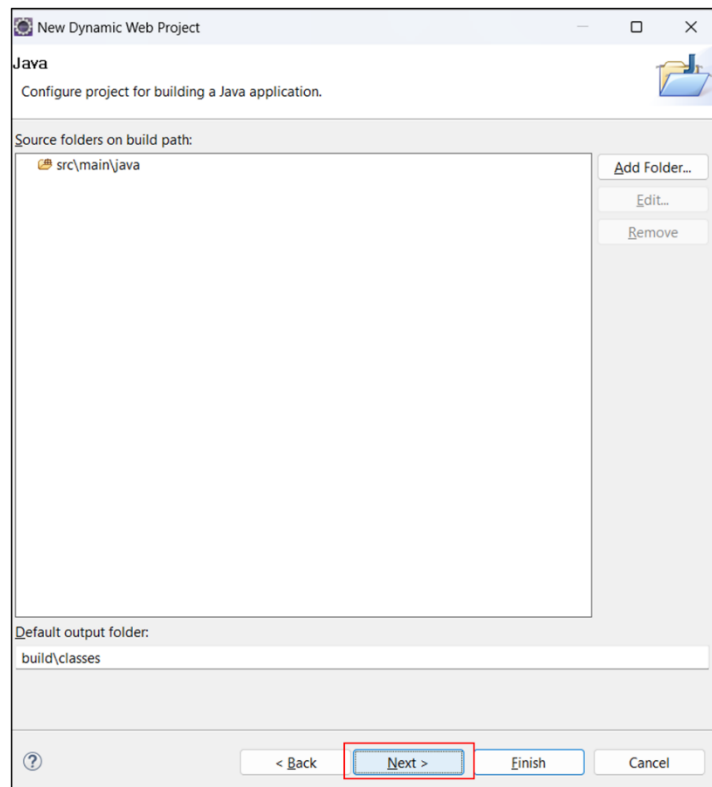


Figure 6.11: Source Folder

5. On the **Web Module** page, select the **Generate web.xml deployment descriptor** check box and click **Finish** as shown in Figure 6.12.

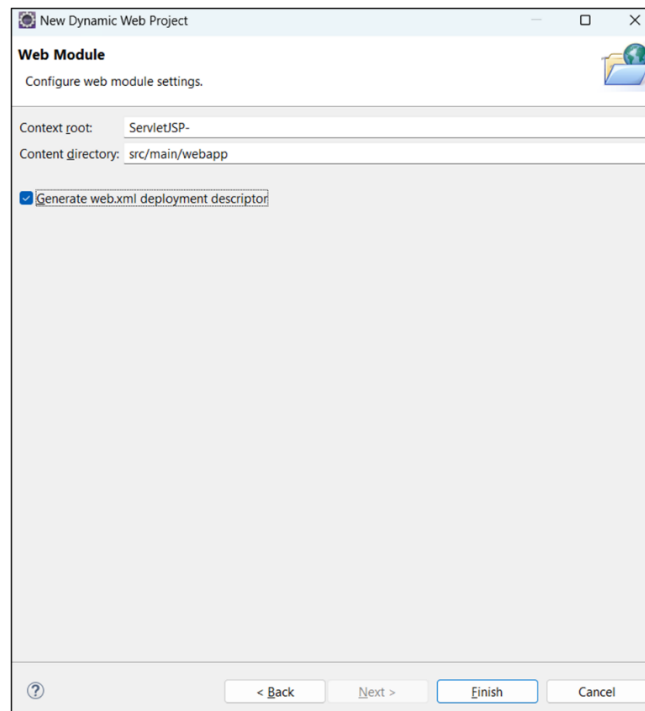


Figure 6.12: Web Module

The project is created and the **ServletJSP** folder is listed, as shown in Figure 6.13.

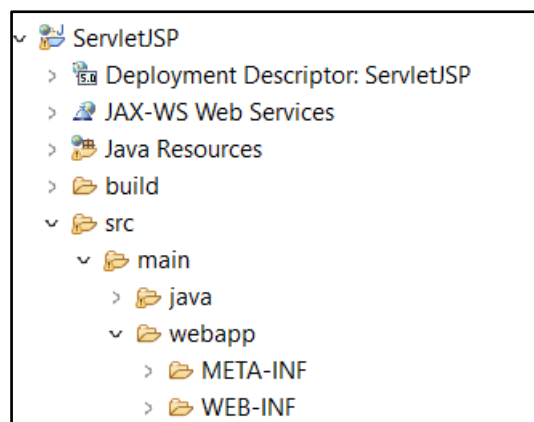


Figure 6.13: Folder Structure

6. To create a Servlet, expand Java **Resources**, right-click **src/main/java**, and select **New** → **Servlet** as shown in Figure 6.14.

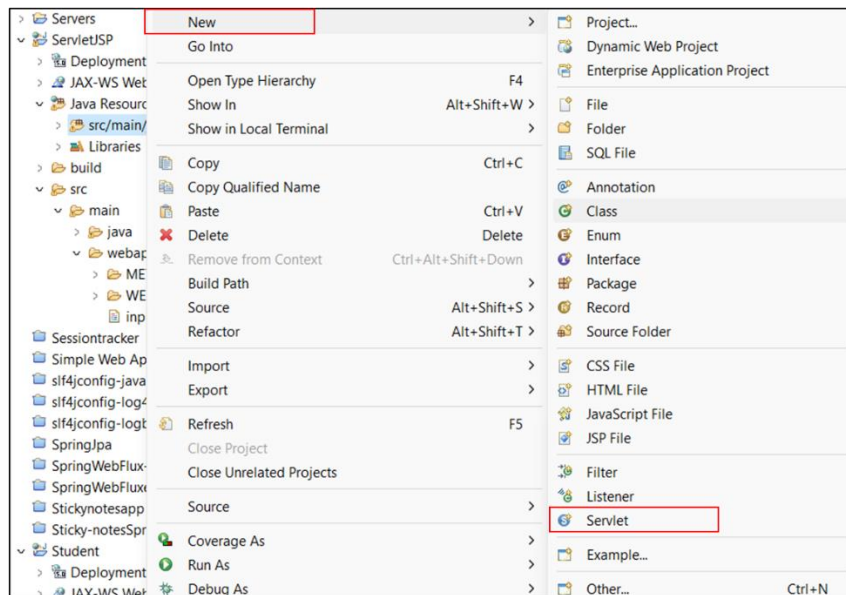


Figure 6.14: Create Servlet

7. **Create Servlet** wizard opens. Enter the class name as **WelcomeServlet** and click **Next** as shown in Figure 6.15.

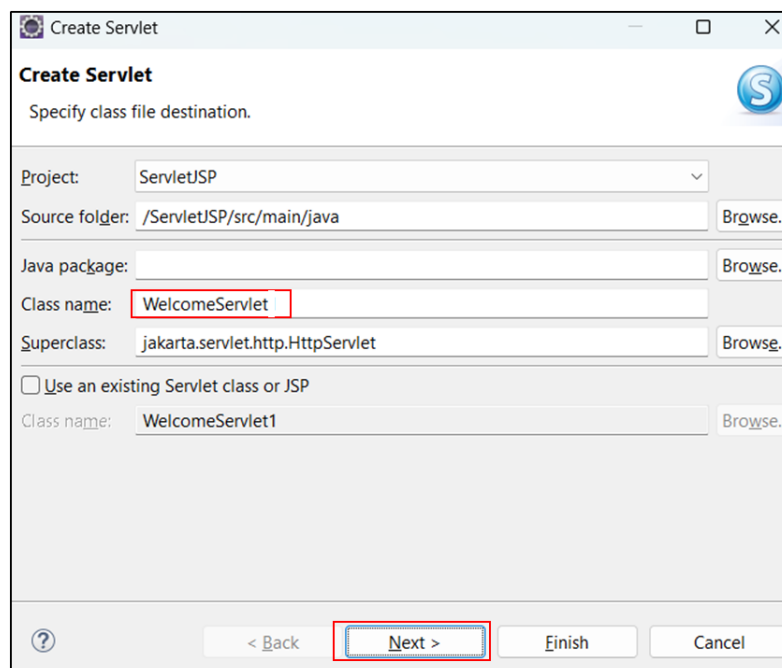
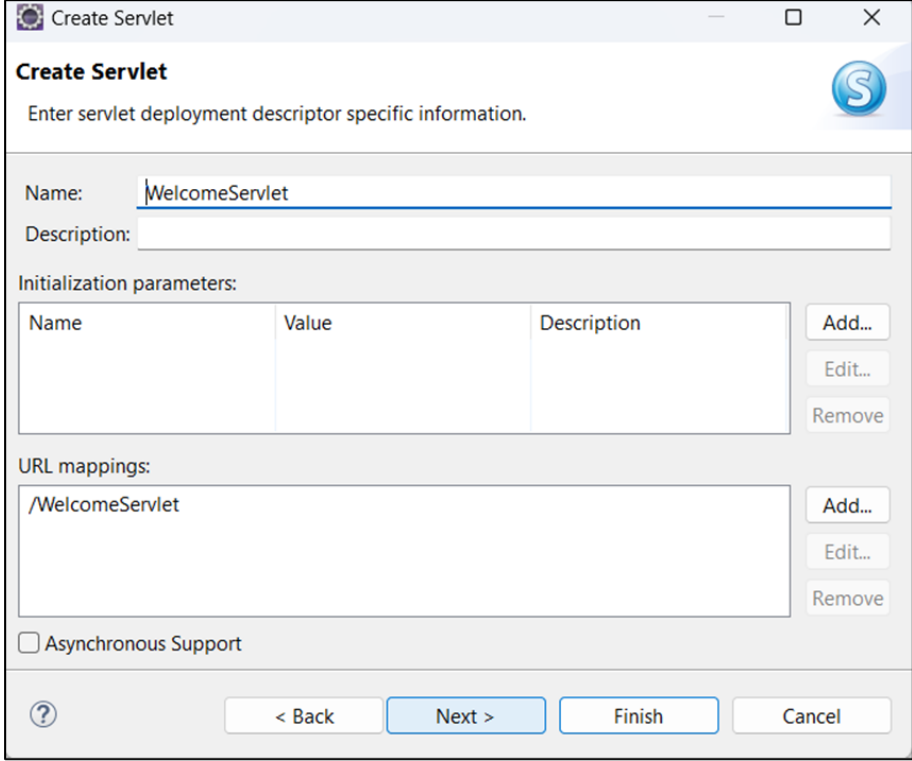


Figure 6.15: Enter Class Name

8. On the **Enter servlet deployment descriptor specific information** page, verify the URL mappings and click **Next** as shown in Figure 6.16.



Create Servlet

Enter servlet deployment descriptor specific information.

Name:

Description:

Initialization parameters:

Name	Value	Description

Buttons: Add..., Edit..., Remove

URL mappings:

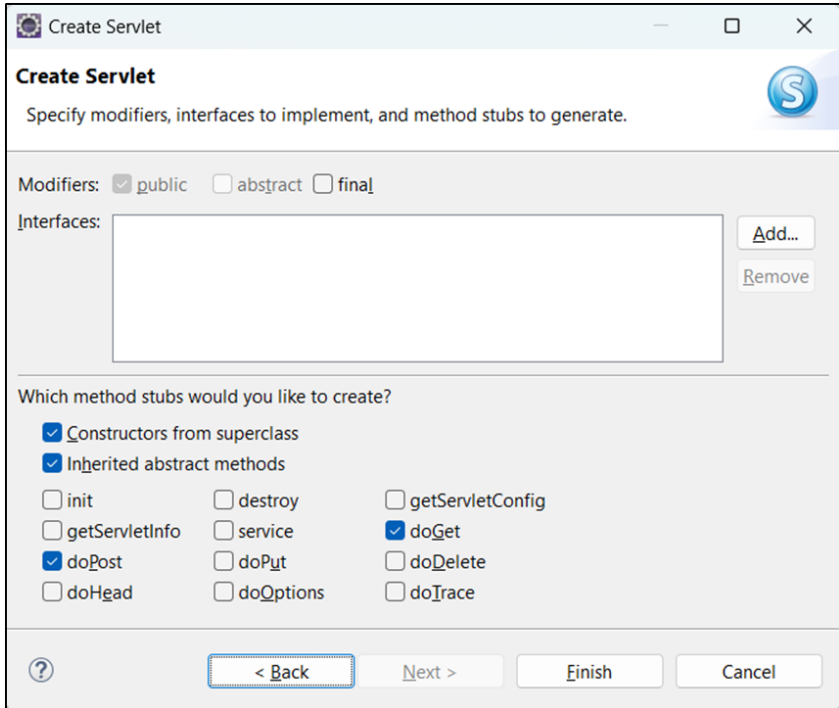
Buttons: Add..., Edit..., Remove

☐ Asynchronous Support

Buttons: ? < Back Next > Finish Cancel

Figure 6.16: URL Mappings

9. On the **Specify modifiers, interfaces to implement, and method stubs to generate** page, check the required method stubs check box and click **Finish** as shown in Figure 6.17.



Create Servlet

Specify modifiers, interfaces to implement, and method stubs to generate.

Modifiers: ☒ public ☐ abstract ☐ final

Interfaces:

Buttons: Add..., Remove

Which method stubs would you like to create?

☒ Constructors from superclass

☒ Inherited abstract methods

☐ init	☐ destroy	☐ getServletConfig
☐ getServletInfo	☐ service	☒ doGet
☒ doPost	☐ doPut	☐ doDelete
☐ doHead	☐ doOptions	☐ doTrace

Buttons: ? < Back Next > Finish Cancel

Figure 6.17: Generate Required Method Stubs

10. Add the code given in Code Snippet 3 to the Servlet class file. In the code, when a **GET** request is made to the `/WelcomeServlet` URL, the `WelcomeServlet` handles the request. After processing, it forwards the request to `Welcome.jsp` using the `RequestDispatcher.forward()` method. The JSP file then, generates the HTML content which is displayed in the user's browser. This is a common pattern for separating business logic from the presentation in Web applications.

Code Snippet 3: `WelcomeServlet.java`

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
@SuppressWarnings("serial")
@WebServlet("/WelcomeServlet")
public class WelcomeServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        String s = request.getParameter("t1");
        request.setAttribute("username", s);

        request.getRequestDispatcher("/welcome.jsp").forward(request, response);
    }
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        doGet(request, response);
    }
}
```

11. To create the `index.jsp` file, right-click the **webapp** directory and select **New → JSP File**, as shown in Figure 6.18.

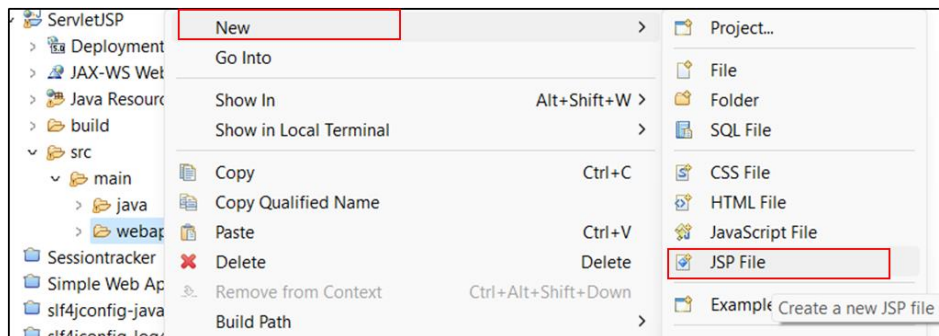


Figure 6.18: Create a JSP File

12. In the **New JSP File** wizard, in the **File name** box, enter **index.jsp** as shown in Figure 6.19.

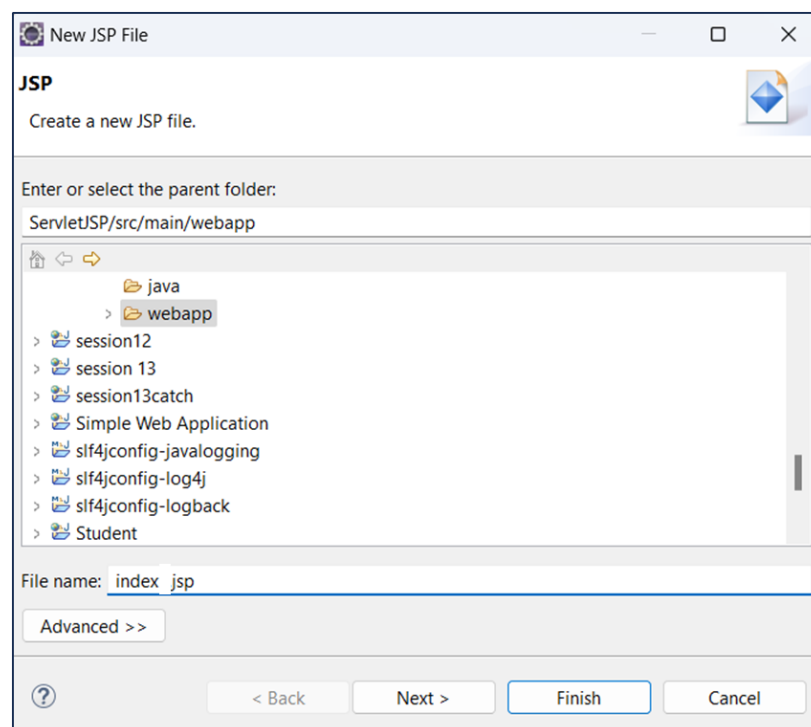


Figure 6.19: index.jsp

13. Add the code given in Code Snippet 4 to the JSP file.

Code Snippet 4:

```

<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Servlet Demo</title>
</head>
<body>
    <form method="get" action="WelcomeServlet">
        <pre>
            Enter your Name: <input type="text"
name="t1">
                <input type="submit"> <input
type="reset">
        </pre>
    </form>
</body>
</html>

```

14. Create another JSP file named **welcome.jsp** and add the code given in Code Snippet 5.

Code Snippet 5:

```

<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Welcome</title>
</head>
<body>
    <h1>Welcome, <%= request.getAttribute("username")
%></h1>
</body>
</html>

```

15. To run the application, right-click the project and select **Run As → Run on Server**, as shown in Figure 6.20.

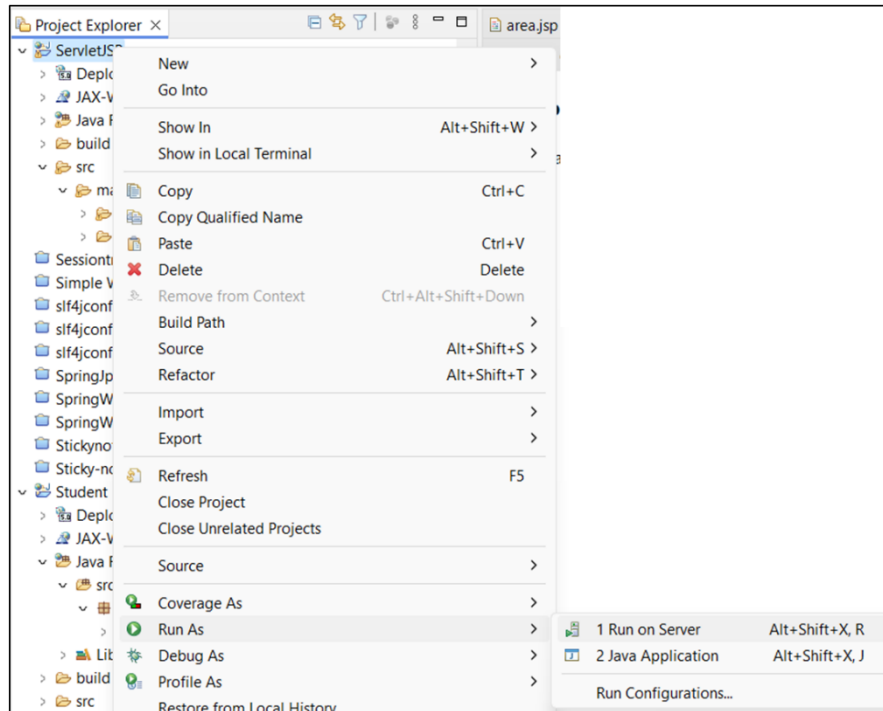


Figure 6.20: Run on Server

16. On the **Run On Server** wizard, select the Tomcat server to use, as shown in Figure 6.21.

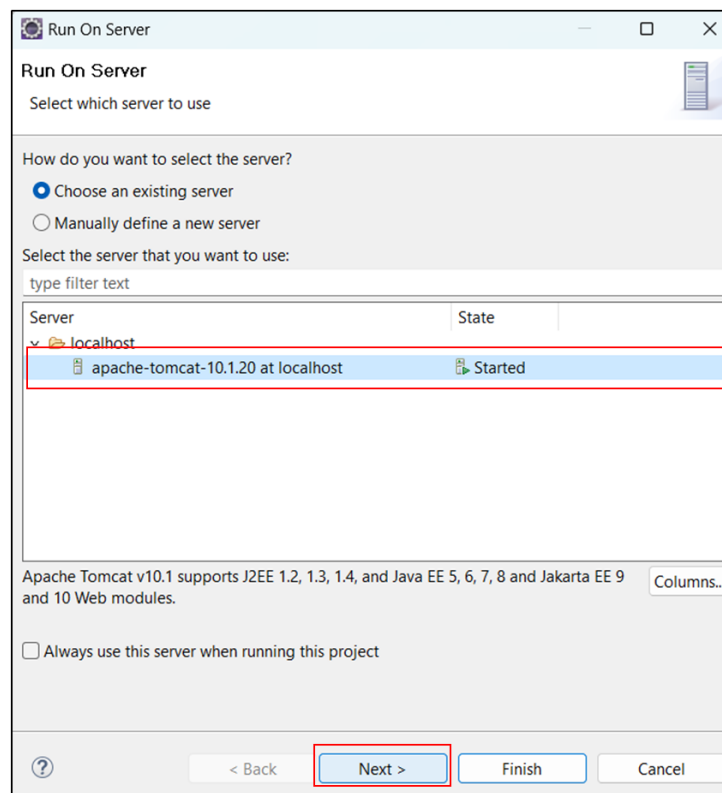


Figure 6.21: Select Tomcat Server

17. On the **Add and Remove** page, ensure that **ServletJSP** is configured and click **Finish** as shown in Figure 6.22.

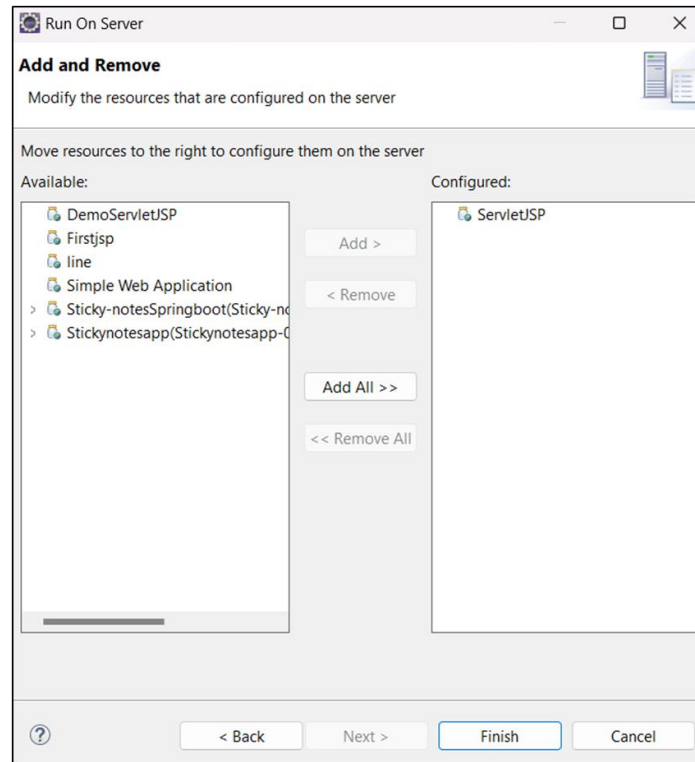


Figure 6.22: Add and Remove

The code executes and the `index.jsp` page is displayed in the Web browser, as shown in Figure 6.23. Enter your name and click **Submit**.

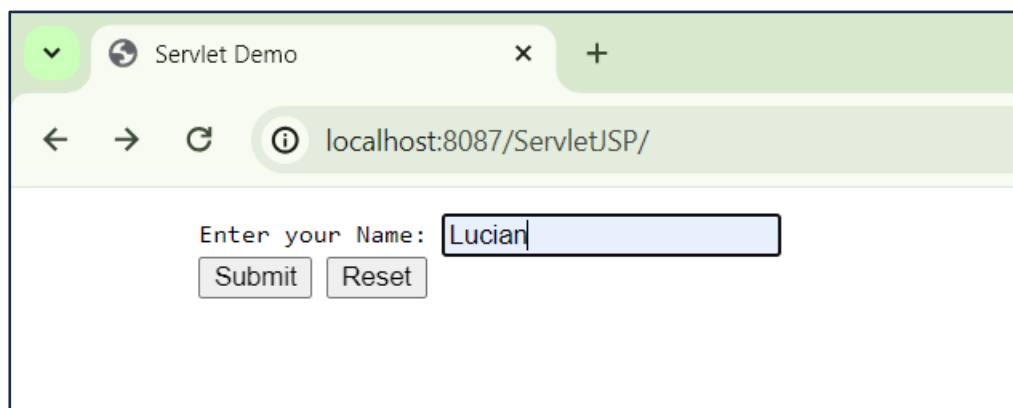


Figure 6.23: Output -Entry Page

The **WelcomeServlet** class processes the request and dispatches it to **Welcome.jsp** to display the response in the browser. Refer to Figure 6.24.

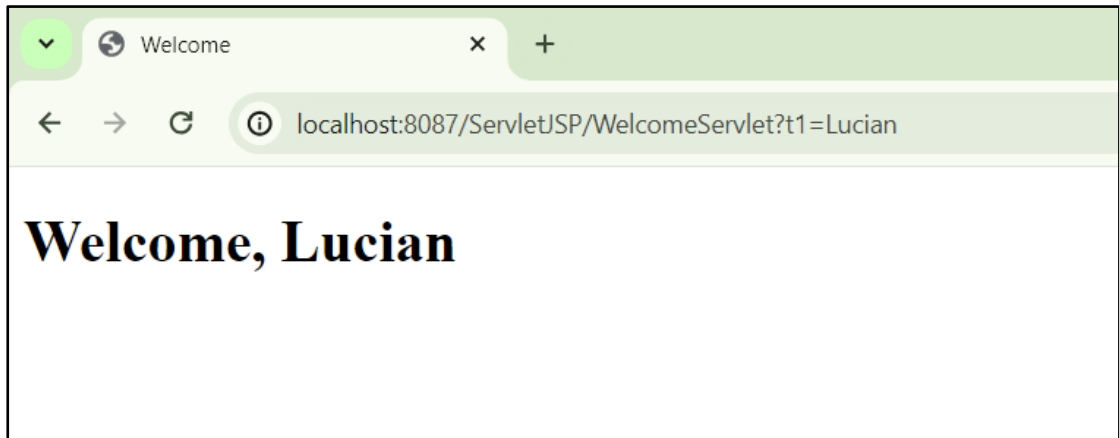


Figure 6.24: Output-Welcome Page

Summary

- JSP technology is used for developing dynamic Web sites and provides server-side scripting support for creating Web applications.
- A JSP page is a mixture of standard HTML tags, Web page content, and dynamic content that is specified using JSP elements.
- The JSP elements are expressions, scriptlets, comments, declarations, directives, and actions.
- Directives are used to specify the structure of the resulting servlet and actions are JSP tags that transfer control to other server objects or perform operations on other objects.
- A JSP scriptlet is used to embed Java code within an HTML code.
- Servlets run faster than JSP since, JSP takes time to compile and convert the program into servlets.
- The page directive is used to set the page attributes, such as the language to be used and encoding format.
- The include directive is used to insert a file referenced in the directive into the JSP page.
- The taglib directive links the JSP page to an XML document that describes a set of custom JSP tags and determines which tag handler class can implement the action of each tag.
- Servlets and JSP work together to build Web applications, with Servlets handling backend tasks and JSP managing frontend presentation.

Check Your Progress

1. Identify which of the following statements related to JSP pages are true.

(A)	JSP implements presentation logic in a JSP page and content logic on server in an EJB.
(B)	EJB cannot be shared and exchanged among Web programmers.
(C)	By using standard tags, even programmers with limited knowledge in scripting language, can design a JSP page.
(D)	Jakarta Server Pages is used in Application layer.
(E)	A JSP page does not contain HTML tags.

A.	A, C	C.	B, E
B.	A, B	D.	A, D

2. Which one is the correct option from the following statements for translation unit?

(A)	A JSP page only includes the contents of servlet pages.
(B)	The translation unit processes the JSP page and converts it to HTML page.
(C)	The translation unit consists of the JSP source file as well as all of its static include files.
(D)	Dynamic include files form a part of the translation unit.

3. Which are the correct options from the following statements for creating JSP expressions?

(A)	JSP expressions are used to display individual variables, or the result of some calculation in a Web page.
(B)	A JSP expression contains a JSP statement whose value will be inserted into the generated Web page.
(C)	At run time, JSP expression is evaluated, and the result is converted into a string.
(D)	The expression is used to access the servlet elements in JSP.
(E)	The evaluated string in a JSP expression is directed to the JSP page.

A.	A, C, D	C.	A, C, E
B.	A, B, C	D.	A, D

4. Which one is the correct option from the following statements for creating JSP scriptlets?

(A)	A JSP scriptlet is used to embed Java code within a JSP code.
(B)	The Java code is inserted into the <code>jspService()</code> method of the servlet.
(C)	The scriptlets are executed when the request of the server is being processed.
(D)	The scriptlets are used to add data to a JSP form.

5. Which are the correct options from the following statements for declaring variables in JSP?

(A)	JSP declarations allow the user to define variables and methods that are inserted into the servlet.
(B)	The methods must be defined outside the <code>init()</code> method.
(C)	A single declaration tag can be used to define multiple variables.
(D)	JSP declarations are used in combination with scriptlets and expressions to display an output.
(E)	The users are allowed to define methods into a JSP file.

A.	A, B, D	C.	A, C, E
B.	A, B, C	D.	A, C, D

6. How does Servlet dispatch work in a Servlet-JSP application?

(A)	Servlet receives client requests and processes them without involving JSP
(B)	Servlet forwards or redirects requests to JSP for further processing
(C)	Servlet sends responses directly to clients without involving JSP
(D)	Servlet manages database interactions independently of JSP

Answers

1.	D
2.	C
3.	C
4.	B
5.	D
6.	B

Try It Yourself

1. Develop a simple Servlet-JSP application in Eclipse by creating a form with fields for Full Name, Email Address, and Mobile Number. Display the entered details on a Jakarta Server Page. Test the application on the Tomcat server. Table 6.1 shows sample data for this program.

Full Name	Email Address	Mobile Number
John Doe	john.doe@example.com	+1 555-123-4567
Jane Smith	jane.smith@example.com	+44 1234 567890
Michael Johnson	michael@example.com	+61 2 1234 5678

Table 6.1: Sample Data

Filters and Annotations

Welcome to the Session, **Filters and Annotations**.

This session introduces the concept of a filter as a vital Web component. The session explains how to create filters that can manipulate client requests and server responses. Further, the session provides a comprehensive study of annotations supported by Java Servlets. Finally, the session concludes with an explanation of how to upload files using HTML forms and Servlets.

In this session, you will learn to:

- Explain the concept and working of filters
- Describe the Servlet filter configuration
- List the benefits of filters
- Explain the Filter API interfaces and their methods
- Explain Filter to Servlet mapping
- Identify the use of wrapper classes in managing Servlets
- Explain how to alter a request and response using filters
- Describe the basic concept of annotations
- List different types of annotations supported in Servlet API
- Outline the steps to upload the file using HTML form elements
- Explain the mechanism to upload files on the server using Servlet
- Describe Server push in Servlet 6.0

7.1 Introduction

There are some situations when data sent or received from the Web resources is to be processed with extra functionality. For example, a common scenario for processing the sensitive data sent in response over the network. The data sent in the response from a Servlet must be encrypted to avoid any malfunction threat. This extra processing of encryption is to be done on the received response from the Servlet.

Java supports filters objects that performs processing of the request before it acquires the resource. Similarly, before sending the response, filters perform manipulation of the responses sent to the client. Filters can be defined as objects that perform the process of filtration and are configured using the `web.xml` deployment descriptor. Filters work both ways on pre-processing client requests and post-processing server responses.

Some of the common scenarios of Web applications where filters can be configured are authentication of the users, logging of the messages, image conversion, compression of data, encryption of responses, transformation to XML, and so on.

7.1.1 Concept of Filter

Filters are Java classes and are typically used to encapsulate the pre-processing and post-processing functionalities that are common to a group of Servlets or JSP pages.

Filters were introduced as a Web component in Java Servlet specification. They reside in the Web container along with the other Web components, such as Servlet or JSP pages.

Figure 7.1 depicts filters in a Web application.

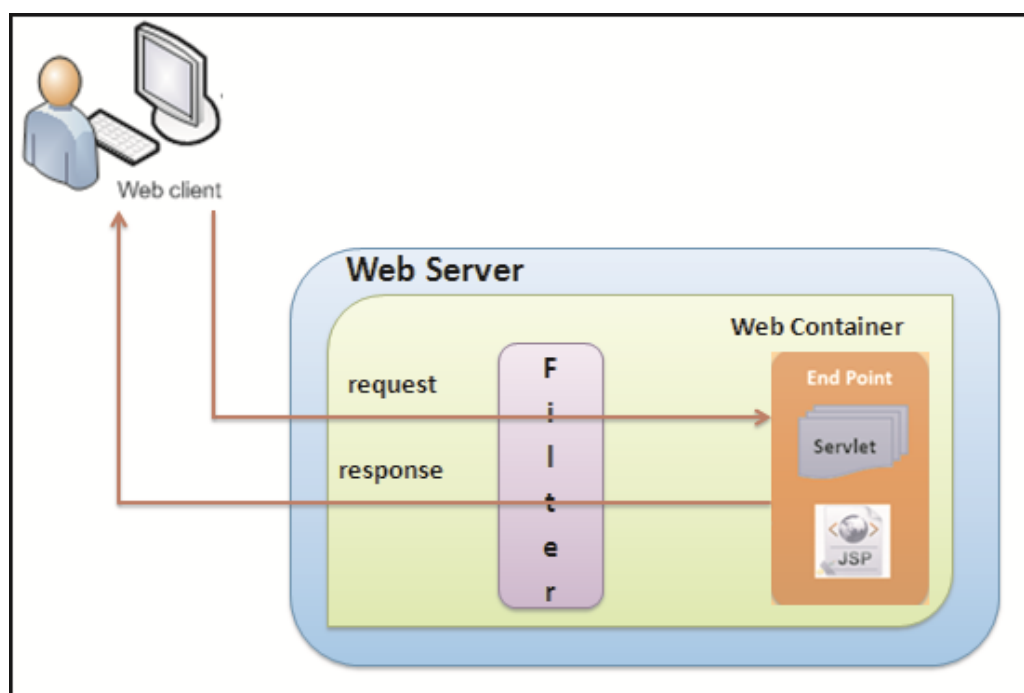


Figure 7.1: Filters

Figure 7.1 shows the filter acting as interceptor for processing the request and response between the client and Web resources or endpoints, such as Servlet and JSP.

Some of the important features provided by the filters to the Web applications include:

Optimization of the time taken to send a response back to a client.

Compression of the content size sent from a Web server to a user over the Internet.

Conversion of images, texts, and videos to the required format.

Optimization of the available bandwidth on the network thereby reducing the network traffic and in turn, increasing the efficiency of all Web-based services.

Authentication of the users in the secured Website.

Encryption of the request and response header for addressing security concerns.

7.1.2 Filter Chaining

There can be more than one filter between the client and the Web resources, thus forming a filter chain. A request or a response is passed through one filter to the next in the filter chain. So each request and response has to be serviced by filters configured in the chain before the Servlet is called by the Web container and after the Servlet responds.

Figure 7.2 shows the chaining of the filters.

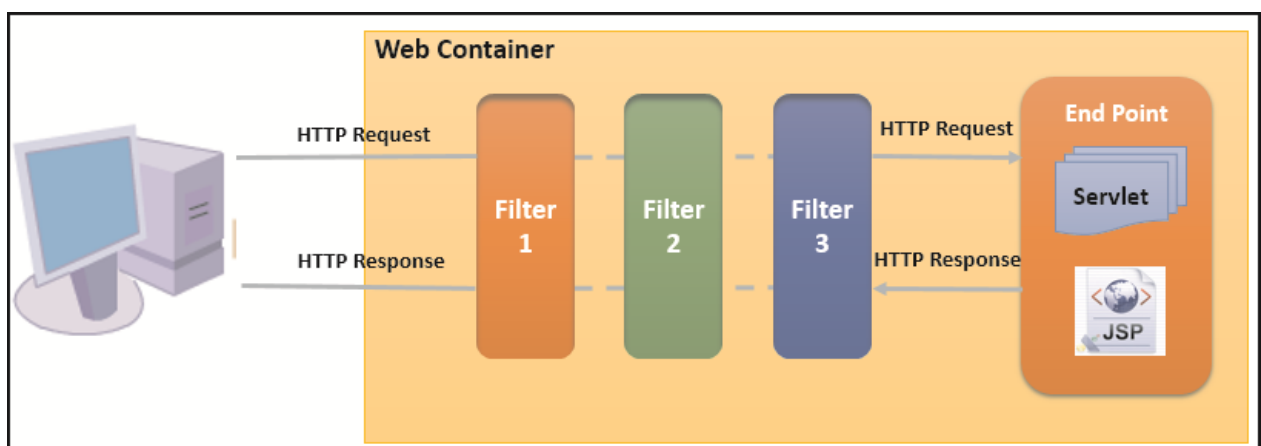


Figure 7.2: Filter Chaining

7.1.3 Working of Filters

Filters dynamically access incoming requests from the user before the servlet processes the request. Filters can also access the outgoing response from the Web resources before it reaches the user. The information about the configured filters is provided when the application is deployed on the server through `web.xml`.

A typical filter operates in following way:

1. When the request for the Web resource is sent to the Web server, the Web container instantiates the filters and loads them for preprocessing.
2. The filter intercepts the request from the user to the servlet.
3. After the request is processed, the filter can do following:
 - Generate response and return to the client.
 - Pass the modified or unmodified request to other filters in the chain.
 - If the request is received by the last filter in the chain, it passes the request to the destination Servlet.
 - It may route the request to a different resource rather than the associated Servlet.
4. Then, the filter sends the serviced request to the appropriate Servlet.

Similarly, when the response is returned to the client, the response passes through the same set of filters in reverse order. Figure 7.3 depicts the working of a typical filter.

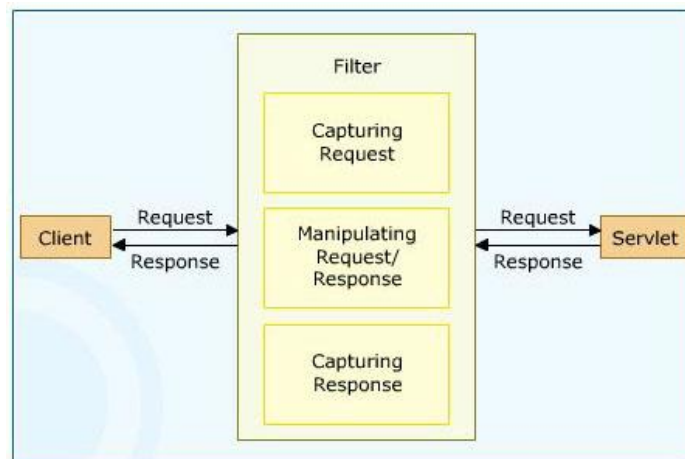


Figure 7.3: Working of a Filter

7.1.4 Usage of Filters

Filters are categorized according to the services they provide to the Web applications. Some of the filter categories are as follows:

Authentication filters	These filters allow the users to access any resource in on secured Websites after providing a proper username and password.
Logging and auditing filters	These filters track the activities of users on a Web application and log them.
Image conversion filters	These filters scale the image size or change the image type as per requirements.
Data compression filters	These filters help in compressing uploaded or downloaded file sizes, thereby reducing the bandwidth requirement and time for downloading.
Encryption filters	These filters help in encrypting the request and response header.
Filters that trigger resource access events	These filters help in registering the particular database for a Web application before they retrieve or manipulate data according to the request made by the client.
XML transformation filters	These filters read the XML data from the response and applies an XSLT transformation before sending it to the client browser.
Character Encoding	Filters can also convert the data requested by users into the required format.

Figure 7.4 depicts a compression filter.

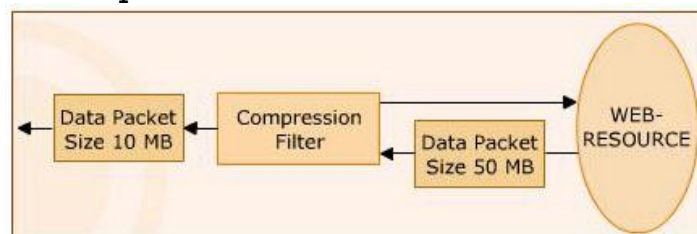


Figure 7.4: Compression Filter

Note: Filters are not Servlets. However, even they have a lifecycle for execution.

7.2 Filter API

The Filter API brings in all the required interfaces namely, `Filter`, `FilterChain`, and `FilterConfig` for creating a filter class into the `jakarta.servlet` package.

Figure 7.5 depicts the hierarchical structure of Filter API.

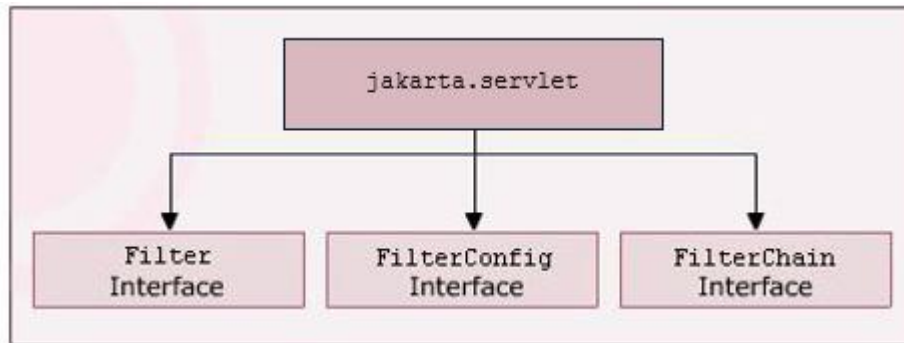


Figure 7.5: Hierarchy of Filter API

Any typical filter has to implement the `Filter` interface and the methods contained in it.

7.2.1 Filter Interface

The first step in creating a filter is to implement a `Filter` interface. The `Filter` interface is part of the Servlet API. The `Filter` interface provides the lifecycle methods of a filter. It must be implemented to create a filter class.

The methods of the `Filter` interface are as follows:

➤ `init()`

This is called by the servlet container to initialize the filter. It is called only once. The `init()` method must complete successfully before the filter is asked to do any filtering work.

Syntax:

```
public void init(FilterConfig filterConfig) throws ServletException
```

where,

- `filterConfig` is the object of `FilterConfig` interface passed as a parameter to the `init()` method.
- `ServletException` is a general exception that can be thrown by a Servlet.

➤ `doFilter()`

The `doFilter()` method of the `Filter` interface is called by the container each time a request or response is processed. It then, examines the request or response

headers and customizes them as per the requirements. It also passes the request or response through the `FilterChain` object to the next entity in the chain.

Syntax:

```
public void doFilter(ServletRequest req, ServletResponse res,
    FilterChain chain) throws IOException, ServletException
```

where,

- `ServletRequest` defines an object to provide client requests to a servlet.
- `ServletResponse` defines an object to help the Servlet in sending a response to the user.
- `FilterChain` interface creates an object chain to invoke the next filter component in the chain.

➤ **destroy()**

This method is called by the servlet container to inform the filter that its service is no longer required. This method is also called only once similar to the `init()` method.

Syntax:

```
public void destroy()
```

Code Snippet 1 demonstrates the implementation of a filter to display a message before the invocation of the configured Servlet.

Code Snippet 1: `MessageFilter.java`

```
package filterdemo;
import jakarta.servlet.*;
import jakarta.servlet.Filter;
public class MessageFilter implements Filter {
    private FilterConfig filterConfig;
    public void doFilter(final ServletRequest request,
                        final ServletResponse response,
                        FilterChain chain)
        throws java.io.IOException,
        jakarta.servlet.ServletException {
        System.out.println("Invoking Filter");
        System.out.println("Preprocessing done by
        filter");
        chain.doFilter(request, response);
    }
}
```

In Code Snippet 1, the `MessageFilter` class implements the `Filter` interface. When the filter will be invoked, the container will pass the request, response, and

chain objects to the `doFilter()` method. These objects are passed to the next component in the chain. It can be a filter or a Servlet.

7.3 Configuring Filters

Each filter is a Web component, which along with others forms a complete Web application. To be a part of a Web application, it is to be configured in the `web.xml` file. The configuration of a filter inside the `web.xml` file ensures that filter is the part of that application.

In the deployment descriptor, the filter is declared by the `<filter>` element. It defines a filter class and its initialization parameters.

Following elements can be defined within a filter element:

<code><display-name></code>	<code><description></code>	<code><filter-name></code>	<code><filter-class></code>	<code><init-param></code>
This element accepts the short name of the filter, which is intended to be displayed by GUI tools. It is not mandatory inside the <code><filter></code> element.	This gives a short description about the filter.	This defines the name of the filter. This is used to refer to the filter definition somewhere in the deployment descriptor. This element is mandatory.	It defines the class name of the filter. This is a mandatory element in a filter definition.	It holds the name and value pair of an initialization parameter. It is optional within the <code><filter></code> element.

7.3.1 Filter-Mapping Element

The `<filter-mapping>` element specifies which filter to execute depending on a URL pattern or a Servlet name. This sets up a logical relation between the filter and the Web application for sequential control flow of requests and responses between them.

Following elements are defined inside a filter-mapping element:

- `<filter-name>`
The element inside the deployment descriptor holds the name of the filter to which a URL pattern or a Servlet is mapped. It is a mandatory element for mapping a filter.
- `<url-pattern>`

The element describes a pattern used to resolve URLs. It is a mandatory element for mapping a filter.

➤ **<servlet-name>**

The element specifies the name of a servlet whose request and response will be modified by the filter. It is a mandatory element.

Figure 7.6 illustrates a filter named `FirstFilter` configured in `web.xml` deployment descriptor. Figure 7.6 declares a filter and identifies the Web app resource that will utilize it.

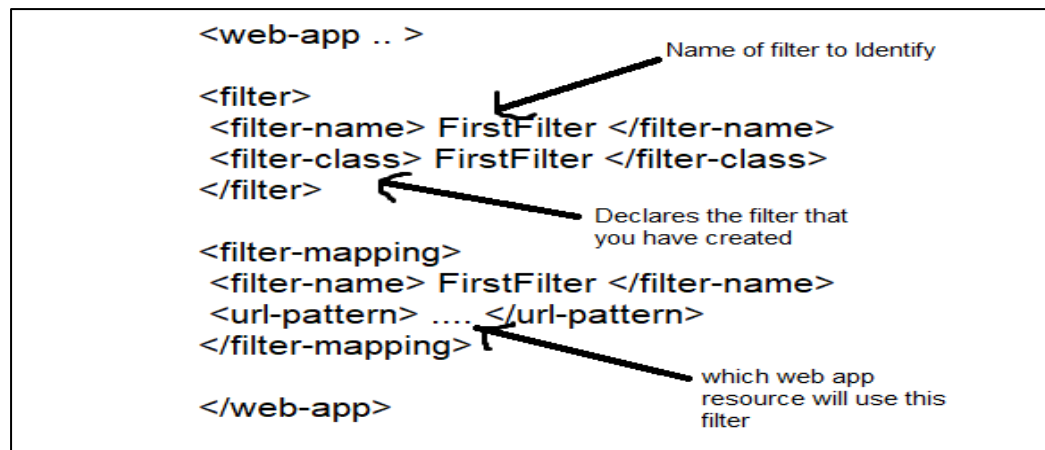


Figure 7.6: Filter Defined in web.xml Deployment descriptor

Hence, a mapped filter named `Message` will be executed when a mapped servlet named `MyServlet` is requested by the client. The `<servlet-name>` in `web.xml` can be replaced with `<url-pattern>` element to match a specific URL pattern.

Let us look at a complete example of using a filter.

By default, a `web.xml` is located under the `WEB-INF` directory of the Web application. For Code Snippet 1, the default content in `web.xml` can be replaced with that shown in Code Snippet 2.

Code Snippet 2: `web.xml`

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
  <display-name>filterdemo</display-name>
  <welcome-file-list>
    <welcome-file>Message.html</welcome-file>
  </welcome-file-list>

```

```

<filter>
  <filter-name>MessageFilter</filter-name>
  <filter-class>filterdemo.MessageFilter</filter-class>
</filter>
<filter-mapping>
  <filter-name>MessageFilter</filter-name>
  <url-pattern>/*</url-pattern>
  <dispatcher>REQUEST</dispatcher>
</filter-mapping>
<servlet>
  <servlet-name>Servlet1</servlet-name>
  <servlet-class>filterdemo.Servlet1</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>Servlet1</servlet-name>
  <url-pattern>/Servlet1</url-pattern>
</servlet-mapping>
</web-app>

```

An HTML file named `Message.html` as shown in Code Snippet 3 can be used with this example.

Code Snippet 3: `Message.html`

```

<!DOCTYPE HTML
<html>
  <head>
    <title>New Document</title>
  </head>
  <body>
    <h1> Servlet with Filter </h1>
    <a href="/filterdemo/Servlet1">
      Click to see the Servlet effect after building the
      filter</a>
    </body>
  </html>

```

Finally, the Servlet class can be created as shown in Code Snippet 4.

Code Snippet 4: `Servlet1.java`

```

package filterdemo;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

```



```

import java.io.PrintWriter;
public class Servlet1 extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<h1>Message Filter Servlet</h1>");
        out.println("</html>");
        out.close();
    }
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }
    public String getServletInfo() {
        return "Short description";
    }
}

```

Figures 7.7 and 7.8 depict the output of the example in the browser.

Servlet with Filter

[Click to see the Servlet effect after building the filter](#)

Figure 7.7: Output showing the HTML Page

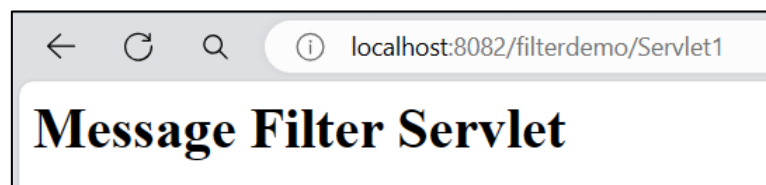


Figure 7.8: Output showing Servlet After Processing the Filter Action

7.3.2 Passing Initialization Parameters to Filter

The `init()` method of a filter initializes the filter. The `init()` method receives the object of `FilterConfig` object created by the Web container. Then, the initialization parameters are read in the `init()` using the methods of `FilterConfig`.

Table 7.1 lists some of the methods of `FilterConfig`.

Method	Description
<code>String getFilterName()</code>	This method returns the name of the filter defined in the <code>web.xml</code> or deployment descriptor file in the Web application.
<code>String getInitParameter(String name)</code>	This method returns the value of the named initialization parameter as a string. It returns null if the servlet has no attribute.
<code>Enumeration getInitParameterNames()</code>	This method returns the names of the initialization parameter as an enumeration of <code>String</code> objects. It returns an empty enumeration if the servlet has no attribute.
<code>ServletContext getServletContext()</code>	This method returns the <code>ServletContext</code> object used by the caller to interact with its Servlet container and filter.

Table 7.1: Methods of `FilterConfig`

Code Snippet 5 exhibits the use of a few methods in `FilterConfig` interface.

Code Snippet 5: `MessageFilter.java`

```
...
public class MessageFilter implements Filter {
    private String message;
    private FilterConfig config;
    public void init(FilterConfig config) throws ServletException
    {
        this.config = config;
        message = config.getInitParameter("message");
    }
    public void doFilter(final ServletRequest request,
                        final ServletResponse response,
                        FilterChain chain)
        throws java.io.IOException,
        jakarta.servlet.ServletException {
        System.out.println("Entering MessageFilter");
        request.setAttribute("messageObj", message);
        chain.doFilter(request, response);
        System.out.println("Exiting MessageFilter");
    }
}
```

The `init()` method of a filter is called only once, when the filter is instantiated by the container. The `getInitParameter()` method retrieves the value passed as a parameter to the filter.

Figure 7.9 shows the configuration of a filter with initialization parameters in the `web.xml` file.

```
<filter>
  <filter-name>Message</filter-name>
  <filter-class>servlet.filter.MessageFilter</filter-class>

  <!-- Initialization parameters -->
  <init-param>
    <param-name>message</param-name>
    <param-value>Welcome to Filter</param-value>
  </init-param>

</filter>

<filter-mapping>
```

Figure 7.9: Filter Initialization

When the application runs, the console will display the messages **Entering MessageFilter** and **Exiting MessageFilter** based on the respective processes.

7.3.3 FilterConfig

The Web container creates a `FilterConfig` object for each filter and the filter uses this object to acquire its initialization parameters and a reference to the `ServletContext`. This object is called by the Servlet container during the initialization phase. Basically the `FilterConfig` object is used to communicate information to a filter.

7.3.4 Use of Wildcard in Mappings

The `<url-element>` element in the deployment descriptor is used to specify the URL pattern for the Web resources. It can either contain a specific URL or even a wild card character (*) can be used to match a set of URLs. All the URLs mapped with filters are applied before the Servlet are invoked. When a Web application is executed, it creates an instance of each filter that has been declared in the deployment descriptor. The sequence for execution of these filters is in the order, as they are declared in the deployment descriptor.

Code Snippet 6 shows how to include multiple filters with a request to a mapped Servlet.

Code Snippet 6: `web.xml`

```
. . .
<filter-mapping>
  <filter-name>LogFilter</filter-name>
  <url-pattern>*.abc</url-pattern>
</filter-mapping>
<filter-mapping>
  <filter-name>AuditFilter</filter-name>
  <url-pattern>*.abc</url-pattern>
```

```
</filter-mapping>
. . .
```

Consider a situation where a Web page sends a request for a URL, `.../pages/add_customers.abc`, then the filters will be applied to the request in the order: `LogFilter` and then, `AuditFilter`.

Servlet API allows one to specify multiple match criteria in the same entry. This implies that there can be multiple entries for the `<url-pattern>` element.

You can also have multiple entries for `<servlet-name>` and `<url-pattern>` elements inside the `<filter-mapping>` element.

Code Snippet 7 shows the mapping for multiple patterns with the filter.

Code Snippet 7: `web.xml`

```
...
<filter-mapping>
  <filter-name>MultipleFilter</filter-name>
  <url-pattern>/abc/*</url-pattern>
  <servlet-name>Servlet1</servlet-name>
  <servlet-name>Servlet2</servlet-name>
  <url-pattern>/xyz/*</url-pattern>
</filter-mapping>
...
```

In Code Snippet 7, the `<filter-mapping>` element allows you to add two different URLs and two servlet classes named, **Servlet1** and **Servlet2**, and then map them with the filter named, **MultipleFilter**.

Code Snippet 8 shows the mapping of the filter for the forward calls to all the Servlets.

Code Snippet 8: `web.xml`

```
...
<filter-mapping>
  <filter-name>DispatchFilter</filter-name>
  <servlet-name>*</servlet-name>
  <dispatcher>FORWARD</dispatcher>
</filter-mapping>
...
```

In Code Snippet 8, the `<filter-mapping>` element defines a filter named **DispatchFilter** in the `<filter-name>` element to handle all the forward calls to the Servlets. This is achieved using the `<dispatcher>` element having the value set to **FORWARD**.

7.4 Manipulating Requests and Responses

Apart from performing pre and post-processing operations on the request and response objects, the filters can also wrap the request or response objects in custom wrappers. This helps to intercept the request or response before they can reach their logical destination.

The wrapper classes create the object to capture the request and response before they reach the server and client respectively. Figure 7.10 shows the working of wrapper objects used for processing of requests and responses in Servlets.

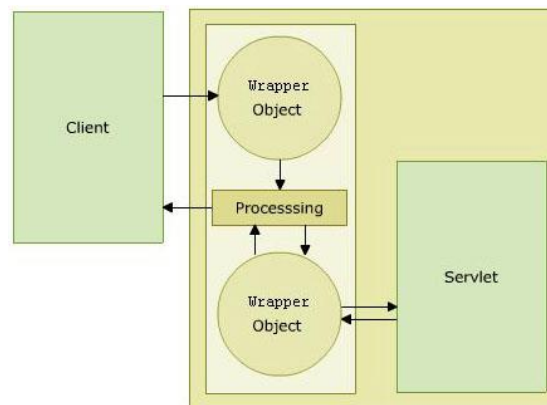


Figure 7.10: Wrapper Objects

The wrapper object generated by the filter implements the `getWriter()` and `getOutputStream()`, which returns a stand-in-stream. The stand-in-stream is passed to the Servlet through the wrapper object. The wrapper object captures the response through the stand-in-stream and sends it back to the filter.

Classes defined by Servlet API for wrapping requests and responses are as follows:

ServletRequestWrapper	ServletResponseWrapper
This class provides a convenient implementation of the <code>ServletRequest</code> interface. The <code>ServletRequest</code> can be sub-classed by developers wishing to send the request to a servlet. To override request methods, one should wrap the request in an object that extends <code>ServletRequestWrapper</code> or <code>HttpServletRequestWrapper</code>.	This class provides a convenient implementation of the <code>ServletResponse</code> interface. The <code>ServletResponse</code> can be subclassed by developers wishing to send the response from a servlet. To override response methods, one should wrap the response in an object that extends <code>ServletResponseWrapper</code> or <code>HttpServletResponseWrapper</code>.

Code Snippet 9 shows how to extend `HttpServletRequestWrapper`.

Code Snippet 9:

```
class MyRequestWrapper extends HttpServletRequestWrapper {
```

```

...
public MyRequestWrapper(HttpServletRequest nested) {
    super(nested);
    ...
}
public void setParams(String key,String value) {
    ...
}
public String getParameter(String name) {
    String response;
    ...
    if (response==null){
        response=super.getParameter(name);
    }
    return response;
}
}

```

Code Snippet 10 shows how to extend `HttpServletRequestWrapper` class.

Code Snippet 10:

```

public class MyResponseWrapper extends
HttpServletRequestWrapper {
    public String toString() {
    ...
    }
    public MyResponseWrapper (HttpServletRequest response){
        super(response);
        ...
    }
    public PrintWriter getWriter(){
        ...
    }
}

```

7.5 Developing a Complete Application

Let us now explore a complete example for a User Login application. Begin by creating a new Dynamic Web Project in Eclipse. Code Snippets 11 to 18 illustrate the code for the application. Code Snippet 11 demonstrates a simple `Login.html` page. This page prompts the user to enter their username and password.

Code Snippet 11: `Login.html`

```

<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">

```

```

<title>Insert title here</title>
</head>
<body>
<form action="Login" method="post">
Enter Username: <input type="text" name="user"><br>
Enter Password: <input type="password" name="pwd"><br>
<input type="submit" value="Login">
</form>
</body>
</html>

```

Code Snippet 12 demonstrates how to create a login page, authenticate the users, and display a LoginSuccess message.

Code Snippet 12: Login.java

```

import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
@WebServlet("/Login")
public class Login extends HttpServlet {
    private static final long serialVersionUID = 1L;
    private final String userID = "admin";
    private final String password = "password";
    public Login() {
        super();
        // TODO Auto-generated constructor stub
    }
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        String user=request.getParameter("user");
        String pwd=request.getParameter("pwd");
        PrintWriter out= response.getWriter();
        if(userID.equals(user) && password.equals(pwd)){
            HttpSession session = request.getSession();
            session.setAttribute("user", "ADMIN");
            Cookie userName = new Cookie("user", user);
            response.addCookie(userName);
            response.sendRedirect("LoginSuccess.jsp");
        }
    }
    else{
        RequestDispatcher rd =

```

```

getServletContext().getRequestDispatcher("/Login.html");
        out.println("<font color=red>Invalid
Login.</font>");
        rd.include(request, response);
    }
}
}

```

Code Snippet 13 demonstrates how to create a login page, authenticate the user at the time of login, and finally, log out the user using a Servlet.

Code Snippet 13: LoginSuccess.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-
8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 5.0 Transitional//EN"
"http://www.w3.org/TR/html5/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=ISO-8859-1">
</head>
<body>
<%
//allow access only if session exists
String user = (String) session.getAttribute("user");
String userName = null;
String sessionID = null;
Cookie[] cookies = request.getCookies();
if(cookies !=null){
for(Cookie cookie : cookies){
    if(cookie.getName().equals("user")) userName =
cookie.getValue();
    if(cookie.getName().equals("JSESSIONID")) sessionID =
cookie.getValue();
}
}
%>
<h3>Hi <%=userName %>, Login successful. Your Session
ID=<%=sessionID %></h3> <br>
User=<%=user %> <br>
<a href="Checkout.jsp">Checkout Page</a>
<form action="Logout" method="post">
<input type="submit" value="Logout" >
</form>
</body>
</html>

```


Code Snippet 14 shows the code for `Logout.java`.

Code Snippet 14: `Logout.java`

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/Logout")
public class Logout extends HttpServlet {
    private static final long serialVersionUID = 1L;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public Logout() {
        super();
        // TODO Auto-generated constructor stub
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        response.setContentType("text/html");
        Cookie[] cookies = request.getCookies();
        if(cookies != null){
            for(Cookie cookie : cookies){
                if(cookie.getName().equals("JSESSIONID")){
                    System.out.println("JSESSIONID="+cookie.getValue());
                    break;
                }
            }
        }
        //invalidate the session if exists
        HttpSession session = request.getSession(false);
        System.out.println("User="+session.getAttribute("user"));
        if(session != null){
            session.invalidate();
        }
        response.sendRedirect("Login.html");
    }
}
```

Code Snippet 15 shows the code for `Checkout.jsp` page.

Code Snippet 15: Checkout.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 5.0 Transitional//EN"
"http://www.w3.org/TR/html5/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<%
String userName = null;
String sessionID = null;
Cookie[] cookies = request.getCookies();
if(cookies !=null){
for(Cookie cookie : cookies){
    if(cookie.getName().equals("user")) userName =
cookie.getValue();
}
}
%>
<h3>Hi <%=userName %>, Click the button for Logout.</h3>
<br>
<form action="Logout" method="post">
<input type="submit" value="Logout" >
</form>
</body>
</html>

```

Code Snippet 16 demonstrates a RequestLogging.java filter.

Code Snippet 16: RequestLogging.java (filter)

```

import java.io.IOException;
import java.util.Enumeration;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import jakarta.servlet.http.*;
@WebFilter("/RequestLogging")
public class RequestLogging implements Filter {
    private ServletContext context;
    public void init(FilterConfig fConfig) throws
ServletException {
        this.context = fConfig.getServletContext();

```

```

        this.context.log("RequestLoggingFilter
initialized");
    }
    public void doFilter(ServletRequest request,
ServletResponse response, FilterChain chain) throws
IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest)
request;
        Enumeration<String> params =
req.getParameterNames();
        while(params.hasMoreElements()){
            String name = params.nextElement();
            String value = request.getParameter(name);
            this.context.log(req.getRemoteAddr() +
"::Request Params::{ "+name+"="+value+"}");
        }
        Cookie[] cookies = req.getCookies();
        if(cookies != null){
            for(Cookie cookie : cookies){
                this.context.log(req.getRemoteAddr() +
"::Cookie::{ "+cookie.getName()+" "+cookie.getValue()+"}");
            }
        }
        // pass the request along the filter chain
        chain.doFilter(request, response);
    }
    public void destroy() {
        //we can close resources here
    }
}

```

Code Snippet 17 demonstrates an Authentication.java filter.

Code Snippet 17: Authentication.java (filter)

```

import java.io.IOException;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebFilter;
import jakarta.servlet.http.*;
@WebFilter("/Authentication")
public class Authentication implements Filter {
    private ServletContext context;
    public void init(FilterConfig fConfig) throws
ServletException {
        this.context = fConfig.getServletContext();
        this.context.log("Authentication Filter started");
    }
    public void doFilter(ServletRequest request,
ServletResponse response, FilterChain chain) throws

```

```

IOException, ServletException {
    HttpServletRequest req = (HttpServletRequest)
request;
    HttpServletResponse res = (HttpServletResponse)
response;

    String uri = req.getRequestURI();
    this.context.log("Requested Resource::"+uri);
    HttpSession session = req.getSession(false);
    if(session == null && !(uri.endsWith("html") ||
uri.endsWith("Login"))){
        this.context.log("Unauthorized ");
        res.sendRedirect("Login.html");
    }else{
        // pass the request along the filter chain
        chain.doFilter(request, response);
    }
}
public void destroy() {
    //close any resources here
}
}

```

Code Snippet 18 demonstrates a web.xml file. The code defines a list of welcome files. A welcome file is a file that is invoked automatically by the server if the user does not specify any file name.

Code Snippet 18: web.xml

```

<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
    <display-name>SF</display-name>
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
    <servlet>
        <servlet-name>Login</servlet-name>
        <servlet-class>Login</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>Login</servlet-name>
        <url-pattern>/Login</url-pattern>
    </servlet-mapping>
    <servlet>

```

```

    <servlet-name>Logout</servlet-name>
    <servlet-class>Logout</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>Logout</servlet-name>
    <url-pattern>/Logout</url-pattern>
  </servlet-mapping>
  <filter>
    <filter-name>RequestLogging</filter-name>
    <filter-class>RequestLogging</filter-class>
  </filter>
  <filter>
    <filter-name>Authentication</filter-name>
    <filter-class>Authentication</filter-class>
  </filter>
  <filter-mapping>
    <filter-name>RequestLogging</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
  <filter-mapping>
    <filter-name>Authentication</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
</web-app>

```

In the application created through Code Snippets 11 to 18, when the user logs in, a session will start, and for each request, the filter checks whether the request is authenticated or not.

Figure 7.11 shows the final Dynamic project created in Eclipse.

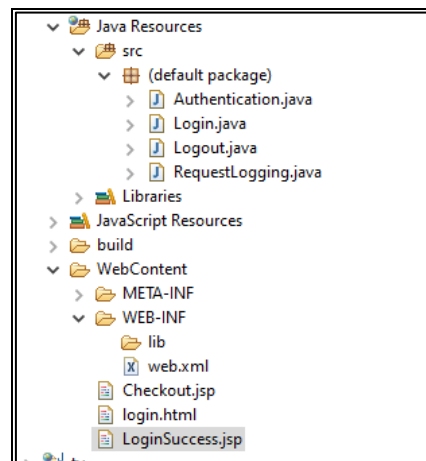


Figure 7.11: Dynamic Project in Eclipse

Save all the files and click **Run As** → **Run on server**. The application will be successfully executed, and one can test the functionality of various pages. User can login, logout, or checkout.

7.6 Introduction to Annotations

There are two approaches adopted in developing Java server-side components namely, imperative programming and declarative programming. The imperative programming approach specifies how to achieve a goal. The declarative programming specifies only the goal, but not the implementation code to achieve that specific goal.

Prior to Java EE 5, the Java EE platform supported the declarative approach where many of the APIs require code to be written in external files such as the deployment descriptors, server-specific configurations, and so on. It would be more convenient if the code to be written in the deployment descriptor is maintained as part of the program itself, but in a declarative way. To avoid writing such kind of unnecessary codes, annotations are used.

Annotations are one of the major advancements from Java EE 5.0 that are used to configure the server components. Using annotation in Java EE makes the standard `application.xml` and `web.xml` deployment descriptors file optional.

7.6.1 Requirement for Annotations

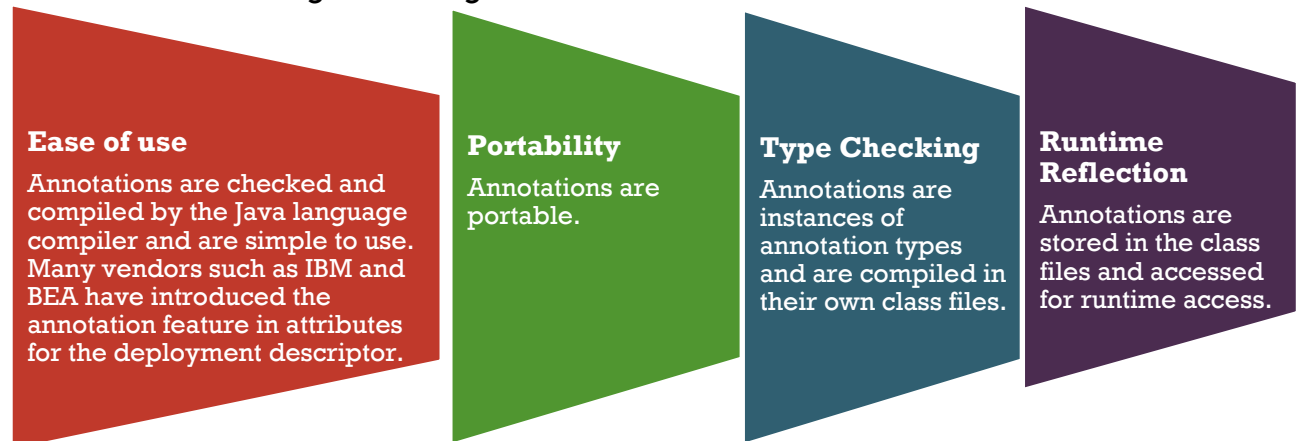
Annotation can be defined as metadata information that can be attached to an element within the code to characterize it. Annotation simplifies the developer's work to a great extent by significantly reducing the amount of code to be written by moving the metadata information into the source code itself.

Annotation can be added to program elements such as classes, methods, fields, parameters, local variables, and packages. Unlike code, annotations are never executed. Annotations are processed when the code containing it is compiled or interpreted by compilers, deployment tools, and so on.

Processing of annotations can result in the generation of code documents, code artifacts, and so on.

7.6.2 Advantages of Annotations

Some of the advantages of using annotations are as follows:



7.6.3 Declaring Annotations

The annotation type is used for defining an annotation and is used when a custom annotation is to be created. An annotation type takes an `at (@)` sign, followed by the interface keyword and the annotation name. Alternatively, an annotation takes the form of an `at (@)` sign, followed by the annotation type. Various standard annotations include `@Deprecated`, `@Override`, `@SuppressWarnings`, `@Documented`, and `@Retention`.

Before using annotation in a Web application, there are some rules and guidelines to be followed by a developer.

They are as follows:

- A field or a method can be assigned to any access qualifier.
- A field or method cannot be static. However, the fields or methods of the main class that have been annotated for injection must be static.
- Resource annotation can be specified in any of the classes or their superclasses.
- Any violation of these rules will generate an error that will result in the logging of message or messages.

7.6.4 Support for Annotation

Servlet API supports different types of annotations to provide information. Some annotation can be used as an alternative to XML entries specified in the deployment

descriptor, `web.xml`. Other annotations act as a request for the container to perform tasks that would otherwise be performed by the Servlet.

The `jakarta.servlet.annotation` package provides annotations to declare Servlets by specifying metadata information in the Servlet class. In addition to this, the `jakarta.servlet.annotation` package also provides annotations to declare filters and listeners.

Figure 7.12 shows how to configure a Servlet by adding annotations to it.

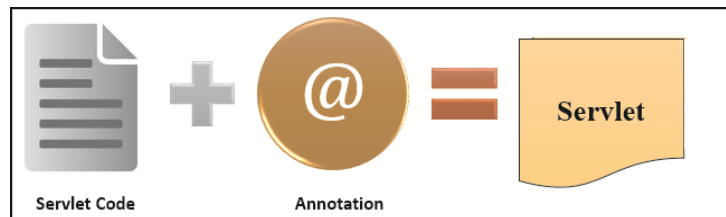


Figure 7.12: Servlet with Annotations

7.6.5 Annotations API

If annotations are used to develop the Web application, there is no requirement of using the deployment descriptor, `web.xml`. You should use a higher version of Eclipse IDE and Web servers to run the Servlets developed using annotations.

The `jakarta.servlet.annotation` package contains a number of annotations Servlets, filters, and listeners. Using these packages, the developers can develop a complete Web application in the annotation.

The `jakarta.servlet.annotation` package provides a different type of annotations as follows:

- Servlet annotations
- Filter annotations
- Listener annotations
- Security annotations

7.6.6 Servlet Annotations

The mapping information of Servlet can be provided in the Servlet file itself, instead of `web.xml`, using annotations. The Servlet mapping annotations are supported with various elements which are optional.

Some of the annotations used by the Servlet are as follows:

➤ **WebServlet**

This annotation is used to provide the mapping information of the Servlet. It is processed by the servlet container at the time of the deployment. The result of processing the annotations provides information about the Servlet to the container such as the Servlet class, specified URL pattern to access the Servlet, and so on.

Table 7.2 shows the attributes of `@WebServlet` annotation.

Attribute Name	Description
<code>name</code>	Specifies the Servlet name. This attribute is optional.
<code>description</code>	Specifies the Servlet description and it is an optional attribute.
<code>displayName</code>	Specifies the Servlet display name, this attribute is optional.
<code>urlPatterns</code>	An array of URL patterns use for accessing the Servlet, this attribute is required and should register one URL pattern.
<code>asyncSupported</code>	Specifies whether the Servlet supports asynchronous processing or not, the value can be true or false.
<code>initParams</code>	An array of <code>@WebInitParam</code> that can be used to pass servlet configuration parameters. This attribute is optional.
<code>loadOnStartup</code>	An integer value that indicates servlet initialization order, this attribute is optional.
<code>smallIcon</code>	A small icon image for the servlet, this attribute is optional.
<code>largeIcon</code>	A large icon image for the servlet, this attribute is optional.

Table 7.2: @WebServlet Annotation

Code Snippet 19 demonstrates how to assign `@WebServlet` annotation to the Servlet class.

Code Snippet 19:

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

@WebServlet(
    name = "webServletAnnotation",
    urlPatterns = {"/hello", "/helloWebApp"},
    asyncSupported = false
)

public class HelloAnnotationServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```

        out.write("<html><head><title>WebServlet
Annotation</title></head>");
        out.write("<body>");
        out.write("<h1>Servlet Hello Annotation</h1>");
        out.write("<hr/>");
        out.write("Welcome" +
getServletConfig().getInitParameter("name"));
        out.write("</body></html>");
        out.close();
    }
}

```

Note that if the Servlet is already configured in the deployment descriptor, then the deployment descriptor will have higher precedence than annotations.

However, this is time-consuming since the container has to find all the annotations. In other words, the container has to scan all the files including the jar files present in the classpath.

To avoid this wastage of time, there is an option to disable annotation using the `metadata-complete` attribute in the `web.xml` file. If one sets the `metadata-complete` attribute value to `true`, then the container will ignore all annotations in the Servlet files. In case, the attribute value is set to `false` or not specified, then by default, it will search for the annotation declaration.

Code Snippet 20 shows the code to disable annotation in the Servlet files.

Code Snippet 20:

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="https://jakarta.ee/xml/ns/jakartaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0" metadata-complete="true">

```

In Code Snippet 20, the `metadata-complete` attribute value is set to `true`, so that the container will disable and ignore all annotations applied in the Servlet files.

➤ `WebInitParam`

This annotation is used to specify the initialization parameters. This can be used with Servlets as well as filters.

Table 7.3 shows the attributes of `@WebInitParam` annotation.

Attribute Name	Description
<code>name</code>	Specifies the names of the initialization parameters.
<code>value</code>	Specifies the value for the initialization parameters.

Table 7.3: Methods of `@WebServlet` Annotation

Code Snippet 21 shows how to apply the `@WebInitParam` annotation to the servlet.

Code Snippet 21:

```
import jakarta.servlet.annotation.WebInitParam;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
@WebServlet(name = "webServletAnnotation", urlPatterns = {
    "/hello",
    "/helloWebApp" }, asyncSupported = false, initParams = {
    @WebInitParam(name = "name", value = "admin"),
        @WebInitParam(name = "param1", value =
"paramValue1"),
        @WebInitParam(name = "param2", value =
"paramValue2") })
public class HelloAnnotationServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        PrintWriter out = response.getWriter();
        out.write("Welcome" +
            getServletConfig().getInitParameter("name"));
    }
}
```

The `initParams` attribute of `@WebServlet` is used that can be used to pass Servlet configuration parameters.

7.6.7 Listener Annotation

The Servlet API provides different types of listener interfaces to handle different events. For example, to handle `HttpSession` lifecycle events, the `HttpSessionListener` can be used. To declare a class as a listener class, the `@WebListener` annotation can be used.

The `@WebListener` annotation is used to register following types of listeners:

Context Listener	<code>jakarta.servlet.ServletContextListener</code>
Context Attribute Listener	<code>jakarta.servlet.ServletContextAttributeListener</code>
Servlet Request Listener	<code>jakarta.servlet.ServletRequestListener</code>
Servlet Request Attribute Listener	<code>jakarta.servlet.ServletRequestAttributeListener</code>
Http Session Listener	<code>jakarta.servlet.http.HttpSessionListener</code>
Http Session Attribute Listener	<code>jakarta.servlet.http.HttpSessionAttributeListener</code>

Code Snippet 22 shows the implementation of the class that will log the session created or destroyed.

Code Snippet 22:

```
. . .
import jakarta.servlet.annotation.WebListener;
import jakarta.servlet.http.HttpSessionEvent;
import jakarta.servlet.http.HttpSessionListener;
@WebListener
public class MySessionListener implements HttpSessionListener{
    @Override
    public void sessionCreated(HttpSessionEvent se) {
        System.out.println("Session Created:: ID="+
se.getSession().getId());
    }
    @Override
    public void sessionDestroyed(HttpSessionEvent se) {
        System.out.println("Session Destroyed:: ID="+
se.getSession().getId());
    }
}
```

Code Snippet 23 shows the Servlet class to create the `HttpSession` object.

Code Snippet 23:

```
@WebServlet("/MyServlet")
public class MyServlet extends HttpServlet {
```

```

        private static final long serialVersionUID = 1L;
        protected void doGet(HttpServletRequest request,
            HttpServletResponse response) throws ServletException,
            IOException {
            HttpSession session = request.getSession();
            session.invalidate();
        }
    }
}

```

The code creates an `HttpSession` object to store the session object. The session object is also destroyed to check the invocation of listeners.

7.6.8 Filter Annotations

In a Web application, the `@WebFilter` annotation is used to define a filter. It is specified in a class and contains metadata regarding the filter being declared. At least one URL pattern must be specified with the annotated filter. The `urlPatterns` or `value` attribute on the annotation accomplishes this. The other attributes are optional and have default settings.

Code Snippet 24 shows the creation of filter using `@WebFilter` annotation.

Code Snippet 24:

```

...
@WebFilter (value="/hello",
            initParams=({
                @WebInitParam(name="message", value="Servlet
says: ")
            }))
public MyFilter implements Filter {
    private FilterConfig _filterConfig;
    public void init(FilterConfig filterConfig)
        throws ServletException
    {
        _filterConfig = filterConfig;
    }
    public void doFilter(ServletRequest req,
                        ServletResponse res,
                        FilterChain chain)
        throws ServletException, IOException
    {
        PrintWriter out = res.getWriter();
        out.print(_filterConfig.getInitParameter("message"));
    }
    public void destroy() {
        // destroy
    }
}

```

7.6.9 Security Annotations

Security annotations can be used to specify permission on the methods of the Servlet class.

Some of the security annotations are as follows:

➤ **ServletSecurity**

This annotation is used in a Servlet class to specify security constraints to be enforced by a Servlet container on HTTP protocol messages.

➤ **HttpConstraint**

This annotation is used within the `ServletSecurity` annotation to represent the security constraints to be applied to all HTTP protocol methods.

For example, to restrict the user from accessing any other method in the Servlet class, when logged in the role as 'User', then the annotation will be:

```
@ServletSecurity(@HttpConstraint(rolesAllowed = "User"))
```

➤ **HttpMethodConstraint**

This annotation is used within the `ServletSecurity` annotation to represent security constraints on specific HTTP protocol messages.

For example,

```
@ServletSecurity(httpMethodConstraint=@HttpMethodConstraint(
value = "POST"), emptyRoleSemantic = EmptyRoleSemantic.DENY)
```

7.6.10 Dependency Injection Annotations

Dependency Injection (DI) is a mechanism that allows the container to inject the dependent objects in the application components. The application components can be dependent on many resources to perform the necessary operations.

For example, a data source object to connect to a database, a JavaBean object, an enterprise bean object, and so on.

The dependent objects are injected before the lifecycle methods are invoked and before any reference is made to the dependent object.

Servlet API supports various dependency injection annotations for different types of resources on which Servlet can be dependent. These are as follows:

- `@Resource`
- `@RJB`
- `@PersistenceUnit`
- `@PersistenceContext`
- `@WebServiceRef`

Code Snippet 25 demonstrates the code that can be used to obtain a reference for the data source in the Servlet class.

Code Snippet 25:

```
// Injecting the Datasource named EmployeeDB
@Resource(name="jdbc/EmployeeDB")
private jakarta.jdbc.DataSource myDB;
// Creating connection with the data source
Connection con;
// Obtaining the connection
con = myDB.getConnection();
```

The `@Resource` annotation is used for accessing data sources registered in JNDI service on the server.

7.7 Uploading Files with Servlet

A very common requirement of a Web application is to upload the files on the server. Files can be uploaded on the server using following two ways:

1. Client-side file upload
2. Server-side file upload

7.7.1 Client-Side Uploading

To upload the files on Web pages, there are some changes required in the HTML form. The requirements are as follows:

- a. Create the input element with the attribute `type="file"`.
- b. The form method supported for file uploading will be `POST`, as `GET` method is not supported in file uploading.
- c. Use the attribute named `enctype` with the `form` element. This element specifies how the form data will be encoded when the form is submitted for processing on the server.

Table 7.4 shows the values that can be assigned to the `enctype` attribute.

Value	Description
<code>application/x-www-form-urlencoded</code>	This is the default value if not specified. The value ensures that all characters are encoded before they are sent to the server.
<code>multipart/form-data</code>	The value is provided when the form is using a file upload control. It does not encode the characters before sending them in the request.
<code>text/plain</code>	The value converts the spaces with the '+' symbols, however, no other special characters are encoded.

Table 7.4: Values of enctype Attribute

Code Snippet 26 shows the `uploadfile.jsp` page containing the HTML form element to upload the file on the server.

Code Snippet 26:

```
<html>
  <head>
    <title>File Uploading Form</title>
  </head>
  <body>
    <h3>File Upload:</h3>
    Select a file to upload: <br />
    <form action="UploadServlet" method="post"
      enctype="multipart/form-data">
      <input type="file" name="file" size="50"/>
      <br />
      <br />
      <input type="submit" value="Upload File" />
    </form>
  </body>
</html>
```

The file element displays the **Browse** button on the form which allows the user to select the file from the system. When the user clicks Upload File, the HTTP request will pass the uploaded file to the Servlet, which will write the uploaded file to the folder on the server.

Figure 7.13 shows the output of the JSP page.

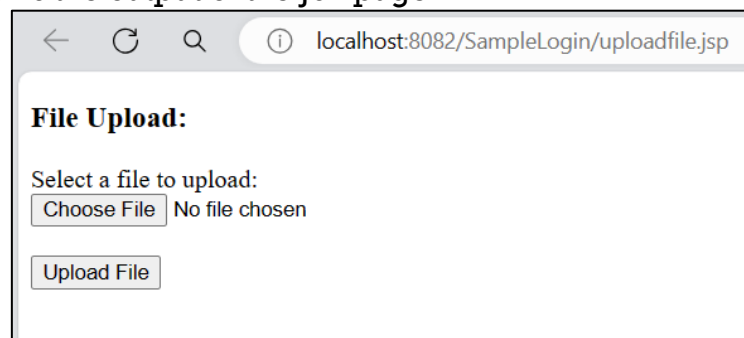


Figure 7.13: JSP File with Single File Upload

7.7.2 Server-side File Upload

In server-side file upload, the files are uploaded using the Servlet class. The Servlet specification provides the classes that are used to handle encrypted form data.

The `HttpServletRequest` class includes two methods for handling the file upload.

These are as follows:

Part getPart(String name)	Collection<Part> getParts()
This method extracts the individual parts of the file and returns each as a Part object. The Part object belongs to the Part interface and provides methods that can be used to extract information from the returned Part object. The information includes: the name, the size, and the content-type of the file returned as a part. It also provides methods for querying the header content submitted with the HTTP request object, writing the part to the external disk, and deleting the part.	This method returns a collection of Part objects which can be iterated to extract an individual Part object from the collection.

Code Snippet 27 demonstrates the Servlet code that uploads the file on the server.

Code Snippet 27:

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.MultipartConfig;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.Part;
import java.io.File;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Collection;
@MultipartConfig()
public class UploadServlet extends HttpServlet {
    public void doPost(HttpServletRequest request,
        HttpServletResponse response) throws IOException,
        ServletException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("You are trying to upload:<br>");

        try {
            Collection<Part> parts = request.getParts();
            for (Part part : parts) {
                if (part.getContentType() != null) { // Check
for file parts
                    String fileName =
part.getSubmittedFileName();
                    // Change the path to your dedicated folder
```

```

        File file = new
File("C:/Users/DELL/OneDrive/Documents/" + fileName);
//Change path for file upload
        part.write(file.getAbsolutePath()); //
Use absolute path for clarity
        out.println("Uploaded Filename: " +
fileName + "<br>");
    }
}
} catch (IllegalStateException e) {
    // Handle non-multipart request
    out.println("No file uploaded.");
} catch (Exception e) {
    e.printStackTrace(out);
}
}
}

```

Code Snippet 28 shows a fragment of the corresponding `web.xml` file for configuring the Servlet.

Code Snippet 28:

```

...
<servlet>
    <servlet-name>UploadServlet</servlet-name>
    <servlet-class>UploadServlet</servlet-
class>
</servlet>
<servlet-mapping>
    <servlet-name>UploadServlet</servlet-name>
    <url-pattern>/UploadServlet</url-pattern>
</servlet-mapping>

    <welcome-file-list>
        <welcome-file>uploadfile.jsp</welcome-file>
    </welcome-file-list>
</web-app>

```

The same `uploadfile.jsp` file that was created in Code Snippet 26 can be used for this example. Figure 7.14 shows the output of the JSP page.

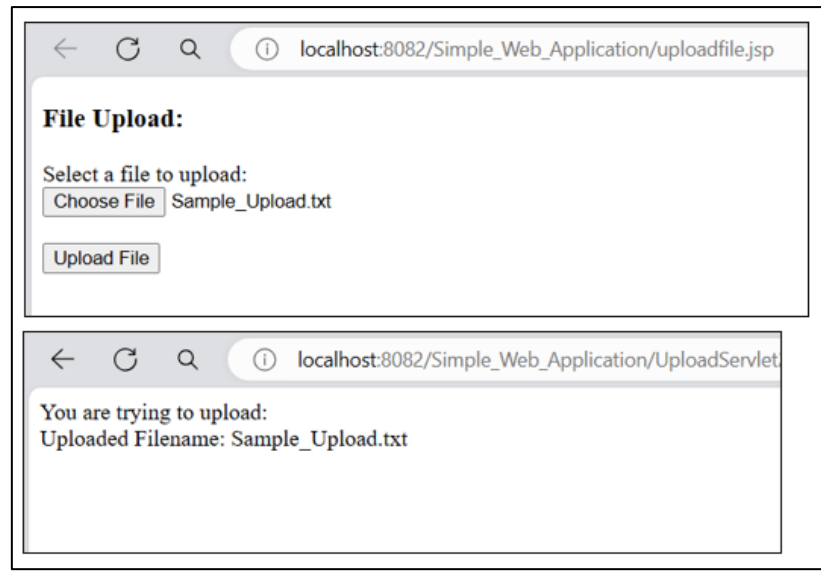


Figure 7.14: UploadServlet Output

7.7.3 MultipartConfig Annotation

The `jakarta.servlet.annotation.MultipartConfig` annotation is specified on a Servlet class. It provides file information to the server while uploading its content.

The information provided by the `MultipartConfig` annotation is as follows:

- The maximum size of the data that can be uploaded on the server
- The default location where the file will be uploaded on the server

Table 7.5 lists the attributes of the `MultipartConfig` annotation.

Attribute	Description
<code>fileSizeThreshold</code>	Specifies the size of the file in bytes. The default size of the file is 0.
<code>maxFileSize</code>	Specifies the maximum file limit allowed to upload the file on the server. If the size of the file to be uploaded is greater than the specified limit, then throws <code>IllegalStateException</code> .
<code>maxRequestSize</code>	Specifies the maximum file size in bytes to be accepted for encrypted file data sent in the request.
<code>location</code>	Specifies the default location for the file upload on the server.

Table 7.5: Methods of MultipartConfig Annotation

Code Snippet 29 shows the attributes of `@multipartconfig` annotation that are applied to the Servlet.

Code Snippet 29:

```
@WebServlet(name = "UploadServlet2", urlPatterns =
{"/Upload"})
```

```
@MultipartConfig(fileSizeThreshold = 100, maxFileSize = 700 *
1024, maxRequestSize = 50 * 1024 * 1024, location = "C:/tmp")
public class UploadServlet2 extends HttpServlet {
    . . .
    . . .
}
```

7.8 Server Push Technology in Servlet 6.0

The Server push, also referred to as push technology, enables the server to push the Website resources to the user even before the user requests it. This is useful for server to know the resources that are required in advance. Server Push feature was introduced as part of HTTP/2 protocol specification. Though HTTP/3 also supports push technology, it handles server push differently.

How does the Server push the resources?

The servlet standard 6.0 creates a new `PushBuilder` object by calling the `HttpServletRequest.newPushBuilder()` method. The purpose of this method is to provide a way to initiate server-push requests, regardless of the specific protocol used for communication. Therefore, this method works independently of the protocol being used (HTTP/1.1, HTTP/2, or HTTP/3).

Once the push is created the push builder object invokes the `push()` method to start the process of moving the content toward the client.

The user just has to input the path of the resource which is supposed to be pushed.

```
var builder = req.newPushBuilder();
...
builder.path(s).push();
```

Server push is the responsibility of the `PushBuilder` interface. In a framework such as Spring MVC, users can inject `PushBuilder` as an argument of the methods marked by `@RequestMapping`.

Maven dependency for server push is demonstrated in Code Snippet 30.

Code Snippet 30:

```
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.0.0</version>
</dependency>
```

Summary

- The filterer acts as an interface between the client and servlet inside the server. It intercepts the request or response to and from the server. More than one filter can also exist between a client and a servlet. This is called a filter chain.
- The Filter API is a part of the servlet package. It contains interfaces namely, Filter, FilterConfig, and FilterChain.
- All the information about the filter is described within the `<filter>` element inside the `web.xml` file. The filter also is to be mapped to the servlet for which it will function. This is done by assigning the URL name and servlet name inside the `<filter-mapping>` element.
- The request and response to and from the servlet are manipulated by the filter. The wrapper object of `RequestWrapper` or `ResponseWrapper` classes created by the filter intercepts the request or response and sends it to the filter.
- Annotation can be defined as metadata information that can be attached to an element within the code to characterize it. Annotation can be added to program elements such as classes, methods, fields, parameters, local variables, and packages.
- The `jakarta.servlet.annotation` package contains a number of annotations, Servlets, filters, and listeners. Apart from this, Servlet API also supports various dependency injection annotations.
- The file can be uploaded on the server using following two ways namely, Client-side file upload and Server-side file upload.
- The `jakarta.servlet.annotation.MultipartConfig` annotation is specified on a Servlet class to provide file information to the server while uploading its content.
- Server Push allows the server to push the Website resources to the client preemptively.

Check Your Progress

1. Which of these characteristics of a filter are true?

(A)	Filter was introduced as a Web component in Java servlet Specification version 2.3.
(B)	A filter optimizes the time it takes to send back a response from Web server.
(C)	A logging and auditing filter allows users to log in to a Web application.
(D)	Filter optimizes the available bandwidth on the network there by reducing the network traffic and in turn increasing the efficiency of all Web-based services.
(E)	An authentication filter tracks the activities of users of a Web application.

2. Match the functions to their appropriate methods.

	Function		Method
(A)	This method returns the names of the initialization parameter as an enumeration of String objects.	(1)	<code>getInitParameter (name)</code>
(B)	This method returns the name of the filter defined in the <code>web.xml</code> or deployment descriptor file inside the Web application.	(2)	<code>getInitParameterNames ()</code>
(C)	This method returns the Servlet Context object used by the caller to interact with its servlet container and filter.	(3)	<code>getFilterName ()</code>
(D)	This method returns the value of the named initialization parameter as a string.	(4)	<code>getServletContext ()</code>

3. Which of these statements is incorrect?

(A)	The <code>FilterChain</code> interface provides an object through the Web container.
(B)	The <code>doFilter (request, response)</code> method invokes the next filter in the filter chain.
(C)	The object invokes the previous filter in a filter chain starting from the first filter from a particular end.
(D)	If the calling filter is the last filter in the chain, it will invoke the Web resource, such as JSP and servlet.

4. Match the features to their appropriate elements.

	Feature		Element
(A)	This element inside the deployment descriptor holds the name of the filter to which a URL pattern or servlet is mapped.	(1)	<icon>
(B)	This specifies the location of a small or a large image used to represent the filter in a GUI tool within a Web application.	(2)	<init-param>
(C)	It holds the name and value pair of an initialization parameter of the filter.	(3)	<filter-name>
(D)	This element specifies the name of a servlet whose request and response will be modified by the filter.	(4)	<display-name>
(E)	This is the short name of the filter, i.e. intended to be displayed by GUI tools.	(5)	<servlet-name>

5. Which of these statements is correct about altering a response using filters?

(A)	The request wrapper object is generated by the filter by extending to <code>ServletResponseWrapper</code> class.
(B)	The stand-in-stream is then, passed to the servlet through the response wrapper object.
(C)	The stand-in-stream allows the servlet to close the response generated by it.
(D)	The request object overrides the <code>getWriter()</code> and <code>getOutputStream()</code> methods.

6. Which of the following annotations will help to create a Servlet?

(A)	@WebListener
(B)	@WebServlet
(C)	@MultipartConfig
(D)	@WebInitparam

Answers

1.	C
2.	B
3.	C
4	D
5.	B
6.	B

Try It Yourself

1. You are working on a Student Management project. Your task is to create a document submission portal where students will upload and submit their pending documents. Perform the given operations.
 - a) Create a `studentlogin.jsp` page where a student will log in using `student_name` and password.
 - b) Create `student_login_servlet.java` page and implement a servlet to authenticate student based on their credentials.
 - c) Create `student_login_sucess.jsp` page.
 - On this page display the message **You have logged in successfully and display** the `session id` as well.
 - Create **Document Upload** button. After clicking this button student will be redirected to `Document_Upload.jsp` page.
 - d) Create `Document_Upload.jsp` page. On this page create a form to upload a document.
 - Add a **Logout** button, it will redirect to `studentlogin.jsp` page.
 - e) Create an `UploadServlet.java` file to handle file upload through an HTTP POST request.
 - f) Create a `Logout.java` servlet to handle student logout functionality after clicking the `Logout` button.
2. Create a `Student.java` servlet page using `@WebServlet` to handle HTTP GET requests. The servlet should respond with a personalized greeting message for students using initialization parameters defined in the annotation. Refer to Table 7.6.

Student Name	Message
Alex	Student_Name + your dedication to learning is impressive.
Ross	
Anna	

Table 7.6: Greeting Message

Database Access and Event Handling

Welcome to the Session, **Database Access and Event Handling**.

This session explains the concept of establishing database connection in the Servlet using Java Database Connectivity (JDBC) API. The session also explains Object Relational Mapping (ORM) and how to persist entities in the database using Java Persistence (JPA) API. Further, the session explains the event-handling mechanism in Servlets. It also explains the difference between JDBC and JPA.

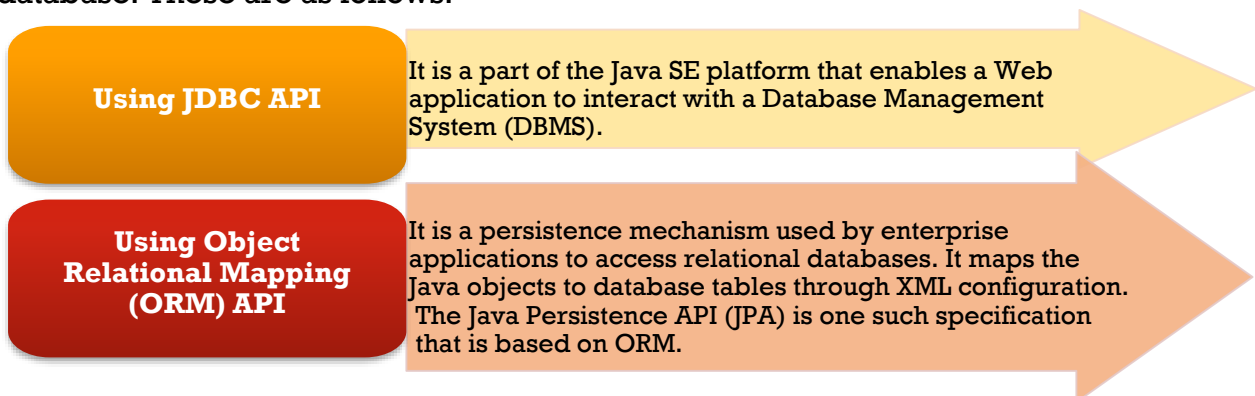
In this session, you will learn to:

- Explain database handling in Servlets using JDBC 4.3
- Describe JPA and its role
- Describe connecting database using JPA
- Explain the significance of session handling and session events
- Identify different types of listener interfaces used in Servlets
- Explain JDBC versus JPA

8.1 Introduction

One of the most common operations performed by a Servlet is namely, storing and retrieving database information.

Java provides various mechanisms using which data can be accessed from the database. These are as follows:



8.2 JDBC API

The JDBC API provides classes and interfaces that allow a Web application to access and perform operations on the databases.

The operations that can be performed on the databases include:

- Connecting to the databases

- Processing SQL statements
- Building result sets
- Executing the parameterized statements
- Invoking callable statements

Figure 8.1 shows the layers of the JDBC architecture.

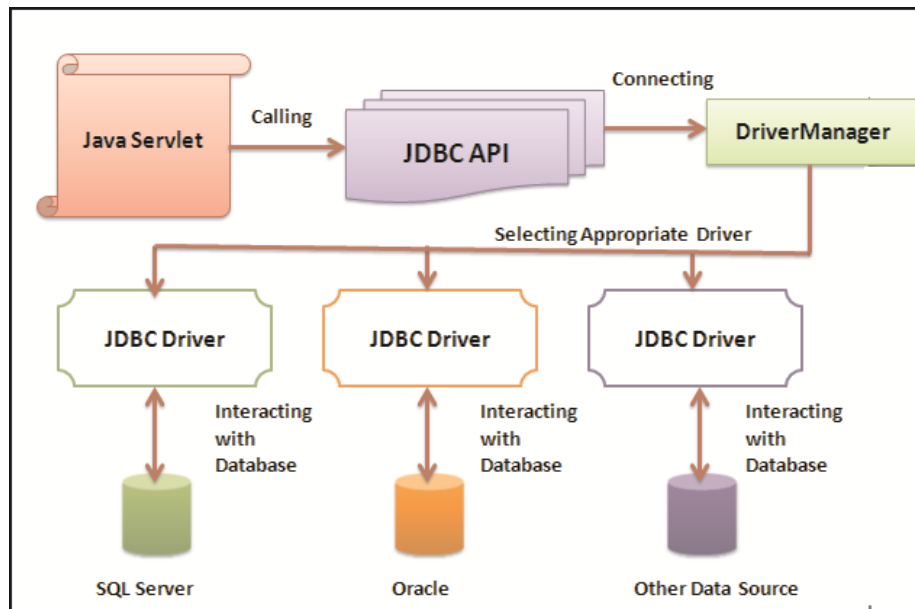


Figure 8.1: JDBC Architecture

The Servlet uses JDBC APIs to access the database. The JDBC API uses a driver manager, which ensures that the correct driver is used for accessing a specific database. A JDBC driver translates standard JDBC calls into a client's API-specific calls. These API calls facilitate communication with the database.

The application developer using JDBC API does not have to bother about repackaging the application, if the database is changed or upgraded.

Note: The JDBC drivers are database specific. For SQL Server 2019 or 2022, developers should download the latest driver from following link:

<https://docs.microsoft.com/en-us/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server?view=sql-server-ver15>

For MySQL, following link can be used:

<https://dev.mysql.com/downloads/connector/j/>

8.2.1 JDBC Driver Types

There are four different types of JDBC drivers. These drivers are as follows:

Type 1:

JDBC-ODBC Bridge Plus ODBC Driver – The JDBC-ODBC bridge provides access to the database through standard ODBC libraries. The driver translates the standard JDBC calls to the corresponding ODBC calls.

This driver is a part of the `sun.jdbc.odbc` package. Its major limitation is that the JDBC driver's capabilities depend on the capabilities of the ODBC driver. This driver is reliant on the native library.

Type 2:

Native API Partly Java Driver – The native APIs partly Java driver calls the native database library that accesses the database. This driver also requires the native database libraries to be installed and configured on the client's machine. These drivers connect to a data source using a native client library specific to just that data source.

Type 3:

JDBC-Net Pure Java Driver – This is a pure Java driver that communicates with the JDBC middle tier on the server through a proprietary network protocol. The Java Servlet sends a JDBC call to the middleware through the JDBC driver without translation. The middleware then, sends the request to the database.

Type 4:

Native-Protocol Pure Java Driver – The type 4 drivers implement a proprietary database driver. It provides database connectivity through the standard JDBC APIs and does not require any native libraries on the client machine. These drivers are the fastest.

8.3 Accessing Database Using JDBC

The Servlet interacts with the database using JDBC APIs and follows certain steps in sequence:

1. Registering the JDBC driver
2. Establishing a database connection
3. Executing an SQL statement
4. Processing the results
5. Closing the database connection

8.3.1 Registering the JDBC Driver

Registering the driver notifies the driver manager that a JDBC driver is available. The JDBC automatically registers itself with the driver manager as soon as it is loaded. The `DriverManager` class is used to establish the database connection. This is done by using the `DriverManager.getConnection()` method, which checks all available drivers for the eligibility to make a connection.

8.3.2 Establishing a Database Connection

The database connection can be established when a valid driver is loaded. The JDBC connection is identified by a database URL.

The syntax for specifying the URL is as follows:

Syntax:

`jdbc:<subprotocol>:<database_identifier>`

The first part specifies that JDBC is used for establishing the connection. The `<subprotocol>` is the name of any valid database driver or any other database connectivity that is being requested. The `<database_identifier>` is the logical name of the database connection that maps to the physical database. It is basically the Data Source Name (DSN) that holds the information to establish a connection.

The database connection is established using the `getConnection()` method of the `DriverManager` class in JDBC API.

Code Snippet 1 demonstrates an example of connecting to an SQL Server database from the Servlet.

Code Snippet 1:

```
public class JDBCServlet implements HttpServlet
{
    ...
    try {
        // Establishing connection to SQL Server 2019
        // database
        String url = "jdbc:sqlserver://WORKHORSE\\MYEXPRESS;
            encrypt=true;trustServerCertificate=true;databaseName=AdventureWorks";
        Connection cn = DriverManager.getConnection(url,
            "sa", "playware");
        ...
    } catch (ClassNotFoundException ex) {
        ...
    }
    ...
}
```

In Code Snippet 1, the `conn` object is created using the `getConnection()` method of the `DriverManager` class.

8.3.3 Executing an SQL Statement

Once the database connection is established, several SQL statements such as retrieving, inserting, updating, or deleting of data can be performed on the database. In order to query the database, the `Statement` interface of JDBC API can be used. The `Statement` object reference can be created using the `createStatement()` method on the obtained connection.

For example, following statement is used to create the `Statement` object:

```
Statement stat = conn.createStatement();
```

The `Statement` interface provides methods to perform various database operations including SQL query execution. The `executeQuery()` method of the `Statement` interface accepts SQL query as an argument and returns a `ResultSet` object.

For example, following statement shows the creation of a query using the `Statement` object.

```
ResultSet rs = stat.executeQuery("SELECT * FROM Product");
```

In case of any error, the `executeQuery()` method throws an exception – `SQLException`. The `ResultSet` object contains all the rows returned by the SQL query.

8.3.4 Processing the ResultSet

`ResultSet` is an interface used to describe `ResultSet` objects. This object represents data in a tabular form that is retrieved by executing an SQL query.

Some of the features of the `ResultSet` object are as follows:

- `ResultSet` follows the iterative pattern.
- `ResultSet` object maintains a cursor that points to the current row in the result table.

There are three types of `ResultSet` as follows:

TYPE_FORWARD_ONLY	TYPE_SCROLL_INSENSITIVE	TYPE_SCROLL_SENSITIVE
In this, the cursor moves only in the forward direction in the resultset.	In this, the cursor scrolls forward and backward. The resultset is not sensitive to changes, which have been made by other users to the database.	In this, the cursor scrolls forward and backward. The resultset is sensitive to changes, which have been made by other users to the database.

Code Snippet 2 demonstrates the code that iterates through the resultset.

Code Snippet 2:

```
// Access column values
while(rs.next()) {
    rs.getInt ("product_id");
    rs.getString ("product_name");
}
```

Iterating the `ResultSet` includes `next()` method which moves the cursor forward one row. This returns `true`, if the cursor is now positioned on a row and `false`, if the cursor is positioned after the last row.

Code Snippet 3 demonstrates the code that is used to update the resultset.

Code Snippet 3:

```
rs.updateString("product_name", "Beverages");
rs.updateRow();
```

8.3.5 Closing the Connection

It is particularly important to close database-related connections. In case we do not close them, there will be unclosed connections to the database, and it could eventually run out of connections.

The statement, `conn.close()` closes the connection explicitly allows garbage collector to recollect memory as early as possible.

8.4 Enhancements in JDBC

JDBC has been enhanced and the new version is JDBC 4.3. It has been upgraded with new methods that are useful when connecting Servlets and JSPs to databases.

JDBC 4.3 is one of the most recent additions to the JDBC API. The JDBC 4.3 API includes the `java.sql` and `javax.sql` packages as well as number of other enhancements. The `javax.sql` package extends the JDBC API from a client-side API to a server-side API and it is a crucial component of Java and Jakarta Enterprise Editions.

In JDBC 4.3, `DatabaseMetaData` is also improved as it distributes the datasets over many databases and can be stored on multiple machines.

New Connection methods in JDBC 4.3 are as follows:

<code>java.sql.connection. beginRequest()</code>	<code>java.sql.connection. endRequest()</code>
Informs the driver that a request, which is a separate unit of work, is about to start.	Informs the driver that a request, which is a separate unit of work, is about to end.

JDBC 4.3 API `Statement` includes several new functions in `java.sql`. Some of these are as follows:

- `enquoteIdentifier`
- `enquoteLiteral`

- `enquoteNCharLiteral`
- `SimpleIdentifier`.

The new classes and interfaces in JDBC 4.3 are as follows:

- `java.sql.ConnectionBuilder`
- `java.sql.ShardingKey`
- `java.sql.ShardingKeyBuilder`
- `javax.sql.PooledConnectionBuilder`
- `jakarta.transaction.TransactionManager`

8.5 Java Persistence API (JPA)

Java Persistence API or JPA is a lightweight, POJO-based Java framework. Plain Old Java Objects (POJOs) are average or ordinary Java objects, not bound to any framework or restrictions. JPA helps to persist Java Objects to the relational database. JPA is an ORM technology that persists entities in the database. The main benefit of using an ORM technology is that it supports mapping of object-oriented software to the database schema through configuration mapping. The configuration mapping information about the entity in the application domain to the database table is specified either in XML file or through annotations.

JPA provides the persistence providers in the JPA application. JPA converts data between incompatible types in object-oriented programming to the appropriate underlying database types. This provides the database and schema independence.

Generally, converting an object-oriented domain schema into relational schema is a time-consuming process. ORM software provides this mapping in less time and requires less or almost no coding. It supports a simple mapping of Java objects to database tables.

Figure 8.2 shows the ORM technology such as JPA support for Java objects to database tables.

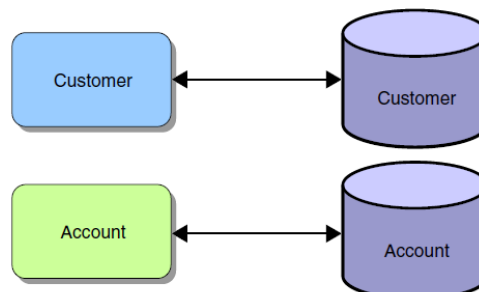


Figure 8.2: Mapping of Java Objects to Tables

Entities are annotated POJOs. They are mapped to the tables in a relational database. While working with entities, it is required to declare an entity explicitly. This is done to distinguish it from other regular Java objects that might be used in the application.

Java annotations or XML is used to define the mapping of entity to existing database tables.

The JPA specification provides the persistence manager, which manages the lifecycle of an entity in the persistence context. It also provides the interfaces to execute both static and dynamic queries.

8.5.1 JPA Components

Figure 8.3 shows the primary components of JPA API.

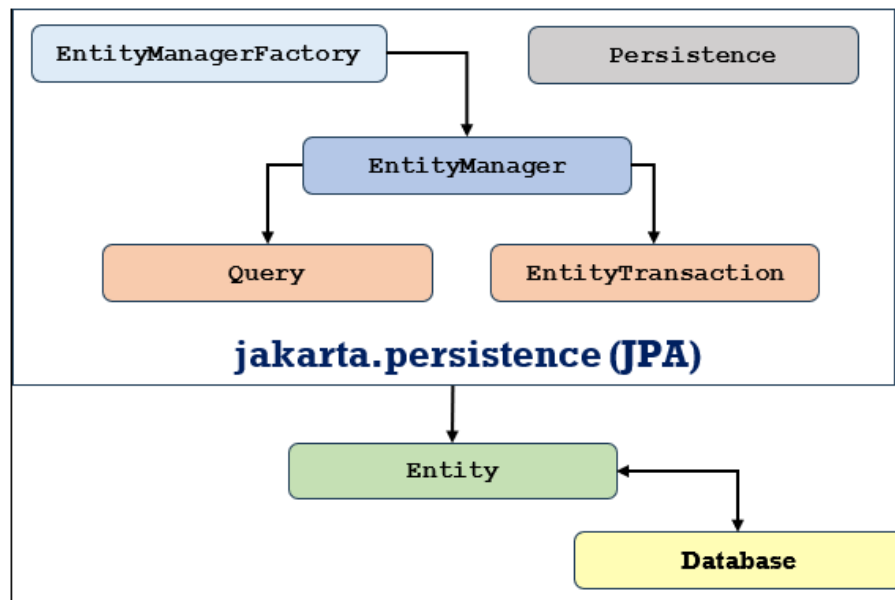


Figure 8.3: Primary Components of JPA API

The components of the JPA specification, including various classes and interfaces, which are as follows:

Persistence (Class)	The <code>persistence</code> provides static methods to obtain an instance of <code>EntityManagerFactory</code> .
EntityManager (Interface)	The <code>EntityManager</code> interface is responsible for managing the lifecycle of the <code>Entity</code> instances. It provides methods to perform CRUD operations on <code>Entity</code> objects. It also interacts with <code>EntityTransaction</code> objects to ensure data consistency.

EntityManagerFactory (Class)	The <code>EntityManagerFactory</code> is responsible for creating instances of <code>EntityManager</code> .
Entity (Class)	The <code>Entity</code> class represents a database table. Each instance of this class represents one row in the database. The attributes of the class represent the database columns. It acts as a bridge between the application and the database.
EntityTransaction (Class)	The <code>EntityTransaction</code> class provides methods to manage transactions on <code>Entity</code> objects.
Query (Interface)	The <code>Query</code> interface is responsible for creating and executing database queries against entities. It is used to retrieve, update, or delete data from a relational database.

8.5.2 Entities in JPA

In JPA, persistent objects are referred as Entities. Entities are plain old Java objects that are persisted to relational databases or legacy systems.

Typically, an entity represents a table in the database and each instance of an entity maps to a row in that table. The entities store data as fields or properties and have getter/setter methods associated with each of the fields. For example, the employee details can be stored in an entity named **Employee** entity.

Some of the requirements of entity class are as follows:

- The entity class must be annotated with `@Entity` annotation defined in `jakarta.persistence` package.
- The class must have a no-argument constructor with public or protected access modifier.
- The properties or methods of the entity class should not be declared as final. The class also should not be declared as final.
- If the entity instance is passed as parameter to a Session Bean remote interface, then it must implement the `Serializable` interface.
- Entities can be inherited from entity or non-entity classes.
- The instance variables of entity class must be declared as private or protected and are accessed only by the methods present in the class.
- Each entity instance must be identified by a unique identifier or a primary key. The primary key helps the client to access the entity instance. The `@Id` annotation applied on the entity property specifies that the property is a primary key.

Creating instances of this class using the `new()` keyword will not create an `Employee` class object in the database. An instance of `EntityManager` is required to create the entity in the database.

8.5.3 Managing Entities in the Persistence Context

A persistence context represents a set of managed entity instances that exist in a particular data store. It is a connection between the entity instances present in the memory and the database. When the entity instance is newly created, it does not have any persistence context associated with it. At this stage, even the database does not have information about them.

When entities are persisted or retrieved from the database, they are associated with the persistence context. To manage the entity instances in the persistence context, an interface named `EntityManager` API is used. The `EntityManager` interface is defined in `jakarta.persistence` package and is used to interact with the persistence context. The `EntityManager` interface provides methods to synchronize the state of the entity instance to the database.

Figure 8.4 shows the working of `EntityManager` in JPA.

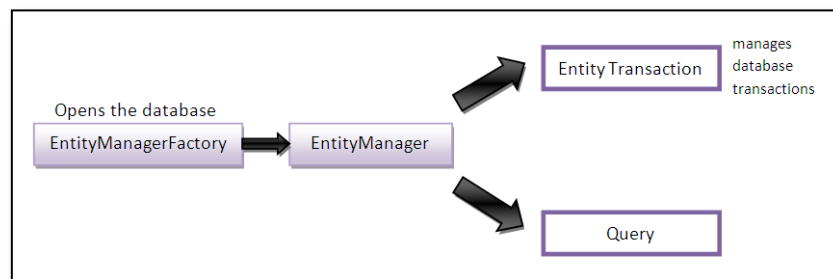


Figure 8.4: EntityManager in JPA

Table 8.1 lists some of the methods defined in the `EntityManager` interface.

Method	Description
<code>public void persist (Object entity)</code>	The method saves an entity into the database and makes the entity managed.
<code>public <T> T merge (T entity)</code>	The method merges an entity to the <code>EntityManager</code> 's persistent context and returns the merged entity.
<code>public void remove (Object entity)</code>	The method removes an entity from the database.
<code>public void flush()</code>	The method synchronizes the state of an entity in the <code>EntityManager</code> 's persistent context with the database.

Table 8.1: Methods in EntityManager Interface

8.5.4 Packaging and Deploying Entity Classes

The packaging and deploying of an entity class is done using persistence units. A persistence unit is a set of class mapped to a particular database. This persistence unit is defined in a special descriptor file named `persistence.xml`. This file contains logical grouping of entity classes, their metadata mappings, and configurations related to a database.

Configuration details in the `persistence.xml` are used while obtaining the `EntityManager` instance in the client program. The `EntityManager` instance accordingly persists the entity instances in the database.

Code Snippet 4 displays the code for a `persistence.xml` file developed as part of an enterprise application.

Code Snippet 4: `persistence.xml`

```
...
<?xml version="1.0" encoding="UTF-8" ?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence">
<persistence-unit name="intro" />
</persistence>
...
```

In Code Snippet 4, the `persistence.xml` file can contain one or more `<persistence-unit>` elements under the root element, `<persistence>`. Each `<persistence-unit>` element defines the configuration for a single data source. Each `<persistence-unit>` element is identified by the `name` attribute. The `name` attribute is mapped to the `unitName` attribute of the `@PersistenceContext` annotation used in the client program.

Some of the attributes defined in the `<persistence-unit>` element are as follows:

<code><description></code>	It describes the persistence unit and is optional.
<code><provider></code>	It must be present in the Java environment or when the application requires a provider specific behavior.
<code><transaction-type></code>	It has a value either as Java Transaction API (JTA) or <code>RESOURCE_LOCAL</code> or by default, the value is JTA.
<code><jta-data-source>/<non-jta-data-source></code>	It is used for specifying the Java Naming and Directory Interface (JNDI) name of the data source. JNDI is used by the persistence provider.
<code><mapping-file></code>	It contains a list of one or more additional XML files that are used for O/R mapping. The mapping file is used to list the entity classes that are available in the persistence unit.

<properties>

It specifies the configuration properties that are vendor-specific for the persistence unit. Any properties that are not recognized by the persistence provider are ignored.

8.5.5 Accessing Database Using JPA

Consider an example of inserting customer records into a database. The Web application will contain an Entity class to represent Customer table and a Servlet class to perform database operation with JPA.

Step by step process to accomplish this is as follows:

1. To begin with, create a table named `Customer` in MySQL database named `Test` as shown in Figure 8.5.

Name	Type
customerid	int(11)
firstname	varchar(50)
lastname	varchar(100)

Figure 8.5: Customer Table with Three Columns

2. Download following jar files into an appropriate location on the computer:
 - Eclipselink-2.7.14.jar
 - jakarta.persistence-api-2.2.3.jar
 - mysql-connector-java-8.0.21.jar

Note: We use Javax Persistence instead of Jakarta Persistence because the Eclipse 2023 version supports JPA 2.2. To access JPA 3.0x the recommended approach is to use Eclipse 2024 or later.

3. Open Eclipse IDE and create a Dynamic Web Project named **JPADemoNew**.
4. Right-click the project created and select **Properties**.
5. Click **Project Facets**.
6. Click **JPA** option and choose JPA version as 2.2 as shown in Figure 8.6.

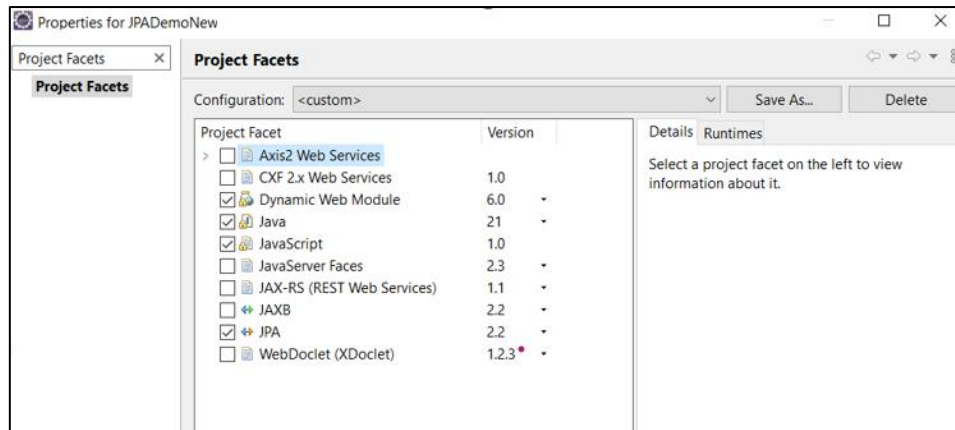


Figure 8.6: Project Facets Page

7. Click **Further configuration available** at the bottom of the dialog box as seen in Figure 8.7.

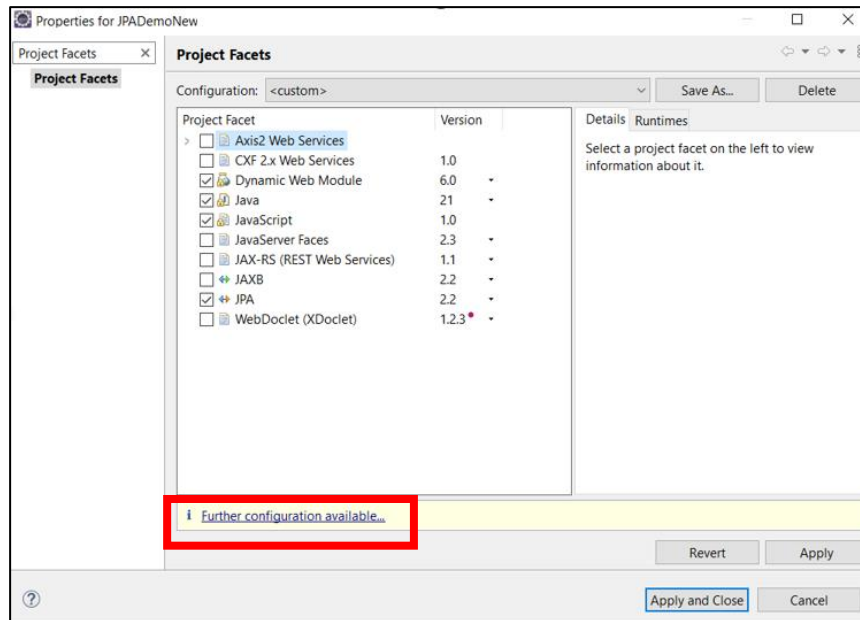


Figure 8.7: Enabling and Configuring JPA

The dialog box shown in Figure 8.8 is displayed.

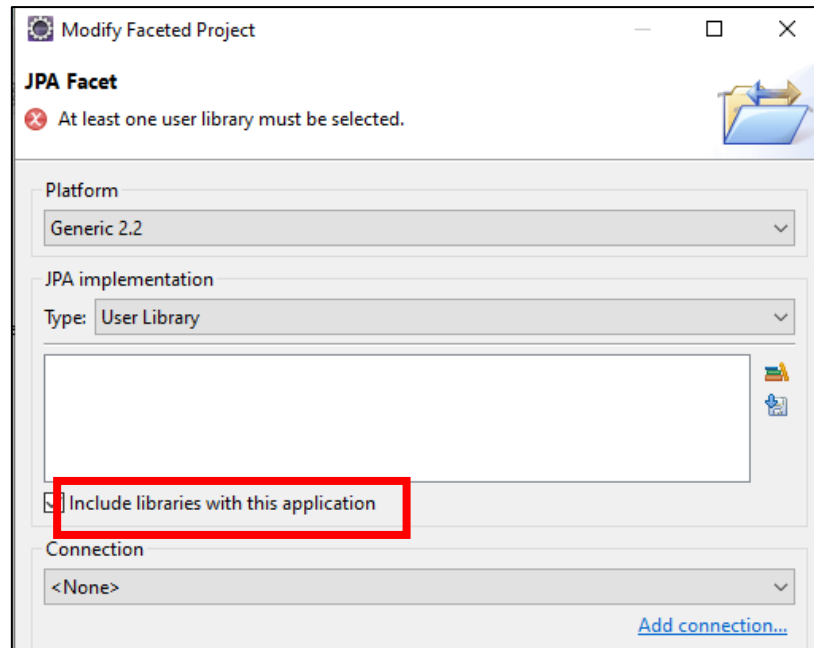


Figure 8.8: User Libraries Option

8. Click **Manage Libraries** → **New** as shown in Figure 8.9. Give a name such as TestLib. Now, the user library is ready for adding jar files.

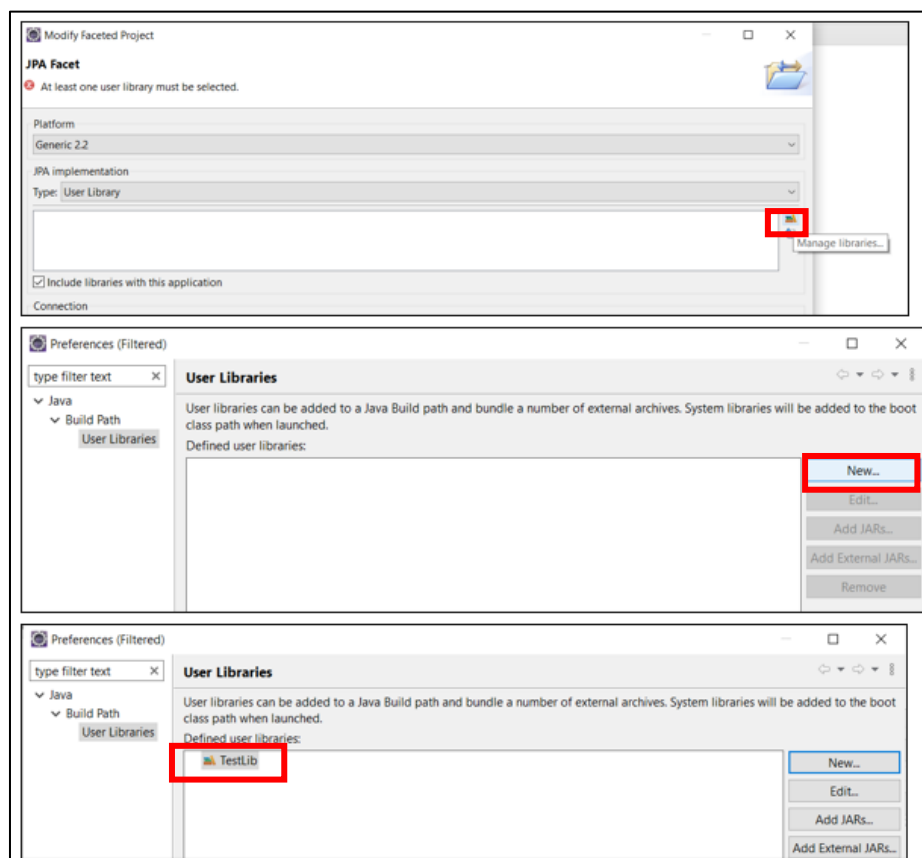


Figure 8.9: Manage Libraries

9. Click **Add External JARs** and add all the jar files that were downloaded earlier as shown in Figure 8.10.

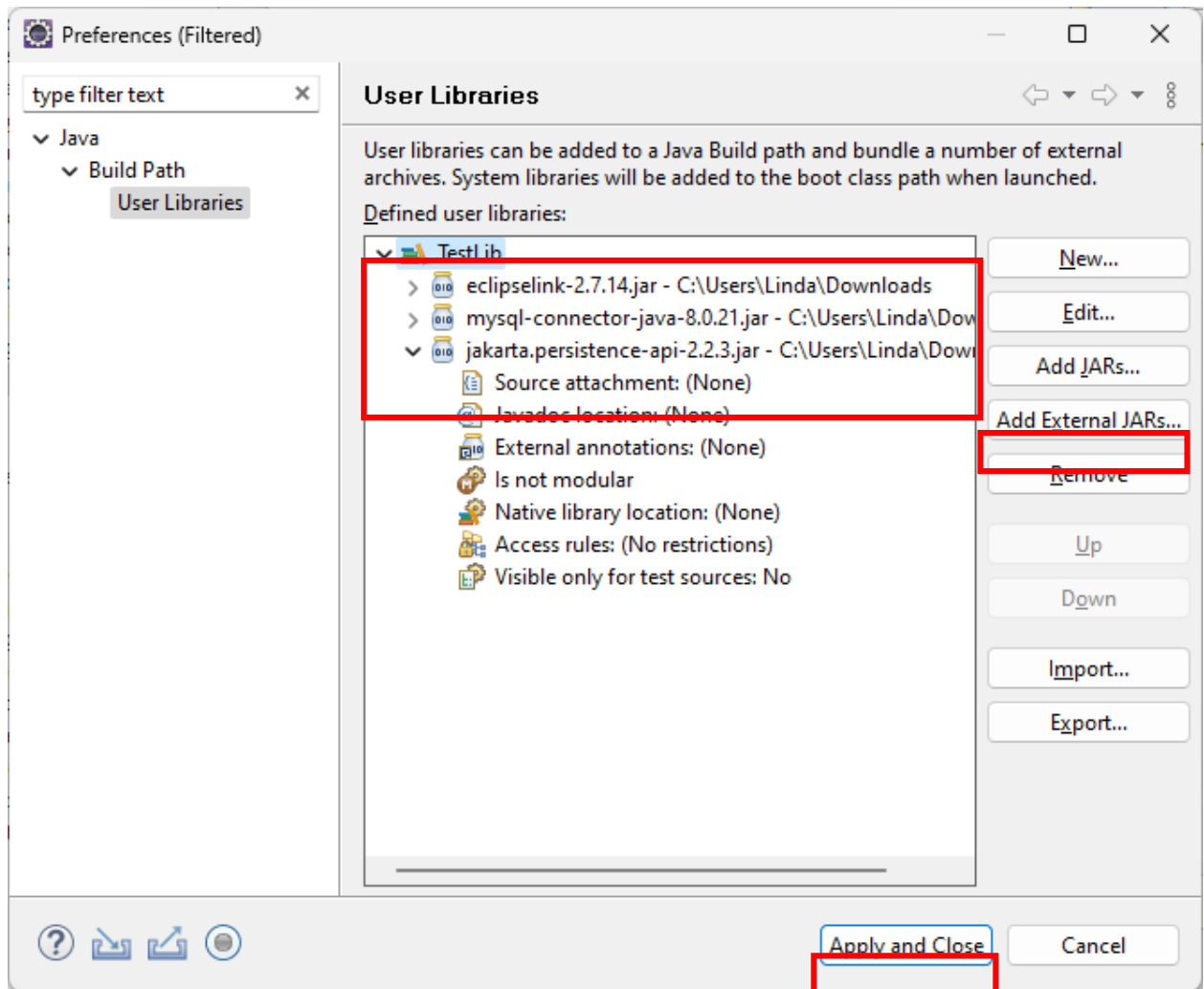


Figure 8.10: User Libraries

10. Click **Apply and Close**.
11. Now, click **Add connection** as shown in Figure 8.11.

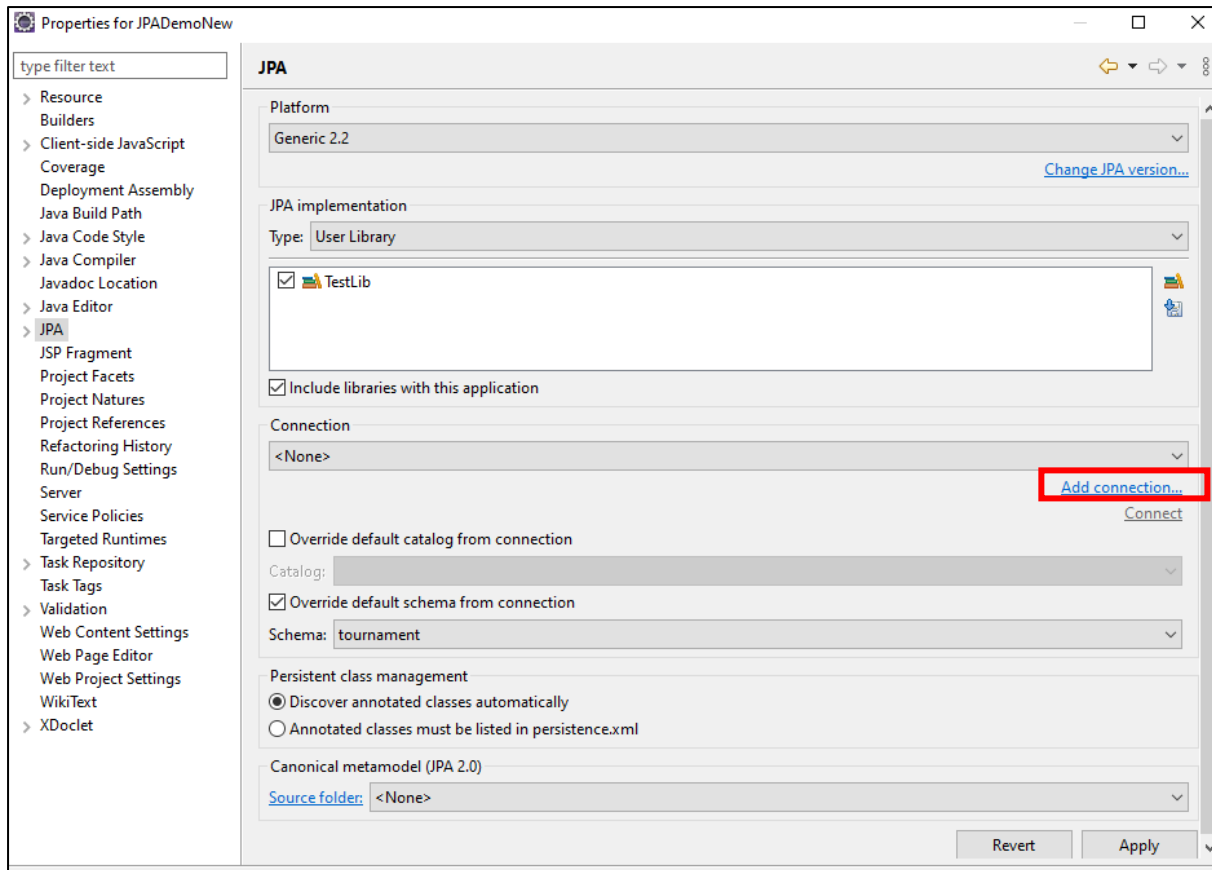


Figure 8.11: Specifying Database Connection

12. Select MySQL, as shown in Figure 8.12.

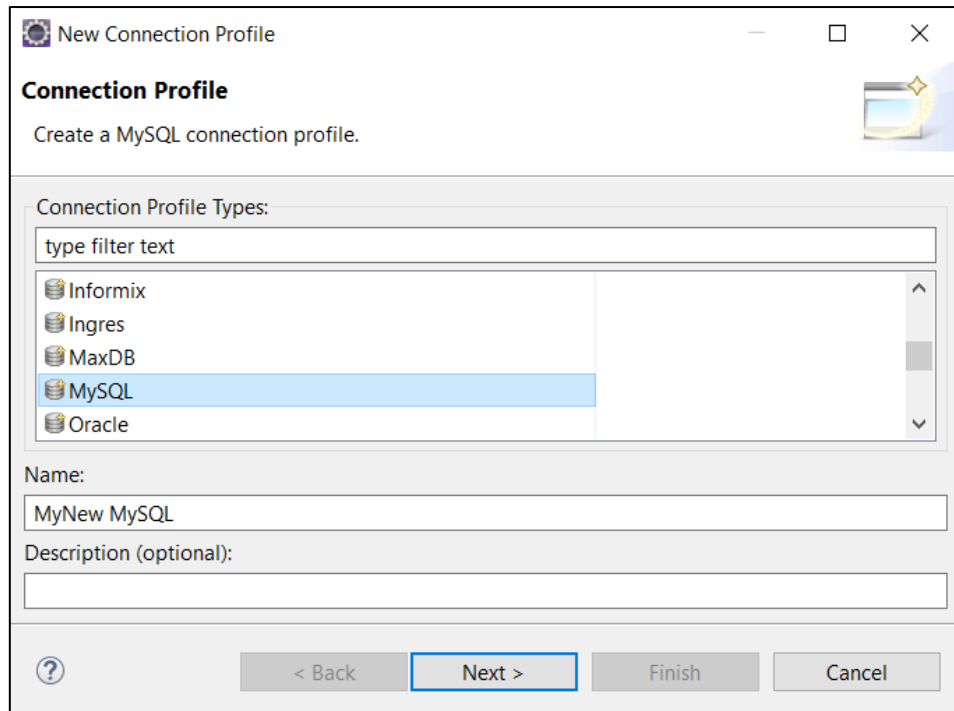


Figure 8.12: New Connection Profile Page

13. Change the database name as test in the URL and provide the connection credentials. Click **Finish** as shown in Figure 8.13.

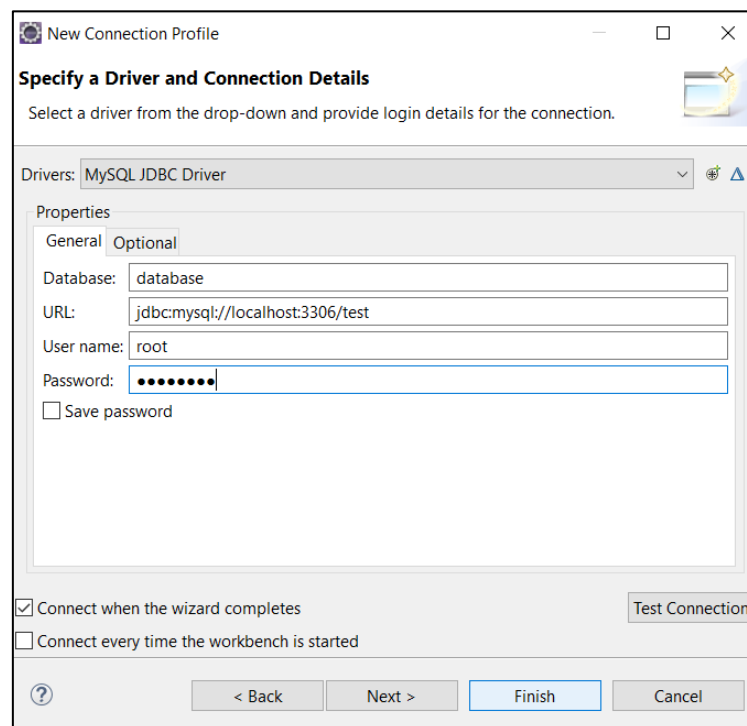


Figure 8.13: Specify Connection Details

14. Click **Apply and Close** again to apply the settings and close the dialog box.

Code Snippet 5 shows the code for `persistence.xml` which should be saved under **java\META-INF** folder of the project. Replace the contents of the existing file with the contents of Code Snippet 5. Change the password for the MySQL login in the property `javax.persistence.jdbc.password`.

Code Snippet 5: `persistence.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.2"
xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd">
    <persistence-unit name="JPADemoNew">

        <provider>org.eclipse.persistence.jpa.PersistenceProvide
r</provider>
        <class>Customer</class>
        <properties>
            <property name="javax.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>
            <property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test"/>
            <property name="javax.persistence.jdbc.user"
value="root"/>
            <property name="javax.persistence.jdbc.password"
value="<password>"/>
        </properties>
    </persistence-unit>
</persistence>
```

The database connection properties are specified here as well as the entity class name.

Now, you will create the entity class named `Customer`. Code Snippet 6 shows the code for the entity class.

Code Snippet 6: Customer.java

```

import java.io.Serializable;
import javax.persistence.*;
@Entity
@Table(name="customer")
public class Customer implements Serializable {
    private static final long serialVersionUID = 1L;
    //Column Mapping
    @Id
    @Column(name="customerid")
    private int customerId;
    @Column(name="firstname")
    private String firstName;
    @Column(name="lastname")
    private String lastName;

    public Customer() { }
    public int getCustomerId() {
        return this.customerId;
    }
    public void setCustomerId(int customerId) {
        this.customerId = customerId;
    }
    public String getFirstName() {
        return this.firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return this.lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}

```

The code defines the entity class `Customer`, which is mapped to the `Customer` table, column by column. Getter and setter methods are created for each column.

Next, create the servlet. Code Snippet 7 shows the code for `TestServlet`.

Code Snippet 7: TestServlet.java

```

import java.io.IOException;
import javax.persistence.EntityManager;

```

```

import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import javax.persistence.PersistenceUnit;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.PrintWriter;
public class TestServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @PersistenceUnit
    private EntityManagerFactory entityManagerFactory=null;
    public TestServlet()
    {
        super();
    }
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        entityManagerFactory =
            Persistence.createEntityManagerFactory("JPADemoNew")
;
        EntityManager entityManager =
            entityManagerFactory.createEntityManager();
        try
        {
            var trans = entityManager.getTransaction();
            if(!trans.isActive()) {
                trans.begin();
                Customer customer = new Customer();
                customer.setCustomerId(35);
                customer.setFirstName("Sarah");
                customer.setLastName("Polley");
                entityManager.persist(customer);
                trans.commit();
                PrintWriter pw = response.getWriter();
                pw.println("<html>");
                pw.println("<head></head>");
                pw.println("<body>");
                pw.println("Record is inserted");
                pw.println("</body>");
                pw.println("</html>");
            }
        }
        catch(Exception ex)
        {
            response.sendError(1, ex.getMessage());
        }
    }
}

```

```

        finally {
            //close the entity manager to release any
resources
            //held up by the persistence provider
            if(entityManager != null) {
                entityManager.close();
            }
        }
    }
}

```

EntityManagerFactory opens the database when it is constructed. An EntityManagerFactory supports the instantiation of EntityManager instances. EntityManagerFactory construct multiple EntityManager instances for a specific database by managing resources efficiently and should be closed when the application ends.

The EntityManager instance represents the following:

- A Remote connection to remote database server in client server mode.
- A local connection to a local database file in embedded mode.

Database transactions are managed by the EntityTransaction interface. As shown in Code Snippet 7, every EntityManager holds a single attached EntityTransaction instance. Here, trans represents the instance.

persist() and commit() methods enable to persist the data from the servlet to the database and save it (commit).

Code Snippet 8 shows the code to be added in web.xml.

Code Snippet 8: web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd
http://xmlns.jcp.org/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
    <display-name>JPADemoNew</display-name>
    <servlet>
        <description></description>
        <display-name>TestServlet</display-name>
        <servlet-name>TestServlet</servlet-name>
        <servlet-class>TestServlet</servlet-class>
    </servlet>
    <servlet-mapping>

```

```

    <servlet-name>TestServlet</servlet-name>
    <url-pattern>/TestServlet</url-pattern>
  </servlet-mapping>
</web-app>

```

The final directory structure is shown in Figure 8.14.

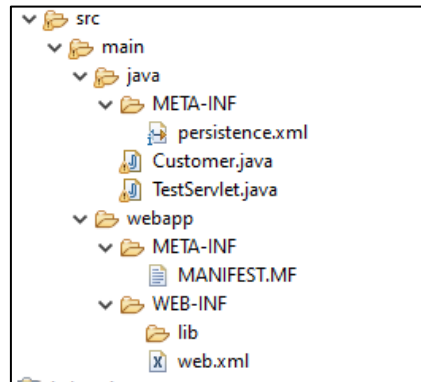


Figure 8.14: Project Structure

Run the application on the server. The record will be successfully inserted into the Customer table. Figure 8.15 shows the output when the application is run with internal Web browser of Eclipse.

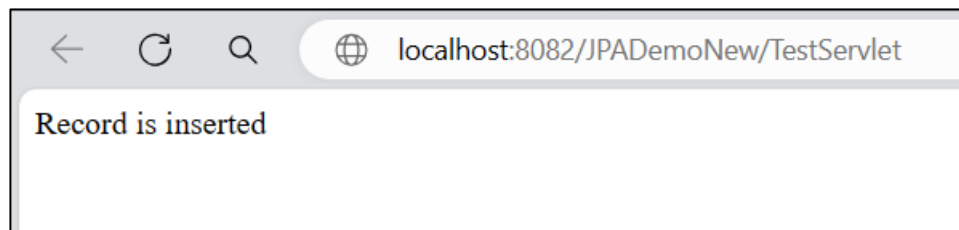


Figure 8.15: Output

Consider another example of inserting records of players in a tournament from a database. The Web application containing JSP pages, Entities, and Servlet classes is created for performing operations on the players' details stored in the database.

Create a table named `Players` in MySQL database named `Test` with column names such as `PlayersID`, `Position`, `PlayerName`, and `Team`.

Code Snippet 9 shows the code for `addPlayers.jsp`.

Code Snippet 9: `addPlayers.jsp`

```

<%@ page language="java" contentType="text/html; charset=UTF-
8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>

```

```

        <h1>New Player record</h1>
        <form id="addNewPlayersForm" action="TestServlet"
method="post">
            <table>
                <tr>
                    <td>id</td>
                    <td><input type="text" id="id" name="id"
/></td>
                </tr>
                <tr>
                    <td>Rank</td>
                    <td><input type="text" id="rank"
name="rank" /></td>
                </tr>
                <tr>
                    <td>Player</td>
                    <td><input type="text" id="playerName"
name="playerName" /></td>
                </tr>
                <tr>
                    <td>Team</td>
                    <td><input type="text" id="team"
name="team" /></td>
                </tr>
            </table>
            <input type="submit" id="CreateRecord"
value="Create Record" />
        </form>
    </body>
</html>

```

Code Snippet 10 shows the code for web.xml.

Code Snippet 10: web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd
http://xmlns.jcp.org/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
    <display-name>JPADemoNew</display-name>
    <welcome-file-list>
        <welcome-file>addPlayers.jsp</welcome-file>
    </welcome-file-list>
</web-app>

```



```

</welcome-file-list>
<servlet>
  <description></description>
  <display-name>TestServlet</display-name>
  <servlet-name>TestServlet</servlet-name>
  <servlet-class>TestServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>TestServlet</servlet-name>
  <url-pattern>/TestServlet</url-pattern>
</servlet-mapping>
</web-app>

```

Code Snippet 11 shows the `persistence.xml` file.

Code Snippet 11:

```

<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.2"
xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd">
<persistence-unit name="JPADemoNew">
  <class>Players</class>
  <properties>
    <property name="jakarta.persistence.jdbc.driver"
value="com.mysql.jdbc.Driver"/>
    <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/tournament"/>
    <property name="jakarta.persistence.jdbc.user"
value="root"/>
    <property name=
"jakarta.persistence.jdbc.password" value=""/>
  </properties>
</persistence-unit>
</persistence>

```

Code Snippet 12 shows the entity class named `Players`.

Code Snippet 12: `Players.java`

```

import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.Column;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity

```

```

@Table(name = "Players")
public class Players implements Serializable {
    private static final long serialVersionUID = 1L;
    //Column Mapping
    @Id
    @Column(name = "PlayerID")
    private int id;
    @Column(name = "Position")
    private String rank;
    @Column(name = "PlayerName")
    private String playerName;
    @Column(name = "Team")
    private String team;
    // no-args constructor
    public Players() {
    }
    // parameterized constructor
    public Players(int id, String rank, String playerName,
String team) {
        this.id = id;
        this.rank = rank;
        this.playerName = playerName;
        this.team = team;
    }
    public int getID() {
        return this.id;
    }
    public String getRank() {
        return this.rank;
    }
    public String getPlayer() {
        return this.playerName;
    }
    public String getTeam() {
        return this.team;
    }
}

```

Code Snippet 12 defines a JPA entity class. To define an Entity class involves declaring and annotating the class.

Code Snippet 13 shows the code for adding new players using a Servlet.

Code Snippet 13: TestServlet.java

```

import java.io.IOException;
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import javax.persistence.PersistenceUnit;

```

```

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.PrintWriter;

public class TestServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @PersistenceUnit
    private EntityManagerFactory entityManagerFactory =
null;
    public TestServlet() {
        super();
    }
    protected void doPost(HttpServletRequest request,
HttpServletResponse response)
        throws ServletException, IOException {
        entityManagerFactory =
Persistence.createEntityManagerFactory("JPADemoNew");
        EntityManager entityManager =
entityManagerFactory.createEntityManager();
        try {
            var trans = entityManager.getTransaction();
            if (!trans.isActive()) {
                trans.begin();
                // Assign the user data from form
                String idval =
request.getParameter("id");
                int id = Integer.parseInt(idval);
                String rank =
request.getParameter("rank");
                String playerName =
request.getParameter("playerName");
                String team =
request.getParameter("team");
                System.out.println(playerName);
                System.out.println(rank);
                System.out.println(id);
                System.out.println(team); // player
instance
                Players player = new Players(id, rank,
playerName, team);
                // Fix the insert query
                entityManager.persist(player);
                trans.commit();
                PrintWriter pw = response.getWriter();
                pw.println("<html>");
                pw.println("<body>");

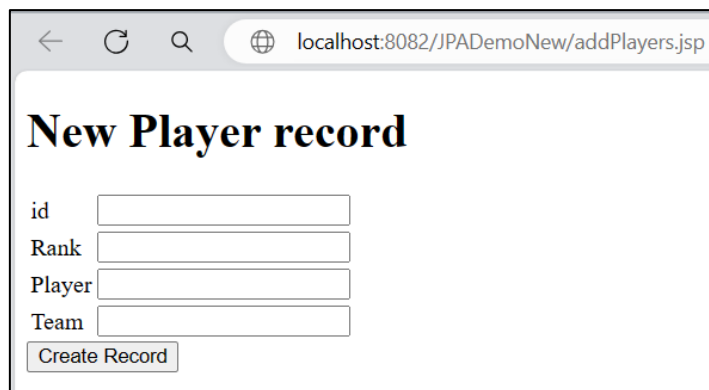
```

```

        pw.println("<h1>Record inserted</h1>");
        pw.println("</body>");
        pw.println("</html>");
    }
} catch (Exception ex) {
    ex.printStackTrace(); // for debugging
    response.sendError(500, "Error inserting data:
" + ex.getMessage());
} finally {
    //close the entity manager to release any
resources
    //held up by the persistence provider
    if (entityManager != null) {
        entityManager.close();
    }
}
}
}
}

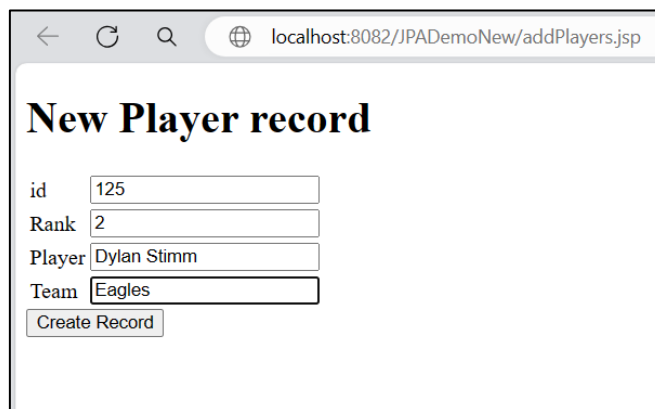
```

The application can now be executed to verify that data entered through the form is inserted into the database table. Figures 8.16 to 8.18 depict the respective outputs.



The screenshot shows a web browser window with the address bar displaying 'localhost:8082/JPADemoNew/addPlayers.jsp'. The page content features a heading 'New Player record' followed by four labeled text input fields: 'id', 'Rank', 'Player', and 'Team'. Below these fields is a button labeled 'Create Record'.

Figure 8.16: Initial JSP Page



This screenshot shows the same 'New Player record' form as Figure 8.16, but with data entered into the input fields. The 'id' field contains '125', 'Rank' contains '2', 'Player' contains 'Dylan Stimm', and 'Team' contains 'Eagles'. The 'Create Record' button remains at the bottom.

Figure 8.17: Data Entered in the Form

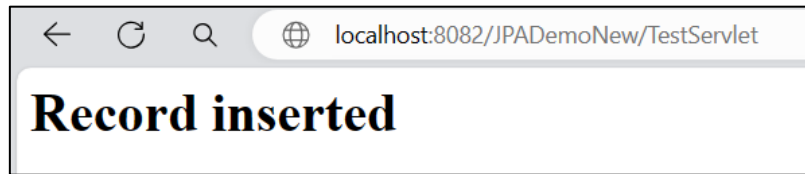


Figure 8.18: Success Message Returned by the Servlet

Upon viewing the table, it will be seen that the data from the form is inserted into the table.

8.6 DAO Pattern for Databases

The DAO pattern provides the easy maintenance of the applications by separating the business logics from the database access. In DAO pattern, the implementation of data access based on JDBC API is encapsulated in the DAO classes. Thus, providing the independence and increasing flexibility between the layers in the application development. The business tier in the Web application includes business service classes, domain object classes, and the DAO classes. The business service uses the DAO to perform the data access functions such as inserting or retrieving an object from the database.

8.7 JDBC versus JPA

Java can connect to databases using JDBC or JPA. The distinctions between **JDBC** and **JPA** are as shown in Table 8.2.

JDBC	JPA
JDBC is database-dependent.	JPA is database-independent.
JDBC uses queries based on database.	JPA can use single query for different databases.
Development is slower as developers do have to write queries.	Productivity is good because developers do not have to write queries.
Does not support POJO model.	Supports POJO model programming.
Java cache is required.	Uses too levels of cache for better performance.
Full SQL query is written to associate database tables.	Annotations are used to create associations, such as one-to-one, one-to-many.

Table 8.2: JDBC Vs JPA

Despite the differences, JPA still uses JDBC APIs. JSP is an abstraction layer used between JDBC calls and database. This is done for ease of programming. JPA uses object-oriented principles for coding leading to better development. Also, with JPA database mapping errors are reduced as JPA handles the interaction between database and Java.

8.8 Session Event Handling

A session is a collection of HTTP requests, between a client and a Web Server. Session handling is used to maintain the state of a user till the lifetime of the session. Event-driven programs are made up of functions which listen and handle events. For example, act of clicking a mouse button. In Jakarta EE, the Web container transfers control to event handlers, that is, event listeners to listen events related to Servlets, JSP, and so on.

For example, when users visit an Online Shopping Website, they add some items to the shopping cart and log off to pay later. When they again login into the shopping Website, they would desire that their shopping cart show the previously selected items. In such a condition, the server has to keep track of the previous session to show the selected items. Now, users just have to pay for the selected items to get them delivered. This is called session handling or tracking.

8.8.1 Listener Types

The `jakarta.servlet` package defines the listener interfaces. These interfaces are of three types as shown in Figure 8.19.

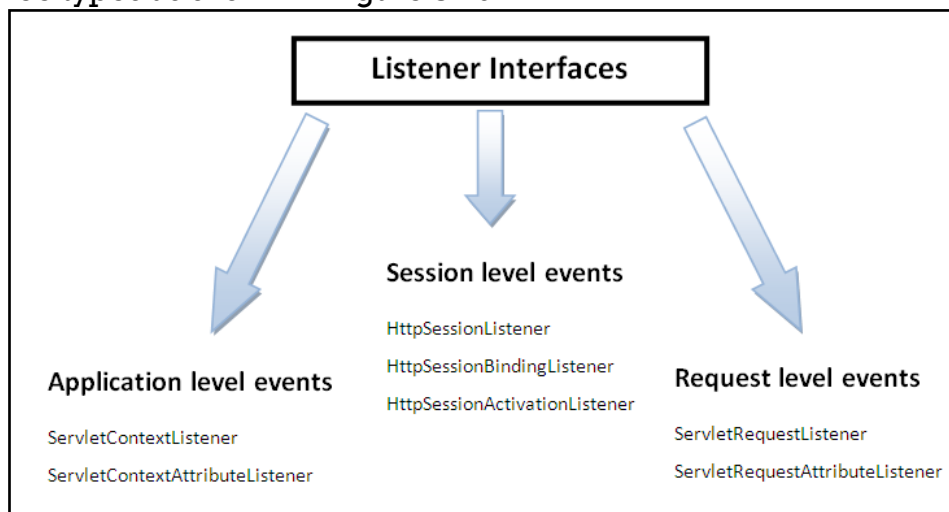


Figure 8.19: Listener Interfaces

8.8.2 Handling Servlet Lifecycle Events

In Web development, listeners assist by recognizing application events. These are also known as observers or event handlers. This helps developers in controlling the lifecycle of `ServletContext`, `ServletRequest`, and `HttpSession` objects.

The events in a servlet lifecycle are monitored by defining listener objects. These objects methods are invoked when lifecycle events occur.

For using listener objects, follow these steps:

- Define the listener class.
- Mention the listener class in `web.xml`, that is, the Web application deployment descriptor.

The servlet context provides access to various resources and facilities, which are common to all Servlets in the application. The Servlet API is used to set the information common to all servlets in an application. The attributes of `ServletContext` class can be used to share information among a group of Servlets.

8.8.3 ServletContextListener

The interface `ServletContextListener` extends `java.util.EventListener` interface. The function of this interface is to receive notifications about changes to the Servlet context of the Web application. The implementation class must be configured in the deployment descriptor of the Web application to receive notification about events.

The methods present in the interface are listed in Table 8.3.

Method	Description
<code>contextInitialized()</code>	The method returns the notification that a Web application initialization has started.
<code>contextDestroyed()</code>	The method returns the notification about closing of the servlet context. All servlets and filters are closed before notification.

Table 8.3: Methods of `ServletContextListener`

Code Snippets 14 and 15 show the use of `ServletContextListener` methods.

Code Snippet 14: `TestListener.java`

```
import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.ServletContextListener;
import java.sql.*;

public class TestListener implements ServletContextListener{
    public void contextInitialized(ServletContextEvent arg0) {
        try{
            Connection con =
                DriverManager.getConnection(
                    "jdbc:mysql://127.0.0.1:3306/test",
                    "root","");
            String query="select COUNT(*) AS COUNT from
                employee";
```

```

        PreparedStatement ps = con.prepareStatement
            (query);
        ResultSet rs= ps.executeQuery();
        rs.next();
        int c = rs.getInt("COUNT");
        System.out.println(c);
    }catch(Exception e){e.printStackTrace();}
    }
    public void contextDestroyed(ServletContextEvent arg0) {
        System.out.println("Project finished.");
    }
}

```

In this code, the `contextInitialized` method contains logic to establish a database connection and perform a query. The `contextDestroyed` method displays a user-defined message.

The JSP file `index.jsp` serves no other purpose than to invoke the listener when the page is launched.

Code Snippet 15:

```

<%@ page language="java" pageEncoding="ISO-8859-1"%>
<!DOCTYPE HTML >
<html>
    <head>
        <title>Start</title>
    </head>
    <body>
        This is the JSP page. <br>
    </body>
</html>

```

Ensure that the `web.xml` has following lines:

```

<listener>
    <listener-class>TestListener</listener-class>
</listener>

```

8.8.4 ServletContextEvent

The `ServletContextEvent` class extends the `java.util.EventObject` class.

This event class notifies about changes made to a Servlet context in a Web application.

Table 8.4 describes the method present in the `ServletContextEvent` class.

Method	Description
<code>getServletContext()</code>	The method returns the modified servlet context.

Table 8.4: Methods of ServletContextListener

Code Snippet 16 shows an example of the method.

Code Snippet 16:

```
// Save and get an application-scoped value
getServletContext().setAttribute("app-param", "app-value1");
value1 = getServletContext().getAttribute("app-param");
```

Figure 8.20 depicts the hierarchy for `ServletContextListener`.

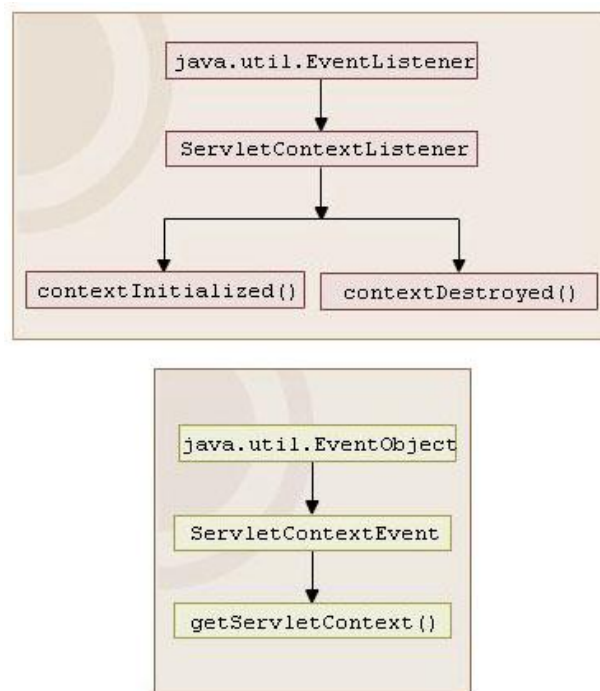


Figure 8.20: ServletContextListener Hierarchy

8.8.5 ServletContextAttributeListener

The `ServletContextAttributeListener` interface extends `java.util.EventListener` interface. The interface receives a notification about any modifications made to the attribute list on the servlet context of a Web application. The implementation class must be configured in the deployment descriptor for the Web application to receive notification events.

Methods present in the interface are as follows:

- **attributeAdded()**
The method notifies that a new attribute is added to the `ServletContext`. The method is invoked after an attribute has been added.
- **attributeRemoved()**
The method notifies that an attribute is removed from `ServletContext`. The method is invoked after an attribute has been removed. Code Snippet 17 shows the method of the `ServletContextAttributeListener`.

Code Snippet 17:

```
public void attributeAdded(ServletContextAttributeEvent
scal) {
    log.append(event1.getName() + ", " + scal.getValue() +
        ", " + new Date() + "\n");
}
public void attributeRemoved(ServletContextAttributeEvent
scal) {
    log.append(event.getName() + ", " + scal.getValue() + ", " + new
Date() + "\n");
}
```

- **attributeReplaced()**
The method notifies that an attribute of `ServletContext` has been replaced. The method is invoked after an attribute has been replaced.

Figure 8.21 depicts the methods of `ServletContextAttributeListener`.

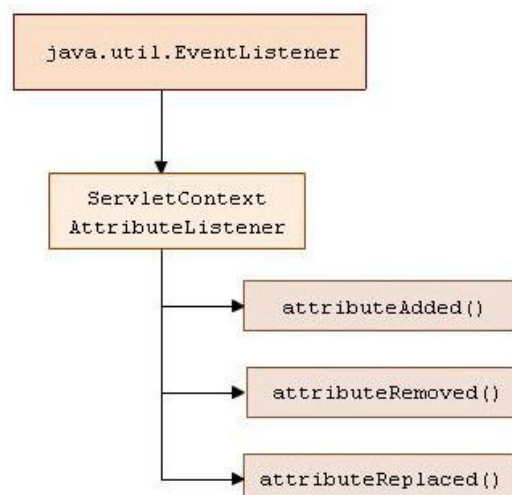


Figure 8.21: Methods of `ServletContextAttributeListener`

8.8.6 ServletContextAttributeEvent

The `ServletContextAttributeEvent` class extends `ServletContextEvent` class. This is the event class for notifications about changes to the attributes of the `ServletContext` of a Web application.

Methods present in the class are as follows:

➤ **getName()**

The name of the altered attribute in the `ServletContext` is returned by the method.

Code Snippet 18 gets the name of the attribute and checks whether the attribute is altered.

Code Snippet 18:

```
public void attributeRemoved(ServletContextAttributeEvent
scal) {
    if (scal.getName().equals("heading")) {
        ServletContext context =
            scal.getServletContext();
        String heading=(String)
            context1.getAttribute("heading");
        if (heading != null) {

            context.log("Heading Replaced="+ heading);
        } else {
            context.log("Attribute 'heading' does not
exist in the servlet context.");
        }
    }
}
```

➤ **getValue()**

The attribute value that has been added, removed, or replaced is returned by the method.

Code Snippet 19 gets the value of the attribute and checks whether the value is not equal to null.

Code Snippet 19:

```
HttpSession session = request.getSession(true);
ShoppingCart1 previousItems1 =
    (ShoppingCart1) session.getValue("previousItems1");
```

```

if (previousItems1 != null) {
    doSomethingWith(previousItems1);
} else {
    previousItems1 = new ShoppingCart1 (...);
    doSomethingElseWith (previousItems1);
}

```

8.8.7 HttpSessionAttributeListener

`HttpSessionAttributeListener` is an interface that extends the `java.util.EventListener` class. It is called whenever some changes are made to the attribute list on the servlet session of a Web application.

`HttpSessionAttributeListener` is used to notify these changes. The listener is used to notify when an attribute has been added, removed, or replaced by another attribute.

The methods belonging to this interface are as follows:

- **`attributeAdded()`**
This method notifies whenever there is an addition of new attributes to a session. It is called after the addition of attribute.
- **`attributeRemoved()`**
This method notifies whenever there is a removal of attributes from a session. It is called after the removal of attribute.
- **`attributeReplaced()`**
This method notifies whenever attributes are replaced in a session. It is called after the attribute is replaced.

Code Snippet 20 implements `HttpSessionAttributeListener` interface and its methods.

Code Snippet 20:

```

public class UserAttributeListener implements
HttpSessionAttributeListener{
// declaring attributeAdded() method
public void attributeAdded (HttpSessionBindingEvent Demoevent)
{
    // Logging the event details
    HttpSession session = Demoevent.getSession();
    ServletContext sc = session.getServletContext();
    sc.log(Demoevent.getName() + ", " + Demoevent.getValue());
}
// Declaring attributeRemoved() method

```

```

public void attributeRemoved(HttpSessionBindingEvent
Demoevent) {}
// Declaring attributeReplaced() method
public void attributeReplaced(HttpSessionBindingEvent
Demoevent) {}
}

```

8.8.8 HttpSessionBindingListener

The `HttpSessionBindingListener` notifies the object when it is being bound to or unbound from a session. This notification can be the result of a forced unbinding of an attribute from a session by the programmer, invalidation of the session, or due to timing out of the session. The implementation class does not require any configuration within the deployment descriptor of the Web application.

The methods belonging to this interface are as follows:

- **valueBound()**
The method notifies the object on being bound to a session and is responsible for identification of the session.
- **valueUnBound()**
The method notifies the object on being unbound from a session and is responsible for identification of the session.

Code Snippet 21 implements the `valueBound` and `valueUnbound` methods of `HttpSessionBindingListener` interface.

Code Snippet 21:

```

public class UserInfo implements HttpSessionBindingListener{
    private String uName;
    public UserInfo(String userName) {
        uName = userName;
    }
    public void valueBound(HttpSessionBindingEvent event) {
        System.out.println(uName+" user bound");
    }
    public void valueUnbound(HttpSessionBindingEvent event) {
        System.out.println(uName+" user unbound");
    }
}

```

8.8.9 HttpSessionBindingEvent

The `HttpSessionBindingEvent` event is sent on occasions to an object that implements `HttpSessionBindingListener` when being bound to or unbound

from the session. It is also sent when `HttpSessionAttributeListener` is configured during the deployment descriptor while being bound, unbound, or replaced in a session. The object is bound to a session by a call to `HttpSession.setAttribute` and unbound from a session by a call to `HttpSession.removeAttribute`.

The methods belonging to this class are as follows:

➤ **`getName()`**

This method returns a string specifying the name with which the attribute is bound to or unbound from the session. Code Snippet 22 gets the name string specifying the name with which the attribute is bound to or unbound from the session.

Code Snippet 22:

```
private void checkAttribute (HttpSessionBindingEvent event,
    String orderAttributeName,
    String keyItemName,
    String message) {
    String demoCurrentAttributeName = event.getName();
    String demoCurrentItemName = (String)event.getValue();
    if (demoCurrentAttributeName.equals(orderAttributeName)
        && demoCurrentItemName.equals(keyItemName)) {
        ServletContext context =
            event.getSession().getServletContext();
        context.log("Customer" + message + keyItemName
            + ".");
    }
}
```

➤ **`getSession()`**

This method returns the session object that has changed.

Code Snippet 23 returns the session object that has changed.

Code Snippet 23:

```
public void sessionCreated (HttpSessionBindingEvent event)
{
    // Retrieves modified session object
    HttpSession session = event.getSession();
    session.getServletContext().log("SessionListener01:
    sessionCreated(" + session.getId() + ")");
}
```

➤ **getValue()**

This method returns the value of the attribute that has been added, removed, or replaced.

Code Snippet 24 returns the value of the attribute that has been added, removed, or replaced.

Code Snippet 24:

```
private void checkAttribute (HttpSessionBindingEvent event,
    String orderAttributeName, String keyItemName,
    String message) {
    String demoCurrentAttributeName =
        event.getName();
    String demoCurrentItemName =
        (String)event.getValue();
    if (demoCurrentAttributeName.equals
        (orderAttributeName) &&
        demoCurrentItemName.equals(keyItemName))
    {
        ServletContext context =
            event.getSession().getServletContext();
        context.log("Customer" + message
            + keyItemName + ".");
    }
}
```

8.8.10 HttpSessionListener

The changes to the list of active sessions in the Web application are notified to the implementations of `HttpSessionListener`. The deployment descriptor for the Web application must have the configured implementation class to receive the notification events.

Various session events are session creation and destruction. The methods belonging to this interface are as follows:

➤ **sessionCreated(HttpSessionEvent se)**

This method provides notification that a session was created.

Code Snippet 25 implements `HttpSessionListener` interface and its methods.

Code Snippet 25:

```
public class SessionTracker extends HttpServlet implements
    HttpSessionListener{
    static private int sessionCount; // count of session
```

```
// Implementing sessionCreated()
public void sessionCreated(HttpSessionEvent se) {
    sessionCount++;
}
```

➤ **sessionDestroyed(HttpSessionEvent se)**

This method provides notification that a session is about to be invalidated.

Code Snippet 26 implements the method `SessionDestroyed()` of the `HttpSessionListener` interface.

Code Snippet 26:

```
public void sessionDestroyed(HttpSessionEvent se) {
    HttpSession session = se.getSession();
    try{
        session.invalidate();
    }
    catch(IllegalStateException e) {
        // Handle or log the exception
        e.printStackTrace();
    }
}
```

The `SessionTracker` class must be configured in the deployment descriptor of the Web application as shown in **Code Snippet 27**.

Code Snippet 27:

```
<listener>
  <listener-class>
    SessionTracker
  </listener-class>
</listener>
```

8.8.11 HttpSessionActivationListener

Sometimes to manage the size of its working set, idle stateful session bean instance is transferred temporarily to the secondary storage. The transfer from the working set to secondary storage is called instance passivation. The transfer back is called activation.

The objects bounded to a session might listen to container events at the same time notifying them about the probability of session being passivated or activated.

The `HttpSessionActivationListener` is implemented when a container migrates the sessions between virtual machines or persists sessions, if required, to

provide notification to all attributes bounded sessions. The implementation class do not require any configuration within the deployment descriptor of the Web application.

The methods present in the listener are as follows:

- **sessionDidActivate (HttpSessionEvent se)**
The method provides notification that the session has just been activated.
- **sessionWillPassivate (HttpSessionEvent se)**
The method provide notification that the session is about to be passivated.

Code Snippet 28 implements `HttpSessionActivationListener` interface and its methods.

Code Snippet 28:

```
public class SessionTracker implements
HttpSessionActivationListener {
    public void sessionWillPassivate (HttpSessionEvent se)
    {
        System.out.println("session is about to be
passivated");
    }
    public void sessionDidActivate (HttpSessionEvent se)
    {
        System.out.println("session has just been
activated");
    }
}
```

8.8.12 HttpSessionEvent

The notifications for the changes to the sessions within a Web application are represented by `HttpSessionEvent`.

The method belonging to this class is `getSession()`. This method returns the session that changed within a Web application.

- **getSession**
Code Snippet 29 shows the `getSession()` method.

Code Snippet 29:

```
// returns modified session object//
public HttpSession getSession()
{
    return session;
}
```

8.8.13 Use of Listeners for Handling Events

Eventlisteners should be executed quickly, because all event listening and drawing methods are executed in the same thread.

While designing the program, one should implement their EventListeners in a class that is not public. For more secure implementation, the implementation can be private.

Code Snippet 30 shows the implementation of `actionPerformed()` method.

Code Snippet 30:

```
public class Beeper ... implements ActionListener {  
    ...  
    //here initialization:  
    button.addActionListener(this);  
    ...  
    public void actionPerformed(ActionEvent e) {  
        ...//Make a beep sound...  
    }  
}
```

Summary

- The most common operation performed by the Servlet is storing and retrieving database information.
- Java provides various mechanisms using which data can be accessed from the database using JDBC or JPA API.
- The JDBC API provides classes and interfaces that allow a Web application to access and perform operations on the databases.
- The JDBC 4.3 API is an improvement in JDBC API, it includes both the `java.sql` and `javax.sql` packages.
- JPA is an ORM technology that persist the entities in the database.
- Java annotations or XML are used to define the mapping of entity to existing database tables.
- In JPA, persistent objects are referred as Entities. Entities are plain old Java objects that are persisted to relational databases or legacy systems.
- A persistence context represents a set of managed entity instances that exist in a particular data store.
- The DAO pattern provides the easy maintenance of the applications by separating the business logics from the database access.
- The events in a servlet lifecycle is monitored by defining listener objects. These objects methods get invoked when lifecycle events occur.

Check Your Progress

1. Which of the following tasks that can be performed using JDBC in Java applications?

(A)	Operate the database transactions
(B)	Request the JDBC connection from the server manager
(C)	Get the results from Database
(D)	Leave the JDBC connection open

A.	A, B	C.	B, C
B.	A, C	D.	A, D

2. Which type of JDBC driver is the fastest?

(A)	Type-1
(B)	Type-2
(C)	Type-3
(D)	Type-4

3. Which class is responsible for creating instances of `EntityManager` in JPA?

(A)	Persistence
(B)	EntityManagerFactory
(C)	EntityManager
(D)	EntityTransaction

4. For using Listener objects, what is the correct option?

(A)	Define listener class
(B)	Define listener class, and specify it in <code>web.xml</code> (deployment descriptor)
(C)	Only specify it in <code>web.xml</code> , no necessity to define it
(D)	None of these

5. Which of the following is the correct syntax of the `getAttribute()` method?

(A)	<code>public Object getAttribute(String name1)</code>
(B)	<code>public Object getAttribute(String name1, Object object1)</code>
(C)	<code>public Object getAttribute(Object object1)</code>
(D)	<code>public Name getAttribute(Object object1)</code>

Answers

1.	B
2.	B
3.	B
4.	B
5.	A

Try It Yourself

1. Install MySQL database 8.0 on your laptop or computer.
2. Add the required JAR files for this project as referenced in the content. Add JPA version settings in the Project Facets options.
3. Develop a Web application using JSP, Servlet, and MySQL database in Eclipse. Insert the purchase records and test it on the Tomcat server to ensure proper functionality. Table 8.5 shows sample data for the purchase history.

Purchase ID	Customer Name	Product	Quantity	Purchase Date
1	John Doe	Laptop	1	2024-04-01
2	Jane Smith	Smartphone	2	2024-04-02
3	Alice Johnson	Headphones	1	2024-04-03
4	Bob Brown	Tablet	1	2024-04-04
5	Emily Wilson	Smartwatch	1	2024-04-05

Table 8.5: Purchase History

This sample data includes columns for the purchase ID, customer name, purchased product, quantity, and purchase date. Use this data to populate the MySQL database table for testing the Web application.

Asynchronous Servlet Communication

Welcome to the Session, **Asynchronous Servlet Communication**.

This session explains in depth about the concept of Asynchronous Servlet and asynchronous listeners. The session explains how to develop asynchronous JavaScript client in Web applications. Further, the session outlines how to perform non-blocking Input/Output operations in a Servlet. Finally, the session concludes with an explanation of upgrading protocols in request and response sent by Servlets.

In this session, you will learn to:

- Explain necessity for Asynchronous Servlet
- Describe advantages and use of Asynchronous servlet
- Explain how to create Asynchronous Servlet and Asynchronous Listener
- Explain how to create an asynchronous JavaScript client using XMLHttpRequest object
- Explain the necessity of non-blocking I/O support in Servlet
- Explain how to implement non-blocking I/O in asynchronous Servlet
- Explain necessity for protocol upgrade
- Elaborate on the process of protocol upgrade in a Servlet

9.1 Introduction

The architecture of the Servlet is based on multithreaded model. This means the Web container creates Servlet threads to process the client requests. Each client will receive its dedicated thread which will serve the request and generate a response to be sent back to the client. The Servlet multithreaded model works fine when the client requests are less for a Servlet. The Web container normally creates a pool of threads ready to serve the client with the Servlet instances.

Consider a situation where the Web server receives multiple client requests for the Servlet. To serve multiple requests, the Web container requires large number of threads in the container pool. A thread associated with the client request may not process the request all the time and may sit idle. This can lead to thread starvation on the server.

Some of the reasons, where the thread can go idle while processing the request are as follows:

- Servlet may take a long running task such as making a database connection call for query execution or invoking a remote Web service. In these

situations, the thread must wait for the resource availability, so that it can generate the response to be sent to the client.

- Servlet may have to wait for some dependent event to proceed for the response generation. The situation can occur when waiting to receive information from another client to proceed with the response generation.

While the threads are engaged in serving clients, the maximum threshold limit to hold threads in the pool may hit. This can affect the Web server scalability, which means either the server will run out of memory or will not remain with the threads in the container pool.

Servlet 3.0 API has provided a new feature in its specification to process the request asynchronously. With Asynchronous processing, one can avoid blocking requests by allowing the thread to perform other operations, while the input is returned to the client.

9.1.1 Asynchronous Processing in Servlets

The Java Servlet API provides asynchronous processing support for servlets and filters in Web applications. In asynchronous processing, a Servlet thread waiting for a resource is released by the container and returned to the pool. The long-running task performed by the thread is carried out asynchronously in the background.

Asynchronous processing is performed in the background thread. Once the task is completed, the background thread can generate a response or a request which is to be dispatched to a Servlet for processing.

To handle the generated result, a notification is sent to the Servlet container. It will allocate a different Servlet thread available in the pool to handle the result and interact with the client with the generated response.

Following are steps to be followed for processing the request asynchronously in the Servlet:

1. The request sent to the Servlet is intercepted by the filters, which perform some pre-processing tasks for the Servlet. For example, authentication of the user can be pre-processed within a Servlet.
2. The Servlet receives the request parameters to process them.
3. The Servlet requests for some external connection or data. For example, sends a request for obtaining a connection with a database. If there are already some requests waiting for the connection, then the Servlet joins a waiting queue for a JDBC connection.
4. While waiting for the JDBC connection, the Servlet returns back to the pool without generating a response.

5. The JDBC connection operation is assigned to an asynchronous context object which is responsible for handling the request in the background thread.
6. When the JDBC connection is available, the asynchronous context object gets the connection object and notifies the Servlet container for a Servlet thread to handle the connection.

9.1.2 Significance of Asynchronous Servlet

In synchronous processing, sometimes it can be challenging for the server to perform optimally, especially for applications with intensive processing requirement, where a high number of threads are devoted to fulfilling the request. If the thread has to wait for an event to occur, such as a user click or connecting to an external database, before it can send its response back, can result in a significant waiting time. Additionally, the Servlet may also run out of threads, in which case the long-running threads will also die.

For the server to perform optimally, it must process such requests asynchronously. Especially requests that take a long time to process and have a long wait time or when the Servlet is overwhelmed with incoming requests and does not want to serve up one thread per request.

A Synchronous operation blocks a process until it completes, whereas Asynchronous operations are non-blocking, allowing the user to perform other tasks simultaneously. One of the key benefits of using asynchronous servlets is that they enable HTTP push, enabling the server to send information to the client at any time it chooses rather than only when the client specifically requests.

In asynchronous processing, the worker thread handles client requests and passes them to a background thread. The background thread then, processes the request and replies to the client. After passing the request to the background thread, the initial worker thread returns to the thread pool to process another request.

9.2 Handling Asynchronous Servlet

To support asynchronous processing, the separation of request from the generated response is to be separated. This is achieved by using the interface `jakarta.servlet.AsyncContext`. This interface is used to process the request asynchronously within the Servlet.

Apart from creating the object of `AsyncContext`, developer must enable the Servlet class with asynchronous processing. This is done by enabling the attribute `asyncSupported` to `true` in the `@WebServlet` annotation.

9.2.1 AsyncContext Interface

The `AsyncContext` interface provides methods that can be used to obtain the instance of the `AsyncContext` within the `Servlet service()` method.

Table 9.1 lists the methods of the `AsyncContext` interface.

Method	Description
<code>void start(Runnable run)</code>	This method gets a thread from the managed thread pool to run the specified <code>Runnable</code> thread.
<code>ServletRequest getRequest()</code>	This method obtains the parameters from the request for the asynchronous context.
<code>ServletResponse getResponse()</code>	This method is used inside the asynchronous context to write to the response with the results of the blocking operation.
<code>void complete()</code>	This method completes the asynchronous operation and closes the response object associated with the asynchronous object.
<code>void dispatch(String path)</code>	This method dispatches the request and response objects to the URL to be forwarded to another Servlet thread.

Table 9.1: Methods of AsyncContext

Following are steps to be performed to handle asynchronous request in the asynchronous Servlet:

- Obtain the object of the `AsyncContext` class by invoking `startAsync()` method on `ServletRequest` object.
- Invoke `asyncContext.start()` method by passing a `Runnable` thread reference which is responsible to execute a running task in the background.
- Call `setTimeout()` on `asyncContext`, after the number of milliseconds the container has to wait for specified task to complete. In case if there is no timeout setting then, container's default will be used. If the task fails to complete within specified time, it throws an exception.
- Call `asyncContext.complete()` or `asyncContext.dispatch()` from the `Runnable` thread to complete the task or forward request and response object to some other Servlet.

Code Snippet 1 shows the skeleton for obtaining asynchronous context within the Servlet's `doGet()` or `doPost()` method.

Code Snippet 1: AsyncServlet.java

```

import java.io.IOException;
import jakarta.servlet.AsyncContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@SuppressWarnings("serial")
@WebServlet(urlPatterns={"/asynctservlet"},
    asyncSupported=true)
public class AsyncServlet extends HttpServlet {
    public void doGet(HttpServletRequest request,
        HttpServletResponse
            response) throws ServletException, IOException
    {
        final AsyncContext asyncContext = request.startAsync ();
        asyncContext.setTimeout (30000); // Set timeout to 30
        seconds (30000 milliseconds)
        asyncContext.start (new Runnable () {
            @Override
            public void run () { // long running task
                // Perform long-running task here
                // For example:
                try {
                    // Simulating a 5-second task
                    Thread.sleep(5000);
                    // This could be replaced with
                    // actual processing logic
                    response.getWriter().println("Long-running task
                    completed.");
                    asyncContext.complete();

                    // Complete the asynchronous processing
                } catch (InterruptedException |
                IOException e) {
                    e.printStackTrace();
                }
            }
        });
    }
}

```

In **Code Snippet 1**, the **Servlet** class is annotated with the attribute `asyncSupported = true` which means the **Servlet** can handle the request and response asynchronously.

Further, to processing asynchronous requests, the object of the `AsyncContext` interface is obtained by calling the method `startAsync()` on the request object of the Servlet. This means that the response will be generated only when the blocked operation is either completed or the request is dispatched to the other Servlet by the `AsyncContext` object.

Then, the statement, `asyncContext.start (new Runnable () { ... })` obtains the new thread from the container pool. The new thread can access the asynchronous context object and process the request and response.

Figure 9.1 shows the method of the `Runnable` interface finished successfully.

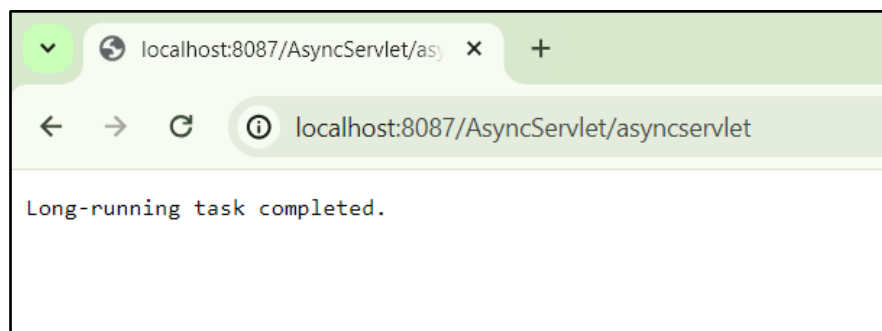


Figure 9.1: Output of AsyncServlet

Code Snippet 2 demonstrates the code that processes the request asynchronously in the Servlet.

Code Snippet 2: `AsyncDispatchServlet.java`

```
import java.io.IOException;
import jakarta.servlet.AsyncContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet(name = "AsyncDispatchServlet",
    urlPatterns = { "/asyncDispatch" },
    asyncSupported = true)
public class AsyncDispatchServlet extends HttpServlet {

    @Override
    public void doGet(final HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        final AsyncContext asyncContext = request.startAsync();
        request.setAttribute("mainThread",
            Thread.currentThread().getName());
        asyncContext.setTimeout(5000);
        asyncContext.start(new Runnable() {
```

```

        @Override
        public void run() {
            // long-running task
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            request.setAttribute("workerThread",
                                Thread.currentThread().getName());
            //Dispatch the request object to a JSP
            asyncContext.dispatch("/threadNames.jsp");
        }
    });
}

```

The servlet in Code Snippet 2 supports asynchronous processing and its long-running task to keep the sleep mode on for three seconds. Servlet's `doGet()` method proves that the long-running task is executed in a different thread than the main thread. `doGet()` method attaches the name of the main thread and that of the worker thread to the `ServletRequest` and dispatches to a **threadNames.jsp** page.

The **threadNames.jsp** page displays `mainThread` and `workerThread` thread names in the output.

Code Snippet 3 shows the `threadNames.jsp` page.

Code Snippet 3:

```

<%@ page language="java" contentType="text/html;
charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE HTML>
<html>
<head>
<title>Asynchronous servlet</title>
</head>
<body>
Main thread: ${mainThread}
<br/>
Worker thread: ${workerThread}
</body>
</html>

```

Remember, at the end of task, call the `dispatch()` or `complete()` method on the `asyncContext` object, so that it will not wait until it times out.

Code Snippet 4 shows the `web.xml` file for the application.

Code Snippet 4: web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd
http://xmlns.jcp.org/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
id="WebApp_ID" version="5.0">
  <display-name>test9</display-name>
  <servlet>
<servlet-name>AsyncDispatchServlet</servlet-name>
<servlet-class>test9.AsyncDispatchServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>AsyncDispatchServlet</servlet-name>
<url-pattern>/asyncDispatch</url-pattern>
</servlet-mapping>
<servlet>
<servlet-name>AsyncServlet</servlet-name>
<servlet-class>test9.AsyncServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>test9.AsyncServlet</servlet-name>
<url-pattern>/asyncservlet</url-pattern>
</servlet-mapping>
</web-app>

```

Figure 9.2 shows the name of the main thread and the name of the worker thread. Both names are different, which proves that the worker thread is different from the main thread.

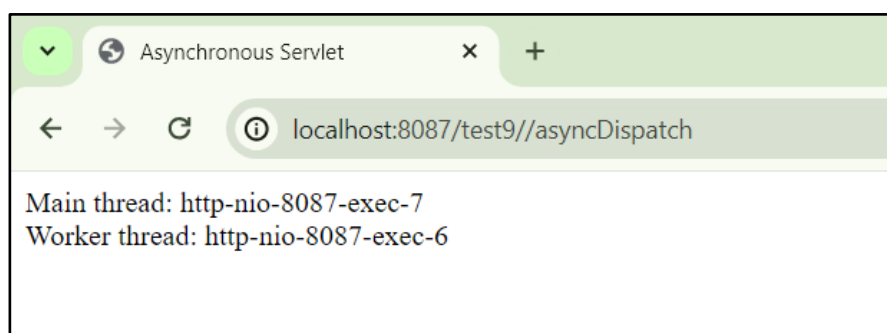


Figure 9.2: Output of AsyncDispatchServlet

Consider a scenario where a servlet sends a progress update every second so that the user can monitor the progress. It sends HTML response and a simple JavaScript code to update the HTML page.

Code Snippet 5 defines a servlet that uses asynchronous processing to update a Web page with progress information.

Code Snippet 5:

```
package servlet;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.AsyncContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
@SuppressWarnings("serial")
@WebServlet(value = "/asyncComplete", asyncSupported = true)
public class AsyncCompleteServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        final PrintWriter writer = response.getWriter();
        writer.println("<html><head><title>" +
            "Async Servlet</title></head>");
        writer.println("<body><div id='progress'></div>");
        final AsyncContext asyncContext =
            request.startAsync();
        asyncContext.setTimeout(90000);
        asyncContext.start(new Runnable() {
            @Override
            public void run() {
                System.out.println("new thread:" +
                    Thread.currentThread());
                for (int i = 0; i < 10; i++) {
                    try{
                        writer.println("<script>");
                        writer.println("document.getElementById("
                            + "'progress').innerHTML = '" +
                                (i * 10) + "% complete'");
                        writer.println("</script>");
                        writer.flush();
                        Thread.sleep(1000);
                    } catch (InterruptedException e) {
                        e.printStackTrace(); // Log or handle the exception
                    } //catch
                } //for
                writer.println("<script>");
```

```

        writer.println("document.getElementById("
            + "'progress').innerHTML = 'DONE'");
        writer.println("</script>");
        writer.println("</body></html>");
        asyncContext.complete();
    } //run
}); //start
}
}

```

The `@WebServlet` annotation in the code specifies that this servlet will handle requests to the `/asyncComplete` URL path and that it supports asynchronous operations (`asyncSupported = true`). The `AsyncCompleteServlet` class extends `HttpServlet`, making it a servlet capable of handling HTTP requests. The servlet writes HTML content to the response, including a `div` element with an ID of `progress`. This is where the progress updates will be displayed. The servlet starts an asynchronous context with `request.startAsync()` and sets a timeout of 90 seconds. A new thread is started to simulate a long-running process. It updates the `progress` element with the percentage of completion every second, for ten seconds. Once the loop is finished, it updates the `progress` element to **DONE** and completes the asynchronous context with `asyncContext.complete()`.

Note that the `AsyncCompleteServlet` can be enabled with the asynchronous behavior by either annotating it using the `@WebServlet` annotation or configuring the deployment descriptor, `web.xml`.

Code Snippet 6 displays the partial code for the deployment descriptor, `web.xml` for configuring the `AsyncCompleteServlet`.

Code Snippet 6:

```

web-app>
. . .
    <servlet>
        <servlet-name>AsyncComplete</servlet-name>
        <servlet-
class>servlet.AsyncCompleteServlet</servlet-class>
        <async-supported>true</async-supported>
    </servlet>
    <servlet-mapping>
        <servlet-name>AsyncComplete</servlet-name>
        <url-pattern>/asyncComplete</url-pattern>
    </servlet-mapping>
</web-app>

```

Code Snippet 7 displays the `index.html` page, which works in conjunction with the `AsyncCompleteServlet`. The HTML page establishes a basic structure with a

title and a `div` element, identified by the ID `progress`. The servlet updates this element asynchronously, allowing users to view the progress in real time without requiring to refresh the page.

Code Snippet 7:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Progress Monitoring</title>
</head>
<body>
  <h1>Progress Update</h1>
  <div id="progress"></div>
  <script>
    // No script is required here as the progress
    updates are handled by the servlet
  </script>
</body>
</html>
```

Figure 9.3 shows result of the HTML page that receives progress updates from the asynchronous Servlet.

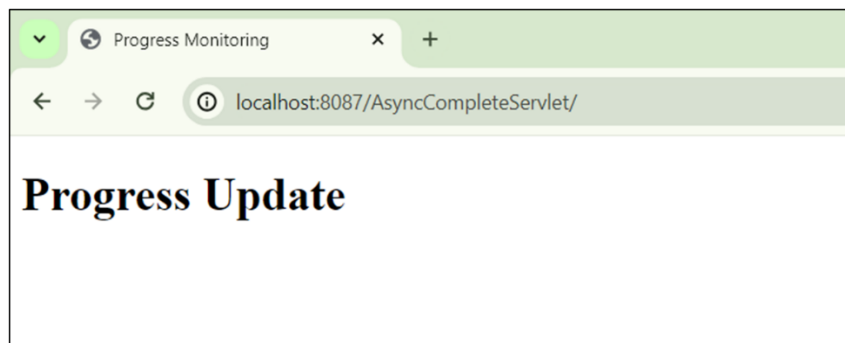


Figure 9.3: Output of index.html

Figure 9.4 shows the result of asynchronous completion from the asynchronous Servlet. You must launch a new browser to view this.

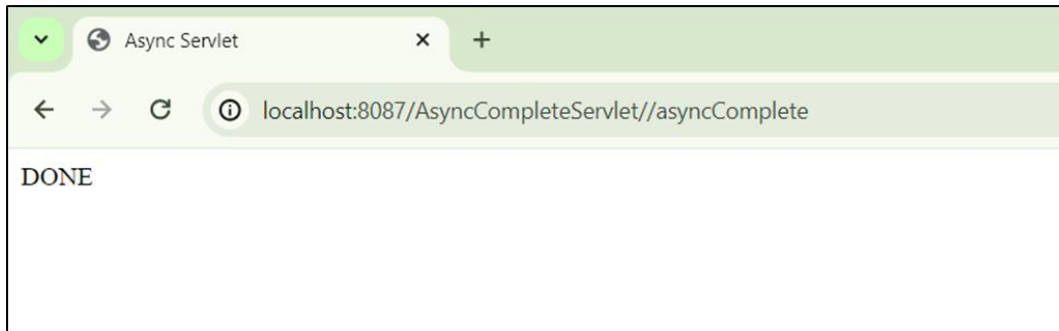


Figure 9.4: Result of Asynchronous Completion

9.2.2 AsyncListener Interface

When the `AsyncContext` object is processing the request and response asynchronously, it passes through a series of events that monitor the lifecycle of the `AsyncContext` object. To handle these events, the `AsyncContext` object can be registered with the `AsyncListener` interface. The `AsyncListener` interface defines the methods that are used to notify the events occurring on the `AsyncContext` object.

Developers can register an `AsyncListener` on an `AsyncContext` object by calling the `addListener()` method on it.

Table 9.2 lists methods of the `AsyncListener` interface.

Method	Description
<code>void onStartAsync(AsyncEvent event)</code>	This method gets called when an asynchronous operation has been initiated.
<code>void onComplete(AsyncEvent event)</code>	This method gets called when asynchronous operation has completed.
<code>void onError(AsyncEvent event)</code>	This method gets called in the event an asynchronous operation has failed.
<code>void onTimeout(AsyncEvent event)</code>	This method gets called when an asynchronous has timed out, especially, when it failed to finish within the specified timeout.

Table 9.2: Methods of AsyncListener Interface

All four methods receive an `AsyncEvent` object that gets fired when the asynchronous operation is started, asynchronous processing is completed, error is encountered, and timeout has happened.

Code Snippet 8 demonstrates an `AsyncListenerServlet` class that registers the listener to receive event notifications.

Code Snippet 8:

```

package servlet;
import java.io.IOException;
import jakarta.servlet.AsyncContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import listener.MyAsyncListener;
@WebServlet(name = "AsyncListenerServlet",
            urlPatterns = {"/asyncListener"},
            asyncSupported = true)
public class AsyncListenerServlet extends HttpServlet {
    private static final long serialVersionUID = 92738L;
    @Override
    public void doGet(final HttpServletRequest request, final
        HttpServletResponse response) throws ServletException,
        IOException {

        // Obtaining the Asynchronous context object
        final AsyncContext asyncContext =
request.startAsync();
        //Setting the time out duration for asyncContext object
        asyncContext.setTimeout(5000);
        // Registering the events with the listener
        asyncContext.addListener(new MyAsyncListener());

        //Obtaining a new thread for long-running task
        asyncContext.start(new Runnable() {

            @Override
            public void run() {
                try {
                    Thread.sleep(3000); // Simulate long-
running task
                } catch (InterruptedException e) {
// Handle potential interruption (optional)
                    e.printStackTrace();
                }
                String greeting = "Hi from listener";

```

```

        System.out.println("wait...");
        request.setAttribute("greeting", greeting);
// Set attribute for JSP
// Dispatch to test.jsp after task completes
        asyncContext.dispatch("/test.jsp");
    }
    });
}
}

```

Code Snippet 9 shows the implementation of the `AsyncListener` interface methods.

Code Snippet 9:

```

package listener;
import java.io.IOException;
import jakarta.servlet.AsyncEvent;
import jakarta.servlet.AsyncListener;
public class MyAsyncListener implements AsyncListener {
    @Override
    public void onComplete(AsyncEvent asyncEvent)
        throws IOException {
        System.out.println("onComplete");
    }
    @Override
    public void onError(AsyncEvent asyncEvent)
        throws IOException {
        System.out.println("onError");
    }
    @Override
    public void onTimeout(AsyncEvent asyncEvent)
        throws IOException {
        System.out.println("onTimeout");
    }
    @Override
    public void onStartAsync(AsyncEvent arg0) throws
        IOException {
        // TODO Auto-generated method stub
    }
}

```

This class listens to any events and displays appropriate messages on the console.

Ensure to create servlet and listener packages for Code Snippets 8 and 9 respectively.

Code Snippet 10 shows the `test.jsp` page that retrieves the `greeting` attribute set by the servlet and displays it within a paragraph element.

Code Snippet 10:

```
<%@ page language="java" contentType="text/html;
charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Async Listener Result</title>
</head>
<body>
    <h1>Listener Message</h1>
    <% String message = (String)
request.getAttribute("greeting"); %>
    <p><%= message != null ? message : "No message
available" %></p>
</body>
</html>
```

Figure 9.5 shows the result of Asynchronous Listener.

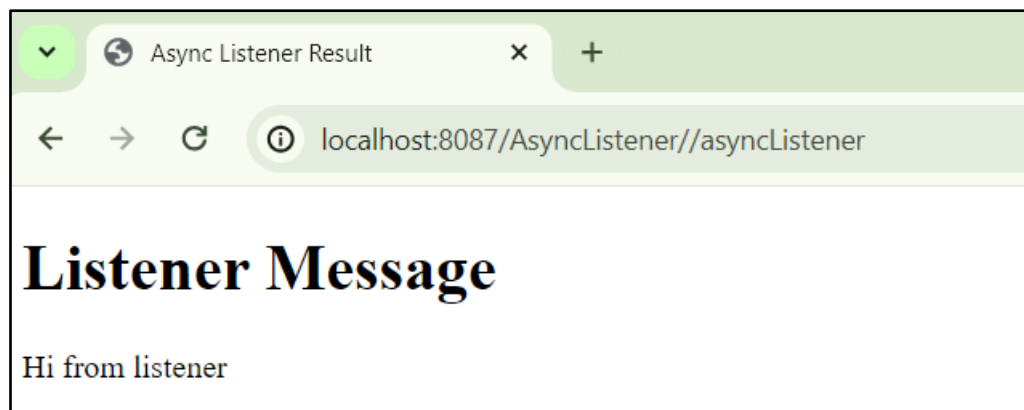


Figure 9.5: Output of Asynchronous Listener

Also, observe the console for the displayed messages.

9.3 Server Push Mechanism in Servlet

The communication between the client and Web server is based on the HTTP protocol. The HTTP protocol implements a strict request-response model, which means that the client sends a HTTP request and waits until the HTTP response is received.

However, as the Web technologies are being updated with the asynchronous behavior, so the requirement for creating more interactive applications have increased. This has led to the development of modern Web clients that allow the page to interact with the server and update the page without loading the whole page. The mechanism of sending the data asynchronously from the Web server to the client, without redrawing the whole page is referred as server push.

There are many techniques to achieve server push mechanism in Web applications. However, one of the most popular programming approach used in modern Web applications is Asynchronous Java and XML (AJAX).

Consider a situation where a Web page is called by a user to search for information about musical albums. The page includes a text field which allows the user to enter the name of the album. As soon as the user enters the first character of the search word, the Web application attempts to complete the name by listing all album names whose first or last name begins with the characters entered. This feature of auto-complete the search text is showing the AJAX feature implementation in the Web application.

Using AJAX, the client Web browser uses a JavaScript code that fetches a small amount of data from the Server and updates or integrates the displayed page with the received response from the server, without fetching the entire page.

Figure 9.6 shows the asynchronous communication of Web client and Web server based on AJAX mechanism in the Web application.

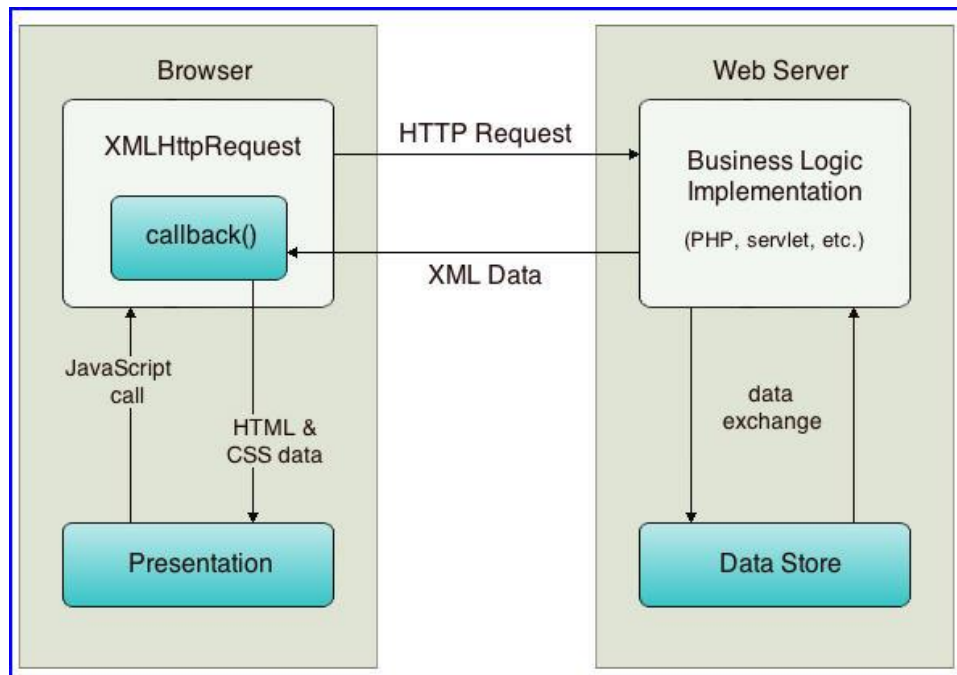


Figure 9.6: Asynchronous Communication in Web Application

As shown in Figure 9.6, the Web application uses JavaScript to modify the client's page, whereas the Web server uses the Java Servlet to process the HTTP request asynchronously.

The AJAX mechanism works with an `XMLHttpRequest` object that is used to pass the requests and responses asynchronously between the client and server. All modern Web browsers support the `XMLHttpRequest` object

9.3.1 Developing Asynchronous Client

The `XMLHttpRequest` object is used in the JavaScript code to interact asynchronously with the Web server.

Following are the steps for creating the asynchronous JavaScript client:

1. Create an `XMLHttpRequest` object.
2. Send the request to the server using the methods of the `XMLHttpRequest` object.
3. Receive and process the response received from the server.

Code Snippet 11 demonstrates the code to check the support for `XMLHttpRequest` object in the HTML page and process the request and response using the `XMLHttpRequest` object.

Code Snippet 11: request.html

```

<!DOCTYPE html>
<html>
<head>
<script type="text/javascript">
    function getXMLHttpRequest() {
        var xmlHttpReq = false;
        // Creating XMLHttpRequest object for non-Microsoft
        //browsers
        if (window.XMLHttpRequest) {
            xmlHttpReq = new XMLHttpRequest();
        } else if (window.ActiveXObject) {

            try {
                // Creating XMLHttpRequest object in later versions
                // of Internet Explorer
                xmlHttpReq = new ActiveXObject("Msxml2.XMLHTTP");
            } catch (exp1) {

                try {
                    // to create XMLHttpRequest object in older versions
                    // of Internet Explorer
                    xmlHttpReq = new ActiveXObject("Microsoft.XMLHTTP");
                } catch (exp2) {
                    xmlHttpReq = false;
                }
            }
        }
        return xmlHttpReq;
    }
    /* AJAX call starts with this function. The makeRequest()
    function sends a GET request to the server endpoint
    helloWorld.do. This would typically correspond to a servlet
    that handles this URL patter*/
    function makeRequest() {
        var xmlHttpRequest = getXMLHttpRequest();
        xmlHttpRequest.onreadystatechange =
            getReadyStateHandler(xmlHttpRequest);
        xmlHttpRequest.open("GET", "helloWorld.do", true);
        xmlHttpRequest.send();
    }

    /* Returns a function that waits for the state change in
    XMLHttpRequest*/
    function getReadyStateHandler(xmlHttpRequest) {

        // an anonymous function returned

```



```
// it listens to the XMLHttpRequest instance
return function() {
    if (xmlHttpRequest.readyState == 4) {
        if (xmlHttpRequest.status == 200) {
            document.getElementById("hello").innerHTML =
xmlHttpRequest.responseText;
        } else {
            alert("HTTP error " + xmlHttpRequest.status + ": " +
xmlHttpRequest.statusText);
        }
    }
};
}
</script>
</head>
<body>
<button onclick="makeRequest()">Make Request</button>
<div id="hello"></div>
</body>
</html>
```

The function `getXMLHttpRequest()` checks if the browser supports the `XMLHttpRequest` object. If the object is supported, then create an `XMLHttpRequest` object. For modern browser, the `XMLHttpRequest()` method is used, whereas for the older version of the Internet Explorer browser, the `XMLHttpRequest` object is obtained using `ActiveXObject()` method.

The next step is to send the asynchronous request to the Web server. To send the request, the two methods namely, `open()` and `send()` of `XMLHttpRequest` object are invoked. These methods are used to prepare the HTTP Request and send it to the server.

The syntax of the `open()` is:

Syntax:

`open(method,url,async)`

where,

method: Specifies the type of request method, that is, GET or POST is used to send the request.

url: Specifies the location of the component handling the request on the server.

async: Specifies whether the asynchronous communication is supported or not. The value can be true or false.

Here, the HTTP request GET method is used to send the data to the Servlet mapped with the URL, `helloWorld.do`, and the asynchronous variable is set to true. The `send()` method sends the data to the request. In case of GET method, the data is

sent as the query string, so nothing is sent with the `send()` method. However, if the method is POST, then `send()` method will have the data.

Then, the JavaScript code will wait for the server response. Once the server is ready with the response, the `onreadystatechange` event is fired which indicates the change in the state. In the `onreadystatechange` event, one can specify what to do when the server response is ready to be processed. When `readyState` is 4 which means server has finished processing request and ready with the response, then check the status is 200, which means response status as OK.

The callback function is invoked, and the response is processed to display the result on the page.

Code Snippet 12 shows a simple servlet code for `helloWorld.do`.

Code Snippet 12:

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/helloWorld.do")
public class HelloWorldServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        response.setContentType("text/plain");
        response.setCharacterEncoding("UTF-8");
        response.getWriter().write("Hello, world!");
    }
}
```

This servlet listens to requests at the URL pattern `/helloWorld.do`. When a GET request is received, it sets the response content type to plain text and writes `Hello, world!` to the response output stream.

Figure 9.7 shows the output of **request.html** page.

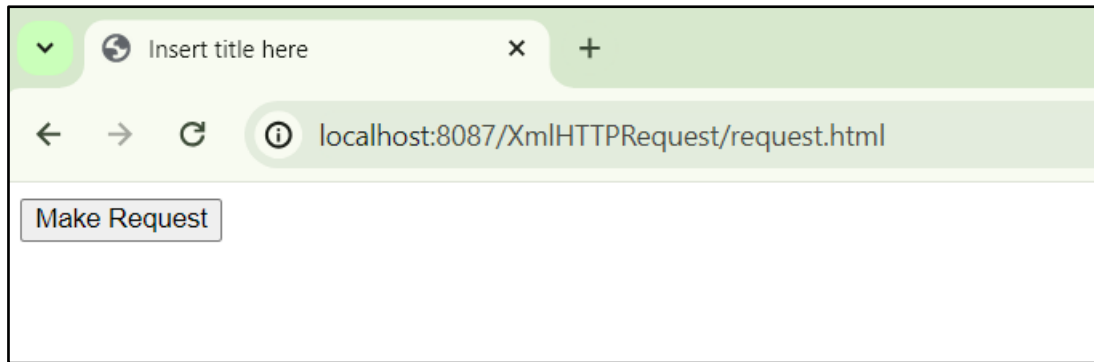


Figure 9.7: Output of request.html page

Figure 9.8 shows the output of AJAX request message.

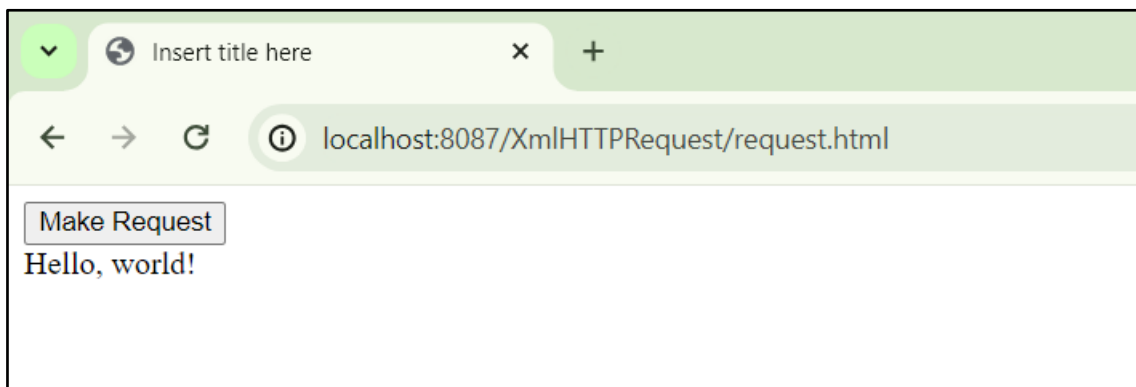


Figure 9.8: Output of AJAX Request Message

9.4 Non-Blocking Input/Output in Servlet

Consider a situation when client is submitting a large HTTP POST request over a slow network connection. Using traditional I/O, the container thread associated with this request would be sometimes sitting idle waiting for the rest of the request. In this process, a large part of CPU memory is wasted. Hence, Servlet API provides non-blocking I/O support for Servlets and filters during asynchronous processing of requests.

9.4.1 Implementation of Non-blocking I/O Support

The non-blocking I/O is supported by following APIs:

- `jakarta.servlet.ServletInputStream`
- `jakarta.servlet.ServletOutputStream`
- `jakarta.servlet.ReadListener`
- `jakarta.servlet.writeListener`

Following steps are used for non-blocking I/O to process requests and write responses inside service methods:

1. Enable the asynchronous mode for the Servlet using the `@WebServlet` annotation. For example,
`@WebServlet(urlPatterns={"/asyncioservlet"},
 asyncSupported=true)`
2. Obtain an input stream and/or an output stream from the request and response objects in the `service()` method.
3. Assign a read listener to the input stream and/or a write listener to the output stream.
4. Process the request and the response inside the listener's callback methods.

Code Snippet 13 demonstrates the use of non-blocking I/O in the asynchronous Servlet.

Code Snippet 13:

```
package com.test9;
import java.io.IOException;
import jakarta.servlet.AsyncContext;
import jakarta.servlet.ReadListener;
import jakarta.servlet.ServletInputStream;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@SuppressWarnings("serial")
@WebServlet(urlPatterns={"/asyncioservlet"},
    asyncSupported=true)
public class AsyncIOServlet extends HttpServlet{

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws IOException {
        response.getWriter().write("Hello! This servlet only
        supports POST requests.");
    }

    @Override
    public void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {
```

```

        final AsyncContext acontextResponse =
request.startAsync();
        final ServletInputStream inputStream =
request.getInputStream();
        inputStream.setReadListener(new ReadListener() {
            byte[] buffer = new byte[4*1024];
            StringBuilder sbuilder = new StringBuilder();
            public void onDataAvailable() {
                try {
                    do {
                        int length = inputStream.read(buffer);
                        if (length == -1) {
                            break; // End of stream reached
                        }
                        sbuilder.append(new String(buffer, 0,
length));
                    } while(inputStream.isReady());
                } catch (IOException ex) {
ex.printStackTrace();
acontextResponse.complete(); // Signal completion with an
error
                }
            }
            public void onAllDataRead() {
                try {
                    String data = sbuilder.toString();
                    // Process data (replace with your logic)
                    String processedData = processData(data);

acontextResponse.getResponse().getWriter().write(processedDat
a);
                } catch (IOException ex) {
ex.printStackTrace();// Signal completion with an error }
                finally {
                    acontextResponse.complete(); // Ensure
completion even on exceptions
                }
            }
        }
    @Override
        public void onError(Throwable t) {
            t.printStackTrace();
            acontextResponse.complete(); // Signal
completion with an error

```

```

        }
    });
}

@SuppressWarnings("unused")
public void onComplete() {
    // No operation required here
}

// Replace this with your actual data processing logic
private String processData(String data) {
    return "Processed data: " + data;
}
}
}

```

In Code Snippet 13, the code enables the servlet with asynchronous processing. The `AsyncContext` object named `acontextResponse` is created to process the request asynchronously. Then, the Servlet obtains the input stream associated with the request and assigns a read listener defined as an inner class. The listener reads parts of the request as they become available and then, writes some response to the client when it finishes reading the request.

Figure 9.9 shows the use of non-blocking I/O in the asynchronous Servlet.

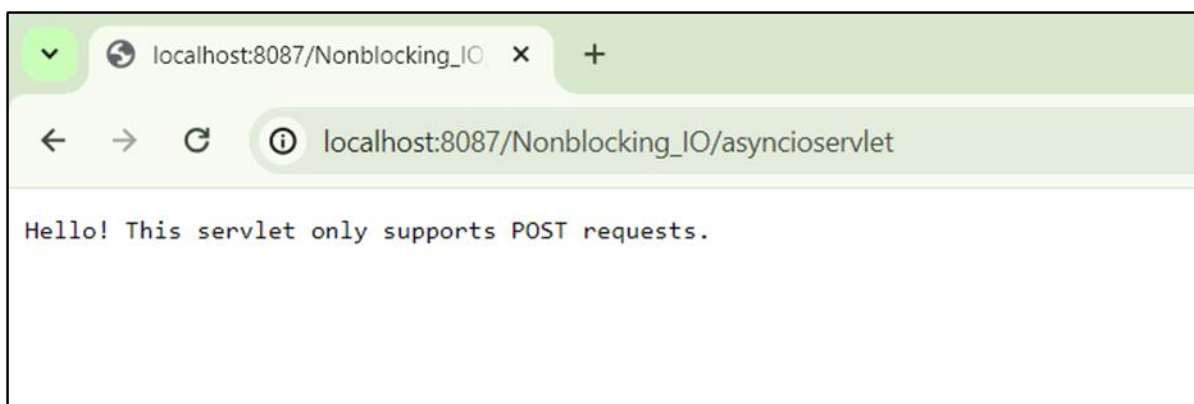


Figure 9.9: Output of Non-Blocking I/O

9.5 Protocol Upgrade Processing

Different client sent requests to server through different protocols. Some client change protocol information in protocol information. With Servlet 3.0 API, the HTTP protocol has been upgraded from HTTP 1.0 to HTTP 1.1. HTTP 1.0 was a stateless protocol, whereas HTTP 1.1 can persist the connection between the client and the server. The advantage of persistent connection is that the connection is alive and can be used for multiple requests with the client.

9.5.1 Upgrading to HTTP 1.1 Protocol

In HTTP/1.1, on a current connection, using the `Upgrade` header field, clients can switch to a different protocol. If the server accepts the request of switching to the protocol as indicated by the client, an HTTP response with status 101 (switching protocols) is generated. After this, the client and the server communicates using the new protocol.

Code Snippet 14 demonstrates the use of headers to upgrade the protocol information in the response.

Code Snippet 14:

```
package servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@SuppressWarnings("serial")
@WebServlet("/upgrade")
public class ProtocolUpgradeServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        // Check if the client requests an upgrade to HTTP
        1.1
        String upgradeHeader = request.getHeader("Upgrade");
        if ("HTTP/1.1".equalsIgnoreCase(upgradeHeader)) {
            // Accept the upgrade request
            response.setStatus(HttpServletResponse.SC_SWITCHING_PROTOCOLS
        );
            response.setHeader("Upgrade", "HTTP/1.1");
            response.setHeader("Connection", "Upgrade");
            // Additional headers or processing can be added
            here
        } else {
            // Reject the upgrade request with an error
            response
```

```
response.setStatus(HttpServletResponse.SC_BAD_REQUEST);  
    response.getWriter().println("Upgrade to HTTP 1.1  
is required.");  
    }  
    }  
}
```

The `ProtocolUpgradeServlet`, mapped to the URL pattern `/upgrade`, accepts an HTTP upgrade request with the **Upgrade** header set to **HTTP/1.1** by responding with status code 101 (`SC_SWITCHING_PROTOCOLS`) and appropriate headers; otherwise, it rejects the request with status code 400 (`SC_BAD_REQUEST`) and indicates that an upgrade to HTTP 1.1 is required as shown in Figure 9.10.

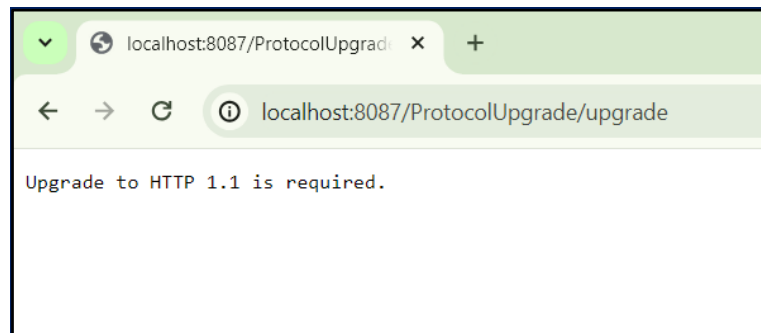


Figure 9.10: Output of Protocol Upgrade

Summary

- The architecture of the Servlet is based on multithreaded model. In this model, the Web container normally creates a pool of threads ready to serve the client with the Servlet instance.
- The Java Servlet API provides asynchronous processing support for servlets and filters in Web applications.
- In asynchronous processing, a Servlet thread waiting for a resource is released by the container and returned to the pool.
- The interface `jakarta.servlet.AsyncContext` is used to process the request asynchronously within the Servlet.
- To handle these events, the `AsyncContext` object can be registered with the `AsyncListener` interface.
- The mechanism of sending the data asynchronously from the Web server to the client without redrawing the whole page, is referred to as server push.
- One of the most popular programming approaches used in modern Web applications to push the server data to the client is performed using AJAX mechanism.
- The AJAX mechanism works with an `XMLHttpRequest` object that is used to pass the requests and responses asynchronously between the client and server.
- Servlet API provides non-blocking I/O support for Servlets and filters to process input and output asynchronously in the request.
- With Servlet 3.0 API, the HTTP protocol has been upgraded from HTTP 1.0 to HTTP 1.1. HTTP 1.0 was a stateless protocol, whereas HTTP 1.1 can persist the connection between the client and the server.

Check Your Progress

1. Which of the following classes does not support the Servlet Non-blocking I/O?

(A)	<code>jakarta.servlet.ServletInputStream</code>
(B)	<code>jakarta.servlet.ServletOutputStream</code>
(C)	<code>jakarta.servlet.ReadListener</code>
(D)	<code>jakarta.servlet.writingListener</code>

2. Which of the following objects is used for performing asynchronous communication in JavaScript?

(A)	<code>XMLHttpRequest</code>
(B)	<code>XMLHttpResponse</code>
(C)	<code>HTMLHttpRequest</code>
(D)	<code>HTMLHttpResponse</code>

3. Which of the following is not the `AsyncListener` method?

(A)	<code>onStartAsync</code>
(B)	<code>onComplete</code>
(C)	<code>onTimeout</code>
(D)	<code>onErrorss</code>

4. Which of the following attribute value is set to true for creating an asynchronous Servlet?

(A)	<code>asyncSupported</code>
(B)	<code>async</code>
(C)	<code>syncSupported</code>
(D)	<code>sync</code>

5. Which of the following methods returns `AsyncContext` object?

(A)	<code>startAsync()</code>
(B)	<code>stopAsync()</code>
(C)	<code>Async()</code>
(D)	<code>firstAsync()</code>

Answers

1.	D
2.	A
3.	D
4.	A
5.	A

Try It Yourself

Implement an asynchronous servlet with an `AsyncListener`:

1. Create a servlet named `AsyncTaskServlet`.
2. In the servlet's **`doGet`** method:
 - a. Start an asynchronous operation using `startAsync()`.
 - b. Set a timeout for the asynchronous context.
 - c. Register an instance of `MyAsyncListener` with the asynchronous context.
 - d. Simulate a long-running task by using `Thread.sleep()`.
 - e. Set an attribute in the request to indicate the completion of the task.
 - f. Forward the request to a JSP page.
3. Create a class named `MyAsyncListener` that implements the `AsyncListener` interface.
4. Implement the `onComplete` method of `MyAsyncListener` to set an attribute in the request to indicate the completion of the asynchronous task.
5. Create a JSP page named `asyncTask.jsp` to display the attribute set by the servlet.
6. Deploy the servlet and JSP page to a servlet container (for example, Apache Tomcat).
7. Access the servlet URL in a Web browser to observe the output.

JSP Implicit Objects and Standard Actions

Welcome to the Session, **JSP Implicit Objects and Standard Actions**.

This session introduces the concept of implicit objects in JSP. It discusses many forms of implicit objects and their uses. This session also provides a summary of the JSP standard actions, their attributes, and their significance. It elaborates on various Scope communication and Servlet object methods. In addition, the session identifies three different categories of directives.

In this session, you will learn to:

- Explain the concept of implicit objects in JSP
- List various types of implicit objects in JSP
- Explain how to use the request and response objects
- Identify use of the out object
- Describe how to use the session object
- Explain the application object
- Identify the use of pageContext object
- Explain how to use the page and config objects
- Outline the concept of standard actions in JSP
- Describe how to use the `<jsp:include>` and `<jsp:forward>` elements

10.1 Introduction

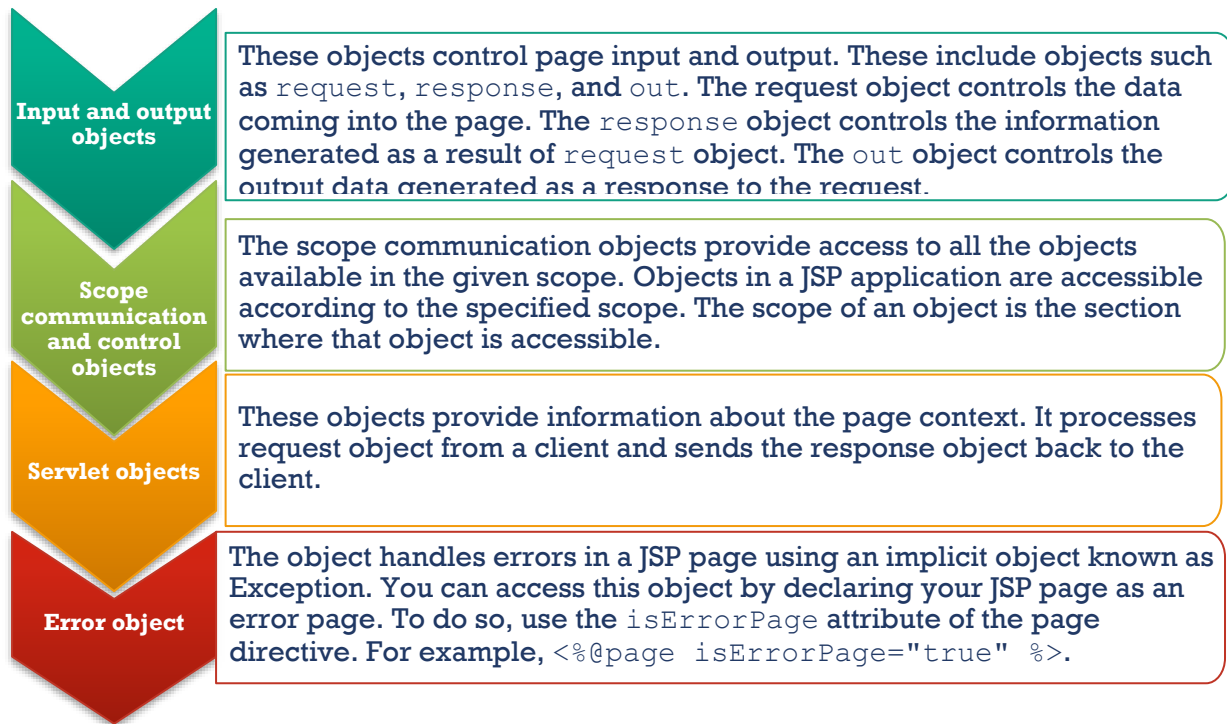
Implicit objects are basically a set of Java objects that are available on every JSP page. These objects are created and loaded by the Web container. This means a Web programmer does not require to create implicit objects in a JSP page, as they are automatically available on the page.

The implicit objects are also referred to as pre-defined variables and are accessible within the scripting elements on the JSP page.

10.1.1 Types of Implicit Objects

When the JSP page is translated into a Servlet class, all the implicit objects declarations are taken within the `_jspService()` method.

Some of the implicit objects provided by JSP are classified into categories that are as follows:



10.2 Input and Output Objects

The input object represents the input data passed through an HTTP request to the JSP page. Similarly, the output object handles the content to be sent to the client in response.

10.2.1 request Object

The `request` object contains the information sent from the client browser through HTTP protocol. The information stored in the `request` object includes the source of the request, the requested URL headers, cookies, and parameters associated with the request.

When the JSP page is translated into the `Servlet` class, the `request` object is passed as a reference of the `jakarta.servlet.http.HttpServletRequest` interface by the container. The scope of the `request` object is page-level, which means that the object will be accessible only on the current page.

Table 10.1 lists some of the methods of the `request` object available on the JSP page for the developers.

Method	Description
<code>public java.lang.String</code>	This method returns value of a request parameter as a string or null if the parameter does not exist.

Method	Description
<code>getParameter(java.lang.String name)</code>	For example, <code>value = request.getParameter("paramName");</code>
<code>public java.lang.String[] getParameterValues(java.lang.String name)</code>	This method returns an array of string objects which contains all the values given request parameter has or null if the parameter does not exist.
<code>public java.util.Enumeration getParameterNames()</code>	For example, <code>String values[] = request.getParameterValues("paramName");</code> This method returns an enumeration of string objects, which contains names of parameters. If the request has no parameters, this method returns an empty enumeration.
<code>public void setAttribute(java.lang.String name, java.lang.Object o)</code>	For example, <code>Enumeration names = request.getParameterNames();</code> This method stores an attribute in the request. Attributes are reset between requests.
<code>public java.lang.Object getAttribute(java.lang.String name)</code>	For example, <code>String personName = new String("Grace");</code> <code>request.setAttribute("name", personName);</code> This method returns name of the string from the named attribute.
<code>public java.lang.String getRemoteHost()</code>	For example, <code>String personName = (String) request.getAttribute("name");</code> This method returns name of the client or the last proxy that sends the request. If the hostname is not resolved, this method returns dotted-string form of the IP address.
<code>public java.lang.String getRemoteAddr()</code>	For example, <code>host: <%=request.getRemoteHost()%></code> This method returns the Internet protocol address of the client or last proxy that sent the request. For example, <code>addr: <%=request.getRemoteAddr()%></code>

Table 10.1: Methods of request Object

Code Snippets 1 and 2 demonstrate the methods of `request` object to receive data from a registration page.

Code Snippet 1: `index.jsp`

```
<%@page import="java.util.Enumeration"%>
```

```

<%@ page language="java" contentType="text/html; charset=UTF-
8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Index Page</title>
</head>
<body>
    <form action="register.jsp">
        First Name:
        <input type="text" name="firstname">
    <br/> <br/>
        Favorite Game:
        <input type="checkbox" name="game"
value="Cricket">Cricket<BR>
        <input type="checkbox" name="game"
value="Hockey">Hockey<BR>
        <input type="checkbox" name="game"
value="Baseball">Baseball<BR>
        <input type="checkbox" name="game"
value="Badminton">Badminton<BR>
    <br/> <br/>
        <input type="submit" value="Submit" name="Submit">
    </form>
</body>
</html>

```

Code Snippet 2: register.jsp

```

<%@page import="java.util.Enumeration"%>
<%@ page language="java" contentType="text/html; charset=ISO-
8859-1" pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Register</title>
</head>
<body>
<%
    // Retrieves data from text field
    String firstName = request.getParameter("firstname");
    // Retrieves data from checkboxes and listbox
    String favGame[] = request.getParameterValues("game");
    // Displays the form data

```



```

out.println("Name is: " + "<font color='blue'>" +
            firstName + "</font>");
out.println("<br/><br/>Favorite Game: " + "\n");
for (int i = 0; i < favGame.length; i++) {
    out.println("\t<font color='blue'>" + favGame[i]
        + "</font>");
}
%>
<br/> <br/>
The parameters sent to the Servlet are: <br/>
<font color="blue">
    <%
        Enumeration enumParam = request.getParameterNames();
        while (enumParam.hasMoreElements()) {
            String str = (String)
                enumParam.nextElement();
            out.println(str);
        }
    %>
</font>
</body>
</html>

```

Figure 10.1 shows the output of Code Snippets 1 and 2.

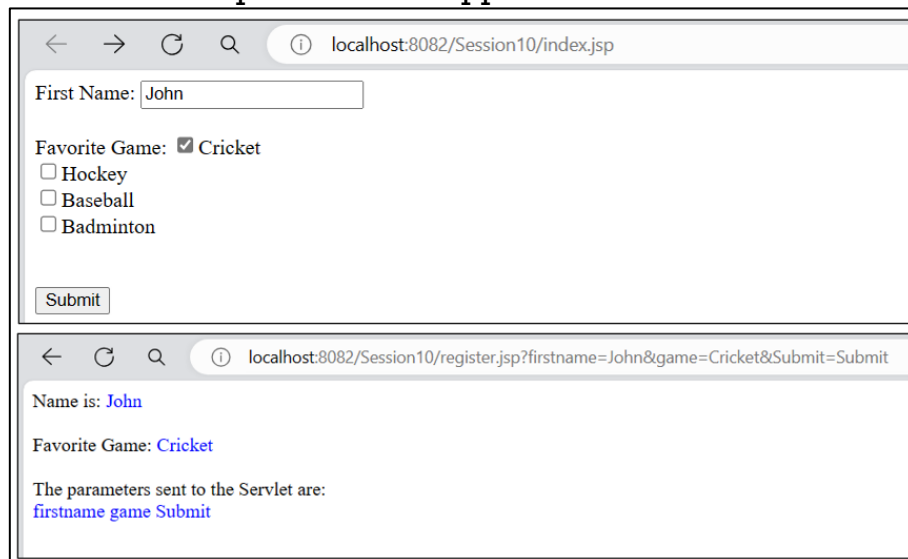


Figure 10.1: Output – request object

In the code, the `getParameter()` method is used to retrieve a value from the text field which is a string value. To retrieve multiple selected values from the checkbox, `getParameterValues()` method is used. This method returns multiple values in an array.

The `getParameterNames()` method is used to retrieve the names of the parameters sent in the request string.

10.2.2 response Object

The `response` implicit object manages the response generated by JSP and uses HTTP protocol to send the response to the client. The `response` object is a reference of the `jakarta.servlet.http.HttpServletResponse` interface.

Table 10.2 lists some of the methods of `response` object to be used on the JSP page.

Method	Description	Example
<pre>public void addCookie(Cookie cookie)</pre>	The method adds the specified cookie to the response. This method can be called more than once to set more than one cookies.	<pre>Cookie MyCookie = new Cookie("RollNumber", "156"); MyCookie.setMaxAge(60 *60*24*7*26); response.addCookie(My Cookie);</pre>
<pre>public void sendRedirect(java.l ang.String location)throws java.io.IOException)</pre>	The method sends a temporary redirect response to the client using the specified redirect location URL. This method can also accept relative URLs. The method is used to encode the specified URL by including session id. This form of the url can be used in html tags that use a url, such as <code><a href.....></code>. This enables the server to keep track of the session.	<pre>response.sendRedirect ("myserver.com/thePag e.htm?ID=737");</pre>
<pre>public java.lang.String encodeURL(java.lang. String url)</pre>	The method returns a rewritten url that can be used with the <code>sendRedirect</code> method of <code>response</code> object.	<pre>response.encodeURL("b uyerPage.jsp");</pre>
<pre>public java.lang.String encodeRedirectUrl(j ava.lang.String url)</pre>		<pre>if (name == null){ response.sendRedirect (response.encodeRedir ectURL("homePage.jsp")) }</pre>

Table 10.2: Methods of response Object

10.2.3 out Object

The `out` object represents the output stream. This output stream will be sent to the client as a response to the request. The `out` object is an instance of the `jakarta.servlet.jsp.JspWriter` class. It uses all standard `write()`, `print()`, and `println()` methods defined in `jakarta.servlet.jsp.JspWriter` class to display the output. It has page scope.

Some of the methods of `out` object are as follows:

➤ **`public abstract void flush() throws java.io.IOException`**

This method flushes the buffer. One invocation of `flush()` will flush all the buffers in a chain of writers and output streams. The `out` object also performs implicit flushing under certain conditions, such as:

- When the buffer reaches its capacity.
- When the JSP page completes its execution.
- When certain JSP tags or expressions are used that trigger output flushing, such as `<jsp:useBean>` or expressions within `<% %>`.

Code Snippet 3 shows how to flush the page content.

Code Snippet 3:

```
String message = "Welcome!";
out.print(message);
out.flush();
```

Note: In Jakarta EE 10, `out.flush()` and `out.getBufferSize` are generally not recommended. There are a few rare cases where you might consider flushing the buffer, but it is usually not necessary for standard Web development. While `out.flush()` can be used for real-time data in some cases, Jakarta EE provides more efficient and modern approaches for this task. Server-Sent Events (SSE) and WebSockets are built-in features that ensure real-time data delivery without manual buffering.

➤ **`public abstract void clear() throws java.io.IOException`**

This method clears the contents of the buffer. If the buffer has already been flushed, then, the clear operation will throw an `IOException`. Code Snippet 4 shows how to clear the buffer.

Code Snippet 4:

```
// If buffer is not empty, buffer is cleared
if (out.getBufferSize() != 0)
    out.clear();
```

➤ **`public abstract void close() throws java.io.IOException`**

For the `JspWriter`, this method does not have to be invoked explicitly as the code generated by the JSP container will automatically include `close()`.

Code Snippet 5 shows the use of `close()` method.

Code Snippet 5:

```
out.print("Welcome!!!");
out.close();
```

- **public abstract void println(java.lang.String x) throws java.io.IOException**

This method prints a string and then, terminates the line. It behaves as a combination of `print(String)` and `println()`.

For example, `out.println("The output is:" + ex);`

- **public abstract void print(java.lang.String s) throws java.io.IOException**

This method prints a string. If the argument is `null`, then the string '`null`' is printed.

Code Snippet 6 demonstrates the use of `print()` method.

Code Snippet 6:

```
out.print("Welcome!!!");
out.print("To Aptech Training");
```

10.3 Scope Communication Objects

Apart from the implicit objects, a JSP page can also access Java objects that are created explicitly. These objects are either created within the scriptlets or through actions. Example of an action can be creation of a `JavaBean` object on the Web page.

Each created object is associated with a scope attribute which defines where the object is accessible and its lifetime.

10.3.1 session Object

The `session` object provides all the objects available in the JSP pages within the session. A session expires when the user does not send any request for a long time or when the user closes the Web browser. This object is an instance of `jakarta.servlet.http.HttpSession` interface. It has a session scope.

Some of the methods are as follows:

➤ **if (session == null)**

In Web applications such as shopping cart, when a visitor opens a page, the server must check whether a session already exists or not. `if` statement checks if the retrieved session object is null then, it means there is no existing session available. This will return false, if a session already exists and true, otherwise.

➤ **public void setAttribute(java.lang.String name, java.lang.Object value)**

In shopping cart, when a visitor opens a page with the username, the `setAttribute` method holds the objects in the existing session.

Code Snippet 7 demonstrates how to associate a new session object with the request.

Code Snippet 7: Login.jsp

```
<%@ page import="com.example.UserLogin1" %>
<%@ page session="false" %>
<html>
<head>
    <title>Login Form</title>
</head>
<body>
    <h2>Login Form</h2>
    <form action="" method="post">
        <table>
            <tr>
                <td>User:</td>
                <td><input type="text" name="username"
size="15"></td>
            </tr>
            <tr>
                <td>Password:</td>
                <td><input type="password" name="password"
size="15"></td>
            </tr>
            <tr>
                <td colspan="2"><input type="submit"
value="Login"></td>
            </tr>
        </table>
    </form>
    <%
        if (request.getMethod().equals("POST")) {
            System.out.println("Entered");
            String username =
request.getParameter("username");
```

```

        String password =
request.getParameter("password");
        System.out.println(username);
        System.out.println(password);
        HttpSession session = request.getSession(false);
        if (session == null) {
            session = request.getSession(); // Create new
session
            UserLogin1 userLoginObj= new
UserLogin1(username,password);
            session.setAttribute("loginuser",
userLoginObj);
            out.print("New Session Created");
            response.sendRedirect("Name.jsp");
        } else {
            // Existing session logic
            UserLogin1 userLoginObj= new
UserLogin1(username,password);
            session.setAttribute("loginuser",
userLoginObj);
            System.out.println(username);
            out.print("Existing Session Found");
        }
    }
%>
</body>
</html>

```

Code Snippet 8: UserLogin1.java

```

package com.example;
public class UserLogin1 {
    private String username;
    private String password;

    public UserLogin1(String username, String password) {
        this.username = username;
        this.password = password;
    }
    public String getUsername() {
        return username;
    }
    public void setUsername(String username) {
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {

```

```

        this.password = password;
    }
}

```

➤ **public java.lang.Object getAttribute(java.lang.String name)**

The method returns the object bound with the specified name in the current session, otherwise returns `null`.

Code Snippet 9 shows how to access the named session object.

Code Snippet 9: Name.jsp

```

<%@ page language="java" import="com.example.UserLogin1"
contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<%
    UserLogin1 user1 = (UserLogin1)
    session.getAttribute("loginuser");
%>
<table border="1">
<tr>
<td> User: </td>
<td> <% out.print(user1.getUsername()); %> </td>
</tr>
</table>
</body>
</html>

```

In the code, the `userLoginObj` is an object of the `UserLogin1` class, which stores session data, such as username and password. The `getAttribute()` method retrieves values stored in the `userLoginObj` object.

Create a Dynamic Web Project with all the three codes and then, run it. Figures 10.2 and 10.3 show the output of Code Snippets 7, 8, and 9.

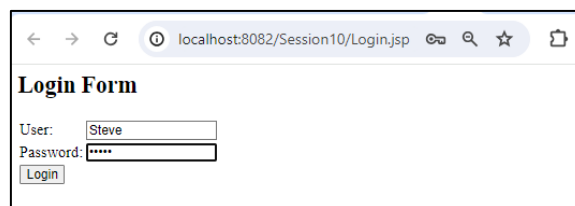


Figure 10.2: Output of Login.jsp

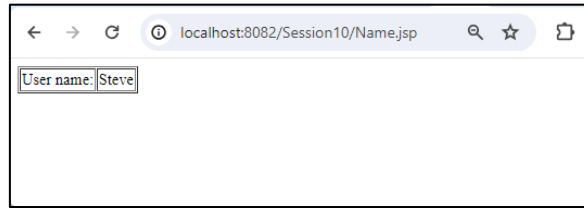


Figure 10.3: Output of Name.jsp

- **public java.util.Enumeration getAttributeNames()**
The method returns an enumeration of string objects. The string contains the names of all the objects bound to this session.
- **public void invalidate()**
The method invalidates a session and then, releases the objects bound to it. Code Snippet 10 shows how to invalidate a session.

Code Snippet 10:

```
<%
request.getSession().invalidate();
%>
```

- **public void removeAttribute(java.lang.String name)**
The method removes the object bounded with the specified name from this session. For example, `session.removeAttribute("loginuser");`

10.3.2 application Object

The application object is used to share the data between all the application pages. So, all users can share the information of a given application using the application object. The application object can be accessed by any JSP present in the application. The application object is an instance of the `jakarta.servlet.ServletContext` interface. It has application scope.

Some of the methods of application object are as follows:

- **public void setAttribute(java.lang.String name, java.lang.Object object)**
The method binds an object to a given attribute name in this servlet context. Code Snippet 11 shows how to set an attribute to be accessed on any JSP page in the application.

Code Snippet 11:

```
. . .
<%
    String Initialized = "yes";
    application.setAttribute("init", Initialized);
%>
```


- **public java.lang.Object getAttribute(java.lang.String name)**
The method returns the servlet container attribute with the given name or null, if there is no attribute.
- **public java.util.Enumeration getAttributeNames()**
The method returns an enumeration containing the attribute names available within this servlet context.

Code Snippet 12 demonstrates how to retrieve all application objects set in the application.

Code Snippet 12:

```
<%@page import="java.util.Enumeration"%>
<%@page import="java.lang.String"%>
...
<%
    String username="john";
    String password="apt001";
    application.setAttribute("username", username);
    application.setAttribute("password", password);
    // Retrieves all attributes
    Enumeration<String> en =
        application.getAttributeNames();
    // Print the attributes
    while(en.hasMoreElements()){
        String attrName=en.nextElement();
        if ("username".equals(attrName) ||
"password".equals(attrName)) {
            Object attrValue =
                application.getAttribute(attrName);
            out.println(attrName + ": " + attrValue + "<BR>");
        }
    }
%>
```

- **public void removeAttribute(java.lang.String name)**
The method removes the attribute with the given name from the servlet context.
For example, `application.removeAttribute("password");`
- **public java.lang.String getServerInfo()**
The method returns the name and version of the servlet container on which the servlet is running. For example, `out.println("Server Information:"+ application.getServerInfo());`

➤ **public void log(java.lang.String msg)**

The method writes the specified message to a servlet log file, usually an event log. For example, `application.log (Hello log Message);`

10.3.3 pageContext Object

The `pageContext` object allows user to access all the implicit objects defined in the **page scope**. The `pageContext` object is an instance of the `jakarta.servlet.jsp.PageContext` class. It uses methods defined in the `PageContext` class to access implicit objects in a Web page.

The `jakarta.servlet.jsp.PageContext` class provides following fields that are used by `pageContext` object to find the scope or specify the scope of the objects:

PAGE_SCOPE	REQUEST_SCOPE	SESSION_SCOPE	APPLICATION_SCOPE
This specifies the scope for attributes stored in the <code>pageContext</code> object. This is the default.	This specifies the scope for attributes stored in the request object. It remains available until current request is complete.	This specifies the scope for attributes stored in the session object. It remains available until <code>HttpSession</code> object is invalidated.	This specifies the scope for attributes stored in the application object. It remains available until it is reclaimed.

The `pageContext` object provides methods to access all the attributes defined by implicit object in a page. The `pageContext` object also provide methods to transfer the control from one Web page to another.

The methods are as follows:

➤ **public abstract void setAttribute(java.lang.String name, java.lang.Object value, int scope)**

The method registers the name and value specified with appropriate scope. If the value passed in is null, this method functions in the same way as `removeAttribute(name, scope)`.

➤ **getAttribute()**

This method returns the object associated with the name in the page scope, otherwise, it returns null. It throws `java.lang.NullPointerException`, if the name is null.

Syntax:

```
public abstract java.lang.Object getAttribute(java.lang.String name)
```

where,

name is the attribute name to be retrieved.

Code Snippet 13 shows the `pageContext` object used to set and retrieve the object.

Code Snippet 13:

```
<%
    String attributeName = "personName"; // Replace with a
                                         //meaningful name

    String personName = "Clive";
    pageContext.setAttribute(attributeName, personName,
PageContext.APPLICATION_SCOPE);%>

. . .

<%
String retrievedName = (String)
pageContext.getAttribute(attributeName,
PageContext.APPLICATION_SCOPE);
    if (retrievedName != null) {
        out.println("Person name: " + retrievedName);
    } else {
        out.println("Attribute 'personName' not found in
application scope.");
    }
%>
```

- **public abstract java.util.Enumeration
getAttributeNamesInScope(int scope)**

This method enumerates all the attributes in a given scope. It throws `java.lang.IllegalArgumentException` if the scope is invalid.

Code Snippet 14 demonstrates how to retrieve all the objects with request scope.

Code Snippet 14:

```
// Prints all attribute names with request scope
<%
    for (java.util.Enumeration e =
pageContext.getAttributeNamesInScope(PageContext.REQUEST_S
COPE);
        e.hasMoreElements(); ) {
        String attrName = (String) e.nextElement();
        out.println(attrName + "<br>");
    }
%>
```

- **public abstract void removeAttribute(java.lang.String name)**

This method removes the object reference associated with the given name from all the scopes. It throws `java.lang.NullPointerException`, if the name is `null`.

Code Snippet 15 demonstrates how to remove the objects from the JSP page.

Code Snippet 15:

```
<%
    out.println("User Name: " + request.getAttribute("Name")
+ "<br/>");
    out.println("Password: " +
request.getAttribute("Password") + "<br/>");
    pageContext.removeAttribute("Password");
%>
```

10.4 Servlet Objects

Servlet object is a representation of the JSP page.

10.4.1 page Object

In JSP, the `page` object (or `this` keyword) provides a reference to the current JSP page itself. It is not directly an instance of `java.lang.Object` but has a connection through the underlying servlet. While not always used explicitly, it can be helpful for accessing page context and attributes within the page scope.

Code Snippet 16 demonstrates how to access the methods of the Servlet through `page` object.

Code Snippet 16:

```
<%
    out.println(this.getServletInfo());
    out.println(((Servlet)page).getServletInfo());
%>
```

As the variable `page` is of the type `Object`, to access the Servlet methods, you have to typecast it with the `Servlet`.

10.4.2 config Object

The `config` object stores the information of the servlet, which is created during the compilation of a JSP page. The `config` object is an instance of the `jakarta.servlet.ServletConfig` interface. The interface provides methods to retrieve servlet initialization parameters. The `config` object represents the configuration of the servlet data where a JSP page is compiled. The configuration data is stored in the form of initialization parameters. It has `page` scope.

Some of the methods are as follows:

- **public java.lang.String getInitParameter(java.lang.String name)**

This method returns a string that contains the value of the named initialization parameter, otherwise it returns null.

- **public java.util.Enumeration getInitParameterNames()**

This method returns the names of the servlet's initialization parameters as an enumeration of string objects. Otherwise, it returns an empty enumeration object, if the servlet has no initialization parameters.

Code Snippet 17 first retrieves all the init parameters `getInitParameterNames()` and then, prints each parameter name using `getInitParameter()`. It demonstrates the method that retrieves the initialization parameters on the JSP page.

Code Snippet 17:

```
public void getParameters(ServletConfig config) throws
ServletException {
    // Retrieves all init parameters
    Enumeration params = config.getInitParameterNames();
    // Prints the init parameter names and values
    while (params.hasMoreElements()) {
        String name = (String) params.nextElement();
        System.out.println("Parameter Name: " + name +
            " Value: " + config.getInitParameter(name));
    }
}
```

- **public ServletContext getServletContext()**

This method returns a reference to the servlet context.

Code Snippet 18 shows how to set a JavaBean object in the `ServletContext`.

Code Snippet 18:

```
ServletContext context = config.getServletContext();
context.setAttribute("dbBean", dbBean);
```

- **public java.lang.String getServletName()**

This method returns the name of this servlet instance. It can get the name that may be provided through the server administration or assigned in the Web application deployment descriptor. For example, `String servletName = config.getServletName();`

10.5 error Object

While executing and processing client requests, runtime errors can occur either inside or outside the JSP page. These errors can be processed using the `exception` object present in the JSP page.

When an error occurs, the execution of a JSP page is terminated. The `exception` object is used to handle errors in a JSP page. The `exception` object is used to trace the exception thrown during the execution.

The `exception` object is available to the JSP page that is assigned as an error page. This object is the instance of `java.lang.Throwable` class. It has page scope.

Some of the methods include:

<pre>public String getMessage()</pre>	<pre>public void printStackTrace(PrintWriters)</pre>	<pre>public String toString()</pre>
This method returns the descriptive error message associated with the exception when it was thrown.	This method prints the execution stack in effect when the exception was thrown to the designated output	This method returns a string combining the class name of the exception with its error message.

The `isErrorPage` attribute of `page` object is set to `true`, which indicates that the page is an error page. The JSP expression (`<%= %>`) is used to create an instance of the `exception` object.

The `out` object is passed as an argument to the `printStackTrace` method which will display the stack trace information.

Code Snippet 19 demonstrates how to handle exceptions in the JSP.

Code Snippet 19:

```
<%@ page isErrorPage="true" %>
<html>
<head>
    <title>Implicit Object</title>
</head>
<body>
    <h1>Implicit Object: Exception</h1>
    Following error has been detected :<br>
    <b><%= exception %></b><br>
    <% exception.printStackTrace(response.getWriter()); %>
</body>
</html>
```

10.5.1 Use of Standard Actions

JSP standard actions are performed when a browser requests for a JSP page.

Properties of standard actions in JSP are as follows:

- It uses `<jsp>` prefix.
- The attributes are case-sensitive.
- Values in the attributes must be enclosed in double-quotes.
- Standard actions can be either an empty or a container tag.

Syntax:

```
<jsp:action_name attribute="value" attribute="value"/>
```

or

Syntax:

```
<jsp:action_name attribute="value" attribute="value">
.
.
.
</jsp:action_name>
```

where,

`action_name` is the action to be used.

`attribute` is replaced by the attributes of the specified action.

Various standard actions that are available in JSP, namely, `<jsp:include>`, `<jsp:forward>`, `<jsp:param>`, `<jsp:params>`, and `<jsp:text>`.

10.5.2 `<jsp:include>`

The `<jsp:include>` element gives developers choice to include either a static or dynamic file in the current requested JSP page is executed. The results are different for each type of inclusion.

If the file is static, then the content is included in the calling JSP file. In case of dynamic, it acts on a request and sends back a result that is included in the JSP page. Developers cannot determine whether a file is static or dynamic from a pathname. When a developer has no idea whether the file is static or dynamic, it is better to use `<jsp:include>` element, because it handles both types of files.

Syntax:

```
<jsp:include page="weburl" flush="true" />
```

where,

`page` specifies the relative Web address of the page to be included in the current page.

`flush` attribute is used to flush out the data stored in the buffer. The value of `flush` attribute is false by default.

Code Snippets 20 and 21 demonstrate the use of `<jsp:include>` action using `index.jsp` and `printdate.jsp` files to display current date and time on the JSP page.

Code Snippet 20: `index.jsp`

```
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html;
charset=UTF-10">
    <title> Dynamic Content Inclusion </title>
  </head>
  <body>
    <h4><font color="BLUE"> Displaying Current Date and
      Time </font></h4>
    <b>Today is: </b> <jsp:include page="printdate.jsp"/>
    <p><i> The Date and Time are displayed as a result of
      evaluation of another JSP page. </i></p>
  </body>
</html>
```

Code Snippet 21: `printdate.jsp`

```
<%@page contentType="text/html" pageEncoding="ISO-8859-1"
import="java.util.*"%>
<html>
  . . .
  <body>
    <%
      Date today = new Date();
      out.print(today.toString());
    %>
  </body>
</html>
```

Figure 10.4 shows the output of Code Snippet 20.

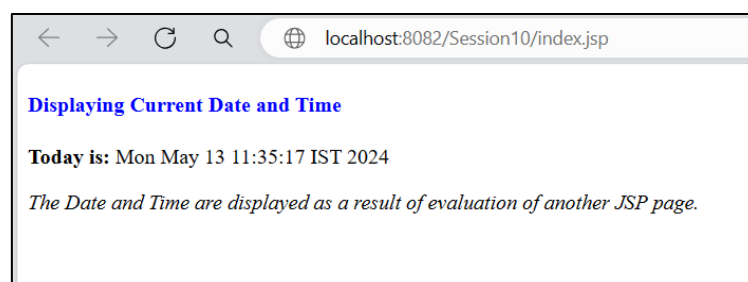


Figure 10.4: Output – `jsp:include` Action

10.5.3 <jsp:forward>

The <jsp:forward> element is used to redirect the request object containing the client request from one JSP to another target page. The target page can be an HTML file, another JSP file, or a Servlet.

In the source JSP file, the code after the <jsp:forward> element is not processed. To pass parameter names and values to the target file, you can use a <jsp:param> clause with <jsp:forward> element.

Syntax:

```
<jsp:forward page="url"/>
```

where,

page specifies the relative Web address of the target page.

Code Snippets 22 and 23 demonstrate the use of <jsp:forward> action using index.jsp and printdate.jsp files to display current date and time on the next JSP page.

Code Snippet 22: index.jsp

```
<%@page contentType="text/html" pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
. . .
    <body>
        <h4><font color="BLUE"> Displaying Current Date and Time
        </font></h4>
        <!-- Forwards request to the other Web resource -->
        <jsp:forward page="printdate.jsp"/>
        <p><i> The request is forwarded to the next page to
            display Date and Time. </i></p>
    </body>
</html>
```

Code Snippet 23: printdate.jsp

```
<%@page contentType="text/html" pageEncoding="ISO-8859-1"
import="java.util.*"%>
<html>
. . .
<body>
    <%! Date today = new Date();%>
    <b> Today is: </b> <% out.print(today.toString());%>
</body>
</html>
```

Figure 10.5 shows the output of Code Snippets 22 and 23.

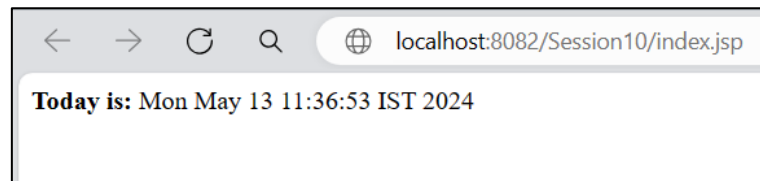


Figure 10.5: Output – jsp:forward Action

10.5.4 <jsp:param>

The `<jsp:param>` element allows you to pass one or more name and value pairs as parameters to an included or forwarded resource such as a JSP page, Servlet, or other resource that can process the parameter.

If you want to send more than one parameter to the included or forwarded resource, you can use more than one `<jsp:param>` clause.

Syntax:

```
<jsp:param name="parameterName" value="{parameterValue | <%=
expression %>}" />
```

where,

name attribute specifies the parameter name and that takes a case-sensitive literal string.

value attribute specifies the parameter value and takes either a case-sensitive literal string or an expression that is evaluated at request time.

Code Snippets 24 and 25 demonstrate forwarding request to the page along with the parameter values.

Code Snippet 24: index.jsp

```
<html>
. . .
<body>
    <jsp:include page="Order.jsp" flush="true">
        <jsp:param name="currency_type" value="Dollar"/>
        <jsp:param name="amount" value="$110"/>
    </jsp:include>
</body>
</html>
```

Code Snippet 25: Order.jsp

```
. . .
<body>
    <% String currency = request.getParameter("currency_type");
```

```
String amount = request.getParameter("amount");
%>
<b><u> Param Values </u></b> <br/><br/>
<% out.println("Currency: " + currency); %> <br/><br/>
<% out.println("Amount: " + amount); %>
</body>
```

Figure 10.6 shows the output of Code Snippets 24 and 25.

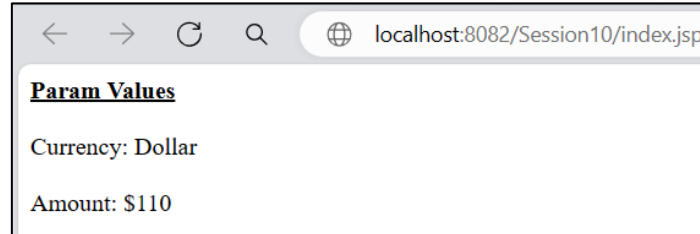


Figure 10.6: Output – jsp:param Action

10.5.5 <jsp:text>

A `<jsp:text>` element encloses contents that are displayed within the body of a JSP page or a JSP document. It does not have any sub-element and can appear anywhere in the JSP page where content has to be displayed in the output.

The `<jsp:text>` element can contain expressions which are evaluated, and their result is displayed in the output. Code Snippet 26 demonstrates the use of expression language with `<jsp:text>` element.

Code Snippet 26: Rectangle.jsp

```
<html>
  <body>
    <jsp:text>
      This is the details page.
    </jsp:text>
    . . .
    <jsp:text>
      Rectangle Perimeter is: ${2* 10 + 2* 20}
    </jsp:text>
  </body>
</html>
```

Figure 10.7 shows the output of Code Snippet 26.

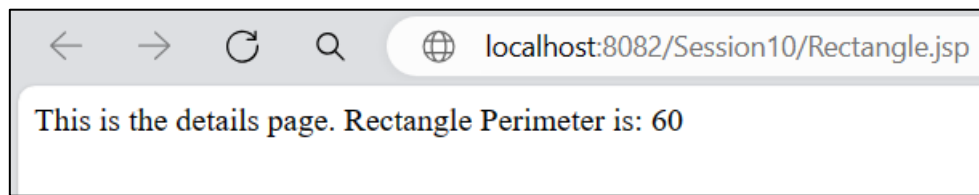


Figure 10.7: Output – `<jsp:text>`

10.6 JSP Directives

JSP directives are a form of messages that provide comprehensive information about the JSP page. The container uses these messages to translate a JSP page into a Servlet code. JSP directives have multiple attributes, and all of these attributes are set up as comma-separated key-value pairs.

Three different categories of directives are as follows:

1. `page` directive
2. `taglib` directive
3. `include` directive

10.6.1 `page` Directive

The information provided by the page directives help the Web container convert a Web page into a Servlet during JSP translation phase.

The page directive specifies properties for a whole JSP page.

Syntax:

```
<%@ page attribute="value" %>
```

Few attributes of `page` directives are as follows:

info

The `info` attribute allows the developer to provide a description of the JSP page. If required this information can be accessed at a later time using the `getServletInfo()` method.

Syntax:

```
<%@ page info="created by abc" %>
```

language

The `language` attribute specifies the language used to design the JSP page. By default, the value of the `language` attribute is set to `java`.

Syntax:

```
<%@ page language = "value" %>
```

session

The `session` attribute gives information about the current session. It reveals details such as whether or not the current JSP page participates in the current HTTP session. The boolean value `true` or `false` can be assigned to the `session` attribute.

Syntax:

```
<%@ page session="true" %>
<%@ page session="false" %>
```

isThreadSafe

The `isThreadSafe` option identifies whether or not the page will handle multiple threads simultaneously without any undesired interactions. If the developer sets the `isThreadSafe` attribute to `false`, the Web container places all the requests in a serial fashion and ensures that the JSP finishes responding to the initial request before delivering another request to it. Typically, Servlet and JSP pages can process multiple threads simultaneously.

Syntax:

```
<%@ page isThreadSafe="false" %>
<%@ page import="xyz" language="xyz" extends="xyz" session="xyz" buffer="xyz" isThreadSafe="xyz" autoFlush="xyz" errorPage="xyz" info="xyz" contentType="xyz" isErrorPage="xyz" %>
```

contentType

The `contentType` attribute specifies the character encoding scheme. Basically, it reveals the HTTP response's Multipurpose Internet Mail Extension (MIME) type. The default value is set to `text/html; charset=ISO-1010510-1`.

Syntax:

```
<%@page contentType = "text/html" %>
```

import

The `import` attribute is responsible for instructing the Web container regarding the classes and packages that must be imported into the program.

Syntax:

```
<%@page import = "java.util.Date"%>
<%@page import = "java.io.*"%>
<%@page import = "java.util.*, java.net.*, java.io.*" %>
```

extends

The `extends` attribute is employed to extend the superclass. Developers use this attribute to inherit a class from a JSP file.

Syntax:

```
<%@ page extends="pack.Demo" %>
```

errorPage

The `errorPage` attribute determines where the user should be redirected if an error occurs on the current JSP page.

Syntax:

```
<%@ page errorPage="myerror.jsp"%>
```

isErrorPage

The `isErrorPage` attribute indicates whether the current JSP page is a URL supplied by the `errorPage` attribute of another JSP page.

Syntax:

```
<%@page isErrorPage="true/false"%>
```

isELIgnored

The `isELIgnored` attribute enables developers to prevent the evaluation of Expression Language (EL) expressions. The default value of the `isELIgnored` attribute is typically set to "true"

Syntax:

```
<%% page isELIgnored="true" %>
```

pageEncoding

The `pageEncoding` attribute specifies the character encoding used for JSP pages.

Syntax

```
<%@ page pageEncoding="ISO-8859-1" %>
```

buffer

The `buffer` attribute is used to set a fixed buffer size for JSP page output. Typically, the default buffer size is set to 10 KB.

Syntax:

```
<%@page buffer = "size in kb"%>
```

autoFlush

The `autoFlush` attribute determines what action will be taken if the buffer overflows. Depending on the value assigned to this attribute, either the buffered output will be automatically flushed when full, or an exception will be thrown instead. The default option is commonly set to true, which indicates that the buffer will be flushed automatically when it reaches its capacity.

Syntax:

```
<%@page autoFlush="true"%>
```

10.6.2 include Directive

The `include` directive is used during page translation to insert content of another resource in a JSP page at run time. The contents are included physically during the translation phase. The resource or content can be a file in a text-based format, such as text, HTML, or JSP. A remote resource if included should be given with the proper relative file path in the `include` directive.

Syntax:

```
<%@ include file="resourceName to be included" %>
```

Code Snippets 27 to 29 display the use of `include` directive.

Code Snippet 27: header.jsp

```
<body>
I am Header part
</body>
```

Code Snippet 28: footer.jsp

```
<body>
I am footer part : copywrite &copy;
</body>
```

Code Snippet 29: index.jsp

```
<body>
    <div> <%@include file="header.jsp" %></div>
    <div><b> Today is: </b> <% out.print(new
Date().toString());%>
    </div>
    <div><%@include file="footer.jsp" %></div>
</body>
```

The output is shown in Figure 10.8.

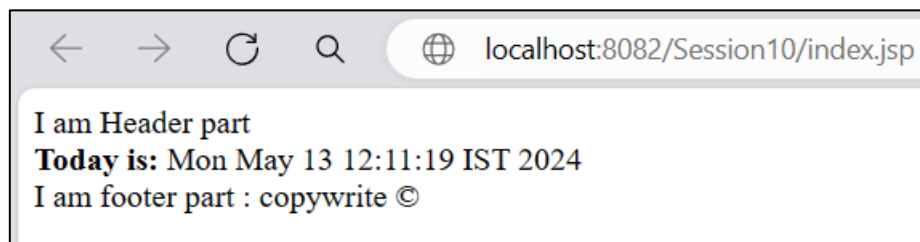


Figure 10.8: Output of include Directive

10.6.3 taglib Directive

JSP taglib directive is used to define or specify the tag library used for custom tags. In other words, the taglib directive allows the JSP page to create custom tags, which are defined by the user.

Custom tags have access to all the objects that are available for a JSP page.

A tag library is a group of custom tags that extend the functionality of a JSP page one after the other. The `taglib` directive specifies name of the tag library, which contains compiled Java code for a tag to be used. Using this tag library, the JSP engine determines what action is to be taken when a particular tag appears in a JSP page.

Syntax:

```
<%@ taglib uri="uri" prefix="prefixForCustomTag">
```

Following are the attributes of JSP `taglib` directive:

1. **URI:** It is used to provide location of the tag.
2. **Prefix:** It is used to specify the markup that is used for custom actions.

Summary

- Implicit objects are a set of Java objects that are available on every JSP page.
- Pre-defined variables are accessible within the scripting elements in the JSP pages.
- The input object represents the data passed through an HTTP request and the output object represents what is sent back to the client in response.
- The session object provides all the objects available in the JSP pages within the session.
- The application object is used to share the data between all the application pages.
- JSP directives are a form of messages that provide comprehensive information about the JSP page. During JSP translation phase, these directives inform the Web container on how to convert a Web page into a Servlet.
- Servlet and JSP pages can process multiple threads simultaneously. The `isThreadSafe` option helps to identify whether a JSP page is thread-safe or not.

Check Your Progress

1. Which of the following statements is correct about the `request` object?

(A)	The <code>request</code> object uses the HTTP protocol to process the request from the server.
(B)	The HTTP protocol implements a subclass of the <code>javax.jsp.HttpServletWriter</code> interface.
(C)	The <code>getParameter()</code> method returns the value of a request parameter as a string.
(D)	The <code>getParameterValues()</code> method returns a string containing the values of all the methods specified in the request parameter.

2. Which of the following statements is correct about the `response` object?

(A)	The <code>response</code> implicit object manages the response generated by JSP and sends the response to the client.
(B)	The <code>sendRedirect(url)</code> method sends a response to the server using the specified redirect location URL.
(C)	The <code>addCookie(cookie)</code> method adds the specified cookie to the response.
(D)	The <code>addCookie(cookie)</code> method is unable to set more than one cookie.
(E)	The <code>encodeURL()</code> method is used to encode the specified URL.

3. Which of the following statements is correct about the `out` object?

(A)	The <code>out</code> object is an instance of the <code>javax.JspWriter</code> class.
(B)	The <code>flush()</code> method flushes all the buffers in a chain of Writers and OutputStreams.
(C)	The <code>clear()</code> method clears the contents of the buffer and prints the string in the same line.
(D)	The <code>print(text)</code> method behaves as the combination of <code>print(String)</code> and <code>println()</code> .

4. Which one of the following statements is correct about the `session` object?

(A)	The <code>session</code> object provides all the objects available in the servlet pages.
(B)	A session expires when the user closes the Web browser.
(C)	The session continues after the user stops sending requests to the Web server.
(D)	The <code>session</code> object is an instance of <code>jakarta.servlet.http.HttpServletRequest</code> interface.

5. Which one of the following statements is correct about the `application` object?

(A)	The <code>application</code> object is used to share the data only between specified pages.
(B)	The <code>getAttribute()</code> method returns the servlet attribute and null, if there is more than one attribute.
(C)	The <code>setAttribute()</code> method binds an object to a given attribute name in this servlet context.
(D)	The <code>getServerInfo()</code> method returns the name of the servlet attribute on which the servlet is running.

6. Which one of the following statements is correct about the `pageContext` object?

(A)	The <code>pageContext</code> object allows the user to access all the implicit objects defined in the page scope.
(B)	The <code>setAttribute()</code> method registers the name and value outside the scope.
(C)	The <code>removeAttribute()</code> method removes the object reference associated with the given name from all the scopes.
(D)	The <code>APPLICATION_SCOPE</code> field specifies the scope for attributes stored in the <code>pageContext</code> object.
(E)	The <code>PAGE_SCOPE</code> field specifies the scope for attributes stored in the <code>pageContext</code> object.

Answers

1.	C
2.	A
3.	A
4.	B
5.	C
6.	A

Try It Yourself

1. Create a JSP file named `Expression.jsp` that uses the JSP Expression Language to display the perimeter of a square and a triangle. .
2. Create JSP files named `CricketMatchDetails.jsp` and `PlayerDetails.jsp` to demonstrate the use of the `<jsp:include>` action for displaying information. Perform the given operations.
 - a. In `CricketMatchDetails.jsp`:
 - i. Display 'Match Location' as 'Cape Town' and 'Match Date' as "14 Jun 2024".
 - ii. Use `jsp:include` to include content from `PlayerDetails.jsp`.
 - b. In `PlayerDetails.jsp`:
 - i. Display 'Player Name' as 'John' and 'Player Age' as '34'.
 - ii. Display the current date (day, month, year) and time (hours, minutes, seconds) using Java code.

Model-View-Controller Architecture

Welcome to the Session, **Model-View-Controller Architecture**.

This session briefly explains the JSP models followed in Web application development. The session explains the Model I architecture with its advantages and disadvantages. Further, the session explains the JSP Model II architecture, which is also referred to as Model-View-Controller (MVC) in modern Web applications. The session explains the MVC architecture and its components.

In this session, you will learn to:

- Describe JSP models
- Describe the advantages and disadvantages of Model I
- Describe the advantages and disadvantages of Model II
- Explain the Model-View-Controller (MVC) architecture
- Explain the relationship between the components of MVC
- Explain Controller and its use
- Explain View and its use
- Explain Model and its use
- Identify how to develop a Web application based on MVC architecture
- Describe MVC and CDI
- Identify MVC events

11.1 Introduction

In most of the Web applications, the user interaction is through forms which are used as an interface for storing and retrieving data from a data store. With time, the data entered might be changed or updated, which the system in turn updates in the data store. In order to improve the application performance and to reduce the Line of Code (LOC), the programmers tend to integrate code for designing and data access in the Web application. This helps to achieve simplicity and flexibility. However, there are significant problems in implementing it. One such problem might be the more frequent change in user interface as compared to the persistence logics leads to the recompilation of whole application.

As a solution, JSP pages were developed to address the problem of tightly-coupled presentation and business logic in servlets. Based on the popularity and benefits of JSP in Web development, the commonly used approaches using JSP were identified. These approaches are also referred to as JSP models.

11.2 JSP Models

The JSP specification presents two approaches for developing Web applications using JSP pages.

There are two types of programming models for developing Web applications:

➤ JSP Model I

The Model I architecture is very simple, browser (HTML/JSP) request to the server and handled by JSP or servlet, and then, that JSP or servlet handles all responsibilities for the request, including processing the request, validating data, handling the business logic, and generating a response back to browser (HTML/JSP).

➤ JSP Model II

Model II, commonly known as MVC, solves many problems of Model I, by providing a clear separation of application responsibilities. In the MVC architecture, a central servlet, known as the Controller, receives all requests for the application from JSP/HTML. The Controller then, processes the request, works with the Model to prepare any data required by the View (JSP/HTML), and forwards the data back to a JSP/HTML. In this architecture, the business and presentation logic are separated from each other, which helps to reuse the logic.

11.2.1 JSP Model Architecture

In the Model architecture, the JSP page is responsible for handling the entire request. This means that the JSP page contains both the presentation logic (HTML and CSS) and the business logic (Java code). The typical flow is:

1. The browser sends a request directly to the JSP page.
2. The JSP page may use JavaBeans to invoke business logic.
3. The JavaBean interacts with the database to retrieve or save data.
4. The response is generated by the JSP page and sent back to the browser.

Figure 11.1 depicts the Model architecture.

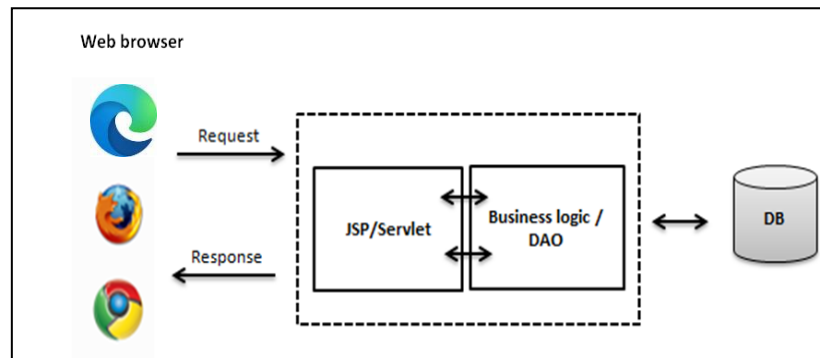


Figure 11.1: Model Architecture

In this model, there is no extra servlet involved in the process. The client request is sent directly to a JSP page, which may communicate with JavaBeans or other services, but ultimately the JSP page selects the next page for the client to view.

JSP Model architecture has a page-centric architecture. In this architecture, the application is composed of a series of interrelated JSP pages and these JSP pages handle all aspects of the application including presentation, control, and business logic. In page-centric architecture, the business process logic and control decisions are hard-coded inside JSP pages in the form of JavaBeans, scriptlets, and expressions.

Figure 11.2 shows an example of JSP Model architecture for an online shopping Web application with all JSP pages.

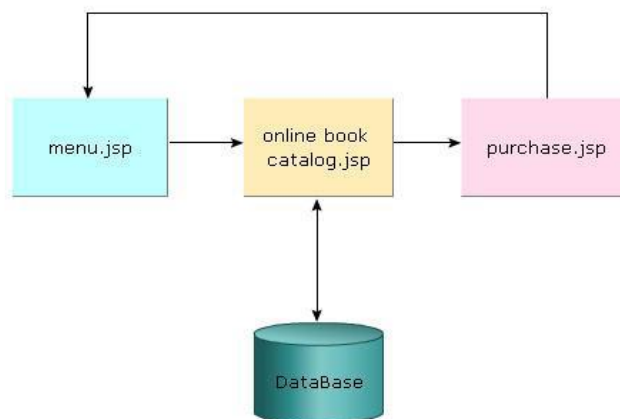


Figure 11.2: JSP Module 1 – Online Shopping Web Application

Advantages of JSP Model1

The advantage of JSP Model1 is that it makes development easier as there are no servlets involved in the application.

Disadvantages of JSP Model1

The disadvantage of this model architecture is:

- As business logic and presentation logic are tied together, it is difficult for building and maintaining a complex enterprise application.
- Each of the JSP pages is individually responsible for control logic, application logic, and also to present results to the user. This makes this model more dependent and less extensible.

11.2.2 JSP Model2 Architecture

Model2 architecture is an approach used for developing a Web application. It separates the Business Logic from the Presentation Logic. Besides this, Model2 has an additional component - a Controller.

Figure 11.3 shows the Model2 architecture.

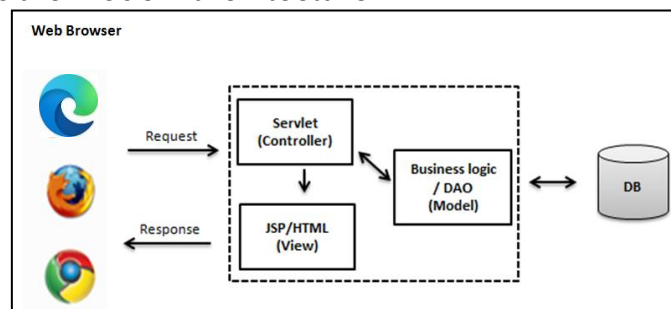


Figure 11.3: Model2 Architecture

Here, a Servlet acts as a Controller. Thus, Model2 has a Servlet-centric architecture. The responsibility of the Controller Servlet is to process the incoming request and instantiate a Model - a Java object or a bean to compute the business logic. It is also responsible for deciding to which JSP page the request should be forwarded.

JSP page is responsible for the View component, retrieves the objects created by the Servlet, and extracts dynamic content for insertion within a template for display.

Advantages of Model2

- Web applications based on this model are easier to maintain and extendable, as business and presentation logic are separated from each other.
- Testing is easy in model2.

11.3 MVC

MVC is a software architectural pattern. This pattern divides the application logic from the User Interface. The division permits independent development, testing, and maintenance of each component. Figure 11.4 shows a brief overview of the components under the MVC architecture.

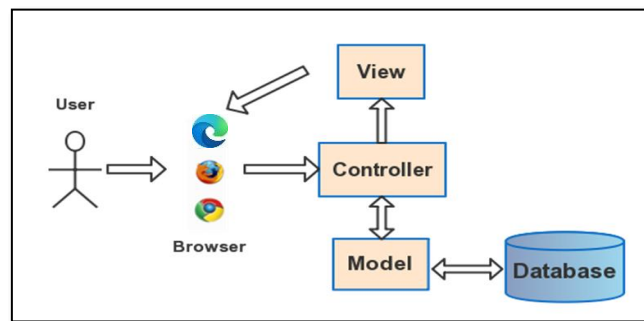


Figure 11.4: Components in MVC Architecture

This pattern divides Web-based applications into three layers:

1. Model

The Model component contains the business logics and functions that manipulate the business data. It is a layer between controller and the controller can access the functionality, which is encapsulated in the model.

2. View

The View component is used to generate the response to the browser, what the user sees. Often the view components are simple JSPs or HTML pages.

3. Controller

At the core of the MVC architecture are the controller components. The Controller is typically a servlet that receives requests from the browser (or any other source) and manages the flow of data between the Model layer and the View layer.

The controller processes the user requests and as per the user request, the Controller calls methods in the Model to fulfil the requested action and after the action has been taken on the data in model, the controller is responsible for redirecting the appropriate view to the user. The client (browser) sends a request to the controller in MVC. Controller then, communicates with the model and passes the response to the View. Finally, the View sends result back to the browser (client).

Benefits of the MVC design pattern are as follows:

- Allows code reusability, as several views can use the same model class.
- Simple to maintain and update the code since the code is organized into three different tiers.
- Quick and easy to debug the program.
- Easy to understand the code because of its simplified structure.

11.3.1 Relationships between Components

Different relationships between MVC components are as follows:

View and Controller Relationship	Model and View Relationship	Model and Controller Relationship
In this relationship, the controller is responsible for creating and selecting views.	In this relationship, the view depends on the model. If a change is made to the model then, there might be a requirement to make parallel changes in the view.	In this relationship, the controller is dependent on the model. If a change is made to the model interface then, there might be a requirement to make parallel changes to the controller.

11.3.2 MVC in Web Applications

Different components and their roles in a Web application that is based on MVC architecture are as follows:

- Model encapsulates data and business logic using JavaBean components or Plain Old Java Object (POJO), a database API, or an XML file.
- View shows the current state of the Model using an HTML or a JSP page.
- Controller updates the state of the Model and generates one or more views using Servlet.

Thus, in MVC design pattern, a controller responds to the user request, interacts with model, and then, displays appropriate view. Figure 11.5 depicts this pictorially.

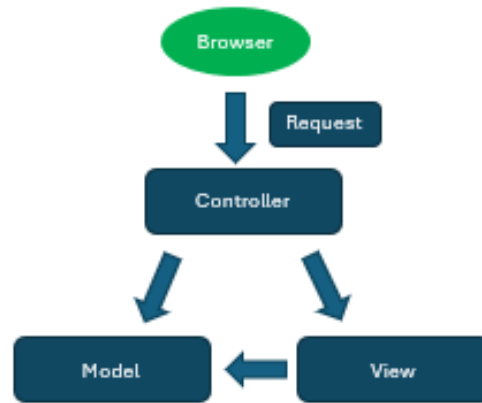


Figure 11.5: MVC Design Pattern

11.3.3 MVC Application Using Jakarta EE

Jakarta MVC 2.1 was released in 2022. It resulted in a package change from `javax.mvc` to `jakarta.mvc`. In MVC design pattern, controller handles the request and after interacting with model, displays the view.

The MVC design pattern can be implemented in Java Web applications in two ways:

1. Component Based
2. Action Based

Table 11.1 describes these implementation frameworks.

Component-based	Action-based
- The framework provides the controller. - Developers must take care of implementing the model and view. - Popular component based frameworks include Jakarta Server Faces.	- Developers must define the model, view, and controller. - Developers can control the flow easily. - Popular action based frameworks are Jakarta MVC, Spring MVC, Eclipse Krazo (an implementation), and Apache Struts.

Table 11.1: Component-based and Action-based Frameworks

Jakarta MVC is an action-based MVC framework. In this, requests are turned into actions through application code. `@Controller` annotation is used to mark a resource as controller. When `@Controller` is used for a class, all the resources of the class become controllers. When `@Controller` is used on a method, one can define controllers and other REST APIs both.

To build an MVC Web application, it is recommended to use an Eclipse Maven project.

Follow these steps to create an MVC Web application with Jakarta EE 10:

1. Launch **Eclipse**.
2. Select **File** → **New** → **Project** → **New Maven project**.
3. Search and select **maven-archetype-webapp** as shown in Figure 11.6.

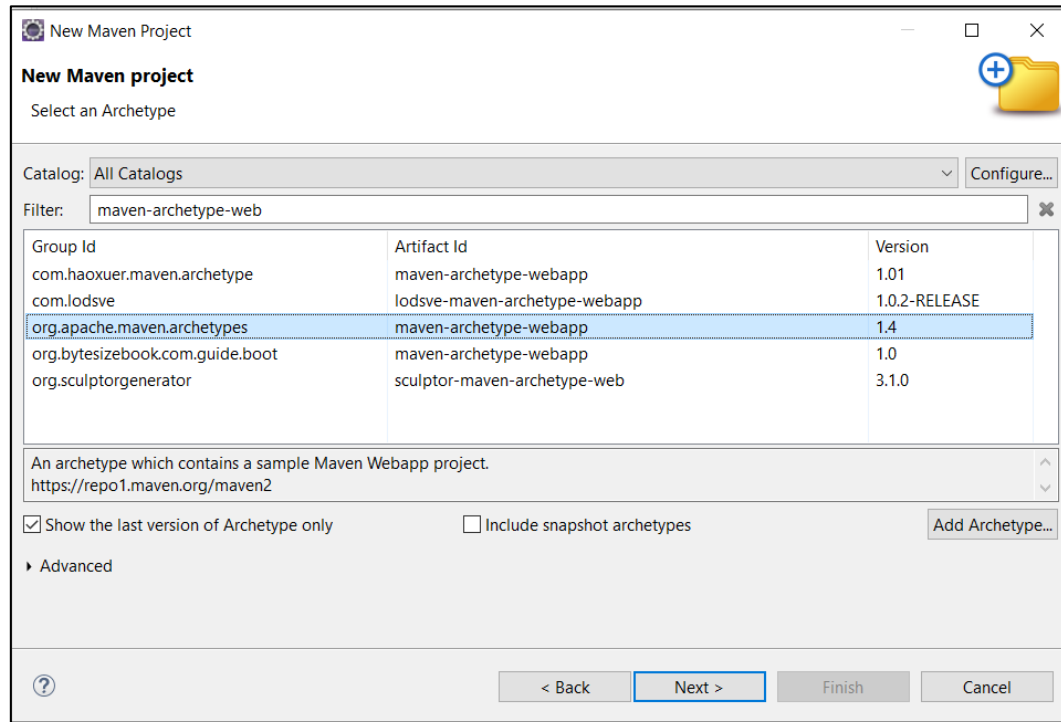


Figure 11.6: Select an Archetype

4. Enter **Group Id** and **Artifact Id** parameters, deselect **run archetype generation interactively** option and then, click **Finish** as shown in Figure 11.7.

New Maven Project

Specify Archetype parameters

Group Id:

Artifact Id:

Version:

Package:

☐ run archetype generation interactively

Properties available from archetype:

Name	Value

Advanced

< Back Next > **Finish** Cancel

Figure 11.7: New Maven Project

Once the project is created, ensure `pom.xml` file will contain elements such as `<groupId>`, `<version>`, and `<artifactId>`. Add controller servlets and views (JSP file) as per requirement.

If you encounter an issue with `jakarta.servlet.http.HttpServlet` not being found on the Java Build Path after creating your project and this is causing problems in your JSP files, perform following steps:

1. Navigate to the project in the IDE.
2. Right-click the project and select **Properties**.
3. In the **Properties** window, go to **Project Facets**.
4. Find **Dynamic Web Module** and change its version to **6.0**. In the same window, in the **Runtimes** section, select **Apache Tomcat v10.1**. Then, click the **Apply** button, followed by the **Apply and Close** button, as illustrated in Figure 11.8.

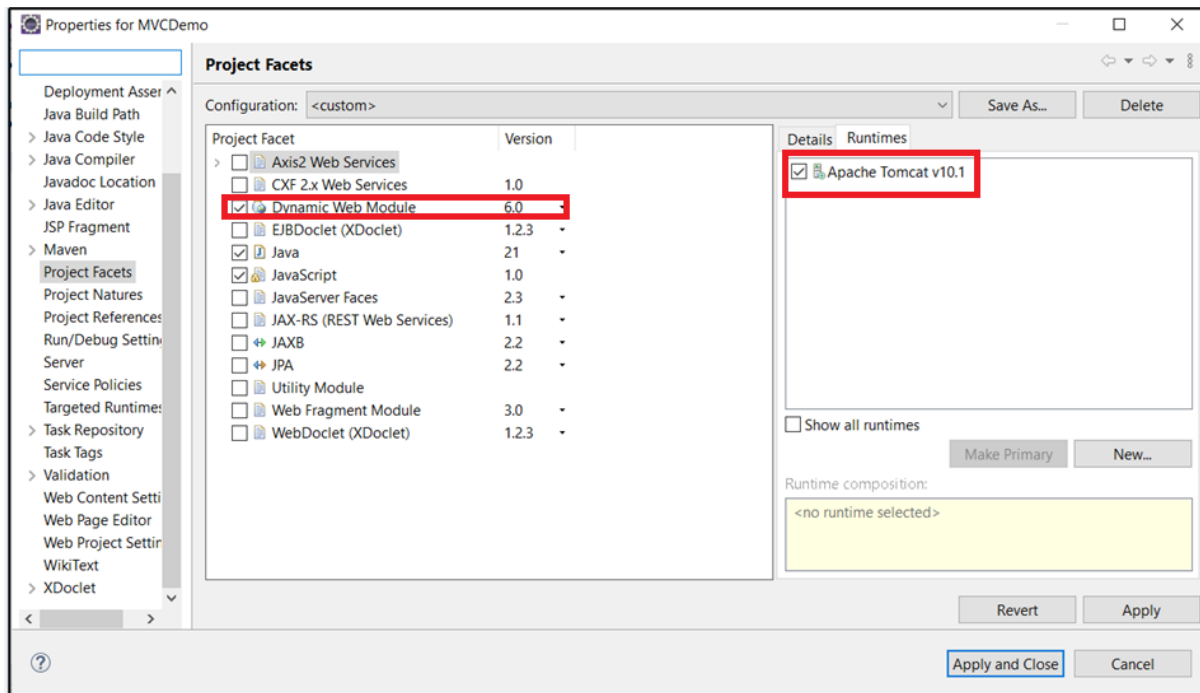


Figure 11.8: Project Facets Settings

Note: Jakarta MVC 2.1 can also be implemented using Eclipse Krazo 3.0.1 archetype.

11.3.4 Implementation of MVC Pattern - Examples

Consider a scenario where a Web application login service will validate login details and accordingly display the appropriate JSP page. To design such a service, developer can use the MVC pattern.

Create an MVC application in Eclipse using steps described earlier.

Code Snippets 1, 2, and 3 are used to develop the JSP pages required to handle presentation and result pages. These will be considered as 'views' in the MVC application.

Code Snippet 1: index.jsp

```
<<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
    <title>MVC Example</title>
</head>
<body>
```

```
<form action="ControllerServlet" method="post">
Name:<input type="text" name="name"><br>
Password:<input type="password" name="password"><br>
<input type="submit" value="login">
</form>
</body>
</html>
```

Code Snippet 2: login-success.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-
8859-1"
    pageEncoding="ISO-8859-1"
    import="com.first.model.LoginBean"%>
<!DOCTYPE html>
<html>
<p>You are successfully logged in!</p>
<%
LoginBean bean=(LoginBean)request.getAttribute("bean");
out.print("Welcome, "+bean.getName());
%>
</html>
```

Code Snippet 3: login-error.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-
8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
    <head>
        <title>Error</title>
    </head>
    <body>
        Login failed, please try again.
    </body>
</html>
```

Codes show the designing of three JSP pages namely, index.jsp, login-success.jsp, and login-error.jsp respectively. These files will be placed under webapp folder of the project.

The index page displays a form to the user for filling username and password. The login-success page will be processed, and a success message is displayed when the validation of the entered details is successful. Otherwise, the login-error.jsp file is processed, and an error message is displayed to the user.

Next, we will design a Controller that takes request from the user, this is the servlet class. The servlet class calls the Model which is a Java Bean. The controller servlet class and the model bean class will be created under folders **controller** and **model** respectively. Refer to Figure 11.9.

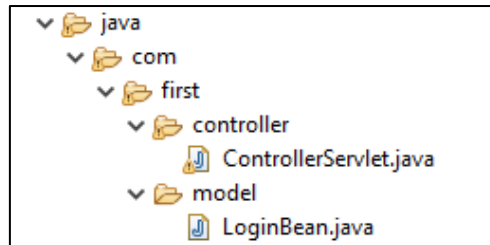


Figure 11.9: Folder Structure in Project Explorer

Code Snippet 4 demonstrates the design of the Controller named ControllerServlet.java.

Code Snippet 4: ControllerServlet.java

```

package com.first.controller;
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.*;
import com.first.model.LoginBean;

public class ControllerServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out=response.getWriter();
        String name=request.getParameter("name");
        String password=request.getParameter("password");
        LoginBean bean=new LoginBean();
        bean.setName(name);
        bean.setPassword(password);
        request.setAttribute("bean",bean);
        boolean status=bean.validate();
        if(status){
            RequestDispatcher rd =
                request.getRequestDispatcher("
                    login-success.jsp");
            rd.forward(request, response);
        }
        else{
            RequestDispatcher rd =
                request.getRequestDispatcher("login-error.jsp");

```

```

        rd.forward(request, response);
    }
}
@Override
protected void doGet(HttpServletRequest req,
    HttpServletResponse resp)
    throws ServletException, IOException {
    doPost(req, resp);
}
}

```

In the code, the user details sent from the `index.jsp` page are received through `request.getParameter()` method. Then, the values are sent to the JavaBean named `LoginBean` which will validate the values and return the appropriate string. Finally, based on the obtained string, the request to appropriate JSP page is sent.

Code Snippet 5 demonstrates the design of `LoginBean` class.

Code Snippet 5: `LoginBean.java`

```

package com.first.model;
public class LoginBean {
    private String name,password;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public boolean validate(){
        if(password.equals("admin123")){
            return true;
        }
        else{
            return false;
        }
    }
}

```

Code Snippet 6 shows the deployment descriptor `web.xml` that is used to configure the servlet in the application.

Code Snippet 6: web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
  <display-name>Archetype Created Web Application</display-
name>
  <servlet>
    <description></description>
    <display-name>ControllerServlet</display-name>
    <servlet-name>ControllerServlet</servlet-name>
    <servlet-
class>com.first.controller.ControllerServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>ControllerServlet</servlet-name>
    <url-pattern>/ControllerServlet</url-pattern>
  </servlet-mapping>
</web-app>
```

The web.xml is placed in WEB-INF folder and describes the deployment of an application to run servlet and JSP pages. The index.jsp page is the default page to open on sending request to the Web application.

Right-click the project, select **Run As** and then, click **Run on Server**. Figure 11.10 shows the outcome, where the form is displayed with the username and password fields.

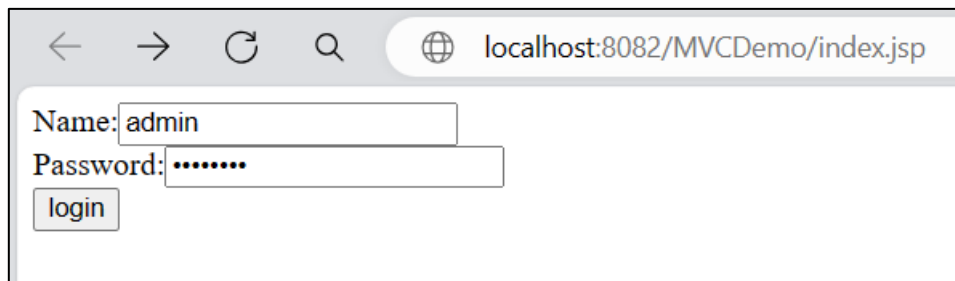


Figure 11.10: Entering Login Details (index.jsp)

The username and password values are sent in request to the Controller, that is, ControllerServlet which invokes the JavaBean method for validating user entered details. As the entered details are correct, the Controller dispatches the request to login-success.jsp page to display the response in the browser.

Upon entering valid credentials, the outcome will be similar to Figure 11.11.

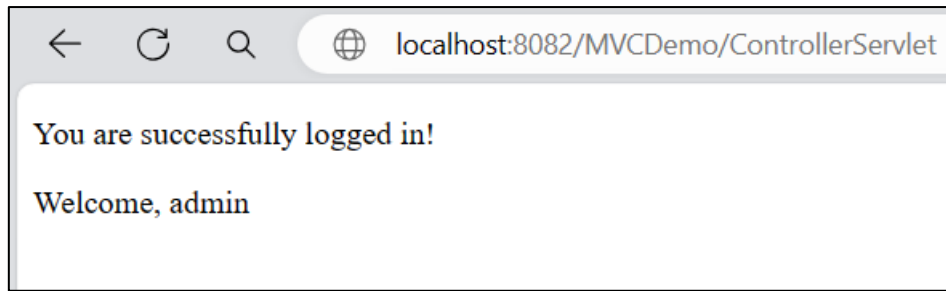


Figure 11.11: Login Successful Message (login-success.jsp)

In case, incorrect credentials are entered, error will be displayed as shown in Figure 11.12.

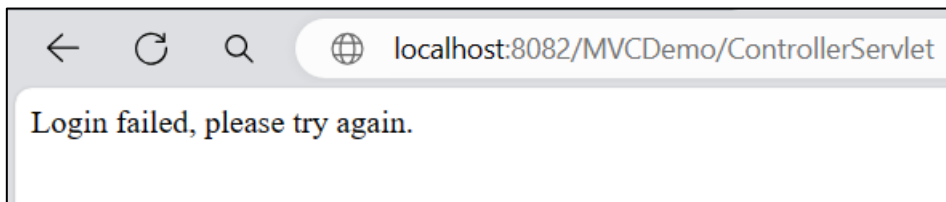


Figure 11.12: Login Error Message (login-error.jsp)

Another approach to create MVC applications with Jakarta EE is by using Eclipse Krazo, which is an implementation of action-based MVC and builds on top of JAX-RS.

To create an MVC application with Eclipse Krazo:

1. Create a new project using Eclipse → File → New → Project → New Maven project.
2. Search and select the `krazo-jakartaee-archetype` Version: 3.0.1 as shown in Figure 11.13.

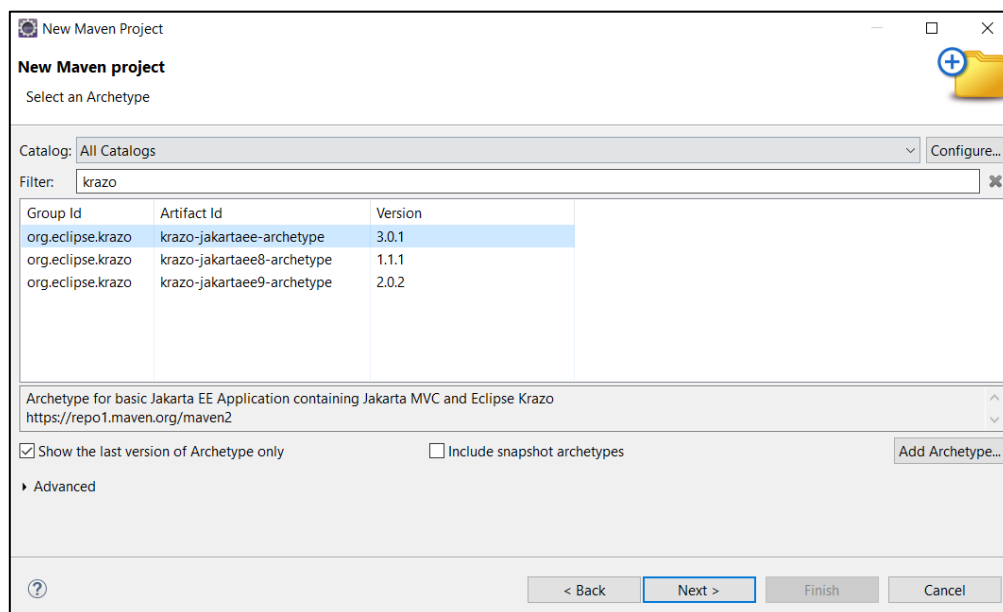


Figure 11.13: Selecting Eclipse Krazo Archetype

3. Enter Group Id and Artifact Id parameters, deselect run archetype generation interactively option, and then, click Next as shown in Figure 11.14.

New Maven Project

Specify Archetype parameters

Group Id:

Artifact Id:

Version:

Package:

☐ run archetype generation interactively

Properties available from archetype:

Name	Value
krazoImpl	jersey
javaRelease	1.8

Advanced

< Back Next > Finish Cancel

Figure 11.14: Specifying Project Details

A Controller (AppController.java), View (index.jsp), and Model (App.java) will be auto-generated as part of the project as shown in Figure 11.15. Later, you can change these according to your requirements. For this example, index.jsp is renamed to Hello.jsp.

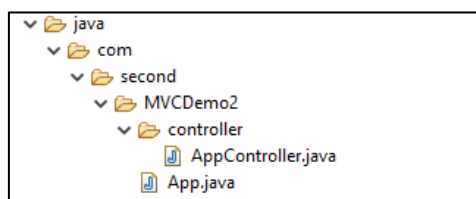


Figure 11.15: Auto-generated Files

Code Snippets 7 to 10 depict the codes added to the files.

Code Snippet 7: AppController.java

```
package com.second.MVCDemo2.controller;
import jakarta.mvc.Controller;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
```

```
@Path("app")
@Controller
public class AppController {
    @GET
    public String sayHello() {
        return "hello.jsp";
    }
}
```

Code Snippet 8: App.java

```
package com.second.MVCDemo2;
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;
/**
 * Default JAX-RS application listening on /mvc
 */
@ApplicationPath("/mvc")
public class App extends Application {
}
```

Code Snippet 9: Hello.jsp

```
<%@ page contentType="text/html; charset=UTF-8"
language="java" %>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Hello!</title>
</head>
<body>
<h1>Hello MVC!</h1>
</body>
</html>
```

Code Snippet 10: Beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"
    bean-discovery-mode="annotated" version="4.0">
</beans>
```

11.3.5 Spring MVC Architecture

Spring MVC is a popular framework used for developing Java based Web applications using MVC design pattern. As shown in Figure 11.16, Spring MVC uses the DispatcherServlet to route all the incoming HTTP requests to the appropriate MVC resource such as, Model, View, or Controller.

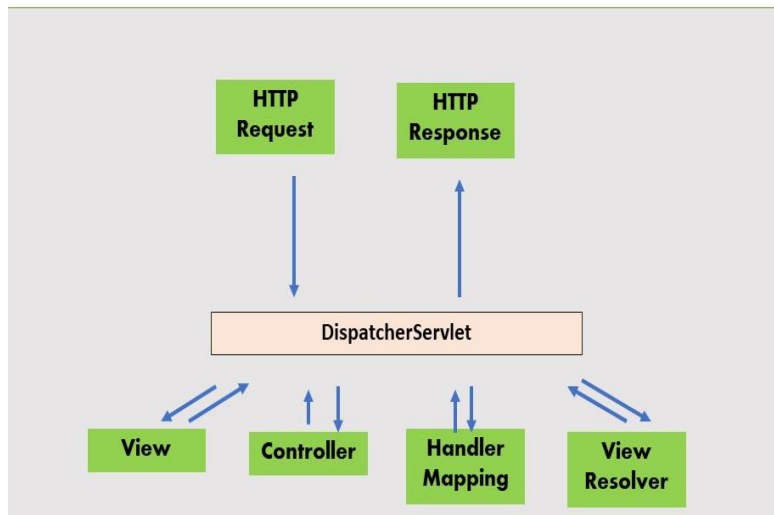


Figure 11.16: Spring MVC Flow Diagram

DispatcherServlet operates in following manner:

1. HTTP request is received.
2. DispatcherServlet, on receiving the request, evaluates the Handler Mapping to decide which Controller to call in order to answer the request.
3. Controller invokes the service methods.
4. The ViewResolver assists the DispatcherServlet to fetch the requested View for the request.
5. View is generated.
6. The DispatcherServlet transfers the model data to the View.
7. View is rendered in the browser.

Spring MVC is a hugely popular action-based framework because it is open-source, lightweight, and robust. Developers save a lot of time when using the Spring MVC framework since it also provides many plug-ins to work with different databases. In addition, it makes the system separable and manageable. Core Spring Framework is based on two design principles – **Dependency Injection (DI)** and **Aspect Oriented Programming (AOP)**.

- **Dependency Injection** is a design pattern that removes the dependency from the programming code, making it easy to manage and test the application. DI makes the programming code loosely coupled. A loosely coupled system is typically a preferred coding style compared to a tightly coupled system.

In a tightly coupled system, the software is developed for a specific requirement and bound to a certain interface, making it less scalable. In a tightly coupled system, if the resource changes, the programmer has to modify the underlying code substantially. On the contrary, in a loosely coupled system, the idea is if the resource is changed then, the programmer does not have to worry about changing the code. The code should work independent of the underlying interface.

Dependency injection can be injected using a constructor or a setter method.

- The second principle used by Core Spring Framework is **Aspect Oriented Programming** (AOP). AOP takes the complete program logic and divides it into multiple smaller units. These smaller units are called concerns. Using the AOP style, a programmer can dynamically plug in the concerned item before, after, or around the actual logic.

The Spring MVC framework uses annotations to establish whether a resource would be used as a Controller or as a View.

`@Controller` annotation marks a resource as the controller. If the annotation is applied to the resource class, all resource methods in the class become controllers. The `@View` annotation marks a resource as a View.

Code Snippet 11 gives a brief demonstration of how to use `@Controller` and `@View` annotations.

Code Snippet 11:

```
@Controller
public class UserController {
    @GET
    @View("userForm.jsp")
    public void showForm() {
        getInitFormData();
    }
}
```

11.3.6 Context and Dependency Injection (CDI)

One of the many features that was introduced with Java EE 6 is Context and Dependency Injection (CDI). CDI is a suite of services, which if used together, eases the task of the developers to integrate Java Server Faces technology and enterprise beans into online applications. Using CDI, developers can link a wide variety of components in a type-safe and loosely coupled manner. CDI can run as a standalone application or can be incorporated into any other application.

The collection of services that are provided by CDI helps to make the code structure better.

By introducing a more advanced lifecycle and interaction model, CDI also helps to extend the lifecycle and interaction model of preexisting Java component types such as managed beans and Enterprise JavaBeans.

A CDI bean can be any of the types:



Following bean types receive contextual lifecycle management and bean injection from the container.

- **Producer Methods and Fields** – This method, can be static or non-static. It gives the application complete control over the bean creation process. Whenever there is no instance of the bean present in the provided context, the container invokes the producer method on the object.
- **Managed Beans** – A managed bean is implemented and managed by a Java class. It executes operations such as handling component events and validating component data. Notably, if the bean class of a managed bean is marked with both `@Interceptor` and `@Decorator`, the container notes the issue and treats it as a definition error.

The bean instances are automatically created and destroyed by the Container as per the requirement. DI is performed on all instances of managed beans. CDI also provides following services:

- Use of annotations such as `@Inject`. This allows the developers to inject any file implementation into the constructor.
- Due to the Expression Language (EL) integration, any component can be utilized directly within a JSP application.

11.3.7 MVC Events

The word 'event' refers to any triggers or actions that take place in the system that the user is now programming. Events can also be defined as objects that are created when something changes in a GUI. For example, if a user clicks a button, selects an option, or enters data in the text field, an event is triggered, resulting in the creation of the corresponding event object. For any program to function efficiently, all the involved components should know of the next step or action. This job of notifying the components of the next step is performed by events.

In Jakarta EE, the Web container is responsible for transfer of control to event listeners. For example, an event listener associated with `JButton` is referred as an `ActionListener`. JavaScript uses the function `addEventListener()` to listen to events.

The syntax of the function `addEventListener()` is as follows:

```
element.addEventListener("event", functionToExecute);
```

where;

- `element` **parameter has an HTML tag**
- `event` **is a specific action**
- `functionToExecute` **refers to an existing function**

Summary

- The JSP specification presents two approaches for developing Web applications namely, JSP Model I and JSP Model II.
- JSP Model II is also known as MVC.
- MVC is a software design pattern, which can be used to design medium and large sized applications.
- Three components of MVC are: Model, View, and Controller.
- In MVC Web application, servlet acts as controller, which receives the request from client.
- The view handles presentation of the content on the Web page and could be an HTML file or a JSP file.
- The Model component contains the business logics and functions that manipulate the business data.
- The Web container transfers control to event listeners in Jakarta EE to listen to events related to Servlets, JSP, and so on.
- ActionListener is an example of an event listener associated with a GUI element such as JButton.

Check Your Progress

1. Which of the following is not a component of MVC architecture?

(A)	Controller
(B)	Model
(C)	View
(D)	Mapping XML file

2. Which of the following statements are correct for Controller component?

(A)	There is only one controller in MVC1
(B)	There can be many number of controllers in MVC2
(C)	There is only one controller in MVC2
(D)	Controller class directly connected to a database

3. Match the components against their descriptions.

	Component		Description
(A)	Controller	(1)	An XML file used for configuring servlet and JSP
(B)	web.xml	(2)	Handles presentation
(C)	View	(3)	Handles business logic
(D)	Model	(4)	Handles requests and responses

(A)	A-3, B-1, C-4, D-2
(B)	A-4, B-1, C-2, D-3
(C)	A-2, B-1, C-4, D-3
(D)	A-4, B-3, C-2, D-2

4. Which of the following MVC components displays the data on browser pages?

(A)	Controller
(B)	View
(C)	Model
(D)	Configuration file

5. Which of the following MVC components handles the request and responses?

(A)	Controller
(B)	View
(C)	Model
(D)	Configuration file

Answers

1.	D
2.	C
3.	B
4.	B
5.	A

Try It Yourself

1. Initialize a new Maven project for a Student Management System. Use the `maven-archetype-webapp` archetype to set up a basic Web application structure.
 - a. `Login.jsp`: Develop the `Login.jsp` page to authenticate students. This page should include fields for students to input their username and password.
 - b. `Error.jsp`: Design the `Error.jsp` page to display error messages. If a student enters incorrect credentials, they should be redirected to this page with a message such as 'Incorrect Credentials.'
 - c. `StudentData.jsp`: Implement a form on the `StudentData.jsp` page to collect student details. The form should accept `student_name`, `student_age`, and `student_percentage`. Students should be redirected to this page after successful authentication.
 - d. `StudentControllerServlet.java`: Create the `StudentControllerServlet.java` servlet to manage form submissions from both `Login.jsp` and `StudentData.jsp`. It should handle the retrieval of the username and password, as well as the student details entered.
 - e. `StudentBean.java`: Define the `StudentBean.java` class to encapsulate the data entered on `Login.jsp` and `StudentData.jsp`. This bean will act as a model to store student information.
2. Implement user authentication logic (for example, checking username and password against a database). Use a database such as MySQL to store student data submitted from `StudentData.jsp`. The `StudentControllerServlet` must connect to the database and perform Create, Read, Update, and Delete (CRUD) operations on student data.
3. Compile and run your code to ensure that the application is functioning as expected. Test the login process, error handling, and data submission flow.

JSP Expression Language

Welcome to the Session, **JSP Expression Language**.

The session explains the Expression Language (EL) in Java. It explains different implicit objects that can be expressed in EL language. The session also explains how to work with the functions, operators, and tag libraries using EL language in the JSP page.

In this session, you will learn to:

- Explain how to use script expressions in JSP
- Describe the implicit objects used in EL
- Describe various operators used in EL
- Explain how to create static method and tag library descriptor using EL
- Explain how to access EL functions within JSP
- Explain the concept of boxing and unboxing
- Explain how to coerce a value to string or number type

12.1 Expression Language

Expression Language (EL) is a primary feature of the JSP technology. The JSP Standard Tag Library (JSTL) expert group and JSP 2.0 expert group at the Java Community Process developed the JSP expression language.

EL is simple and robust. It can handle both expressions and literals, which are constants and are assigned some memory location. The advantage of EL is that it provides cleaner syntax and is specifically designed for JSP. EL makes it possible to easily access application data stored in JavaBeans components.

EL is a great help to the page authors in accessing and manipulating the application data without mastering the complexities of the programming language, such as Java and JavaScript. JSP expression language can be used to display the generated dynamic content in a table on a Web page. In addition, EL can also be used in HTML tags.

Syntax:

```
$ {EL expression}
```

where,

- \$ indicates the beginning of an expression in EL.
- { is the opening delimiter.
- EL expression specifies the expression.
- } is the closing delimiter.

Code Snippet 1 demonstrates the use of the expression language on JSP.

Code Snippet 1: result.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html;
      charset=UTF-8">
    <title>Result</title>
  </head>
  <body>
    <h1>Qualifying Exam Criteria:</h1>
    <br />
    <br />
    <p>Student should atleast score:</p>
    <pre>
      Subject      Marks
      Maths        ${40+30}
      Java          ${40+35}
      C++           ${40+35}
      Database      ${40+35}
    </pre>
    <hr />
    <p>You are accessing the Website on: ${header["User-
Agent"]} </p>
  </body>
</html>
```

Figure 12.1 shows output where EL expression prints qualifying marks and user agent.



Figure 12.1: Expression Language

12.2 EL Implicit Objects

JSP implicit objects are a standard set of classes. JSP container provides the methods and variables in the classes to the user. EL expressions access implicit objects by name. The user creates an instance of an implicit object to use available methods and variables.

Implicit objects defined in JSP EL are as follows:

pageContext

The `pageContext` object can be used without creating an instance of the object. The `pageContext` object provides access to page attributes. For example, `${pageContext.request}` will return the request object of `pageContext`. This request object can be used to access different page attributes.

servletContext

The `servletContext` object specifies the JSP page, servlet, and Web components contained in the same application.

session

The `session` object represents the session created for the client sending a request.

request

The `request` object represents the request accepted by the JSP page from client. Information of the request is accessed using request object.

12.2.1 Request Headers and Parameters

Several implicit objects are available for easy access to following objects:

➤ param

The `param` returns a value that maps a request parameter name to a single string value.

Code Snippet 2 demonstrates the use of EL to read parameters.

Code Snippet 2:

```
<!-- If the request parameter name is null or an empty string,
this snippet returns true.
-->
${empty param.Name}
```

➤ paramvalues

The `paramValues` returns an array of values, which is mapped to the request parameters from client.

Code Snippet 3 demonstrates the use of EL to read parameter values from an array.

Code Snippet 3:

```
<!-- Takes the address line as the input -->
<br>Address Line 1: ${paramValues.address[0]}
<br>Address Line 2: ${paramValues.address[1]}
```

➤ **header**

The **header** returns a request header name and maps the value to single string value.

Code Snippet 4 demonstrates the use of header.

Code Snippet 4:

```
<!--returns the host as a header name -->
${header["host"]}
```

➤ **headerValues**

The **headerValues** returns an array of values that is mapped to the request header.

Example: <!--Returns multiple cookies with the same name -->
`${headerValues.name}`

➤ **cookie**

The **cookie** returns the cookie name mapped to a single cookie object.

Example: <!--Returns the value of the cookie -->
`${cookie.name.value}`

➤ **initParam**

The **initParam** returns a context initialization parameter name, which is mapped to a single value.

12.2.2 Scoped Variables

The JSP API provides the facility of storing data and retrieving it from four different scopes within the JSP container. The EL enhances this facility by further supporting the retrieval of the stored objects as scoped variables. The term **scoped variable** means that the variable is confined to the mentioned context only.

For example, for a specific request, the object will be retrieved only during the processing of the page in question. If the object is stored as session scope, then it will be retrieved by any page during the single interactive session with the Web application.

The four scopes for implicit objects in JSP are as follows:

pageScope	The <code>pageScope</code> returns page-scoped variable names, which are mapped to their values. The <code>pageScope</code> is accessible from the JSP page that creates the object. Example: <code><!-- Accesses the page-scoped attribute, book--></code> <code>\${pageScope.book}</code>
requestScope	The <code>requestScope</code> provides access to the attributes of request object. The <code>requestScope</code> object returns request-scoped variable names, which are mapped to their values. The <code>requestScope</code> is accessible from Web components handling a request that belongs to the session. Example: <code><!-- Returns the value of the request-scoped attribute, name.--></code> <code>\${requestScope.student.name}</code>
sessionScope	The <code>sessionScope</code> returns session-scoped variable names, which are mapped to their values. The <code>sessionScope</code> is accessible from Web components handling a request that belongs to the session. Example: <code><!-- Returns the value of the numberOfPages property of the session-scoped attribute named book. --></code> <code>\${sessionScope.book.numberOfPages}</code>
applicationScope	The <code>applicationScope</code> returns application-scoped variable and maps the variable name to their values. Example: <code><!-- Checks the value of application-scoped attribute, booklist --></code> <code>\${applicationScope.booklist == null}</code>

12.2.3 Page Context

The `pageContext` implicit object defines the context for the JSP page. It provides access to page attributes.

The `pageContext` object provides Web page information using following objects:

servletContext	session	request	response
The <code>servletContext</code> object specifies the servlet of the JSP page and the Web components contained in the same application. It provides methods that a servlet uses to communicate with its servlet container.	The <code>session</code> object represents the session created for the client to send requests. It is helpful in maintaining the client state where series of requests are inter-related.	The <code>request</code> object represents the request accepted by the JSP page from a client and information of the request.	The <code>response</code> object represents the response sent to the client by a JSP page. The response contains the data passed between a client and servlet.

12.3 EL Operators

EL helps with easy access of application data stored in JavaBeans components.

Table 12.1 shows all operators used by the JSP EL.

Category	Operators
Variable	. and []
Arithmetic	+, - (binary), *, / and div , % and mod , - (unary)
Conditional	A ? B : C
Relational	==, eq , !=, ne , <, lt , >, gt , <=, le , >=, ge
Logical	and , && , or , , not , !
Empty/Null checking	empty

Table 12.1: Operators used by the JSP Expression Language (EL)

12.3.1 EL Arithmetic Operators

Operators are used to perform different arithmetic, relational, and logical operations. Dot operator (.) or [] is used to access the value of a variable.

An arithmetic statement written in JSP EL may contain more than one operator. The EL supports following arithmetic operators:

- + (Addition)
- - (Subtraction)
- * (Multiplication)
- / and **div** (Division)
- % and **mod** (Modulo division)

The EL arithmetic operators accept strings that can be converted into numbers as parameters. Thus, the expression `${ "2"+"2" }` will evaluate to output 4.

Code Snippet 5 demonstrates the use of EL with arithmetic operators.

Code Snippet 5:

```
<!--Following code illustrates how you can use the math
operators --%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<html>
<head>
  <title>EL Math Operations</title>
</head>
<body>
  <h1>Results of Using Math Operators in JSP EL</h1>
  <p>2 + 2 = ${2 + 2}</p>
  <p>3 - 1 = ${3 - 1}</p>
  <p>2 * 2 = ${2 * 2}</p>
  <p>10 / 2 = ${10 / 2}</p>
```

```

<p>10 / 9 = ${10 / 9}</p>
<p> 10 mod 9 = ${10 mod 9} </p>
</body>
</html>

```

Figure 12.2 shows output with arithmetic operators.

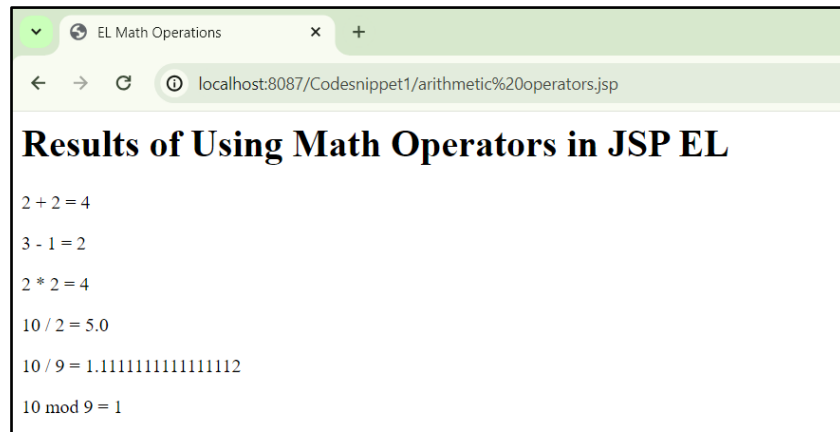


Figure 12.2: Arithmetic Operators

12.3.2 EL Relational Operators

By using various relational operators, comparisons can be made against other values, such as boolean, string, integer, or floating point literals.

Various relational operators used in Expression Language are as follows:

- == Equal to
- != Inequality
- < Lesser than
- > Greater than
- <= Lesser than or equal to
- >= Greater than or equal to

Code Snippet 6 demonstrates the EL with relational operators.

Code Snippet 6:

```

<!--comparing numbers-->
4 > '3'    ${4 > '3'}<br/>
'4' > 3    ${'4' > 3}<br/>
'4' > '3'  ${'4' > '3'} <br/>
4 >= 3     ${4 >= 3}<br/>
4 <= 3     ${4 < 3}<br/>
4 == '4'   ${4 == '4'}<br/>

```

12.3.3 EL Logical Operators

The logical operators supported by Expression language are as follows:

- and, && (Logical AND)
- or, || (Logical OR)
- ! or not (Boolean complement)

Code Snippet 7 demonstrates the use of EL with logical operators.

Code Snippet 7:

```
<!--Test the condition-->
<c:if test="${(guess >= 10)  && (guess <= 20)}">
    <b>You are in the range!</b><br/>
</c:if>
<c:if test="${(guess < 10)  || (guess > 20)}">
    <b>Try again!</b><br/>
</c:if>
```

12.3.4 EL Empty Operators

Whether the value is `null` or empty can be determined using the `empty` operator, which is a prefix operation.

The `empty` operator returns `true` if the string is empty. The string is said to be empty if it contains no character. If the string is not empty, then `empty` operator returns `false`.

Code Snippet 8 demonstrates the use of empty operators.

Code Snippet 8:

```
<% this code returns true if the string is empty and string
"sometext" %>
<b>
empty " " ${empty ""}<br/>
empty "sometext" ${empty "sometext"}<br/>
</b>
```

12.3.5 EL Dot Operators

The Dot (.) operator is used to access attribute values of JavaBean and map values within the EL. When you use the dot operator, the code to the left of the operator must specify a JavaBean or a map, and the code to the right of the operator must specify a JavaBean or a map key and the property name follows the conventions of Java identifiers.

Syntax:

```
${mapObject.keyName}
```

The EL allows to replace dot notation with array notation (square brackets).

Example,

`${param.header}` can be replaced with `${param["header"]}`

The array notation operator works similar to the dot operator. However, it has its own advantages as follows:

- Array notation operator can be used to access elements of an array or a list by specifying an index. That is the value inside the brackets can itself be a variable in array notation while with dot notation the property name must be a literal value.
- The array notation lets you use values as property names.

Code Snippet 9 demonstrates the use of dot operator.

Code Snippet 9: `home.jsp`

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html;
            charset=UTF-8">
        <title>Home</title>
    </head>
    <body>
        <p>Enter Username and Submit:</p><br /><br />
        <form action="welcomeuser.jsp" method="post">
            <input name="txtval" type="text" size="20"
                maxlength="60" placeholder="Enter Username..."
                required/>
            <input name="sbtName" type="submit" value="submit"
        />
        </form>
    </body>
</html>
```

Code Snippet 10 demonstrates the code for `welcomeuser.jsp`.

Code Snippet 10: `welcomeuser.jsp`

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
```

```

    <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
    <title>Employee Work Details</title>
</head>
<body>
    <h1>Work Details!</h1> <br /><br />
    <b>Employee:    <%=    request.getParameter("txtval")
%></b> <br />

    <p>Please refer to the given topics and complete it
before
Monday:</p>
    <ul>
        <li> Contact Form in PHP </li> <br />
        <li> HTML5 new tags </li> <br />
        <li> Complete Review of all the documents </li>
<br />
    </ul>
    <!-- Hidden form field to pass Username --%>
    <form action="welcomeuser.jsp" method="post">
        <input    type="hidden"    name="Username"    value="<%=
request.getParameter("txtval") %>">
    </form>
</body>
</html>

```

Figure 12.3 shows the username entered as `Madison` during submitting the form.

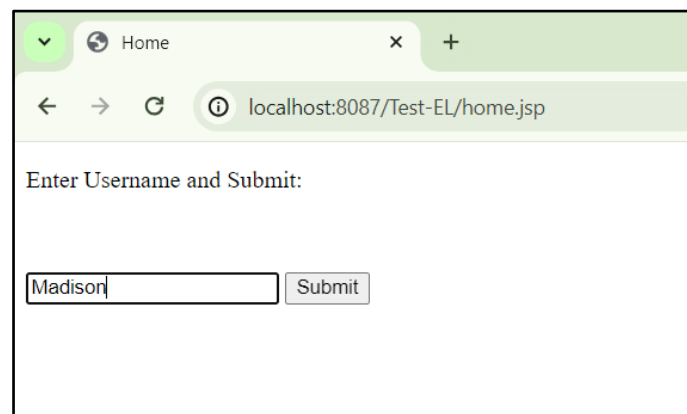


Figure 12.3: Username Details

Figure 12.4 shows the output after submitting the form successfully.

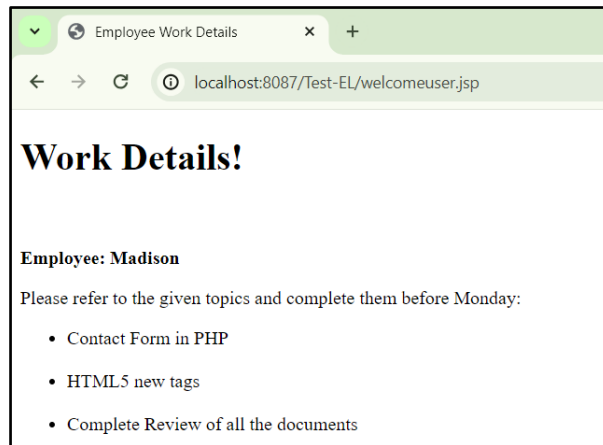


Figure 12.4: Output Showing Instructions to Employee

12.4 Functions Using EL

JSP allows the users to define a function that can be invoked in an expression. Functions are defined using the same technique as custom tags.

12.4.1 Creating static Methods

The `static` java methods can be called within the EL expression. To access the function using EL, the function must be implemented as a `static` function in a java class. You can define many functions in a single class. After defining the functions, you must map the function name with EL using a Tag Library Descriptor (TLD) file.

Syntax:

```
ns:funcName(arg1, arg2, . . .)
```

where,

- `ns` refers to name space and function name. Name space is generally a class or a tag library or a function name to access a static method in some class.

Figure 12.5 depicts the EL.

```
package mypackage;
public class MyFunctions {
    public static double
    average(double [] values){
        double dblSum=0.0;
        int iCount = values.length();
        for (int i=0;i<iCount;i++){
            dblSum+=values[i];
        }
        return dblSum/iCount;
    }
}
```

Figure 12.5: Expression Language Method

12.4.2 Creating Tag Library Descriptor

A TLD file uses XML syntax to map the name of functions defined in a class with EL. In the `<function>` tag in a tag library descriptor file, you must mention the name of the function using element `<name>`. Also, you must mention the class in which the function is defined using `<function-class>` element and the signature of the function in the TLD file using `<function-signature>` element. You then, save this TLD file in the `/WEB-INF/tlds` folder, where `tlds` is a user-created folder.

Figure 12.6 depicts EL function configuration in the Tag Library descriptor.



```
<function>
  <name>average</name>
  <function-class>mypackage.
    MyFunctions
  </function-class>
  <function-signature>
    double average
    (double[])
  </function-signature>
</function>
```

EL Function Configuration in Tag Library Descriptor

Figure 12.6: EL Function Configuration in a Tag Library Descriptor

12.4.3 Modifying Deployment Descriptor

The default mode for JSP version 1.2 technology or before is to ignore EL expressions. Starting with JSP version 2.0, the default mode is to evaluate EL expressions.

Setting the value of the `<el-ignored>` element in the deployment descriptor can explicitly change the default mode. The `<el-ignored>` element is a sub element of `<jsp-property-group>`. It has no sub elements. Its valid values are `true` and `false`.

In deployment descriptor, `web.xml` file, declare as follows:

Syntax:

```
<el-ignored>>false</el-ignored>
```

where,

- `true` indicates that EL expressions will be ignored.

- `false` indicates that EL expressions will be enabled for interpretation by servlet container.

Code Snippet 11 shows the deployment descriptor, `web.xml`, which has to be modified as follows:

Code Snippet 11:

```
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="https://jakarta.ee/xml/ns/jakartaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
    <jsp-config>
        <jsp-property-group>
            <url-pattern>*.jsp</url-pattern>
            <el-ignored>>false</el-ignored>
        </jsp-property-group>
    </jsp-config>
</Web-app>
```

- `<jsp-config>`: Includes JSP configuration, such as interpretation of tag library and property information.
- `<jsp-property-group>`: Defines a set of properties that applies to a set of files representing the JSP pages.
- `<url-pattern>`: Specifies that JSP properties defined in `<jsp-property-group>` to specific JSP pages, `*.jsp` indicates that these apply to all JSP pages.
- `<el-ignored>`: Enables interpretation of JSP EL in JSP pages.
- `<scripting-enabled>`: Allows JSP scripting.

12.4.4 Accessing EL Functions within JSP

To access the function created in a TLD file using a JSP file, you import the TLD file using the `taglib` directive. In the directive statement, you mention a prefix for the tags and the location of the TLD file.

After importing the TLD file, you can access the function using an EL expression.

For the `taglib` directive, declares as follows:

Syntax:

```
<%@ taglib prefix = "prefix" uri="path"%>
```

where,

- `prefix` is the prefix to be used for the tags defined in the TLD file.
- `path` is the location of the TLD file.

For accessing the function:

```
<%= ${prefix:functionName(arguments)} %>
```

Code Snippet 12 shows how to access EL function in JSP.

Code Snippet 12:

```
<%@ taglib prefix="fn" uri="/WEB-INF/tlds/functions"%>
.....
Average of the values is : <%= ${fn:average(values)} %>
.....
```

12.5 Coercion

Automatic conversion of a data from one data type to another data type within an expression is called Coercion. Coercion occurs when the datum is stored as one data type, but its context requires a different data type.

12.5.1 Coercion Concept in EL

Coercion means that the parameters are converted to the appropriate objects or primitives automatically. The JSTL defines appropriate conversions and default values. For example, a string parameter from a request will be coerced to the appropriate object or primitive.

If a parameter that represents the month is passed in the request as a string, the value of the month variable will be correct because the string will be coerced to the correct type when used.

Figure 12.7 depicts types of coercion.

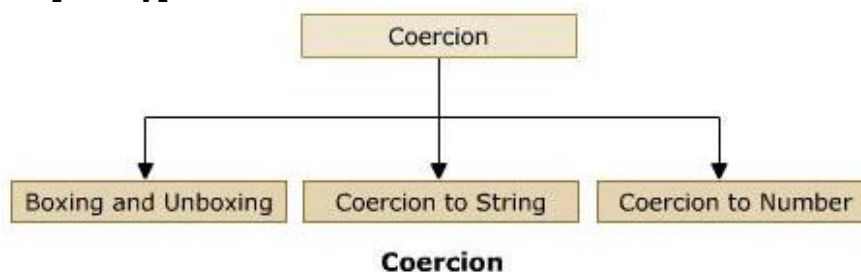


Figure 12.7: Types of Coercion

If the value of the parameter does not parse correctly to a number, for example, the value is `September` instead of `9`, an exception will be thrown. Having automatic type conversions can save unnecessary exceptions from happening.

There is a difference between type coercion and type conversion. Coercion is implicit type conversion and is usually performed automatically by the compiler, whereas type conversion is an explicit type conversion inserted by the programmer.

12.5.2 Boxing and Unboxing

Boxing converts values of primitive type to corresponding values of reference type.

Unboxing converts values of reference type to corresponding values of primitive type.

There are precise rules for boxing and unboxing. They are as follows:

➤ **The precise rules for boxing**

- A) If `i` is a `boolean` value, then boxing conversion converts `i` into a reference `r` of class and type `Boolean`, such that `r.value() == i`.
- B) If `i` is a `byte` value, then boxing conversion converts `i` into a reference `r` of class and type `Byte`, such that `r.value() == i`.

➤ **The precise rules for unboxing**

- A) If `r` is a `Boolean` reference, then unboxing conversion converts `r` into a value `v` of type `boolean`, such that `r.value() == v`.
- B) If `r` is a `Byte` reference, then unboxing conversion converts `r` into a value `v` of type `byte`, such that `r.value() == v`.
- C) If `r` is a `Character` reference, then unboxing conversion converts `r` into a value `v` of type `char`, such that `r.value() == v`.

12.5.3 Coercion to String

The rule to coerce a value to `String` type is as follows:

- `A` is `String`, return `A`
- `A` is `null`, return `""`
- `A.toString()` throws exception, return `error`. Otherwise return `A.toString()`.

12.5.4 Coercion to Number

The rule to coerce a value to number type is as follows:

- If `A` is `null` or `""`, return `0`

`A` is character and is converted to short, you apply following rules:

- If `A` is `Boolean`, return `error`
- If `A` is number type, return `A`

`A` is number, coerce occurs quietly to type `N` using following algorithms:

- If `N` is `BigInteger`
- If `A` is `BigDecimal`, return `A.toBigInteger()`
- Otherwise, return `BigInteger.valueOf(A.longValue())`
- If `N` is `BigDecimal`
- If `A` is a `BigInteger`, return `new BigDecimal(A)`
- Otherwise, return `new BigDecimal(A.doubleValue())`

- If N is Byte, return new Byte(A.byteValue())
- If N is Short, return new Short(A.shortValue())
- If N is Integer, return new Integer(A.integerValue())
- If N is Long, return new Long(A.longValue())
- If N is Float, return new Float(A.floatValue())
- If N is Double, return new Double(A.doubleValue())
- Otherwise return error

If a variable `myInteger` is to be declared and its value in an expression is used as a number, a variable is declared with the value using `<c:set>` as shown in Code Snippet 13.

Code Snippet 13:

```
<c:set var="myInteger" value="${param.month}"/>
<p>
The value of myInteger is: <c:out value="${myInteger}"/>.
</p>
```

The given statement can be demonstrated by performing a multiplication operation to show that the type is correct: `<c:out value="${myInteger * 2}"/>`

Summary

- EL is simple and robust. It can handle both expressions and literals, which are constants and are assigned some memory location.
- EL is a great help to the page authors in accessing and manipulating the application data without mastering the complexities of the programming language such as Java and JavaScript.
- JSP implicit objects are a standard set of classes. The user creates an instance of an implicit object to use available methods and variables.
- Operators are used to perform different arithmetic, relational, and logical operations. Dot operator (.) or [] is used to access value of a variable. Various operators used in Expression Language are arithmetic operators, relational operators, logical operators, empty operators, and dot operators.
- In Expression language the static java methods can be called within the EL expression. To access the function using EL, the function must be implemented as a static function in a java class.
- The TLD file uses XML syntax to map the name of functions defined in a class with EL. Setting the value of the `<el-ignored>` element in the deployment descriptor can explicitly change the default mode.
- The accessing of the function created in a TLD file using a JSP file is possible by importing the TLD file using the `taglib` directive.
- Coercion means that the parameters are converted to the appropriate objects or primitives automatically. Coercion is an implicit type conversion.

Check Your Progress

1. Which of the following statements relating to JSP script expression is incorrect?

(A)	JSP is used to display the dynamic content on a Web page.
(B)	The dynamic content generated using JSP language can be formatted.
(C)	The JSP expression is inserted into the page after being evaluated and converted to string.
(D)	Semicolon is not used at the end of the JSP script expression.

2. Which option is the correct one for `<jsp:include>` from the following statements?

(a)	EL is simple and robust.
(b)	EL can handle both expressions and literals.
(c)	The application data stored can be accessed but cannot be manipulated by EL.
(d)	The advantage of EL is that it provides cleaner syntax and is specifically designed for JSP.
(e)	JSP expression language cannot be used to display the generated dynamic content in a table on a Web page.
(f)	EL makes it possible to easily access application data stored in JavaBeans components.
(g)	JSP expression language can be used to display the generated dynamic content in a table on a Web page.
(h)	The page authors should be master in programming language, such as Java and JavaScript to utilize the benefits of EL.

A.	a, c, d, e, f	C.	a, b, c, d, e
B.	a, b, d, f, h	D.	a, c, d, e

3. Which implicit object defined in JSP EL can be used without creating an instance of the object?

(A)	<code>servletContext</code>
(B)	<code>pageContext</code>
(C)	<code>session</code>
(D)	<code>request</code>

4. Match the different implicit objects against their corresponding description.

Implicit Object		Description	
(a)	<code>initParam</code>	(1)	This object maps a request parameter name to a single value.
(b)	<code>headerValue</code>	(2)	This object maps a context initialization parameter name to a single value.
(c)	<code>param</code>	(3)	This object maps a request header name to an array of values.
(d)	<code>header</code>	(4)	This object maps a request parameter name to an array of values.
(e)	<code>paramvalues</code>	(5)	This object maps a request header name to a single value.

A.	a-2, b-3, c-4, d-5, e-1	C.	a-3, b-1, c-2, d-5, e-4
B.	a-4, b-5, c-1, d-2, e-3	D.	a-5, b-1, c-2, d-3, e-4

5. Which scope is accessible from the JSP page that creates the object?

(A)	<code>requestScope</code>
(B)	<code>pageScope</code>
(C)	<code>sessionScope</code>
(D)	<code>applicationScope</code>

6. Which among the following statements related to `pageContext` is incorrect?

(A)	The <code>servletContext</code> object specifies the servlet of the JSP page and Web components contained in the same application.
(B)	The <code>session</code> object represents the session created for the client sending a request.
(C)	The <code>request</code> object represents the request accepted by the JSP page from a client and information of the request.
(D)	The <code>response</code> object represents the response sent by the client using a JSP page.

Answers

1.	B
2.	B
3.	B
4.	C
5.	B
6.	A

Try It Yourself

1. Create a simple JSP page that retrieves sample order data and displays it on a Web page using JSP and EL. Table 12.2 shows sample data representing orders in a bookstore Web application. Utilize this data to populate the Web page showing order details. Deploy the JSP files to a Web container such as Tomcat. Access the OrderDetails page in a Web browser to see the displayed sample order data.

Order ID	Customer Name	Book Title	Quantity	Total Price
101	John Doe	The Great Gatsby	2	\$40.00
102	Jane Smith	To Kill a Mockingbird	1	\$25.00
103	Alice Johnson	Pride and Prejudice	3	\$60.00

Table 12.2: Sample Data

2. Explore how to loop through a list of orders instead of manually defining each row.

Standard Tag Library and Internationalization

Welcome to the Session, **Standard Tag Library and Internationalization**. This session describes the Jakarta Server Pages Standard Tag Library (JSTL), its various elements, and its benefits. The session lists and describes the JSTLs core and SQL library tags and their usage on the JSP page. This session also describes the concept of internationalization and localization and the resource bundling mechanism and various locales. Additionally, it explains various formatting techniques in servlets for internationalization and reactive JAX-RS Client API.

In this session, you will learn to:

- List the advantages of using JSTL
- Describe different tag libraries available in JSTL
- Explain how to configure the JSTL library in Eclipse
- Explain general-purpose tags
- Explain decision-making in the tags
- Explain iteration tags in the core tag library
- List different tags available in the SQL tag library
- Recall in brief concept of internationalization and localization
- Explain the resource bundling mechanism and resource bundle for various locales
- Explain various formatting techniques in servlets for internationalization
- Explain Reactive JAX-RS Client API

13.1 Introduction

Consider a situation in a shopping cart Web application, where a JSP page generates a dynamic list displaying user-selected products on the Web page. To display such a dynamic list in a cart with selected or removed products, only HTML tags will not be useful. In such a case, the Web designer has to involve himself/herself in coding the scripts, such as iterating through the list elements or displaying data based on a condition. This may not be feasible in all situations.



To help Web designers, Sun developed a pre-defined tag library, which contains tags related to the core common functionalities performed in Java applications. The pre-defined tag library is now known as Jakarta Server Pages Standard Tag Library

(JSTL) that helps to reuse standard tags that work in a similar manner in every Java Web application. JSTL allows programming using tags rather than scriptlet code.

13.1.1 Advantages of JSTL

JSTL is a very beneficial technology for the development of JSP applications. Some of the benefits of using JSTL are as follows:

JSTL provides most of the functionality necessary for development of the JSP application, so you do not have to develop your own tag.

JSTL has a number of tags for common tasks, such as interacting with databases, iterating through lists, formatting data, handling XML data, and more. You can use these tags and create a program with business logic that saves lots of development time that you could waste in the developing the common tags.

JSTL is based on XML, which is very similar to HTML. Hence, HTML programmers easily start programming using JSTL.

Since JSTL is always expressed in XML-compliant tags, it will be easier for HTML generation tools to parse the JSTL code that is contained within the

JSTL includes tags that support both formatting and multilingual support, which provides a consistent approach to formatting of numbers and strings, and internationalization (I18N) support features in JSP scriptlet code.

JSTL provides a mechanism that enables the programmer to develop his or her own custom tags as well.

13.1.2 JSTL Tag Libraries

JSTL provides various tag libraries that can be used for many functionalities. Figure 13.1 shows the JSTL tag libraries.

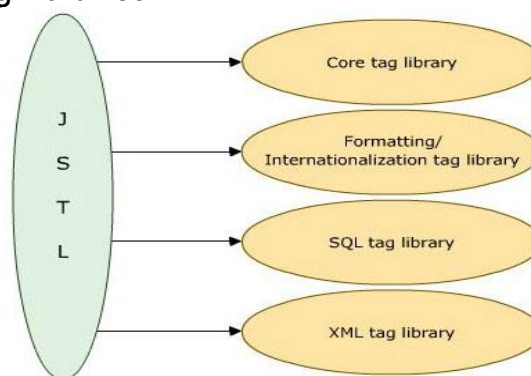


Figure 13.1: JSTL Tag Libraries

Table 13.1 lists the functional areas and the URIs and the prefixes used to reference the libraries.

Functional Area	URI	Prefix
core	http://java.sun.com/jsp/jstl/core	c
XML processing	http://java.sun.com/jsp/jstl/xml	x
l18N capable formatting	http://java.sun.com/jsp/jstl/fmt	fmt
Relational db access (SQL)	http://java.sun.com/jsp/jstl/sql	sql

Table 13.1: JSTL Functional Areas and URIs

Source: <https://jakarta.ee/specifications/tags/2.0/jakarta-tags-spec-2.0.html#multiple-tag-libraries>

Core Tag Library

The library contains the tags for looping, expression evaluation, and basic input/output. It can be declared with `<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core" %>`

**Formatting/
Internationalization
(l18N) Tag Library**

The library contains the tags which are used to parse the data, such as dates and time, based on the current location. It can be declared with `<%@ taglib prefix="fmt" uri="http://java.sun.com/jstl/fmt" %>`

SQL Tag Library

The library contains tags which are used to access SQL databases. It provides an interface for executing SQL queries on the database through JSP. It can be declared with `<%@ taglib prefix="sql" uri="http://java.sun.com/jstl/sql" %>`

XML Tag Library

The library contains tags for accessing XML elements. In JSTL, XML processing is an important feature, as it is used in many Web applications for managing data. It can be declared with `<%@ taglib prefix="x" uri="http://java.sun.com/jstl/xml" %>`

13.2 jakarta.servlet.jsp.tagext

`jakarta.servlet.jsp.tagext` contains all the classes and interfaces that describe a JSP Tag Library. The `taglib` directives are JSP Page directives that are part of the JSP source code. The use of the `taglib` directive on a JSP page informs the servlet that the page uses a tag library. The tag library contains multiple user-defined tags to implement a customized behavior. The `taglib` directive contains information about the location specifics of the library but also gives a mechanism for identifying the custom tags that are being used in the JSP page.

A JSP tag library primarily consists of four elements:

- A Tag Library Descriptor (TLD)
- Multiple tag files that establish request-time behavior
- Additional classes and resources used at runtime
- Some additional classes to provide extra translation information

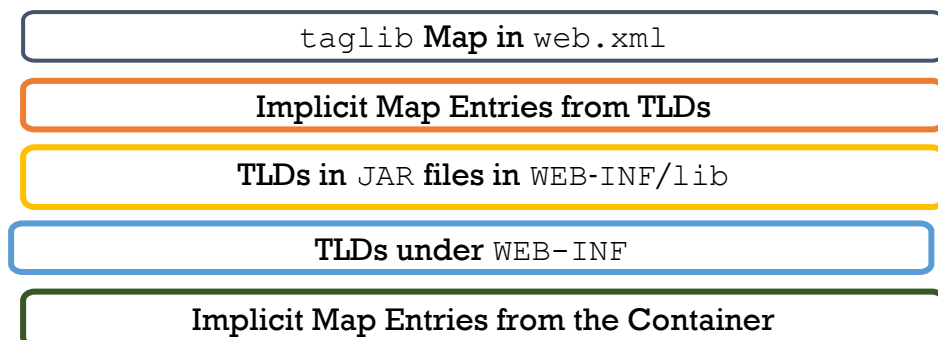
13.2.1 Tag Library Descriptor (TLD)

A Tag Library Descriptor (TLD) as the name indicates describes the tags and connects them to their related Java classes. It includes information on the tag library and its tags. The information of TLD is used by the JSP container to parse JSP pages with `taglib` directives.

Some of the essential attributes of the TLD are as follows:

- TLD file is written in an XML notation and has the `.tld` extension.
- If deployed straight into a Web application, TLD files must always be in the `WEB-INF` directory or any of its subdirectories, whereas if deployed within a Java Archive (JAR), the tag library descriptor files must be in the `META-INF` directory or any of its subdirectory.
- The URI in the `taglib` directive points to a context relative path; this path hosts components that start with/and is called the TLD resource path.
- A direct map from URIs to TLD resource paths may be specified with the help of `taglib` elements and included in the `web.xml` file as an explicit `taglib` map. The Map entries for the tag libraries are included in the Java EE platform. This includes the JSTL as well as the Java Server Faces libraries.

Following sequence must be observed during `taglib` map construction:



13.2.2 Tag Handler

Tag handler analyzes the custom actions of the JSP page while it is being executed. It is controlled by the container. Some of the important attributes of the tag handler include:

Every tag handler has a `pageContext` attribute for the JSP page in which the tag is located, in addition to a `parent` property that connects the tag handler to the action that is immediately around it.

A tag handler that does not want to process the body of its tags can simply implement the tag interface. It is possible that the tag handler does not analyze the body if the tag is empty or if the body is only supposed to be passed through. `DoStartTag` and `DoEndTag` are two of the most important methods in the tag interface that are used to start or end a tag ().

To create a tag handler, implement

`jakarta.servlet.jsp.tagext.SimpleTag` interface as demonstrated in Code Snippet 1. Here, a class 'Sess13' is created as a tag handler and extends `SimpleTagSupport`.

Code Snippet 1:

```
import jakarta.servlet.jsp.JspException;
import jakarta.servlet.jsp.JspWriter;
import jakarta.servlet.jsp.tagext.SimpleTagSupport;
import java.io.IOException;
public class Sess13 extends SimpleTagSupport {
    public void doTag() throws JspException, IOException {
        JspWriter out = getJspContext().getOut();
        out.write("Hello students");
    }
}
```

The complete steps to create and use a tag handler with a JSTL library project are as follows:

1. In Eclipse, create a Dynamic Web Project named **session_13**.
2. Add a tag handler class with the name **Sess13.java** having code given in Code Snippet 1 to the project.
3. Create a TLD file for the application with code as given in Code Snippet 2.

Code Snippet 2: mytag.tld

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib xmlns="https://jakarta.ee/xml/ns/jakartaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
        xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-
jsptaglibrary_3_1.xsd"
        version="3.1">
    <tlib-version>2.0</tlib-version>
    <short-name>simple</short-name>
    <uri>http://localhost:8087/mytaglib</uri>

    <tag>
        <name>MyMsg</name>
        <tag-class>Sess13</tag-class>
        <body-content>empty</body-content>
    </tag>
</taglib>
```


4. Create an HTML file and a JSP file for core tags used in a login application. Code Snippets 3 and 4 demonstrate the code for these files.

Code Snippet 3: welcome.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Welcome Page</title>
</head>
<body>
<h1>Welcome</h1>
  <form action="login.jsp">
    User Name:
    <input type="text" name="username"/>
    <br/> <br/>
    Password:
    <input type="password" name="password"/>
    <br/> <br/>
    <input type="submit" value="Submit" name="Submit">
  </form>
</body>
</html>
```

Code Snippet 4: login.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-
8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"
%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Login Page</title>
</head>
<body>
  <!-- Set variables from request parameters --%>
  <c:set var="name" value="${param.username}"/>
  <c:set var="password" value="${param.password}"/>

  Entered Name: <c:out value="${name}"/> <br/><br/>
  Entered Password: <c:out value="${password}"/>
</body>
</html>
```

The `login.jsp` page assigns the variables `name` and `password` with the value from the request parameters `username` and `password` respectively. The `out` tag outputs the value of `name` and `password` respectively.

5. Download **`jakarta.servlet.jsp.jstl-2.0.0.jar`** and **`jakarta.servlet.jsp.jstl-api-2.0.0.jar`** from these links:

<https://mvnrepository.com/artifact/org.glassfish.web/jakarta.servlet.jsp.jstl/2.0.0>

<https://mvnrepository.com/artifact/jakarta.servlet.jsp.jstl-api/2.0.0>

6. Add the jar files to the build path as shown in Figure 13.2. To perform this, right-click **Project**. Click **Configure Build Path** and then, click **Add External Jars**. Locate the jar files and add them to the project.

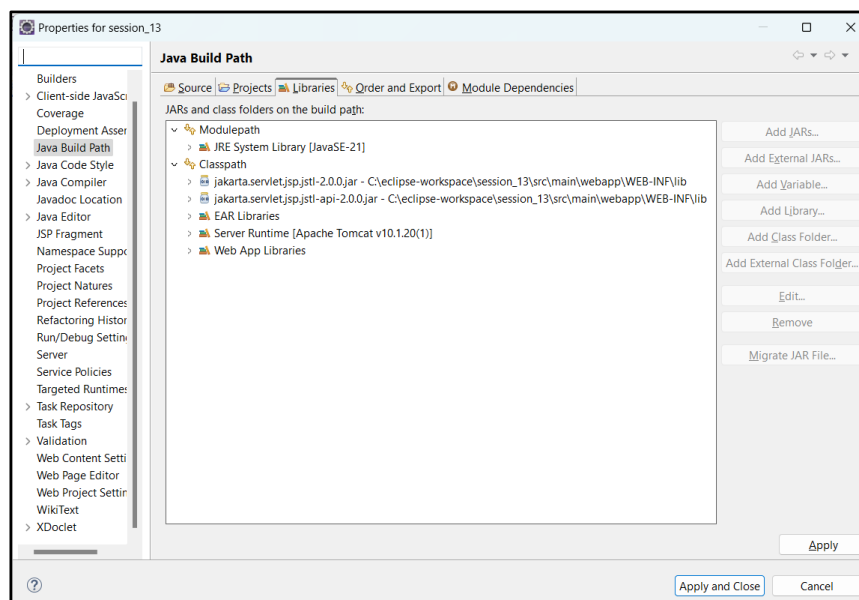


Figure 13.2: Build Path

The project structure will now be as shown in Figure 13.3.

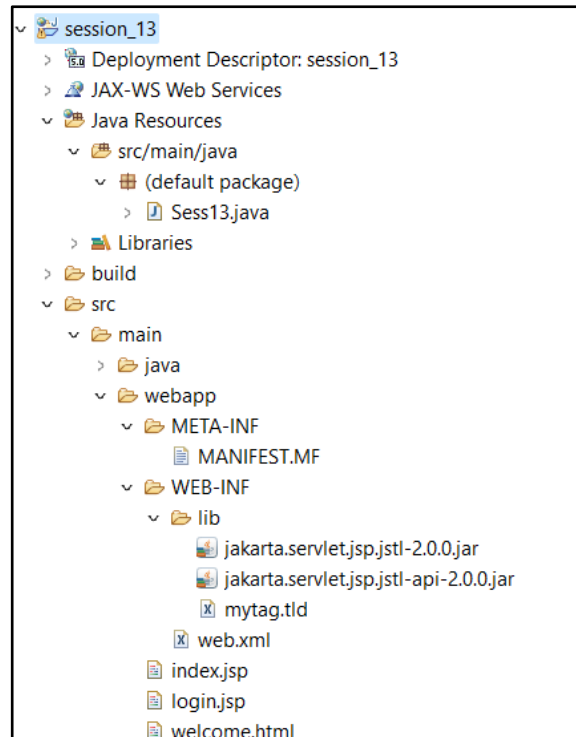


Figure 13.3: Project Structure

7. Create `web.xml` as shown in Code Snippet 5.

Code Snippet 5: `web.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:jsp="http://java.sun.com/xml/ns/javaee/jsp"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
  <display-name>session_13</display-name>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <jsp-config>
    <taglib>
      <taglib-uri>http://tomcat.apache.org/example-
taglib</taglib-uri>
      <taglib-location>/WEB-INF/mytag.tld</taglib-location>
    </taglib>
  </jsp-config>
  <servlet>
    <description></description>
    <display-name>Sess13</display-name>
    <servlet-name>Sess13</servlet-name>
```

```

    <servlet-class>Sess13</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>Sess13</servlet-name>
    <url-pattern>/Sess13</url-pattern>
  </servlet-mapping>
</web-app>

```

8. Create `index.jsp` as shown in Code Snippet 6.

Code Snippet 6: `index.jsp`

```

<%@ page language="java" contentType="text/html; charset=UTF-
8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <title>Entry Page</title>
</head>
<body>
    <h1>Welcome to My Website</h1>
    <p>This is the homepage of my Website.</p>
    <p>Please proceed to the login page:</p>
    <a href="welcome.html">Login</a>
</body>
</html>

```

Run the project and provide some random text for username and password. The entered data will be displayed in the browser using JSTL tags. Figures 13.4, 13.5, and 13.6 show the output of JSTL tags.

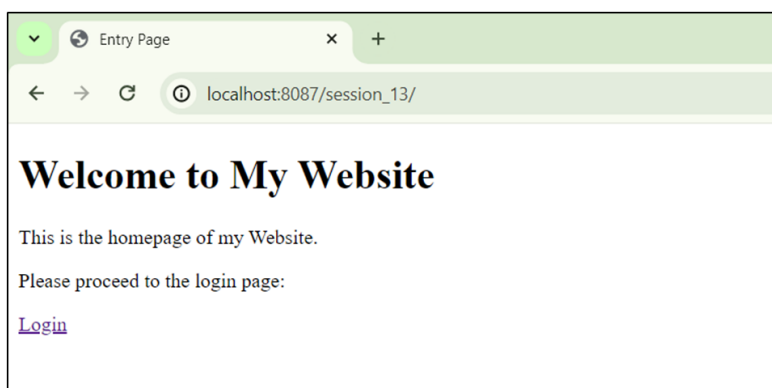


Figure 13.4: Output of `index.jsp`

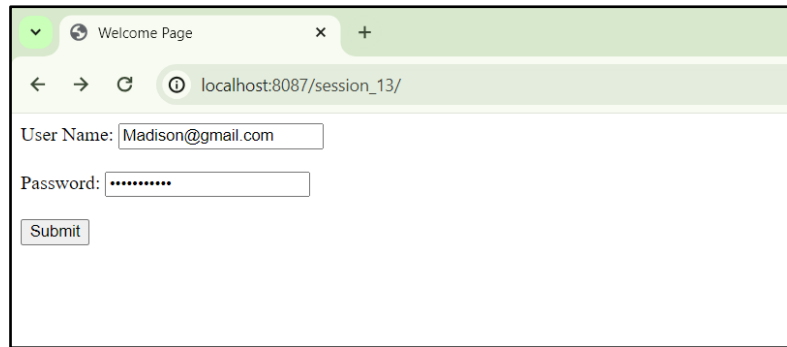


Figure 13.5: Output of welcome.html

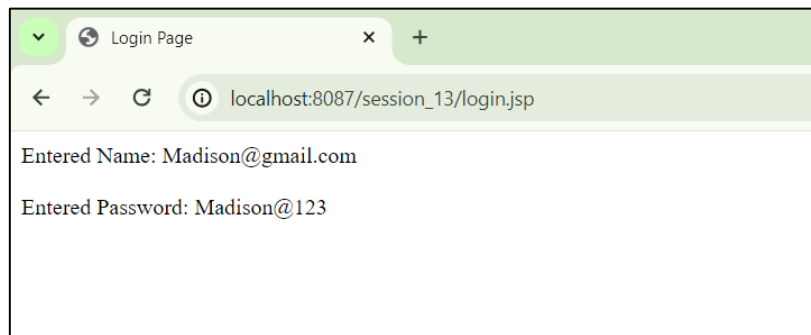


Figure 13.6: Output of login.jsp

13.3 Core Tag Library

The Core tag library provides support for conditional logic, iteration, forward, or redirect response, URL, catch exception, and so on. It is the most frequently used JSTL tag library containing core group of tags. The core tag library tags are prefixed with `c`.

13.3.1 General-Purpose Tags

General-purpose tags are used to set, remove, and display variable values that are created within a JSP page. The core tag library contains tags for getting, setting, and displaying attribute values.

The general-purpose tags are as follows:

➤ `<c:set>`

This tag assigns a value to a variable in scope.

Syntax:

```
<c:set var = "varName" value = "expression" scope =
"page/request/ session/ application"/>
```

where,

- `value` specifies the expression.

- **var** specifies the name of the exported scope to hold the value specified in the tag.
- **scope** specifies the scope of variable such as page, request, session, and application. The default scope is page.

Example: `<c:set var = "sessionvariable" value = "${80+8}" scope = "session" />`

➤ **<c:remove>**

This tag is used to remove a scoped variable. This is an empty tag.

Syntax:

```
<c:remove var = "varName" scope =
"page/request/session/application"/>
```

where,

- **var** specifies the name of the variable to be removed.
- **scope** specifies the scope of the variable.

Example: `<c:remove var="simple" scope="page"/>`

➤ **<c:out>**

This tag is used to evaluate an expression and store the result in the current `JspWriter` object.

Syntax:

```
<c:out value = "value" escapeXml = "boolean" default =
"defaultvalue"/>
```

where,

- **value** specifies the expression.
- **default** specifies the default value if the value of the result is null.
- **escapeXml** determines that special characters in the result, such as `<`, `>`, `&`, `'`, and `"` should be converted to their character entity codes. The default value of `escapeXml` is true.

Example: `<c:out value = "${sessionvariable}"></c:out>`

➤ **<c:catch>**

This tag provides an exception handling functionality such as `try-catch`, inside JSP pages without using scriptlets.

Syntax:

```
<c:catch [var="varName"]>
    nested actions
    . . .
</c:catch>
```

where,

- **var** specifies the name of the variable to be caught.

Code Snippet 7 demonstrates how to catch an exception using `<c:catch>` tag.

Code Snippet 7:

```
<body>
<c:catch var="e">
    100 divided by 0 is
    <c:out value="${100/0}" />
<br />
</c:catch>
```

The `catch` tag provides an error handling mechanism for the division operation. The exception raised by dividing the number from 0 is stored in the variable **var**.

13.3.2 Decision-Making Tags

JSTL provides decision-making tags to support conditions in a JSP page. Decision-making tags are necessary as the contents or the output of the JSP page is often conditional based on the value of the dynamic application data.

The two types of decision-making tags are as follows:

➤ `<c:if>`

The tag is used for the conditional execution of the code. This tag is a container tag that allows the execution of the body if the test attribute evaluates to true.

Syntax:

```
<c:if test = "testCondition" var = "varName" scope =
"page/request/session/application">
    Body Content
</c:if>
```

where,

- **test** specifies the test condition.
- **var** specifies the name of the variable of the test condition.
- **scope** specifies the scope of the variable, **var**.

In `<c:if>` tag, attribute **var** and **scope** are optional.

Code Snippet 8 demonstrates the use of the `if` tag to evaluate.

Code Snippet 8:

```
<!-- Test the username -->
```

```

<c:set var = "name" scope = "session" value = "admin"/>
...
    <c:if test="${name=='admin'}">
        <h4> Valid User </h4>
    </c:if>

```

The code uses the `if` tag to check the value of the `name` variable. If the condition evaluates to true, that is, the value assigned to the `name` variable is `admin`, then the body is executed.

<c:choose>

The tag is similar to the `switch` statement in Java. The `<c:choose>` tag performs conditional block execution. The `<c:choose>` tag processes body of the `<c:when>` tag. Multiple `<c:when>` tags can be embedded in a `<c:choose>` tag. If none of the conditions evaluates to true, then the body of `<c:otherwise>` tag is processed.

Syntax:

```

<c:choose>
    <c:when test = "testcondition">
        Body Content
    </c:when>
    . . .
    . . .
    <c:otherwise>
        Body Content
    </c:otherwise>
</c:choose>

```

where,

- `<c:when>` is the body of `<c:choose>`. It is a container tag and has test conditions. It will execute the body content if the test condition evaluates to true.
- `<c:otherwise>` is executed when none of the test conditions of `<c:when>` evaluates to true.

Code Snippet 9 demonstrates the `choose` tag.

Code Snippet 9:

```

<!-- Performs the checking of the condition -->
    <c:choose>
        <c:when test="${name == 'admin'}">
            <h4> Valid User </h4>
        </c:when>
        <c:otherwise>

```



```

        <h4> Invalid User </h4>
    </c:otherwise>
</c:choose>

```

The `choose` tag performs an evaluation of the condition to test if the name entered is 'admin'. If the condition evaluates to true, then the nested `when` tag is executed; else, the nested `otherwise` tag is executed.

13.3.3 Iteration Tags

The iteration tag is required for performing the looping function. The object can be retrieved from a collection in the JavaBeans components and assigned to a scripting variable by using iteration tags.

The two iteration tags are as follows:

➤ **<c:forEach>**

This tag is used to repeat the body content over a collection of objects. The iteration will continue for the number of times specified by the user in the code.

Syntax:

```

<c:forEach var = "varName" item = "collection" varStatus =
"varStatusName" begin = "begin" end = "end" step = "step">
    Body Content
</c:forEach>

```

where,

- `var` specifies the name of the exported scoped variable.
- `item` specifies the collection of items to iterate over.
- `varStatus` specifies the name of the variable for the status of iteration.
- `begin` specifies the index from which the iteration is to begin.
- `end` specifies the index at which the iteration is to end.
- `step` specifies that iteration will process every item of the collection.

Code Snippet 10 demonstrates the `forEach` tag.

Code Snippet 10:

```

// Displays objects from collection "companies"
<c:forEach var= "company" items = "${companies}">
    ${company} <br>
</c:forEach>

// displays values from 10 to 100
<c:forEach var="i" begin="10" end="100">
    ${i}
</c:forEach>

```

➤ `<c:forTokens>`

This tag is used to iterate over a collection of tokens separated by user-specified delimiters. It is a container tag.

Syntax:

```
<c:forTokens items = "stringofToken" delims = "delimiters"
var = "varName" varStatus = "varStatusName" begin = "begin"
end = "end" step = "step">
    Body Content
</c:forTokens>
```

where,

- `items` specify the string of values to iterate.
- `delims` specifies the character that separates the tokens in the string.
- `var` specifies the name of the scope variable for the item of iteration.
- `varStatus` specifies the name of the scope variable for the status of iteration.

Code Snippet 11 demonstrates the use of `<c:forTokens>` tag.

Code Snippet 11:

```
// Displays each token at a time separated by ',' delimiter
<c:forTokens var="token" items="Tom,Dick,Harry" delims=",">

    <c:out value="${token}"/>
</c:forTokens>
```

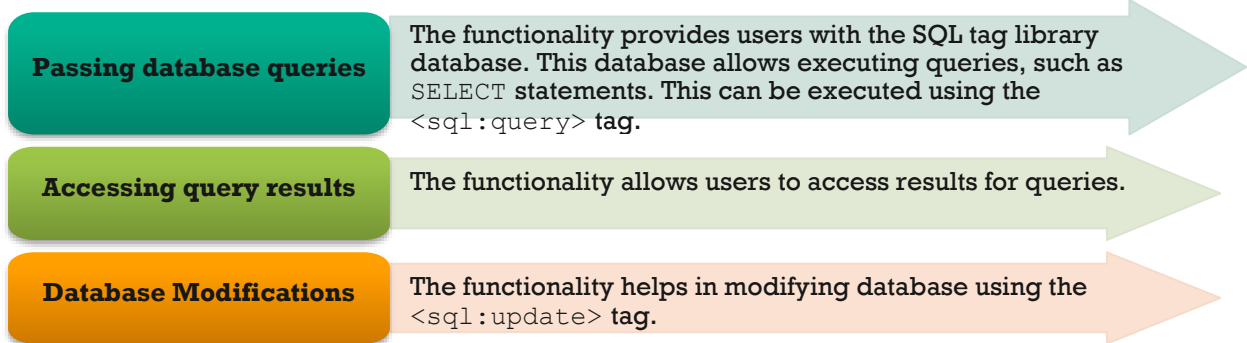
13.4 SQL Tag Library

To interact with other databases such as Oracle, MySQL, or Microsoft SQL Server, the JSTL SQL tag library is used. These tags are prefixed by `sql`.

13.4.1 `<sql:setDataSource>`

The JSTL SQL tags are used to access databases and are designed for low-volume Web-based applications. SQL tag library provides tags that allow direct database access within a JSP page.

The JSTL SQL tag provides following functionalities:



Syntax:

```
<sql:setDataSource dataSource = "datasource" | url = "jdbcurl"
driver =
"driverclassdriver" user = "username" password = "userpwd" var
= "varname" scope = "page/request/session/application"/>
```

where,

- `dataSource` can either be the path to Java Naming and Directory Interface (JNDI) resource or a JDBC parameter string.
- `url` is the URL associated with the database.
- `driver` is a JDBC parameter and takes the driver class name.
- `user` takes the user of the database.
- `password` takes the user password.
- `var` is the name of exported scoped variable for the data source specified.
- `scope` specifies the scope of the variable.

Code Snippet 12 demonstrates the use of `<sql:setDataSource>` SQL tag to make the connection to SQL Server database.

Code Snippet 12:

```
// Sets the data source for the database
<sql:setDataSource
    driver="com.microsoft.jdbc.sqlserver.SQLServerDriver"
    url="jdbc:microsoft:sqlserver://10.1.3.27:1433;DataBaseName=pu
bs;" user="sa" password="playware" var="conn"/>
```

In the code, `<sql:setDataSource>` is used to set a data source for the database. This is an empty tag and allows the user to set data source information for the database. In `<sql:setDataSource>` tag, then `url` attribute provides the JDBC url string for the SQL Server database.

13.4.2 <sql:query>

The <sql:query> tag searches the database and returns a result set containing rows of data. The tag can either be an empty tag or a container tag. The SELECT statement is used to select data from the table.

Syntax:

```
<sql:query sql = "sqlQuery" var = "varName" scope =
"{page|request|session|application}" dataSource = "dataSource"
maxRows = "maxRows" startRow = "startRow"/>
```

where,

- sql specifies the SQL query statement.
- var specifies the name of the exported scope variable for the query result.
- scope specifies the scope of the variable.
- dataSource specifies the data source associated with the database to query.
- maxRows specifies the maximum number of rows to be included in the result.
- startRow specifies the row starting at the specified index.

Syntax:

```
<sql:query var = "varName" dataSource = "dataSource" scope =
"{page|request|session|application}" maxRows = "maxRows"
startRow = "startRow">
    SQL Statement
    <sql:param/>
</sql:query>
```

where,

- param takes the parameter for the query.

Following is the syntax of the JSTL Core library and SQL tag library on a JSP page.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core"
prefix="c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql"
prefix="sql"%>
```

Code Snippet 13 demonstrates the use of <sql:query> tag.

Code Snippet 13:

```
// Gets the records about the 'product' from data source 'conn'
<sql:query var="products" dataSource="{conn}">
    select * from Products
</sql:query>
```

Code Snippet 14 demonstrates SQL core tag libraries that can be used with the login example shown earlier.

Code Snippet 14: index.jsp

```

<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core"
prefix="c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql"
prefix="sql"%>
<!DOCTYPE html>
<html>
<body>
<sql:setDataSource var="db"
driver="com.mysql.cj.jdbc.Driver"
    url="jdbc:mysql://localhost:3306/test"
    user="root" password=""/>
    <sql:query dataSource="${db}" var="rs">
SELECT * from login;
</sql:query>

<table border="2" width="100%">
<tr>
<th>User Name</th>
<th>Password</th>

</tr>
<c:forEach var="table" items="${rs.rows}">
<tr>
<td><c:out value="${table.uname}"/></td>
<td><c:out value="${table.pwd}"/></td>
</tr>
</c:forEach>
</table>
</body>
</html>

```

13.4.3 <sql:update>

The `<sql:update>` tag executes the INSERT, UPDATE, and DELETE statements. It returns 0, if no rows are affected by the DML statements.

Syntax:

```

<sql:update dataSource = "dataSource" var = "varName" scope =
"{page|request|session|application}">
    SQL Statement
    <sql:param value = "value"/>
</sql:update>

```

where,

- **dataSource:** The name of the data source that provides the connection to the database.
- **var:** The name of the variable that will store the number of rows affected by the SQL operation.
- **scope: (Optional)** The scope in which the variable will be available. It can be page, request, session, or application.
SQL Statement: The SQL command to be executed (INSERT, UPDATE, or DELETE).
- **sql:param: (Optional)** The parameters for the SQL statement being executed.

Code Snippet 15 demonstrates the use of `<sql:update>` tag.

Code Snippet 15:

```
// Adds the product details in the product table
<sql:update var="newrow" dataSource="${conn}"
INSERT INTO Products(ProductName, ProductType, Price,
Brand,
Description)
VALUES('Jeans', 'Clothes', '1000', 'Lee', 'Good Quality
Jeans')
</sql:update>
```

Figure 13.7 shows the **product** table with existing data. Figure 13.8 shows the **product** table with newly added data using the `sql:update` tag.

```
mysql> select * from product;
+-----+-----+-----+-----+-----+
| ProductName | ProductType | Price | Brand | Description |
+-----+-----+-----+-----+-----+
| shirt      | cotton      | 100.00 | otto  | Good Product |
+-----+-----+-----+-----+-----+
```

Figure 13.7: Products Table with Existing Data

```
mysql> select * from product;
+-----+-----+-----+-----+-----+
| ProductName | ProductType | Price | Brand | Description |
+-----+-----+-----+-----+-----+
| shirt      | cotton      | 100.00 | otto  | Good Product |
| Jeans     | Clothes     | 1000.00 | Lee   | Good Quality Jeans |
+-----+-----+-----+-----+-----+
```

Figure 13.8: Products Table with Newly Added Data

13.4.4 `<sql:transaction>`

The `<sql:transaction>` is used to establish a transaction context for `<sql:query>` and `<sql:update>` tags. The connection object is obtained from `<sql:transaction>` as this tag is responsible for managing access to the database.

Syntax:

```
<sql:transaction dataSource = "dataSource" isolation =
isolationLevel>
    <sql:update> or <sql:query > statements
</sql:transaction>
```

where,

- **dataSource** sets the SQL data source which can be a string or a data source object.
- **isolation** sets the transaction isolation level. Isolation level can be read_committed, read_uncommitted, repeatable_read, or serializable.

Code Snippet 16 demonstrates the use of <sql:transaction> tag.

Code Snippet 16:

```
/** The code snippet performs a transaction by first
accessing a data source to create a table and then inserting
a row. It then performs a SQL queries on the table
**/
<sql:transaction dataSource="${mydatasource}">
    <sql:update var="newTable">
        create table emp (
            id int primary key,
            name varchar(80)
        )
    </sql:update>
    <sql:update var="updateCount">
        INSERT INTO emp VALUES (1,'Jenny')
    </sql:update>
    <sql:update var="updateCount">
        INSERT INTO emp VALUES (2,'Christina')
    </sql:update>
    ...
    <sql:query var="empQuery">
        SELECT * FROM emp
    </sql:query>
</sql:transaction>
```

13.4.5 <sql:param>

<sql:param> is used to set values for parameters markers ('?') in SQL statements. It acts as a sub tag for **<sql:query>** and **<sql:update>**.

Syntax:

```
<sql:param value = "value"/>
```

where,

value sets the value for the parameter.

Code Snippet 17 demonstrates how to use the `<sql:param>` in database updation.

Code Snippet 17:

```
//This snippet sets the values for the parameters
<sql:update>
    INSERT INTO Employee (DemoFirstName, DemoLastName,
EmpDate)
    VALUES(?, ?, ?)
    <sql:param value="${param. DemofirstName}" />
    <sql:param value="${param. DemolastName}" />
    <sql:dateParam value="${empDemoDate}" type="date" />
</sql:update>
```

Figure 13.9 depicts the use of `<sql:param>` tag.



```
<sql:query var="product"
  sql="select * from PUBLIC.product where id = ?" >
  <sql:param value="${product.id}" />
</sql:query>

<c:forEach var="productRow" begin="0"
  items="${product.rows}">
  <sql:update var="product" sql="update PUBLIC. product set
  inventory = inventory - ? where id = ?" >
  <sql:param value="${item.quantity}" />
  <sql:param value="${product.id}" />
  </sql:update>
</c:forEach>
```

`<sql:param>`

Figure 13.9: `<sql:param>` Tag

13.5 Internationalization and Localization

Internationalization can be defined as the method of designing and developing an application that can be deployed internationally. The concept where companies adapt and customize their products and services to quickly access multiple national markets is called Internationalization.

One of the examples of Internalization is the Amazon Website. Amazon caters to an international market. Therefore, if someone from UK accesses Amazon, the response is in a format that can be comprehended by people in the UK. The currency is in Pound sterling. On the other hand, Amazon in US shows prices in US dollars.

Internationalization in Java is also referred to as I18N.

Localization refers to adapting of the application for a particular region or locality. This also means that the application adapts to the language and culture of the region.

The concepts associated with localization and internationalization are Unicode, Locale, and Resource bundle.

13.6 Unicode

As there are numerous geographical areas with various languages and conventions, a uniform standard is required for encoding these diversities. The Unicode standard serves this purpose.

Unicode is a coding system that has codes for all the major languages of the world. The coding system also includes punctuation marks, symbols, and classical as well as historical texts of many written languages.

Unicode in Java is a 16-bit character encoding. By default, java uses UCS-2. This is a fixed-width two-byte encoding that simply encodes each code point from U+0000 to U+FFFF as itself. It has the advantage of being simple and fast but has the disadvantage of being byte-order dependent and of being incompatible with 7-bit ASCII. It allows java to handle international characters for most of the languages of the world.

Figure 13.10 depicts the Unicode table.



Figure 13.10: Unicode Table

13.7 Locale

Locale refers to an object for a particular language or region. The region can be political, geographical, or cultural. `java.util.Locale` is the class through which locale is implemented.

Some of the fields of `Locale` object are as follows:

- **Country:** This is used to represent the country code. This is also case insensitive.
- **Language:** It is used to represent the local language. This is case insensitive. It uses language codes.
- **Script:** It is used to represent the script. For example, Latin. This is also case insensitive.

An internationalized application can be adapted to any locale. The applications can be localized using resource bundles, which contain locale-specific objects. This helps in coding programs, which are independent of the location of the user.

Some of the examples of locales are: `zh_CN` for Shanghai, China, `ko_KR` for Seoul, Korea, and `en_US` for English, US.

Note: `Locale` in Jakarta Standard Tag Library are the same as defined in the class `java.util.locale`.

13.8 Resource Bundles

Content in Web applications can be displayed in any particular language (provided it is supported by the underlying platform). This can be done easily using property files, which are also called message resources.

Internationalization of an application can be achieved with the help of resource bundles. They are especially important for developing applications for different languages. Resource bundles contain locale-specific data.

Resource bundle is a set of related classes that inherit from `ResourceBundle` class of `java.util` package.

`ResourceBundle` enables creating internationalized and localized programs that can display UI and output in different languages. An object of the `ResourceBundle` class represents locale-specific information. When a program requires a locale-specific resource, for example, a `String`, the program loads it from the resource bundle based on the current locale of the user.

The subclass of `ResourceBundle` has the same base name with an additional component that identifies locales. For example, if resource bundle is named `DemoResources`, along with it, locale-specific classes can be related as in `DemoResources_en_US`.

Resource bundling helps to build server-side code that gives output based on the location and language of the user. It makes the job easier by avoiding the writing of multiple version of a class for different locales.

There are several methods in `ResourceBundle` class as follows:

➤ **getBundle()**

In order to get resource bundle for getting locale-specific data, the overloaded `getBundle()` method is used. This method gets a resource bundle using the specified base name, the default locale, and the class loader of the caller.

Syntax:

```
public static final ResourceBundle getBundle(String baseName)
```

where,

- `baseName` is the name of the resource bundle.

```
public static final ResourceBundle getBundle(String baseName,
Locale locale)
```

where,

- `baseName` is the base name of the resource bundle, a fully qualified class name.
- `locale` is the locale for which a resource bundle is desired.

Code Snippet 18 demonstrates a complete example to retrieve a resource bundle using a specified base name and generate corresponding output based on locale.

Code Snippet 18: `TestInternationalization.java`

```
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Locale;
import java.util.ResourceBundle;
import jakarta.servlet.annotation.WebServlet; // Import
the necessary annotation

@SuppressWarnings("serial")
@WebServlet("/TestInternationalization") // Define the
servlet mapping
public class TestInternationalization extends HttpServlet
{
    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
        response.setContentType("text/html; charset=UTF-
8");
        PrintWriter out = response.getWriter();
```

```

        Locale locale = Locale.of("ja", "JP");
        out.println(locale.getDisplayName());

        ResourceBundle bundle =
ResourceBundle.getBundle("DemoLabelsBundle", locale);
        out.println(bundle.getString("greetings") +
"<br>");
        out.println(bundle.getString("inquiry") + "<br>");
        out.println(bundle.getString("farewell") +
"<br>");
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {
        doGet(request, response);
    }
}

```

For this example, it is assumed that **.properties** files named `DemoLabelsBundle` have already been created under **src/main/webapp/WEB-INF/classes** path for five locales with following content:

DemoLabelsBundle.properties

```

greetings = Hello.
farewell = Goodbye.
inquiry = How are you?

```

DemoLabelsBundle_de_DE.properties

```

greetings = Hallo.
farewell = Tschüß.
inquiry = Wiegeht's?

```

DemoLabelsBundle_en_US.properties

```

greetings = Hello.
farewell = Goodbye.
inquiry = How are you?

```

DemoLabelsBundle_fr_FR.properties

```

greetings = Bonjour.
farewell = Au revoir.
inquiry = Comment allez-vous?

```

DemoLabelsBundle _ja_JP.properties

```
greetings = Ohayōgozaimasu.
farewell = sayonara/sayounara.
inquiry = O genkidesuka?
```

Here in this particular example, the values were hard-coded

```
locale = Locale.of("ja", "JP");
```

However, in a practical scenario, it should detect the locale from the system and produce the output accordingly.

Following statement retrieves the specific resource bundle corresponding to the locale:

```
ResourceBundle bundle = ResourceBundle.getBundle(
"DemoLabelsBundle", locale);
```

Thus, if the locale were German, the contents of **DemoLabelsBundle_de_DE.properties** would be retrieved, and output would be displayed according to that.

The `web.xml` file should contain code as shown in Code Snippet 19.

Code Snippet 19:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
  <display-name>Resourcebundlesession13</display-name>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>TestInternationalization</servlet-name>
    <servlet-class>TestInternationalization</servlet-
class>
  </servlet>
  <servlet-mapping>
    <servlet-name>TestInternationalization</servlet-name>
    <url-pattern>/TestInternationalization</url-pattern>
  </servlet-mapping>
</web-app>
```

The file **index.jsp** should have code as shown in Code Snippet 20.

Code Snippet 20:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>ResourceBundle</title>
</head>
<body>
<a href="TestInternationalization">Click Here</a>
</body>
</html>
```

Figure 13.11 displays the output in Japanese language if the language code is specified as ja JP.

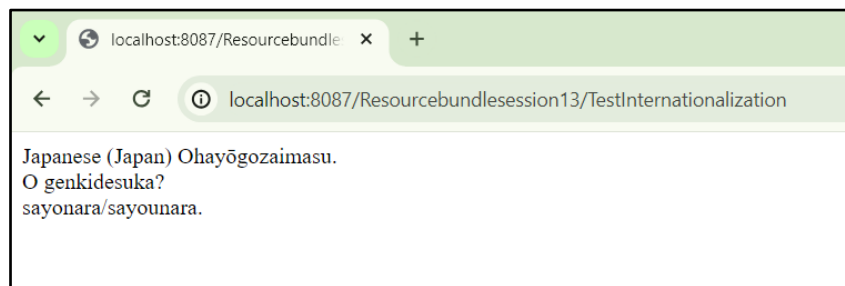


Figure 13.11: Output Demonstrating Internationalization

➤ **getKeys ()**

This method returns an enumeration of the keys. The keys present in the property file are returned using this method.

Syntax:

```
public abstract Enumeration getKeys()
```

Code Snippet 21 shows the use of the `getKeys ()` method.

Code Snippet 21:

```
// This snippet displays the value of the keys
static void iterateKeys(Locale currentLocale) {
    ResourceBundle labels =
        ResourceBundle.getBundle("LabelsBundle",
            currentLocale);
    Enumeration bundleKeys = labels.getKeys();
    while (bundleKeys.hasMoreElements()) {
        String key = (String)bundleKeys.nextElement();
        String value = labels.getString(key);
    }
}
```

```

        System.out.println("key = " + key + ", " +
            "value = " + value);
        . . .
    }
}

```

➤ **getLocale()**

This method returns the locale of this resource bundle. This method can be used after the calling of `getBundle()` to check whether the returned resource bundle really corresponds to the requested locale or not. It returns the locale for the current resource bundle.

Syntax:

```
public Locale getLocale()
```

Code Snippet 22 shows the use of the `getLocale()` method.

Code Snippet 22:

```

// This snippet gets the user's Locale
Locale locale = request.getLocale();
ResourceBundle bundle = ResourceBundle.getBundle (
    "WelcomeDemoBundle", locale);
String welcome = bundle.getString("DemoWelcome");

```

➤ **getObject()**

This method gets an object for the given key from the resource bundle. It tries to get the object from the resource bundle and if it fails, the parent resource bundle is called using parent's `getObject()` method. It throws a `MissingResourceException` if it fails.

Syntax:

```
public final Object getObject(String key)
```

where,

- `key` is the key for the desired object.

Code Snippet 23 shows an example.

Code Snippet 23:

```

// This snippet casts the result into myDemoIntegers
int[] myDemoIntegers = (int[])
myDemoResources.getObject("intList");

```

➤ **getString()**

This method returns a string for the given key from the resource bundle or one of its parents.

Syntax:

```
public final String getString(String key)
```

where,

- `key` is the key for the desired string.

Code Snippet 24 shows the use of `getString()` method.

Code Snippet 24:

```
/** Retrieves the translated value from the ResourceBundle
 * by invoking the getString method.
 */
String value = labels.getString(key);
```

13.9 Formatting in Servlets

Formatting of numbers, currency, date, and percentage helps the programmer to format these values based on the locale or the region of the user.

13.9.1 Formatting Dates

There are several formats in which dates can be displayed, such as:

Using Predefined Formats	Using Customized Formats
The <code>DateFormat</code> style is predefined and locale-specific and is easy to use. The styles are as follows: • SHORT - it is completely numeric, such as 11.14.50 or 3:30pm. • MEDIUM - it is longer, such as Jan 10, 1954. • LONG - it is longer, such as January 10, 1954, or 3:50:32pm. • FULL - it is completely specified, such as Tuesday, April 14, 1954, AD or 3:50:42pm PST. There are two steps in formatting of date using the <code>DateFormat</code> class. The first step is to create a formatter with the <code>getDateInstance()</code> method. The second step consists of invoking the <code>format()</code> method, which returns formatted date in the form of string.	Most of the time, the predefined formats are enough but sometime customized format is required. To do that <code>SimpleDateFormat</code> class is used. A <code>SimpleDateFormat</code> object is created with specified pattern strings. The format for the date is determined by the contents of the pattern string. The code in the snippet formats the date as per the pattern string. The formatted date is contained in string, which returned by the <code>format()</code> method.

Several classes can facilitate formatting of dates. They are as follows:

➤ **DateFormat**

This is an abstract class, which formats and parses the dates and time in a language independent manner.

Code Snippet 25 demonstrates how to format the date.

Code Snippet 25:

```
// This snippet formats the date
String date =
DateFormat.getDateTimeInstance(DateFormat.FULL,
    DateFormat.SHORT, locale).format(new Date());
```

➤ **DateFormat.Field**

This is a nested class that is used as attribute keys in the `AttributedCharacterIterator`. The `AttributedCharacterIterator` allows iteration through both text and related attribute information. It is returned from `DateFormat.formatToCharacterIterator` and is also used as field identifiers in `FieldPosition`.

➤ **DateFormatSymbol**

It is a public class, which encapsulates localizable date-time formatting data, such as names of months, days of week, and the time zone data. Both `DateFormat` class and `SimpleDateFormat` class use `DateFormatSymbols` to encapsulate localizable date-time formatting data information.

➤ **DateFormatter**

The `DateFormatter` class formats the date as per the format of the current locale.

➤ **SimpleDateFormat**

The `SimpleDateFormat` is a class that formats and parses dates in a locale-sensitive manner.

Code Snippet 26 demonstrates the use of the `SimpleDateFormat` class.

Code Snippet 26:

```
// This snippet formats the date as per the specified style
SimpleDateFormat fmt = new SimpleDateFormat("yyyy-MM-dd",
    Locale.US);
```

13.9.2 Formatting Currency

The formatting of the currency is necessary so that the users can be benefited from the availability of the locale specific formatting. Conversion of one currency to another requires some conversion in value. The formatting of currency value is much more involved compared to formatting date and time. Similarly, formatting of numbers as per the locale or particular region is also very necessary.

Some of the classes used for formatting currency are as follows:

➤ **Currency**

This class represents currency. The currencies are identified by ISO 4217 currency codes.

➤ **NumberFormat**

This is an abstract base class for all number formats. `NumberFormat` provides methods to determine the number formats of the locales and their name. It helps to format and parse numbers for any locales.

The class `Currency` represents currencies. These currencies are identified by their ISO 4217 currency codes.

Some of the methods of currency class are as follows:

➤ **`getCurrencyCode()`**

This method gets the currency code of the currency as per the ISO 4217 codes.

Syntax:

```
public String getCurrencyCode()
```

Code Snippet 27 shows the use of the `getCurrencyCode()` method.

Code Snippet 27:

```
// This snippet returns the currency code of the currency
// as per the ISO 4217 codes
public String getCurrencyCode() {
    return currency.getCurrencyCode();
}
```

Here, it is assumed that `currency` object is already created.

➤ **`getSymbol()`**

This method gets the currency symbol for the default locale.

Syntax:

```
public String getSymbol()
```

Code Snippet 28 shows the use of `getSymbol()` method.

Code Snippet 28:

```
// This snippet gets the currency symbol for default locale
//
public String getCurrencySymbol() {
    return currency.getSymbol();
}
```

➤ **getInstance(Locale locale)**

This method returns the `Currency` instance for the country of the specified locale.

Syntax:

```
public static Currency getInstance(Locale locale)
```

where,

- `locale` is the locale representing the country for which a `Currency` instance is required.

Code Snippet 29 shows the use of `getInstance()` method.

Code Snippet 29:

```
// This snippet gets the currency instance
import java.text.NumberFormat;
import java.util.Currency;
. . .
. . .
@Override
protected void doGet(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException
{
    response.setContentType("text/html;charset = UTF-8");
    request.setCharacterEncoding("utf8");
    try {
        PrintWriter out = response.getWriter();
        Locale locale = request.getLocale();
        Currency localeCurrency =
            Currency.getInstance(locale);
        out.println(localeCurrency.getSymbol());
    } catch (Exception e) {}
}
```

The output will display the currency symbol of the current locale in the browser.

13.9.3 Formatting Numbers

Some of the methods of `NumberFormat` class are as follows:

➤ **format(double number)**

This method is the specialization of the `format`.

Syntax:

```
public final String format(double number)
```

Code Snippet 30 demonstrates the use of `format()` method.

Code Snippet 30:

```
// This snippet formats the number
// getPercentInstance() returns a percentage format for the
//current default locale
NumberFormat nft = NumberFormat.getPercentInstance(locale);
String formatted = nft.format(0.51);
```

➤ **`getCurrency()`**

This method provides the number format while formatting currency values. The value derived initially is locale dependent. If no valid currency is determined and no currency has been set using `setCurrency`, the returned value may be null.

Syntax:

```
public Currency getCurrency()
```

Code Snippet 31 demonstrates the use of `getCurrency()` method.

Code Snippet 31:

```
// This snippet display the currency as per the locale
import java.text.NumberFormat;
import java.util.Currency;
. . .
    NumberFormat formatter = NumberFormat.getInstance ( )
;
    out.println ( formatter.getCurrency ( ) ) ;
}
}
```

➤ **`getInstance()`**

This method returns the default number format for the current default locale.

Syntax:

```
public static final NumberFormat getInstance()
```

Code Snippet 32 demonstrates the use of `getInstance()` method.

Code Snippet 32:

```
// This snippet display the currency as per the locale
import java.text.NumberFormat;
import java.util.Currency;
```

```

. . .
    NumberFormat formatter =
NumberFormat.getInstance();
    System.out.println ( formatter.getCurrency());
    }
}

```

➤ **parse ()**

This method parses text from the beginning of the specified string to produce a number.

Syntax:

```
public Number parse(String str) throws ParseException
```

where,

- **str** is a string whose beginning is parsed.

For example,

```

NumberFormat nf = NumberFormat.getInstance ( );
Number myDemoNumber = nf.parse(myDemoString);

```

➤ **setCurrency ()**

This method sets the currency used by the number format when formatting currency values. This does not update the minimum or maximum number of fraction digits used by the number format.

Syntax:

```
public void setCurrency(Currency currency)
```

where,

- **currency** is the new currency to be used by this number format.

For example,

```

// sets the currency to new amountCurrency
NumberFormat formatter = NumberFormat.getInstance();
formatter.setCurrency(amountCurrency);

```

13.9.4 Formatting Percentages

Data such as dates, currencies, and numbers are locale dependent. They must conform to the end user language and region. Therefore, formatting is necessary.

The format for displaying percentages can be changed using `getPercentInstance ()` method of `NumberFormat` class. For example, with this format a fraction as `0.82` can be displayed as `82%`.

The two methods for formatting percentage as follows:

getPercentInstance()

This method returns a percentage format for the current default locale.

Syntax:

```
public static final NumberFormat getPercentInstance()
```

getPercentInstance(Locale inLocale)

This method returns a percentage format for the specified locale.

Syntax:

```
public static NumberFormat getPercentInstance(Locale inLocale)
```

where,

inLocale is the specified locale according to which percentage is to be formatted.

Figure 13.12 depicts the code for formatting of percentages.

```
static public void displayPercent(Locale currentLocale) {  
  
    Double percent = new Double(0.75);  
    NumberFormat percentFormatter;  
    String percentOut;  
  
    percentFormatter = NumberFormat.getPercentInstance(currentLocale);  
    percentOut = percentFormatter.format(percent);  
    System.out.println(percentOut + " " + currentLocale.toString());  
}  
}
```

Output:

```
75% en_US
```

Figure 13.12: Formatting Percentages

13.9.5 Formatting Messages

Message formatting is required so that the end user is benefited. The formatting provides message in language neutral way. Formatting also helps provide messages in user's language.

In order to format a message the `MessageFormat` object is used. The array of objects using the format specifiers embedded in the pattern is formatted by `MessageFormat.format()` method. It returns the result as a `StringBuffer`. The classes for formatting the message are as follows:

MessageFormat	MessageFormat.Field
This class provides a means to generate related messages in a language-neutral way. <code>MessageFormat</code> takes a set of objects, formats them, then inserts the formatted strings into the pattern at the appropriate places. It can also be used to construct messages displayed for end users.	This class defines constants that are used as attribute keys in the <code>AttributedCharacterIterator</code> object returned from <code>MessageFormat.formatToCharacterIterator()</code>.

Figure 13.13 depicts an example of message formatting through a console application. Similar logic can be applied in a Web application.



```

C:\WINNT\system32\cmd.exe

C:\Program Files\Java>java ConvertLocale
currentLocale = en_US

This is Aptech Demo

currentLocale = fr_FR
C'est d mo d'Aptech

C:\Program Files\Java>
  
```

Figure 13.13: Formatting a Message

Summary

- The JSTL tag library helps Web designers to integrate the Java technology code into JSP, by writing a simple code.
- The tag extension API in the JSP adds the tag functionalities to the language. It includes information on the tag library and its tags.
- `taglib` directive on a JSP page informs the Servlet that the page uses a tag library.
- The Tag extension API in the JSP adds the tag functionalities to the language.
- The Tag interface defines the lifecycle and the methods to be invoked at the start and end tag.
- The URI in the `taglib` directive points to a context relative path; this path hosts components that starts with / and is called the TLD resource path.
- JSTL provides decision-making tags to support conditions in a JSP page.
- The `BodyTag` interface extends `IterationTag` and adds new functionality that lets the tag handler evaluate its body content in a temporary buffer.
- JSTL is based on XML, which is very similar to HTML.
- The use of tag libraries, such as JSTL, helps people involved in the application development to focus on their area of expertise.
- Internationalization can be defined as the method of designing an application that can be adapted to a region or a language without much change in the technology
- Localization is a process of making a product or service, language, culture, and local 'look-and-feel' specific.
- Internationalization of server side code reduces the task of writing multiple versions of a class for different locales.
- Resource bundles are used to achieve the locale specific output.
- Internationalization requires formatting of dates, numbers, currencies, and messages with the help of resource bundle.

Check Your Progress

1. Match the different description against their corresponding tag libraries.

	Description		Library
(A)	It contains the tags for looping, expression evaluation and basic input and output.	(1)	Formatting/Internationalization (I18N) Tag Library
(B)	It contains the tags, which are used to parse the data, such as dates and time, based on current location.	(2)	XML Tag Library
(C)	It contains tags, which are used to access databases.	(3)	Core Tag Library
(D)	It contains tags, which are used to access XML elements.	(4)	SQL Tag Library

(A)	A-2, B-3, C-4, D-1	(C)	A-3, B-1, C-2, D-4
(B)	A-3, B-1, C-4, D-2	(D)	A-4, B-1, C-2, D-3

2. Match the syntax against the corresponding tags.

	Syntax		Tag
(A)	<code><x var = "varName" value = "expression" scope = "page/request/session/Application"/></code>	(1)	<code><c:remove></code>
(B)	<code><x var = "varName" scope = "page/request/session/application"/></code>	(2)	<code><c:catch></code>
(C)	<code><x value = "value" escapeXml = "boolean" default = "defaultValue"/></code>	(3)	<code><c:set></code>
(D)	<code><x [var="varName"]></code> nested actions	(4)	<code><c:out></code>

(A)	A-2, B-3, C-4, D-1	(C)	A-3, B-1, C-2, D-4
(B)	A-3, B-1, C-4, D-2	(D)	A-4, B-1, C-2, D-3

3. Identify the incorrect statement relating to `<sql:setDataSource>` tag.

(A)	<code><sql:setDataSource></code> is an empty tag and allows the user to set data source information for the database.
(B)	<code>dataSource</code> tag is a path to JNDI resource not a JDBC parameter string.
(C)	In <code><sql:setDataSource></code> tag, if the <code>dataSource</code> attribute is used, then <code>url</code> attribute cannot be used.
(D)	In <code><sql:setDataSource></code> tag <code>scope</code> specifies the scope of the variable.

4. Which among the following statements relating to the internationalization and localization are true?

(A)	Internationalization can be defined as the method of designing an application that can be adapted to only English speaking region without much change in the technology.
(B)	Internationalization helps in creating Web applications that overlooks formatting of numeric and date-time output.
(C)	Localization is a process of making a product or service language, culture, and local 'look-and-feel' specific.
(D)	A Locale is a simple object, which identifies a specific language and a geographic region.
(E)	Locales are used in entire Java class libraries data for formatting and customization.

A.	A, B, C	C.	A, C, E
B.	C, D, E	D.	A, B, D

5. Which among following options returns the enumeration of keys present in the property file?

(A)	<pre>static void iterateKeys(Locale currentLocale) { ResourceBundle labels = ResourceBundle.getBundle("LabelsBundle",currentLocal e); while (bundleKeys.hasMoreElements()) { String key = (String)bundleKeys.nextElement(); String value = labels.getString(key); System.out.println("key = " + key + ", "+"value = " + value); } }</pre>
(B)	<pre>static void iterateKeys(Locale currentLocale) { ResourceBundle labels = ResourceBundle.getBundle("LabelsBundle",currentLocal e); Enumeration bundleKeys = labels.getKeys(); while (bundleKeys.hasMoreElements()) { String key = (String)bundleKeys.nextElement(); String value = labels.getString(key); System.out.println("key = " + key + ", "+"value = " + value); } }</pre>
(C)	<pre>static void iterateKeys(Locale currentLocale) { ResourceBundle labels = ResourceBundle.getBundle("LabelsBundle",currentLocal e); Enumeration bundleKeys = labels.getKeys(); while (bundleKeys.hasMoreElements()) { String key = (String)bundleKeys.nextElement(); System.out.println("key = " + key + ", " + "value = " + value); } }</pre>
(D)	<pre>static void iterateKeys(Locale currentLocale) { Enumeration bundleKeys = labels.getKeys(); while (bundleKeys.hasMoreElements()) { String key = (String)bundleKeys.nextElement(); String value = labels.getString(key); System.out.println("key = " + key + ", "+"value = " + value); } }</pre>

Answers

1.	B
2.	B
3.	B
4.	B
5.	B

Try It Yourself

Implement a Web application named **PatientProfile** that allows users to perform Create, Read, Update, and Delete (CRUD) operations on patient records using SQL Core tags.

1. Database Configuration

- Establish a MySQL database and create a table named patients (or another name of your choosing).
- Design the table to include columns for id (as the primary key), patient name, email, phone, and diagnosis.

2. Web Application Structure

Set up the directory structure for the **PatientProfile** Web application in Eclipse. Obtain the following JAR files:

- jakarta.servlet.jsp.jstl-2.0.0.jar
- jakarta.servlet.jsp.jstl-api-2.0.0.jar
- mysql-connector-java-8.0.21.jar (or an equivalent MySQL driver)

Store these JAR files in the **WEB-INF/lib** directory of the project.

3. CRUD using SQL Core Tags

- Design a simple Web interface for **PatientProfile** with forms for each CRUD operation.
- Demonstrate the use of SQL Core tags for CRUD operations on patient records.
- Test the functionality of the application.

Refer to Table 13.2 for sample data to populate the patient records.

Patient ID	Patient Name	Email	Phone	Patient Diagnosis
101	John Doe	john@gmail.com	+1(212) 555-1234	Dehydration
02	Jane Smith	smith@gmail.com	+1(212)323- 9878	High Blood Presser
103	Alice Johnson	alice@gmail.com	+1(212)555- 2345	Arthritis

Table 13.2: Sample Data

Securing Java Web Applications

Welcome to the Session, **Securing Java Web Applications**.

This session explains the concept of security in Java Web applications. The session explains the three pillars of security mechanisms. Further, the session describes different authentication and security methods used in securing Java Web components. The session also details the steps for implementing declarative and programmatic security, `HttpServletRequest` methods for identifying users, and the use of SSL Certification.

In this session, you will learn to:

- Explain the necessity for and features of securing Web applications
- Describe Jakarta EE Security API
- Describe the `HTTP` basic, digest, client, and form-based authentication method of ensuring security
- Explain how to configure users in Tomcat
- Explain how to specify authentication mechanisms using annotations
- Describe the steps to implement declarative security
- Explain the steps to implement programmatic security
- Describe the `HttpServletRequest` methods for identifying users
- Explain the use of SSL certification

14.1 Introduction

A Web application is an application accessed using a Web browser over a network, such as Intranet or the Internet. The reason for popularity of the Web applications is the ability to maintain the application and easy accessibility to information.

Easier accessibility to Web applications opens door to attackers to access or modify confidential information. Hence, it is necessary to secure the Web application to keep the information safe using some of the security mechanisms.

Figure 14.1 depicts unauthorized access to the Internet.

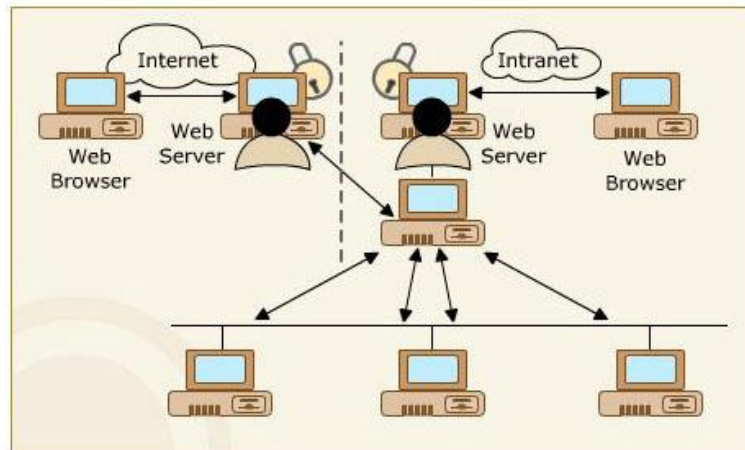


Figure 14.1: Unauthorized Access to the Internet

14.1.2 Pillars of Security/Security Mechanisms

The security mechanisms are based on authentication and authorization techniques. The authorization mechanism specifies who is allowed to access the resources in the application. Authentication is used to verify the users' identity. It authenticates the user by some reusable methods, which do not require any modification.

Some of the security mechanisms used in Web applications are as follows:

- **HTTP basic authentication method**
In HTTP basic authentication, the Web browser pops-up a login page, which prompts the user for credentials in the form of a username and password. A user will be allowed to access information only if the credentials match a stored pair of username and password.
- **HTTP digest authentication method**
In HTTP digest authentication, the server has the password based on a hash function. By using the hash function, the database stored the hash value rather than the password in plain text, so that it is difficult to decipher the data.
- **Form-based authentication method**
In form-based authentication, the browser provides a login form, which appears in response to a request. The user can enter a username and password in the form. If the entered credentials match a stored pair of username and password, the user will be allowed to access information. The user will be directed to an error page if the login fails.
- **HTTPS client authentication method**
In HTTPS client authentication, the browser uses HTTPS protocol, which is identical to the HTTP, but the URL indicates to use of a default TCP port and an

extra authentication layer in between HTTP and TCP. This extra security layer conforms the client's authentication.

14.2 Jakarta EE Security API

With Jakarta EE Security, servlet annotations can be used to establish basic authentication, form authentication, and custom form-based authentication. It also includes the `SecurityContext` API that inspects the application code for programmatic security flaws. Security context APIs can also be used to launch a Web-based authentication process. Security context also assures that all servlet and EJB containers share the same security context.

Therefore, if you are developing a Web application that relies on the security features provided by Jakarta EE, you would typically only require to include the Jakarta EE Web Profile API Maven artifact, as shown in Code Snippet 1, in your project.

Code Snippet 1 shows the Jakarta EE 10 Web Profile API Maven artifact.

Code Snippet 1:

```
<parent>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakartaee-api-parent</artifactId>
    <version>10.0.0</version>
</parent>
<artifactId>jakarta.jakartaee-web-api</artifactId>
<name>Jakarta EE Web Profile API</name>
<description>Jakarta EE Web Profile API</description>
```

Code Snippet 2 shows the explicit implementation in the Security API for Java EE Applications.

Code Snippet 2:

```
<dependencies>
    <dependency>
</dependency>
    <groupId>jakarta.security.enterprise</groupId>
    <artifactId>jakarta.security.enterprise-api</artifactId>
    <version>3.0.0</version>
</dependency>
</dependency>
</dependencies>
```

Prior to Jakarta EE, the developers specified the security mechanism in the `web.xml` file. However, Jakarta EE has a new `HttpAuthenticationMechanism` interface that allows Web applications to specify authentication methods.

As described in the Servlet specification, the container has three authentication mechanisms. These are as follows:

- Form-based authentication
- Basic HTTP authentication
- Custom form-based authentication

In addition, it offers following annotations to activate each implementation:

- `@BasicAuthenticationMechanismDefinition`
- `@FormAuthenticationMechanismDefinition`
- `@CustomFormAuthenticationMechanismDefinition`

14.3 Authentication Mechanisms

In developing Web applications, the authentication process confirms the user's identity to access the necessary Web resources deployed on the Web server.

Some of the commonly used authentication mechanisms are as follows:

- HTTP basic authentication
- HTTP digest authentication
- HTTPS client authentication
- Form-based authentication

14.3.1 HTTP Basic Authentication

In HTTP basic authentication, the Web browser pops up a login page in response to a request. The login page prompts the user for credentials, such as username and password. This authentication method works on the assumption that the client-server communication is reliable.

This mechanism does not provide any protection for the information communicated between the client and the server. The credentials are transmitted in plain text, so any attacker can easily retrieve the data. To prevent this, the data is encoded before the transmission. The advantage of using basic authentication is that it is browser-friendly.

Figure 14.2 depicts basic authentication.



Figure 14.2: Basic Authentication

14.3.2 HTTP Digest Authentication

The HTTP digest authentication builds over the basic authentication, which authenticates the user identity without sending the password as plain text to the server. The password is first encrypted and then sent.

This type of authentication is based on a hash function. By using the hash function, the database stores a hash value rather than the password. This process is a one-way system because it is difficult to know the original password according to the output. So the attacker cannot breach this authentication mechanism.

Figure 14.3 depicts digest authentication.

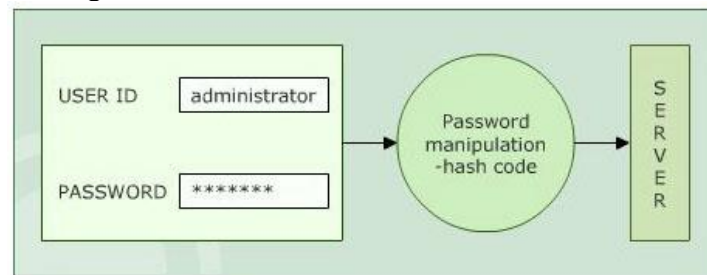


Figure 14.3: Digest Authentication

14.3.3 HTTPS Client Authentication

The HTTPS client authentication is a secured client authentication technique, which is based on Public Key Certificates. This authentication is similar to http but uses the https, which is HTTP over Secure Socket Layer (SSL). The URL instructs the browser to use a default TCP port with an extra authentication layer in between HTTP and TCP.

Figure 14.4 depicts HTTPS client authentication.

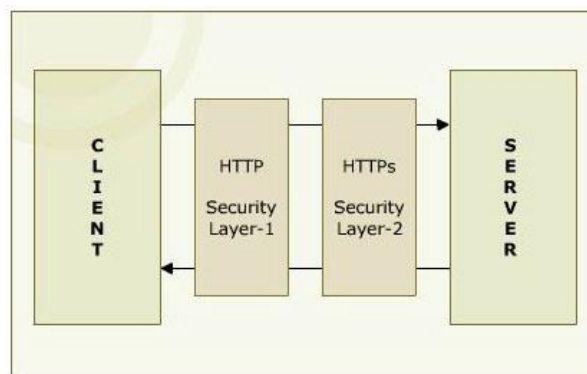


Figure 14.4: HTTPS Client Authentication

14.3.4 Form-Based Authentication

The form-based authentication provides a login page, which appears in response to a request from the user. An error page is also available if the login fails from the user side. After getting the username and password validated by the server, access

is provided to the user. Here, you can define how to protect the Web content for the URLs you are planning to protect.

Figure 14.5 depicts form-based authentication.

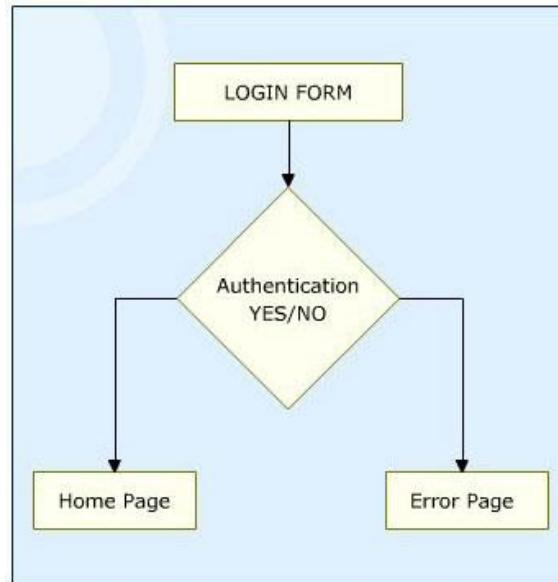


Figure 14.5: Form-based Authentication

14.4 Authentication in Jakarta EE

In modern Jakarta EE applications, annotations can be used to configure security settings directly in the code, replacing the requirement for the `web.xml` file. This approach is more flexible and keeps the configuration closer to the code it affects.

However, security configuration can also be configured using `web.xml`. This offers flexibility and backward compatibility for applications that may have been developed with earlier versions of Java EE or for developers who prefer declarative security configuration.

14.4.1 Configuring Users in Tomcat

Tomcat is flexible to configure authentication as per your requirements. By configuring Tomcat, you can create usernames and passwords for user authentication.

Following steps are used to access a new user in Tomcat:

- Ensure Tomcat is up and running.
- Configure users and roles in the **tomcat-users.xml** file as shown in Code Snippet 3.

Code Snippet 3:

```
<?xml version='1.0' encoding='utf-8'?>
<tomcat-users>
// The purpose of role is to give access to a designated
user. //If the rolename is manager, then the access is given
to that //area, which is specified for manager.
  <role rolename="manager"/>
  <role rolename="admin"/>
  <user username="admin" password="app_tech"
roles="admin,manager"/>
</tomcat-users>
```

- **Open Web browser.** In the address bar, type `http://localhost:8080` to access Apache Tomcat home page.
- **Click the Tomcat Manager app,** which prompts a login screen.
- **Enter the username and password** which you have created. This will redirect you to Tomcat Web Application Manager Page.

Code Snippet 4 shows the addition of a new user in the Tomcat's configuration file.

Code Snippet 4:

```
// To include a new user
<?xml version='1.0' encoding='utf-8'?>
<tomcat-users>
  <role rolename="manager"/>
  <role rolename="admin"/>
  <user username="Thomas" password="tempo"
roles="admin,manager"/>
  <user username="admin" password="aptech"
roles="admin,manager"/>
</tomcat-users>
```

Figure 14.6 depicts the home page of Tomcat Web Server.

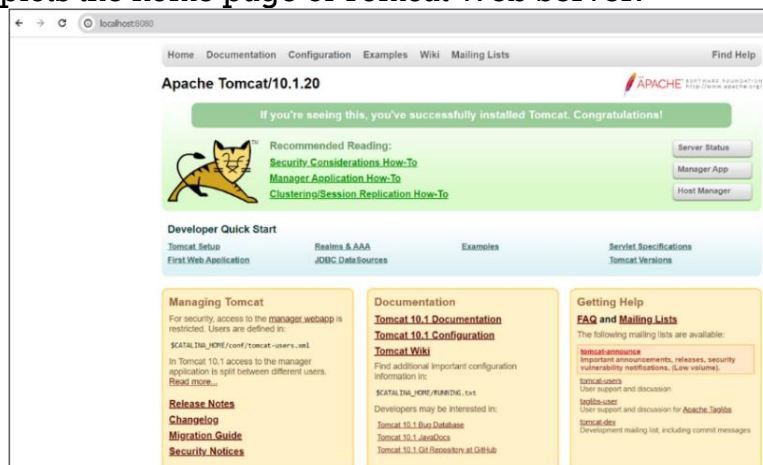


Figure 14.6: Home Page of Tomcat Web Server

14.4.2 Specifying Authentication Mechanisms using web.xml

The traditional `web.xml` file can still be used for security configuration in Jakarta EE applications. To know the use of basic authentication in an application, use the `login-config` element and its sub-elements in the `web.xml` file. Note that the browser prompts the user for a username and password when using basic authentication. These credentials are then, sent to the server with each request. Before granting access, the server verifies the credentials against its store (such as using Tomcat). With form-based authentication, the server sends a login page to the client when a protected resource is requested. The user enters their credentials into the form, which is then submitted to the server.

The sub-elements of `login-config` element are as follows:

- **<auth-method>**
This sub-element names the authentication method used in the element.
- **<realm-name>**
This sub-element names the Web resource where the `<login-config>` maps.
- **<form-login-config>**
This sub-element is optional and is specified only when using form-based authentication that is the `<auth-method>` value is set to FORM.
- **<form-login-page>**
This sub-element specifies the form to display when a request is made to a protected Web resource in the Web application. The form-login page is usually an HTML or JSP file, but it can also be a servlet.

Code Snippet 5 uses the form-based authentication mechanism.

Code Snippet 5:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app      xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"      xmlns="https://jakarta.ee/xml/ns/jakartaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
    <display-name>Session14SecurityMechanism</display-name>
    <!-- Configure Basic Authentication -->
    <web-app>
        <login-config>
            <auth-method>BASIC</auth-method>
            <realm-name>Managers</realm-name>
        </login-config>
    <!-- Define Security Constraint -->
    <security-constraint>
        <web-resource-collection>
            <web-resource-name>Protected Area</web-resource-name>
            <url-pattern>/protected/*</url-pattern>
        </web-resource-collection>
```

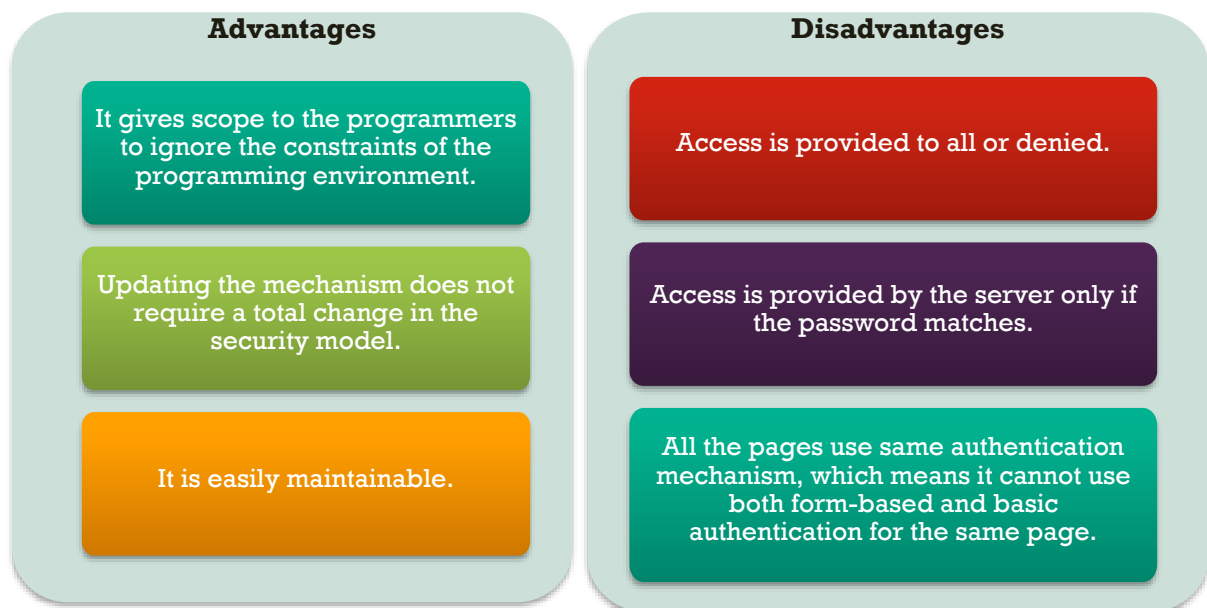
```

    <auth-constraint>
        <role-name>admin</role-name>
    </auth-constraint>
</security-constraint>
<!-- Define Security Roles -->
<security-role>
    <role-name>admin</role-name>
</security-role>
<!-- Servlet Mapping -->
<servlet>
    <servlet-name>ProtectedServlet</servlet-name>
    <servlet-class>ProtectedServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ProtectedServlet</servlet-name>
    <url-pattern>/protected/*</url-pattern>
</servlet-mapping>
</web-app>

```

14.5 Declarative Security

The declarative security provides security to a resource with the help of the server configuration. Declarative security works as a different layer from the Web component with which it works.



14.5.1 Implementing Declarative Security

To implement the declarative security, perform following eight steps:

1. **Set up usernames, passwords, and roles**

When a user tries to access a secured resource in a Web application, which uses form-based authentication, the system uses a form to ask for a username and a password. After verifying the password, the system gives access to the user. Then, it determines the role, such as user, administrator, or executive, which is defined by the user.

2. Tell the Server that you are using Form-Based authentication and designate locations of login and login failure Pages

After validating the user and the role of the user, it uses a form-based authentication, which supplies a value for the `auth-method` sub-element and uses the `form-login-config` sub-element to give the locations of the login and login-failure pages. These pages can consist of either JSP or HTML.

3. Create a Login Page

The `login-config` element informs the server to use form-based authentication for creating a login page, and also redirects unauthenticated users to a designated error page. The login page requires a form for security check, a text field named `username`, and a password field named `password`.

4. Create a Page to Report Failed Login Attempts

The login-failure page contains a link to an unrestricted section of the Web application. The link shows 'login failed' or 'username and password not found' message.

5. Specifying URLs That Should Be Password Protected

Specifying the URLs and describing the protection they should have the purposes of the `security-constraint` element. The `security-constraint` element should come before the `login-config` element in `web.xml` and contains four sub-elements, `display-name`, `web-resource-collection`, `auth-constraint`, and `user-data-constraint`.

6. Specifying URLs That Should Be Available Only with SSL

If your server supports SSL, you can define that certain resources are available only through encrypted HTTPS (SSL) connections. Use of SSL does not obstruct the basic way that form-based authentication works. None of your servlets or JSP pages to be modified or moved to different locations when you enable or disable SSL. This is an extra advantage of declarative security.

7. Turning Off the Invoker Servlet

The most portable approach of turning off the invoker servlet is to remap the servlet pattern in the Web application so that all requests that include the pattern are sent to the same servlet. To remap the pattern, first create a simple servlet that prints an error message. Then, use the `servlet` and `servlet-mapping` elements to send requests that include the servlet pattern to that servlet.

8. Write Authentication Filter

Write the Servlet filter and mention in `web.xml`. This `Filter` class filters the HTTP requests based on its type.

14.5.2 Annotations for Declarative Security

In Jakarta EE 10, several annotations are provided to simplify the configuration of declarative security. These annotations allow developers to manage access control directly within their code, ensuring that security policies are clear and easy to maintain. Key annotations used for declarative security in Jakarta EE 10 are shown in Table 14.1.

Annotation	Description	Purpose	Syntax
@RolesAllowed	Used to restrict access to methods or classes to users with specified roles.	Specifies that only users with certain roles are allowed to access the annotated component.	<code>public @interface RolesAllowed</code>
@PermitAll	Permits all security roles to invoke specified method(s), indicating that no specific security checks are required. It can be applied to an entire class or individual methods.	Allows access to everyone, regardless of whether they are authenticated or not.	<code>public @interface PermitAll</code>
@DenyAll	Specifies that no security roles are allowed to invoke the specified method(s), meaning the methods are excluded from execution in the Jakarta EE container.	Denies access to everyone, including authenticated users.	<code>public @interface DenyAll</code>
@RunAs	Defines the identity under which the application executes in a Jakarta EE container. This allows developers to specify a particular security role for the	Defines the security identity (role) under which the component should run. This is useful for running certain operations with a specific role that might have different	<code>Public @interface RunAs</code>

Annotation	Description	Purpose	Syntax
	execution of the application.	permissions than the calling user.	

Table 14.1: Security Annotations Overview

14.6 Programmatic Security

When declarative security alone is not sufficient to express the security model of the application, then the programmatic security model can be used by security-aware applications.

14.6.1 Concept of Programmatic Security

Programmatic security authenticates the users and grants access to the users. The servlet or JSP page either authenticates the user or verifies that the user has authenticated earlier, to check the unauthorized access. To ensure the safety of network data, the servlet or JSP page redirects the HTTP connection to HTTPS of the URLs.

The advantages of programmatic security are as follows:

- Ensures total portability
- Allows password matching strategies

The disadvantages of programmatic security are as follows:

- Much harder to code and maintain
- Each resource must use the code

14.6.2 HttpServletRequest Methods for Identifying User

Some of the methods in the `HttpServletRequest` interface are used for identifying a user with the help of cookies. Cookies are texts that are sent by the server to a browser, and then, sent back unchanged by the browser each time it accesses the server.

There are various methods available in `HttpServletRequest` interface as follows:

➤ `getAuthType()`

This method returns the authentication scheme name.

Syntax:

```
public java.lang.String getAuthType()
```

➤ **getCookies()**

It returns an array, which has all the information of the `Cookie` objects sent by the client.

Syntax:

```
public Cookie[] getCookies()
```

➤ **getHeader()**

It returns the value of the header as a `String`, which is requested.

Syntax:

```
public java.lang.String getHeader(java.lang.String name)
```

➤ **getRemoteUser()**

If the user is authenticated, it returns the login name of the user, or else it returns `null`.

Syntax:

```
public java.lang.String getRemoteUser()
```

➤ **getRequestedSessionId()**

This method returns the session ID that is defined by the client.

Syntax:

```
public java.lang.String getRequestedSessionId()
```

➤ **getSession()**

This method returns the current session or creates one if there is no session.

Syntax:

```
public HttpSession get Session()
```

➤ **isUserInRole()**

It returns a `boolean` value, which indicates whether the authenticated user is included in the logical 'role'.

Syntax:

```
public boolean isUserInRole(java.lang.String role)
```

➤ **getUserPrincipal()**

This method returns a `java.security.Principal` object.

Syntax:

```
public java.security.Principal getUserPrincipal()
```

➤ **isRequestedSessionIdValid()**

It checks the validity of the requested session ID.

Syntax:

```
public boolean isRequestedSessionIdValid()
```

14.6.3 Implementing Programmatic Security

Six steps to implement programmatic security are as follows:

1. Check whether there is an authorization request header.

It checks whether the header exists or not.

2. Get the string, which contains the encoded username/password.

If the authentication header exists it returns the string, which contains the username and password in base64 encoding format.

3. Reverse the base64 encoding of the username/password string.

Reverse the base64 encoding of username or password by using the decodeBuffer method of BASE64Decoder class.

4. Check the username and password.

The common approach of checking the username and password is to use the database to get the original username and password. In other way, put the information of the password in the servlet. You will get access if the incoming username and password matches the reference username and password pairs.

5. If authentication fails, send the proper response to the client.

It returns a 401 response code, which is an unauthorized one in the form of WWW-Authenticate: BASIC realm="some-name". This response pop-up a dialog box requesting the user to enter a name and password for some-name.

6. Servlet Annotations.

Based on the Jakarta EE 10 API, the `jakarta.servlet.annotation` package provides a number of classes and interfaces. Using these classes and interfaces, developers can declare filters, servlets, and listeners as part of the mechanism to authenticate HTTP requests.

14.7 ServerAuthModule Interface

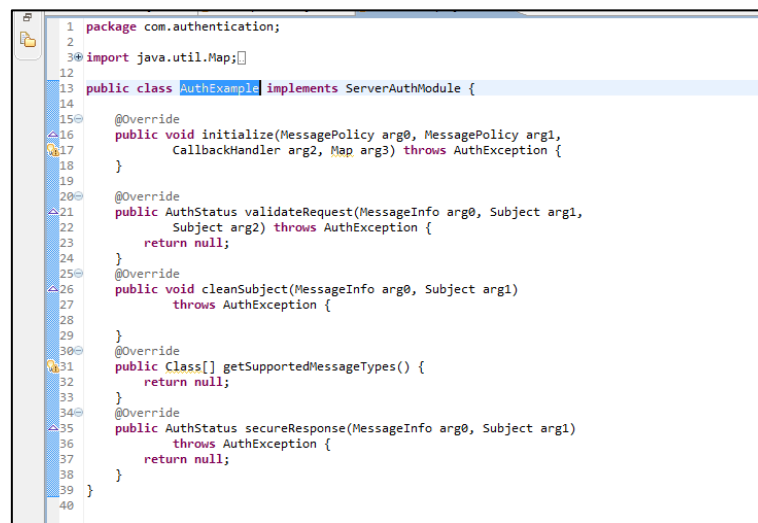
Different clients send different requests to server. For application security, it is necessary to validate the client requests. Different type of security methods are used for validating the client requests. `ServerAuthModule` interface helps to validate the client requests.

14.7.1 ServerAuthModule Interface Methods

`ServerAuthModule` interface have five methods that are as follows:

1. **Initialize ()**: This method is used to initialize the authentication module and declare all the necessary objects for authentication. This method has four arguments namely, `messagepolicy1`, `messagepolicy2`, `callbackhandler`, and `Map` objects.
2. **getSupportedMessageTypes ()**: This method returns the array of objects, and it contains the messages.
3. **ValidateRequest ()**: This method is call by servlet container and validates the client sent http Servlet requests. This method has argument `messageinfo`, `subject 1` and `subject2` objects.
4. **SecureResponse ()**: This method is also called by Servlet container. This method returns the response code.
5. **CleanSubject ()**: This method is used to clean the subject of requests.

Figure 14.7 demonstrates the code for authentication module.



```

1 package com.authentication;
2
3 import java.util.Map;
4
5
6
7
8
9
10
11
12
13 public class AuthExample implements ServerAuthModule {
14
15     @Override
16     public void initialize(MessagePolicy arg0, MessagePolicy arg1,
17         CallbackHandler arg2, Map arg3) throws AuthException {
18     }
19
20     @Override
21     public AuthStatus validateRequest(MessageInfo arg0, Subject arg1,
22         Subject arg2) throws AuthException {
23         return null;
24     }
25
26     @Override
27     public void cleanSubject(MessageInfo arg0, Subject arg1)
28         throws AuthException {
29     }
30
31     @Override
32     public Class[] getSupportedMessageTypes() {
33         return null;
34     }
35
36     @Override
37     public AuthStatus secureResponse(MessageInfo arg0, Subject arg1)
38         throws AuthException {
39         return null;
40     }
41 }

```

Figure 14.7: Authentication Module

14.8 SSL Certification

SSL stands for Secure Sockets Layer, which is a security protocol that establishes a secure communication channel over the Internet. An SSL certification is a digital certificate that authenticates the identity of a Website and enables encrypted connections between Web servers and Web browsers. SSL certification is essential for securing online transactions, protecting user privacy, and maintaining trust in the integrity of the Internet. It plays a crucial role in creating a safer and more

secure online environment for businesses and users. Here are the important benefits of SSL certification:

Authentication: SSL certificates verify the identity of a Website, ensuring that users are connecting to the legitimate site they intend to visit. This authentication helps prevent phishing attacks and assures users of the Website's authenticity.

Encryption: SSL certificates enable encryption of data transmitted between users and Websites, protecting sensitive information from being intercepted and read by unauthorized parties. This encryption ensures confidentiality of data in transit.

Data Integrity: SSL ensures the integrity of data during transmission, preventing unauthorized tampering or modification of information. This ensures that the data received by users is exactly as it was sent by the Website.

Trust Indicators: Websites secured with SSL certificates display trust indicators such as HTTPS in the URL and a padlock icon in the address bar. These visual cues reassure users that their interactions with the Website are secure and trustworthy.

Summary

- A Web application is an application, which is accessed with the help of a Web browser.
- The reason for popularity of Web applications is the ability to maintain them without changing the client computers. When you access the Web, hackers can get your information. So, to keep your information secret, it is necessary to secure Web applications.
- There are four authentication Mechanisms available. These are HTTP Basic Authentication, HTTP Digest Authentication, HTTPS Client Authentication, and Form-Based Authentication.
- In modern Jakarta EE applications, using annotations to configure security settings has become a common practice.
- Traditional `web.xml` file can still be used for security configuration in Jakarta EE applications, offering flexibility and backward compatibility for applications.
- The declarative security provides security to a resource with the help of server configuration.
- Programmatic security authenticates the users and grants access to the users.
- An SSL certificate authenticates a Website and encrypts data between servers and browsers, ensuring security, data integrity, and user trust through HTTPS and visual indicators such as padlock icon.

Check Your Progress

1. Identify the correct options for different security mechanisms used in the Web application.

(A)	In the Web application, it is difficult to maintain the data without changing in the client systems through the server.
(B)	In HTTP digest authentication, the server has the password based on a hash function.
(C)	In a basic security mechanism, the Web browser never pops-up a login page.
(D)	In form-based authentication, an error page is shown if the login fails from the users' side.
(E)	In HTTPS client authentication, the browser uses a default TCP port and an extra authentication layer in between HTTP and TCP.

A.	A, B, E	C.	B, D, E
B.	A, B, D	D.	A, C, D

2. Identify the correct option from the following statements for basic authentication.

(A)	The user credentials are transmitted in plain text, so it is secure from any attacker.
(B)	This mechanism provides protection for the information communicated between client and server.
(C)	The login page prompts the user for credentials, such as username and password.
(D)	In basic authentication, the login page directly prompts the user to the home page.

3. Identify the correct options from the following statements for HTTP digest authentication.

(A)	Digest authentication authenticates the user identity without sending the password as plain text to the server.
(B)	The hash value is vulnerable for the attacker to breach.
(C)	In digest authentication, the password is first encrypted and then, sent.
(D)	It is quite easy to know the original password according to the output.
(E)	Digest authentication is based on a hash function.

4. Identify the correct option from the following statements for client authentication.

(A)	This authentication is similar to TCP and Secure Socket Layer (SSL).
(B)	The HTTPS client authentication is a secured client authentication technique.
(C)	HTTPS client authentication based on Private Key Certificate.
(D)	The URL instructs the browser not to use a default HTTP port with an extra authentication layer.

5. Identify the wrong option for configuring and specifying users in Tomcat.

(A)	To configure user in Tomcat, first select the Tomcat 10.0 from Apache Software Foundation in your C: drive.
(B)	In URL, put typing <code>http://localhost</code> you will get Apache Tomcat home page.
(C)	To know the use of basic authentication in an application, use the login-config element.
(D)	When the browser loads a resource, which is secured by web.xml file, the browser responds in different ways.

6. Identify the correct options for declarative security from the following statements.

(A)	The login-config element tells the server to use form-based authentication and to redirect unauthenticated users to a designated error page.
(B)	Use of SSL hampers the basic way that form-based authentication works.
(C)	The login page requires a form for security check, a text field named username and a password field named password.
(D)	Specifying URLs and describing the protection they should have, is not the purpose of the security-constraint element.
(E)	In declarative security, any updating does not require the total change in the security model.

7. Which annotation is used to specify the list of roles permitted to access method(s) in a Jakarta EE application?

(A)	@PermitAll
(B)	@DenyAll
(C)	@RolesAllowed
(D)	@RunAs

Answers

1.	C
2.	C
3.	B
4.	B
5.	B
6.	B
7.	C

Try It Yourself

Create a Jakarta EE 10 Web Application using Tomcat. Set up a new dynamic Web project in Eclipse. Develop a servlet that will handle basic authentication. This servlet will act as a gatekeeper, ensuring only authenticated users can access certain resources. Configure the URL mappings in the `web.xml` file. Deploy and test your application. Access your application via a Web browser and ensure that it prompts for authentication and that the authentication mechanism works as expected.

Building an Application Using JSP and Servlet

Welcome to the Session, **Building an Application Using JSP and Servlet**.

This session describes how to set up a database connection in a Servlet using the JDBC API. This session also explains how to build a full-fledged application that can perform Create, Read, Update, and Delete (CRUD) operations through JSP and Servlet. Furthermore, this session elaborates on the development of a Web application project titled 'Employee Information' using JSP and Servlet. Finally, it provides a complete walkthrough of the application creation process, covering tasks such as adding and viewing employee details as well as updating and deleting records.

In this session, you will learn to:

- Explain the process of creating a Dynamic Web application using JSP and Servlet
- Describe connecting a database using JPA
- Describe the steps to connect the JSP application to a MySQL database

15.1 Overview

In this session, a simple employee information application is created using Jakarta Server Pages, Servlets and a MySQL database called `employee_information`.

Figure 15.1 shows the characteristics of the employee information application, which are as follows:

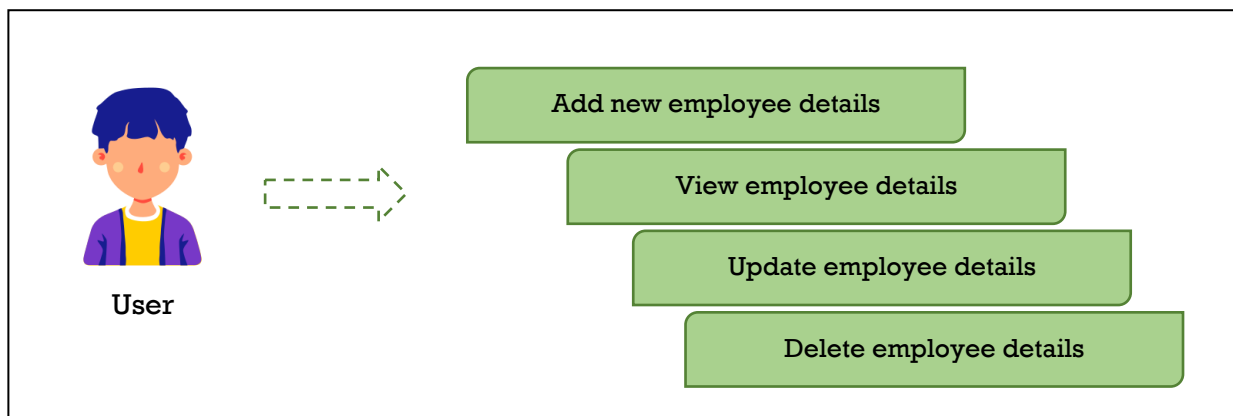


Figure 15.1: Employee Information Application

Figure 15.2 shows the site map of the Jakarta Server Pages and Servlets application, listing each page along with its description.

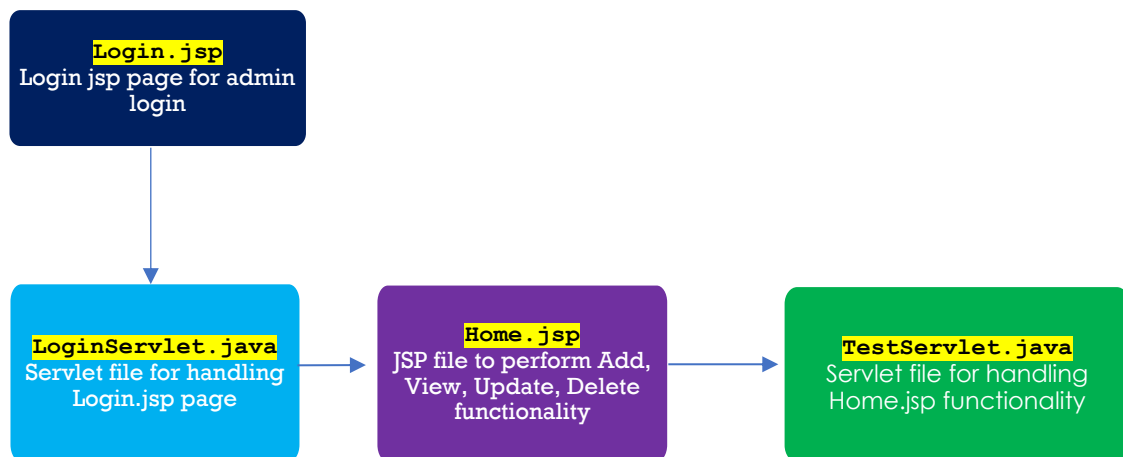


Figure 15.2: Site Map

Steps to create the Employee Information application are as follows:



15.2 Creating MySQL Database

The prerequisite to create the Employee Information application is to:

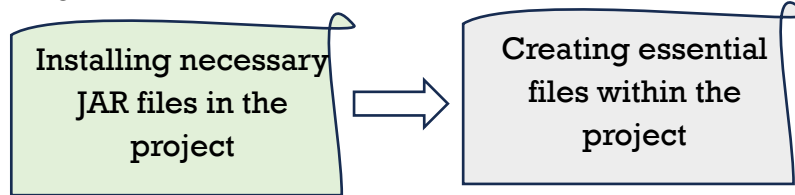
- Create a database in MySQL named `employee_information`.
- After creating the database, create a table named 'employee_data' using columns such as `Employee_id`, `Employee_Name`, `DOB`, `DOJ`, `Salary`, and `Department` as shown in Figure 15.3.

Field	Type
Employee_id	int
Employee_Name	varchar(255)
DOB	date
DOJ	date
Salary	decimal(10,2)
Department	varchar(50)

Figure 15.3: employee_data Table Structure

15.3 Creating Employee Information Application

After creating the database, the next step is to create the Employee Information application in Jakarta Server Pages and Servlets. This process involves several steps, including:



15.3.1 Installing necessary JAR Files in the Project and Setting Up Connection

First, you must install some important JAR files, create a Dynamic Web Project, and configure the database connection. These actions can be accomplished as follows:

1. Download following JAR files into an appropriate location on the computer:
 - Eclipselink-2.7.14.jar
 - jakarta.persistence-api-2.2.3.jar
 - mysql-connector-java-8.0.21.jar
2. Create a Dynamic Web project:
 - a. Open **Eclipse** and click **File**→**New**→**Dynamic Web Project**.
 - b. Name the project as **Employee_Information**.
 - c. Click **Finish** to create the project.
3. Once Project is created, right-click the project and select **Properties**.
4. In the Properties window, click **Project Facets**.
Under the **Project Facets** tab, click the JPA option and choose JPA version 2.2. You may refer to steps and figures from Session 8 under 8.5.5 to recollect how these actions are performed.
5. Click **Further configuration available** at the bottom of the **Properties** window.
6. Click **Manage Libraries** → **New**. Give a name, such as `TestLib`. Now, the user library is ready for adding JAR files.
7. Click **Add External JARs** and add all the JAR files that were downloaded earlier as shown in Figure 15.4.

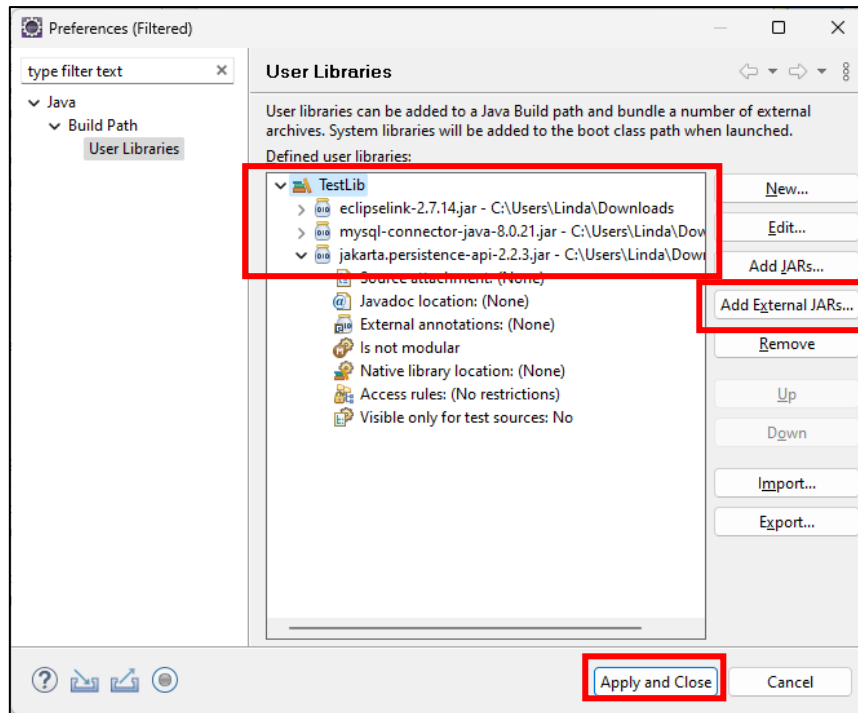


Figure 15.4: User Libraries

8. Click **Apply and Close**.
9. Now, click **Add connection** as shown in Figure 15.5.

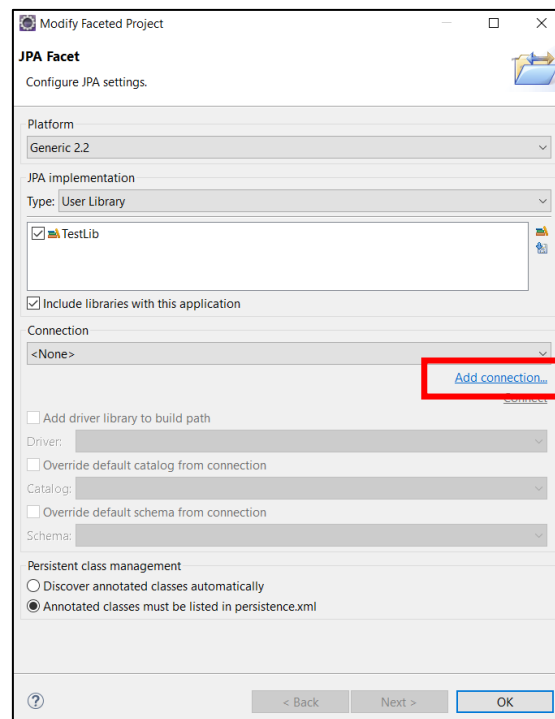


Figure 15.5: Specifying Database Connection

10. Select **MySQL**.

11. Change the database name to `employee_information` in the URL and provide the connection credentials. Click **Finish**.

12. Click **Apply and Close** again to apply the settings and close the dialog box.

Figure 15.6 shows the listing of the `Employee_Information` folder.

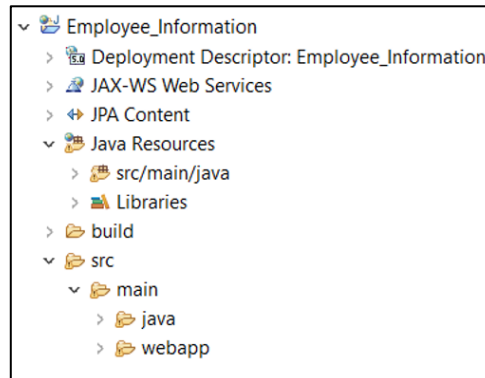


Figure 15.6: Folder Structure

15.3.2 Creating Essential Files within the Project

To define the logic and functionality of the project, create the required source code files.

➤ `persistence.xml`

Code Snippet 1 shows the code for `persistence.xml` which should be saved under **Java Resources** → **src/main/java** → **META-INF** folder of the project. Replace the contents of the existing file with the contents of Code Snippet 1. Change the password for the MySQL login in the property `javax.persistence.jdbc.password`.

Code Snippet 1:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.2"
xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd">
    <persistence-unit name="Employee_Information">

<provider>org.eclipse.persistence.jpa.PersistenceProvider</pr
ovider>
```

```

        <class>com.emp.model.Employee</class>
        <properties>
            <property name="javax.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>
            <property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/employee_information"/>
            <property name="javax.persistence.jdbc.user"
value="root"/>
            <property name="javax.persistence.jdbc.password"
value="password"/>
            <property name="javax.persistence.schema-
generation.database.action" value="none"/>
        </properties>
    </persistence-unit>
</persistence>

```

The database connection properties are specified here, as well as the entity class name.

Next, create a package named `com.emp.model`. Then, proceed to create the class named `Employee.java`.

➤ **Employee.java**

Code Snippet 2 shows the code for `Employee.java`.

Code Snippet 2:

```

package com.emp.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "employee_data")
public class Employee implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    @Column(name = "Employee_Name")
    private String employeeName;

```



```
@Column(name = "DOB")
private String DOB;

@Column(name = "DOJ")
private String DOJ;

@Column(name = "Salary")
private String salary;

@Column(name = "Department")
private String department;

// Getters and Setters
public String getEmployeeName() {
    return employeeName;
}

public void setEmployeeName(String employeeName) {
    this.employeeName = employeeName;
}

public String getDOB() {
    return DOB;
}

public void setDOB(String DOB) {
    this.DOB = DOB;
}

public String getDOJ() {
    return DOJ;
}

public void setDOJ(String DOJ) {
    this.DOJ = DOJ;
}

public String getSalary() {
    return salary;
}

public void setSalary(String salary) {
    this.salary = salary;
}

public String getDepartment() {
    return department;
}
```

```

    public void setDepartment(String department) {
        this.department = department;
    }
}

```

Employee.java:

- This class is used as a JPA entity that maps to the `employee_data` table in a database.
- It allows users to perform Create, View, Update, and Delete operations on the `employee_data` table using JPA.

Save the code with the file name **Employee.java** in the `com.emp.model` package.

Note:

Save all the `.jsp` files under `src→webapp` folder.

Create folder under `src→webapp→CSS` and save all `.css` files.

➤ **Login.jsp**

Code Snippet 3 shows the code for `Login.jsp`.

Code Snippet 3:

```

<%@ page language="java" contentType="text/html;
charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
<link rel="stylesheet" type="text/css"
href="${pageContext.request.contextPath}/CSS/Login.css">
</head>
<body>
    <div class="container">
        <h1>Login</h1>
        <form action="LoginServlet" method="post">
            <label for="username">User Name:</label>
            <input type="text" id="username" name="username"
required>
            <label for="password">Password:</label>
            <input type="password" id="password"
name="password" required>
            <input type="submit" class="button-style"
value="Login">

```

```

        </form>
    </div>
</body>
</html>

```

Login.jsp:

- This JSP form submits data to LoginServlet for processing admin login.
- The form includes two input fields: one for username and password and one submit button.

➤ Login.css

Consider the code in Code Snippet 4 for Login.css file.

Code Snippet 4:

```

@charset "ISO-8859-1";
body {

    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
    font-size: 16px;
    background:
url('https://cdn.pixabay.com/photo/2018/05/18/04/00/work-
3410297_1280.jpg');
    background-attachment: fixed;
    background-repeat: no-repeat;
    background-size: cover;
    background-attachment: fixed;
    backdrop-filter: blur(5px); /* Blur effect */
    -webkit-backdrop-filter: blur(4px);
    color: #6f2a3b;

}
.container {

    width: 300px;
    padding: 20px;
    border-radius: 10px;
    background-color: rgba(255, 255, 255, 0.1);
/* Semi-transparent background */
    -webkit-backdrop-filter: blur(1px);
}
form {

```

```

        display: flex;
        flex-direction: column;

    }
    label, input {
        margin-bottom: 10px;
        font-size: 18px;
    }
    input[type="submit"] {
        width: 100px;
        align-self: center;
    }
    h1{
        text-align: center;
    }
    .button-style {
        background-color: #6f2a3b;
        border-radius: 12px;
        cursor: pointer;
        color: white; /* Ensures text is visible on
dark background */
        padding: 10px 15px;
        font-size: 16px;
        text-align: center;
    }
    input {
        padding: 10px;
        text-align: left;
        border-radius: 40px;
    }
}

```

Save the code with the file name Login.css.

➤ Home.jsp

Code Snippet 5 shows the code for Home.jsp.

Code Snippet 5:

```

<%@ page contentType="text/html; charset=UTF-8"
language="java" import="com.emp.model.Employee"
import="java.util.List" %>
<!DOCTYPE html>
<html>
<head>
    <title>Employee Information</title>

```

```

<link rel="stylesheet" type="text/css"
href="${pageContext.request.contextPath}/CSS/Home.css">
<script>
    function showErrorPopup(message) {
        alert(message); // Display the error message as
an alert popup
    }

    function enableEditing() {

document.querySelectorAll('.editable').forEach(function(element) {
        element.removeAttribute('disabled');
    });
}

    function deleteEmployee(employeeName) {
        if (confirm("Are you sure you want to delete this
employee?")) {
            document.getElementById('actionForm').action
= 'TestServlet?action=delete';
document.getElementById('actionForm').submit();
        }
    }

    function toggleViewButton(disable) {
        document.getElementById('viewButton').disabled =
disable;
    }

    function toggleViewButtonOnAdd(disable) {
        document.getElementById('viewButton').disabled =
disable;
    }
</script>
</head>
<body>
    <div class="container">
        <div class="left">
            <h1>Select Employee</h1>
            <form id="viewForm" action="TestServlet"
method="post">
                <label for="Employee_Name">Employee
Name:</label>
                <select name="Employee_Name"
id="Employee_Name" onchange="toggleViewButton(false);">
                    <option value="">Select Option</option>

```

```

        <%
            List<String> employeeNames =
(List<String>) request.getAttribute("employeeNames");
            if (employeeNames != null) {
                for (String name : employeeNames)
{
                    %>
                    <option value="<%= name %>"><%= name
%></option>
                    <%
                        }
                    }
                %>
            </select><br><br>

            <button type="button" class="button-style"
onclick="window.location.href='Home.jsp?action=add';
toggleViewButton(true);">Add</button>
            <input type="submit" class="button-style"
value="View" id="viewButton" disabled>
        </form>
        <%-- Check if error message attribute is set in
request and display popup if present --%>
        <%
            String errorMessage = (String)
request.getAttribute("errorMessage");
            if (errorMessage != null) {
                %>
                <script>alert('<%= errorMessage %>');</script>
                <%
                    }
                %>
            </div>

            <div class="right">
                <h2>Employee Information</h2>
                <%-- Check if employee attribute is set in
request and display employee details if present --%>
                <%
                    String action =
request.getParameter("action");
                    Employee employee = (Employee)
request.getAttribute("Employee");
                    if ("add".equals(action)) {
                        %>
                        <form id="actionForm" action="TestServlet"
method="post">
                            <table>

```

```

        <tr>
            <th>Employee Name</th>
            <td><input type="text"
name="employeeName" value=""></td>
        </tr>
        <tr>
            <th>Date of Joining (DOJ)</th>
            <td><input type="text" name="doj"
value=""></td>
        </tr>
        <tr>
            <th>Department</th>
            <td><input type="text"
name="department" value=""></td>
        </tr>
        <tr>
            <th>Date of Birth (DOB)</th>
            <td><input type="text" name="dob"
value=""></td>
        </tr>
        <tr>
            <th>Salary</th>
            <td><input type="text" name="salary"
value=""></td>
        </tr>
    </table>
    <div class="buttons">
        <button type="submit" class="button-
style" formaction="TestServlet?action=add"
onclick="toggleViewButton(false);">Add Employee</button>
    </div>
</form>
<%
    } else if (employee != null) {
%>
    <form id="actionForm" action="TestServlet"
method="post">
        <table>
            <tr>
                <th>Employee Name</th>
                <td><input type="text"
name="employeeName" value="<%= employee.getEmployeeName() %>"
readonly></td>
            </tr>
            <tr>
                <th>Date of Joining (DOJ)</th>

```

```

                                <td><input type="text" name="doj "
value="<%= employee.getDOj() %>" class="editable"
disabled></td>
                                </tr>
                                <tr>
                                    <th>Department</th>
                                    <td><input type="text"
name="department" value="<%= employee.getDepartment() %>"
class="editable" disabled></td>
                                </tr>
                                <tr>
                                    <th>Date of Birth (DOB)</th>
                                    <td><input type="text" name="dob"
value="<%= employee.getDOB() %>" class="editable"
disabled></td>
                                </tr>
                                <tr>
                                    <th>Salary</th>
                                    <td><input type="text" name="salary"
value="<%= employee.getSalary() %>" class="editable"
disabled></td>
                                </tr>
                            </table>
                            <div class="buttons">
                                <button type="button" class="button-
style" onclick="enableEditing()">Edit</button>
                                <button type="submit" class="button-
style" formaction="TestServlet?action=update"
onclick="toggleViewButton(false);">Submit</button>
                                <button type="button" class="button-
style" onclick="deleteEmployee('<%=
employee.getEmployeeName() %>')">Delete</button>
                            </div>
                        </form>
                        <%
                            } else {
                        %>
                        <p>No employee found with the given details.</p>
                        <%
                            }
                        %>
                    </div>
                </div>
            </body>
        </html>

```


The Home.jsp file:

- **function showErrorPopup():** This function displays an error message to the user in the form of a pop-up alert.
- **function enableEditing():** This function enables all form fields that use the class 'editable'.
- **function deleteEmployee():** This function deletes the selected employee from the database.
- **Function toggleViewButton():** This function enables or disables the view button based on the given parameter. If a true parameter is passed, button is disabled, if a false parameter is passed, the button is enabled.
- **In left div class:**
 - The onchange event calls the JavaScript function toggleViewButton(false) whenever a new option is selected from the drop-down, which enables the view button.
 - In this `List<String> employeeNames = (List<String>) request.getAttribute("employeeNames");` JSP scripting retrieves the list of employee names from the employeeNames attribute.
 - It checks if employeeNames is not null and iterates over each name in the employeeNames list.
 - The button `<button type="button" class="button-style" onclick="window.location.href='Home.jsp?action=add'; toggleViewButton(true);">Add</button>` is used to redirect to the Home.jsp page with a query parameter action=add and to disable the view button.
 - The button `<input type="submit" class="button-style" value="View" id="viewButton" disabled>` is disabled when the page loads, until an employee is selected from the drop-down. Once an employee is selected, the JavaScript function toggleViewButton(false) is called enabling the button and allowing the user to submit the form to view the selected employee's details.
- **In the right div class:**
 - The line `String action = request.getParameter("action");` is used to retrieve action parameter from the request.
 - The line `Employee employee = (Employee) request.getAttribute("Employee");` retrieves the Employee object from the request attribute.
 - The First `<form id="actionForm" action="TestServlet" method="post">` form is used to add new employee when action is add.

- **The Second** `<form id="actionForm" action="TestServlet" method="post">` form is used for displaying and editing employee information when the employee object is not null.
- The Edit button calls the `enableEditing()` function to enable editing of the fields.
- The Submit button submits the form with the `action=update` parameter to update the employee information. It calls `toggleViewButton(false)` function to disable the view button.

Code Snippet 6 shows the code for `Home.css`.

Code Snippet 6:

```
@charset "ISO-8859-1";
body {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
    font-size: 16px;
    background:
url('https://cdn.pixabay.com/photo/2017/08/07/03/07/interior-
2599130_1280.jpg');
    background-attachment: fixed;
    background-repeat: no-repeat;
    background-size: cover;
    background-attachment: fixed;
    backdrop-filter: blur(7px); /* Blur effect */
    -webkit-backdrop-filter: blur(4px);
    color: #6f2a3b;
}
.container {
    width: 70%;
    height: 410px; /* Increased height for better
visibility */
    display: flex;
    justify-content: space-between; /* Adjusted to
space-between for better layout */
    border: 1px solid #ccc;
    padding: 20px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    overflow-y: auto; /* Add vertical scroll if
content overflows */
}
.left, .right {
    width: 48%;
```

```

    }
    .right {
        margin-left: 2%;
    }
    table {
        width: 100%;
        border-collapse: collapse;
        font-size: 16px;
    }

    th, td, input {
        padding: 10px;
        text-align: left;
        border-radius: 40px;
    }
    .buttons {
        margin-top: 20px;
    }
    .button-style {
        background-color: #6f2a3b;
        border-radius: 12px;
        cursor: pointer;
        color: white; /* Ensures text is visible on dark
background */
        padding: 10px 15px;
        font-size: 14px;
    }
    .button-style:disabled {
        background-color: #ccc#6f2a3b;
        border: 3px solid #999;
        cursor: not-allowed;
    }

    select {
        width: 50%;
        margin-left: 20px; /*Space between label and
dropdown*/
        padding: 10px; /*This is used for the height of
the dropdown*/
        font-size: 12px;
        border: solid #6f2a3b;
        border-radius: 40px;
        background-color: #f0f0f0;
        color: #6f2a3b;
        cursor: pointer;
    }

```

Code Snippet 7 shows the code for `LoginServlet.java`.

Code Snippet 7:

```

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    private static final String CORRECT_USERNAME = "Admin";
    private static final String CORRECT_PASSWORD =
"admin123";

    protected void doPost(HttpServletRequest request,
HttpServletResponse response)
        throws ServletException, IOException {
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // Validate the username and password
        if (username.equals(CORRECT_USERNAME) &&
password.equals(CORRECT_PASSWORD)) {
            // Credentials are correct, redirect to Home.jsp
            response.sendRedirect("Home.jsp");
        } else {
            // Credentials are incorrect, display error
message
            request.setAttribute("error", "Invalid username
or password");

            request.getRequestDispatcher("login.jsp").forward(request,
response);
        }
    }
}

```

LoginServlet.java file:

- **This file checks if the entered username and password match the saved credentials in the LoginServlet file.**
- **If the credentials are correct, it will redirect to the Home.jsp file.**
- **If the credentials are not correct, it will redirect back to the Login.jsp page.**

Code Snippet 8 shows the code for TestServlet.java.

Code Snippet 8:

```

import java.io.IOException;
import java.util.List;
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import javax.persistence.PersistenceUnit;
import javax.persistence.TypedQuery;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import com.emp.model.Employee;

public class TestServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @PersistenceUnit
    private EntityManagerFactory entityManagerFactory = null;

    public TestServlet() {
        super();
    }

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        entityManagerFactory =
        Persistence.createEntityManagerFactory("Employee_Information"
        ); // This is Project name
        EntityManager entityManager =
        entityManagerFactory.createEntityManager();
        try {
            var trans = entityManager.getTransaction();
            if (!trans.isActive()) {
                trans.begin();
                // Get the "action" parameter from the request
                String action =
                request.getParameter("action");
                if (action == null) {
                    action = "view";
                }

                if ("view".equals(action)) {
                    String employeeName =
                    request.getParameter("Employee_Name");

```

```

        if (employeeName != null &&
!employeeName.isEmpty()) {
            // Fetch the employee data from the
            database based on the selected employee name
            Employee employee =
entityManager.find(Employee.class, employeeName);

            // Check if employee is found
            if (employee != null) {
                request.setAttribute("Employee",
employee);
            } else {

request.setAttribute("errorMessage", "Employee not found or
incorrect details");
            }
            } else {
                request.setAttribute("errorMessage",
"Employee name is required");
            }
        } else if ("update".equals(action)) {
            String employeeName =
request.getParameter("employeeName");
            String doj = request.getParameter("doj");
            String department =
request.getParameter("department");
            String dob = request.getParameter("dob");
            String salary =
request.getParameter("salary");

            Employee employee =
entityManager.find(Employee.class, employeeName);

            if (employee != null) {
                employee.setDOj(doj);
                employee.setDepartment(department);
                employee.setDOB(dob);
                employee.setSalary(salary);
                entityManager.merge(employee);
            }
        } else if ("delete".equals(action)) {
            String employeeName =
request.getParameter("employeeName");

            Employee employee =
entityManager.find(Employee.class, employeeName);

```

```

        if (employee != null) {
            entityManager.remove(employee);
        }
    } else if ("add".equals(action)) {
        String employeeName =
request.getParameter("employeeName");
        String doj = request.getParameter("doj");
        String department =
request.getParameter("department");
        String dob = request.getParameter("dob");
        String salary =
request.getParameter("salary");

        Employee employee = new Employee();
        employee.setEmployeeName(employeeName);
        employee.setDOj(doj);
        employee.setDepartment(department);
        employee.setDOB(dob);
        employee.setSalary(salary);

        entityManager.persist(employee);
    }

    // Fetch all employee names to display in the
dropdown
    TypedQuery<String> query =
entityManager.createQuery("SELECT e.employeeName FROM
Employee e", String.class);
    List<String> employeeNames =
query.getResultList();
    request.setAttribute("employeeNames",
employeeNames);

    trans.commit();

request.getRequestDispatcher("Home.jsp").forward(request,
response);
    }
    } catch (Exception ex) {
        ex.printStackTrace(); // for debugging
        response.sendError(500, "Error fetching data: " +
ex.getMessage());
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }
}
}

```

```

@Override
protected void doGet(HttpServletRequest request,
HttpServletResponse response)
    throws ServletException, IOException {
    doPost(request, response);
}
}

```

TestServlet.java file:

- The line `EntityManager entityManager = entityManagerFactory.createEntityManager();` initializes the `EntityManagerFactory` with the persistence unit named **Employee_Information** and creates an `EntityManager` instance.
- The line `var trans = entityManager.getTransaction();` starts a new transaction if one is not already active.
- The line `String action = request.getParameter("action");` retrieves the value of the **action** parameter from the HTTP request.
- The function `("view".equals(action))` checks if the action is 'view' means View button is clicked, then it will fetch the selected employee details from the database.
- The function `("update".equals(action))` checks if Update button is clicked in the jsp file, then it will update the selected field in the database.
- The function `("add".equals(action))` checks if Add button is clicked in the jsp file, then it will add the entered details in the database.
- The line `TypedQuery<String> query = entityManager.createQuery("SELECT e.employeeName FROM Employee e", String.class);` is used to create a JPA query that selects the employee name from the database.
- The line `List<String> employeeNames = query.getResultList();` is used to execute the query and retrieve employee name from the database.

Code Snippet 9 shows the code for `web.xml` after adding all the files in the project.

Code Snippet 9:

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="https://jakarta.ee/xml/ns/jakartaee"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
id="WebApp_ID" version="6.0">
    <display-name>Employee_Information</display-name>
    <welcome-file-list>
        <welcome-file>Home.jsp</welcome-file>
    </welcome-file-list>
</web-app>

```



```

    <welcome-file>Login.jsp</welcome-file>
</welcome-file-list>
<servlet>
    <description></description>
    <display-name>TestServlet</display-name>
    <servlet-name>TestServlet</servlet-name>
    <servlet-class>TestServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>TestServlet</servlet-name>
    <url-pattern>/TestServlet</url-pattern>
</servlet-mapping>
<servlet>
    <description></description>
    <display-name>LoginServlet</display-name>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>LoginServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/LoginServlet</url-pattern>
</servlet-mapping>
</web-app>

```

15.4 Running the Application

To view the application:

1. Right-click `Login.jsp` page, then select **Run As** → **Run on Server**. Enter Username as **Admin** and Password as **admin123**. Click **Login** button. The Output will be displayed as shown in Figure 15.7.

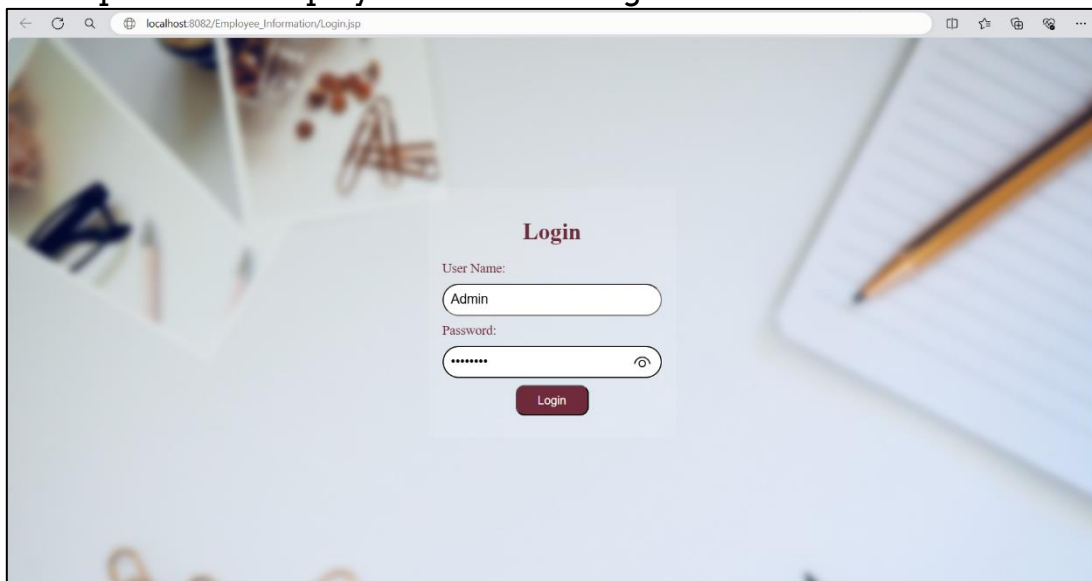


Figure 15.7: Login Page

2. After clicking **Login** button, it will redirect to the `Home.jsp` page as shown in Figure 15.8.

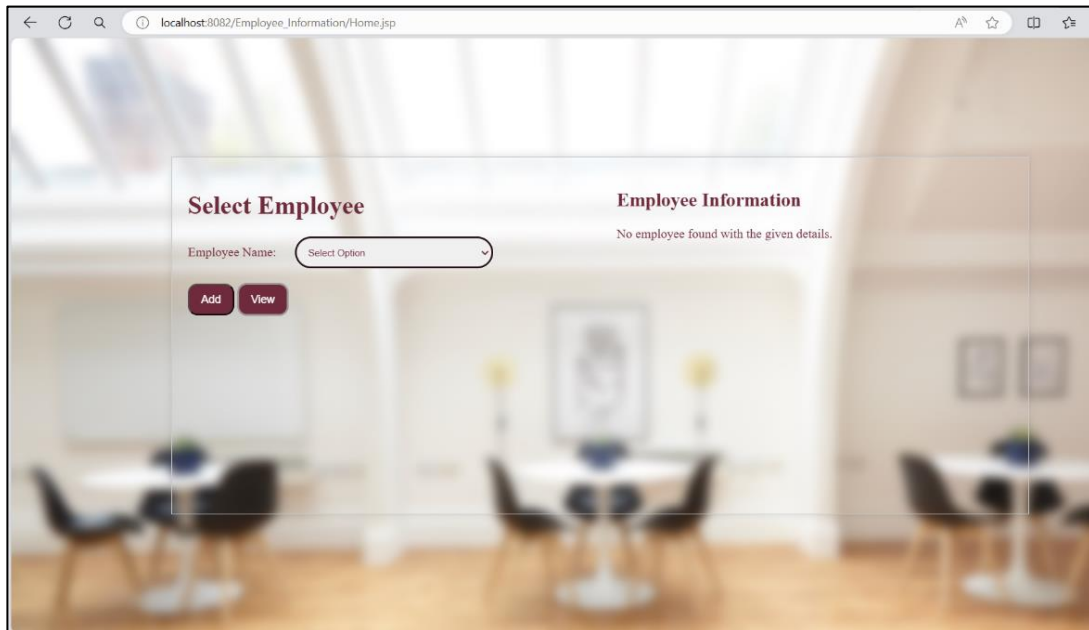


Figure 15.8: Home Page

3. Click **Add** button, a form will be displayed on the right side of the container. Add Employee details and click **Add Employee** button as shown in Figure 15.9.



View button will be disabled here. After adding an employee, **View** button will be enabled.

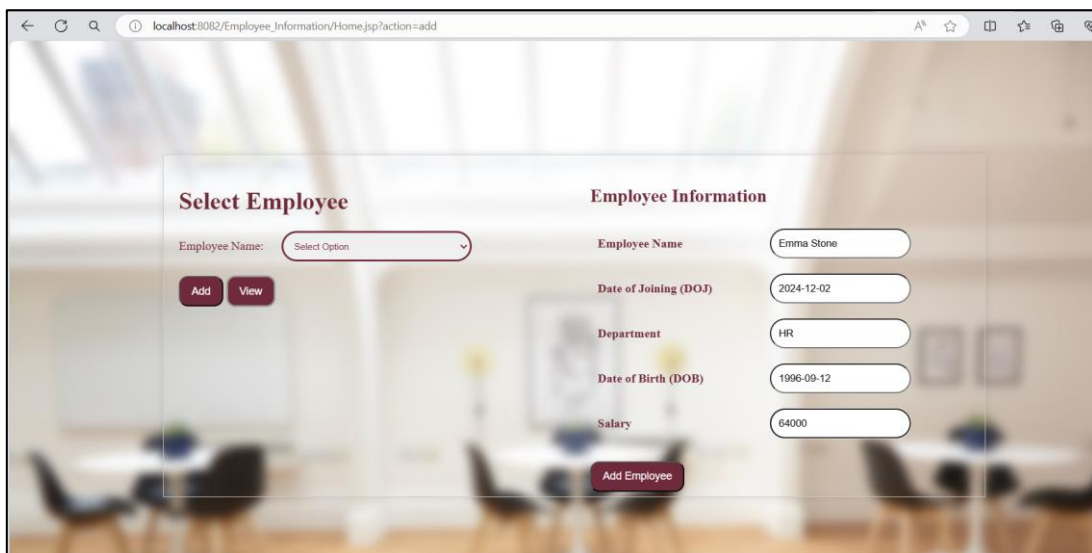
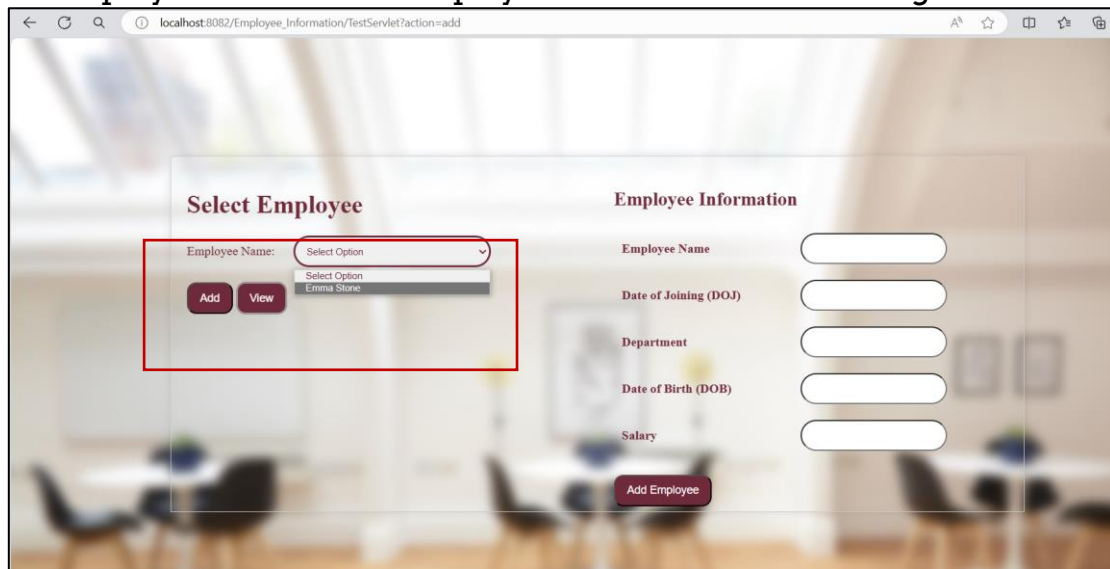


Figure 15.9: Home Page Add Operation

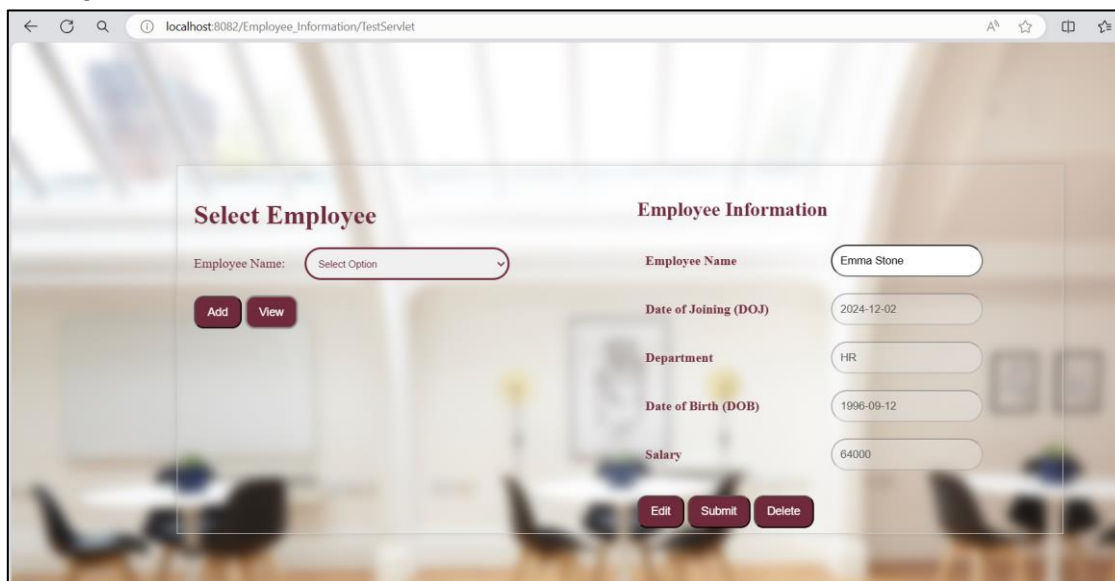
4. Now, click drop-down on the left side of the container. The entered employee name will be displayed in the list as shown in Figure 15.10.



The screenshot shows a web browser window with the URL `localhost:8082/Employee_Information/TestServlet?action=add`. The page has a background image of an office. On the left, there is a 'Select Employee' section with a label 'Employee Name:' and a dropdown menu. The dropdown menu is open, showing three options: 'Select Option', 'Select Option', and 'Emma Stone'. Below the dropdown are two buttons: 'Add' and 'View'. On the right, there is an 'Employee Information' section with five input fields: 'Employee Name', 'Date of Joining (DOJ)', 'Department', 'Date of Birth (DOB)', and 'Salary'. At the bottom right of this section is an 'Add Employee' button.

Figure 15.10: Home Page

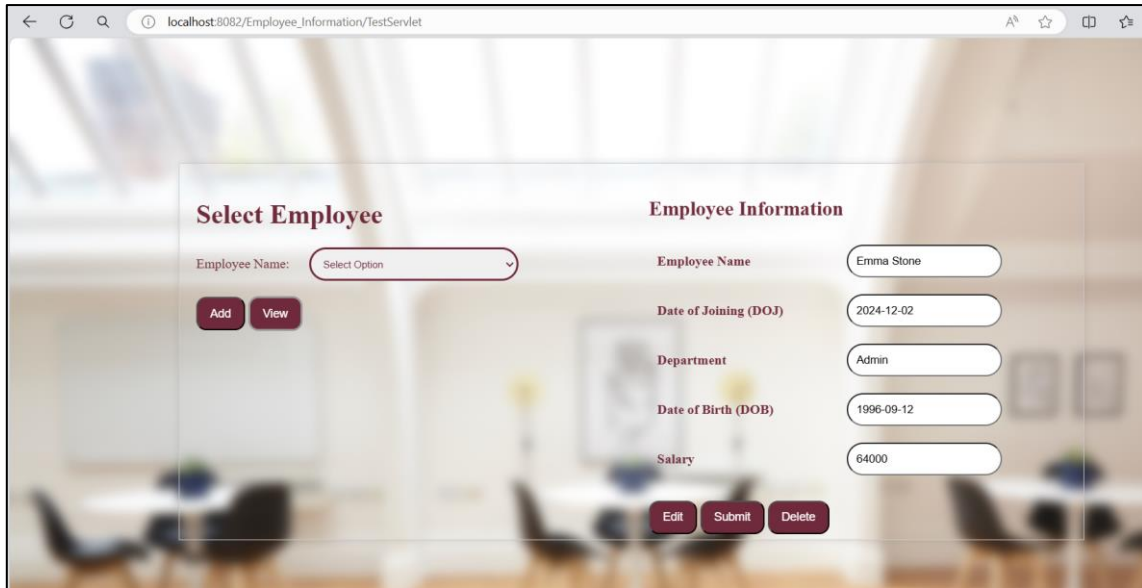
5. Select the employee name and click **View** button, the selected employee details will be displayed on the right side of the container as shown in Figure 15.11.



The screenshot shows the same web browser window. The 'Select Employee' section now has the dropdown menu closed, and the 'View' button is highlighted. The 'Employee Information' section now displays the details for 'Emma Stone':
Employee Name: Emma Stone
Date of Joining (DOJ): 2024-12-02
Department: HR
Date of Birth (DOB): 1996-09-12
Salary: 64000
At the bottom of this section are three buttons: 'Edit', 'Submit', and 'Delete'.

Figure 15.11: Home Page

6. To edit the employee details, click **Edit** button and edit the **Department HR** to **Admin**. After completing the edit, click the **Submit** button as shown in Figure 15.12.



The screenshot shows a web browser window with the URL `localhost:8082/Employee_Information/TestServlet`. The page is divided into two main sections: "Select Employee" and "Employee Information".

Select Employee:

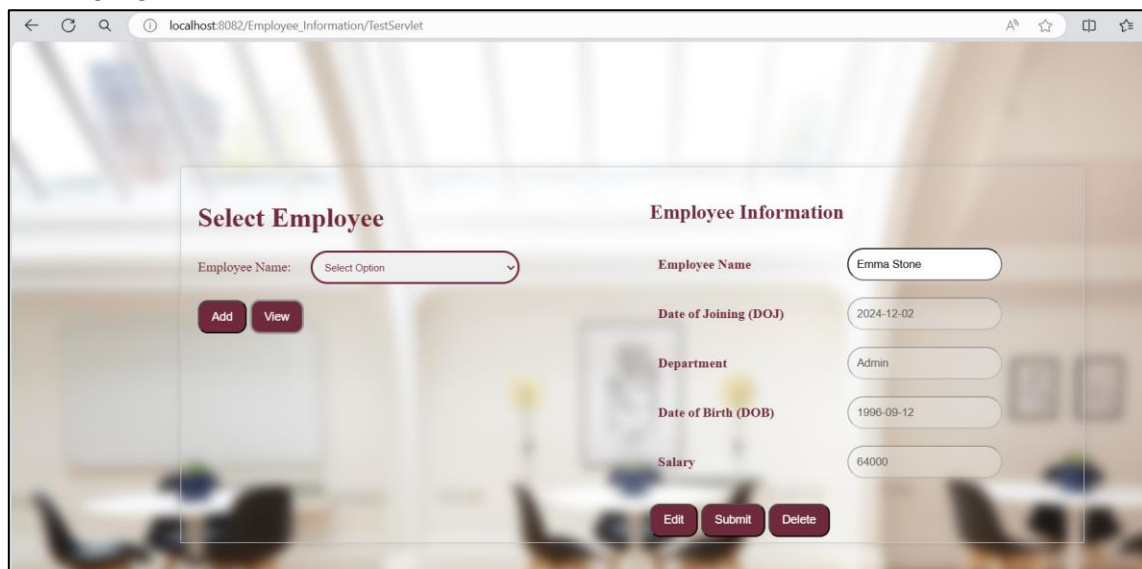
- Employee Name:
-

Employee Information:

Employee Name	Emma Stone
Date of Joining (DOJ)	2024-12-02
Department	Admin
Date of Birth (DOB)	1996-09-12
Salary	64000

Figure 15.12: Home Page Edit Operation

7. To recheck if the edited file is stored in the database, select employee name from the dropdown and click the View button as shown in Figure 15.13.



The screenshot shows the same web browser window as Figure 15.12, but the "View" button is highlighted, indicating the next step in the process.

Figure 15.13: Home Page

8. To perform the delete operation → select the employee name from the drop-down → View → Click **Delete** button. A popup will be displayed. Click **OK** button as shown in Figure 15.14.

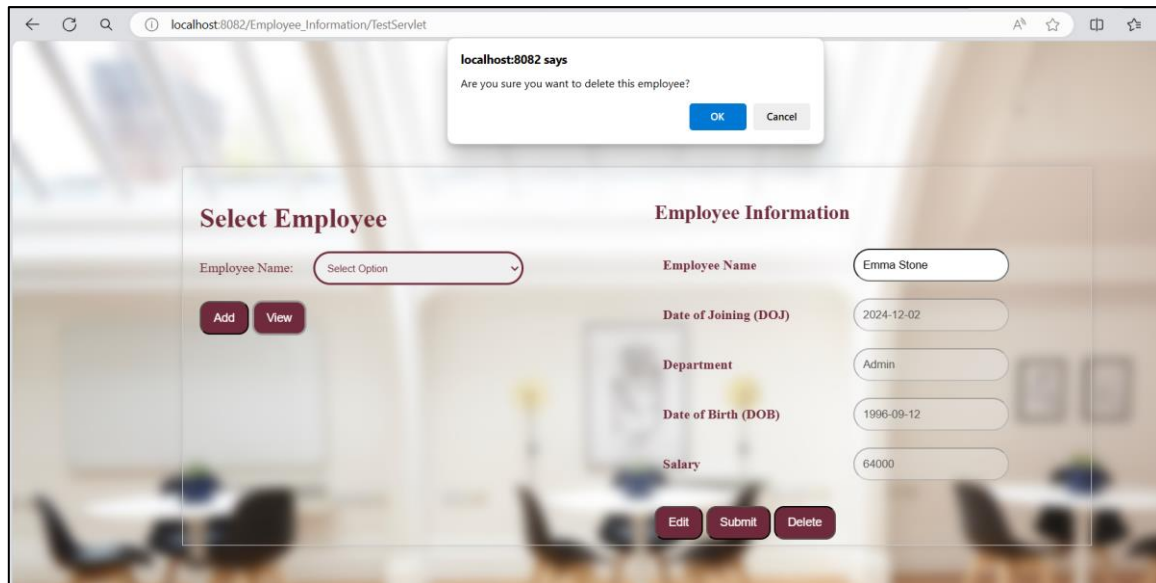


Figure 15.14: Home Page Delete Operation

Summary

- A simple employee information application is created using Jakarta Server Pages, Servlets, and a MySQL database called `employee_information`.
- Steps to create the Employee Information application involve creating a Dynamic Web Page application, connecting the application to database, and running the application.
- The prerequisite to creating the Employee Information application is to create a database in MySQL.
- Two processes to create Employee Information application involve installing necessary JAR files in the project and creating essential files within the project.
- The required source code files, including `Login.jsp` for admin login, `LoginServlet.java` for handling login page, `Home.jsp` for Add, View, Update, Delete functionality, and `TestServlet.java` for managing `Home.jsp` functionality.
- Steps to run the application involve logging in using the provided credentials, navigating through various pages to add, view, edit, and delete employee details, and observing the corresponding changes in the interface.

Check Your Progress

1. Which of the following is the prerequisite for creating the Employee Information application?

(A)	Set up local development Server
(B)	Design the use interface
(C)	Create a database in MYSQL
(D)	Define servlet mappings

2. Which of the following statements is not true regarding the creation of Employee Information application?

(A)	After creating the database, the next step is to create the Employee Information application.
(B)	This process involves installing the necessary JAR files in the project.
(C)	This process involves creating essential files within the project.
(D)	Both A and B are not true.

3. Which of the following JAR files is not necessary for this project?

(A)	Eclipselink-2.7.14.jar
(B)	commons-lang3.jar
(C)	jakarta.persistence-api-2.2.3.jar
(D)	mysql-connector-java-8.0.21.jar

4. Which of the following is not a step in the installation of JAR files?

- Download the necessary JAR files into an appropriate location on the computer.
- Create a Dynamic Web project in Eclipse.
- Configure JPA version 2.2 in the project facets.
- Click **Add connection** after applying and closing the properties.
- Install Web Server for this project.

(A)	a, c
(B)	b, c
(C)	e
(D)	d, e

5. What is the pupose of `TestServlet.java` Servlet file?

(A)	To display employee information
(B)	To handle user authentication
(C)	To perform database operations
(D)	To handle the functionality of the Home.jsp page

Answers

1.	C
2.	D
3.	B
4.	C
5.	D

Try It Yourself

1. Create a Bank Management System Project by developing a Dynamic Web Application using JSP, Servlet, and MySQL database in Eclipse.

Customer ID	Customer Name	Customer Age	Customer Date of Birth
1	Jack	25	1999-01-15
2	Ross	26	1998-06-06
3	Anna	27	1997-08-11

Table 15.1: Customer Details

- Install all the required jar files.
 - Create a Dynamic Web Application in Eclipse named **Bank_Management_System**.
 - Create a Database named `Bank_Management` and create a table named `Customer_Details`.
 - Add JBC MySQL connections details in `persistence.xml` file.
 - Create two jsp files named `UserLogin.jsp` and `Bank_Home_Page.jsp`.
 - Create two .java servlet files named `UserServlet.java` and `HomeServlet.java`.
 - Create one entity file named `Bank.java`.
 - `UserLogin.jsp`: In this file add two fields one for Username and other for Password. Add one **Submit** button. After clicking the Submit button Page will redirect to the `Bank_Home_Page.jsp`.
 - **Bank_Home_Page.jsp**: On this page, perform the following actions:
 - Add one Div Container.
 - Add four buttons: Add, View, Update, and Delete.
 - The Add button should display the form on the page. The form must contain the following fields: Customer ID, Name, Age, and Date of Birth.
 - Display the names of the customers in a dropdown menu. The values for the customers' names should be fetched from the database.
 - The on-click event of the View button should display the customer's details based on the value selected from the dropdown.
 - The on-click event of the Edit button should allow users to edit the fields.
 - The on-click event of the Delete button should enable users to delete the selected customer's information.
 - **Bank.java**: This class uses a JPA entity that maps to the `Customer_Details` table in a database. This allows student to perform CRUD operations on the `Customer_Details` table using JPA.
2. Execute the application and use the customer details as given in Table 15.1 for CRUD operation.

Appendix

Sr. No.	Case Studies
1.	Create a Web application using Jakarta Servlets where users can upload a PDF document containing their employee profile information. Design a servlet that handles these uploads with following functionalities: - Create a JSP file named <code>uploadProfile.jsp</code>. This JSP file provides the user interface for uploading PDF profiles. It includes an HTML form with a file input field, a submit button, and an optional element to display the upload status message received from the servlet. - Create a Servlet file named <code>uploadProfileServlet.java</code>. This servlet should handle the server-side logic for uploading PDF documents. - The Servlet retrieves the uploaded file, performs validations (file existence, type, and size), normalizes the filename, generates a unique filename (optional), stores the uploaded PDF securely on the server, and communicates the upload status (success or error) back to the JSP file. - Create informative messages for success and error scenarios: - Success: Profile document uploaded successfully! - Error: Invalid file type or file size exceeds the limit! Please upload a PDF document of the specified size. - Store the uploaded PDF documents on the server in a designated directory (<code>employee_profiles</code>). Generate unique filenames to avoid overwriting existing files with the same name.
2.	Consider an example of accessing a database using JPA to insert employee details. The Web application will contain an <code>Entity</code> class to represent the employee table and a Servlet class to perform database operations with JPA. Perform these step-by-step instructions to accomplish this task: - Create a table named <code>employee</code> in a MySQL database named <code>employeeDB</code>. The <code>employee</code> table should have columns for employee ID, name, department, and position. - Download the following JAR files and place them in an appropriate location on the computer: <code>eclipselink-2.7.14.jar</code> <code>jakarta.persistence-api-2.2.3.jar</code> <code>mysql-connector-java-8.0.21.jar</code> - Create a <code>persistence.xml</code> file under the <code>src/main/resources/META-INF</code> directory of the project. - Create a class named <code>Employee.java</code> and a servlet named <code>EmployeeServlet.java</code> under the <code>src/main/java</code> directory.

Sr. No.	Case Studies																																								
	e. Create a JSP file named <code>employee.jsp</code> which is a form to enter employee details. Ensure the form action attribute points to the servlet for processing and insertion into the database. f. Refer to Table 1 for sample employee details, which are used in this program. <table><tr><th>Employee ID</th><th>Name</th><th>Department</th><th>Position</th></tr><tr><td>E001</td><td>John Doe</td><td>Engineering</td><td>Software Engineer</td></tr><tr><td>E002</td><td>Jane Smith</td><td>HR</td><td>HR Manager</td></tr><tr><td>E003</td><td>Alice Brown</td><td>Sales</td><td>Sales Representative</td></tr><tr><td>E004</td><td>Bob Johnson</td><td>Marketing</td><td>Marketing Analyst</td></tr><tr><td>E005</td><td>Emily Davis</td><td>Finance</td><td>Financial Analyst</td></tr><tr><td>E006</td><td>Michael Wilson</td><td>IT</td><td>IT Specialist</td></tr><tr><td>E007</td><td>Sarah Taylor</td><td>Operations</td><td>Operations Manager</td></tr></table> Table 1: Employee Details	Employee ID	Name	Department	Position	E001	John Doe	Engineering	Software Engineer	E002	Jane Smith	HR	HR Manager	E003	Alice Brown	Sales	Sales Representative	E004	Bob Johnson	Marketing	Marketing Analyst	E005	Emily Davis	Finance	Financial Analyst	E006	Michael Wilson	IT	IT Specialist	E007	Sarah Taylor	Operations	Operations Manager								
Employee ID	Name	Department	Position																																						
E001	John Doe	Engineering	Software Engineer																																						
E002	Jane Smith	HR	HR Manager																																						
E003	Alice Brown	Sales	Sales Representative																																						
E004	Bob Johnson	Marketing	Marketing Analyst																																						
E005	Emily Davis	Finance	Financial Analyst																																						
E006	Michael Wilson	IT	IT Specialist																																						
E007	Sarah Taylor	Operations	Operations Manager																																						
3.	Create a simple Web application for displaying employee details using JSP and Expression Language with the data given in Table 2. <table><tr><th>ID</th><th>Name</th><th>Age</th><th>Gender</th><th>Mail ID</th></tr><tr><td>E001</td><td>John Doe</td><td>30</td><td>Male</td><td>john.doe@example.com</td></tr><tr><td>E002</td><td>Jane Smith</td><td>28</td><td>Female</td><td>jane.smith@example.com</td></tr><tr><td>E003</td><td>Alice Brown</td><td>35</td><td>Female</td><td>alice.brown@example.com</td></tr><tr><td>E004</td><td>Bob Johnson</td><td>32</td><td>Male</td><td>bob.johnson@example.com</td></tr><tr><td>E005</td><td>Emily Davis</td><td>27</td><td>Female</td><td>emily.davis@example.com</td></tr><tr><td>E006</td><td>Michael Wilson</td><td>40</td><td>Male</td><td>michael.wilson@example.com</td></tr><tr><td>E007</td><td>Sarah Taylor</td><td>33</td><td>Female</td><td>sarah.taylor@example.com</td></tr></table> Table 2: Employee Personal Details a. Create a JSP file named <code>employeeDetails.jsp</code>. This JSP file presents the employee details using EL expressions to dynamically display the content.	ID	Name	Age	Gender	Mail ID	E001	John Doe	30	Male	john.doe@example.com	E002	Jane Smith	28	Female	jane.smith@example.com	E003	Alice Brown	35	Female	alice.brown@example.com	E004	Bob Johnson	32	Male	bob.johnson@example.com	E005	Emily Davis	27	Female	emily.davis@example.com	E006	Michael Wilson	40	Male	michael.wilson@example.com	E007	Sarah Taylor	33	Female	sarah.taylor@example.com
ID	Name	Age	Gender	Mail ID																																					
E001	John Doe	30	Male	john.doe@example.com																																					
E002	Jane Smith	28	Female	jane.smith@example.com																																					
E003	Alice Brown	35	Female	alice.brown@example.com																																					
E004	Bob Johnson	32	Male	bob.johnson@example.com																																					
E005	Emily Davis	27	Female	emily.davis@example.com																																					
E006	Michael Wilson	40	Male	michael.wilson@example.com																																					
E007	Sarah Taylor	33	Female	sarah.taylor@example.com																																					

Sr. No.	Case Studies																																								
	b. Create a Java class file named <code>Employee.java</code>. This class represents the Employee entity with properties such as ID, name, age, gender, and email ID. c. Create a JSP file named <code>index.jsp</code>. This JSP file serves as the entry point for the Web application, providing a link to access employee details.																																								
4.	Create an MVC Web application to manage employee details with the data given in Table 3. The application uses Jakarta EE 10 with JSP 3.1 and Servlets 6.0. <table><tr><th>Employee ID</th><th>Name</th><th>Department</th><th>Joining Date</th><th>Salary</th></tr><tr><td>E001</td><td>John Doe</td><td>Engineering</td><td>2021-01-15</td><td>\$75,000</td></tr><tr><td>E002</td><td>Jane Smith</td><td>HR</td><td>2019-03-22</td><td>\$65,000</td></tr><tr><td>E003</td><td>Alice Brown</td><td>Sales</td><td>2020-05-10</td><td>\$55,000</td></tr><tr><td>E004</td><td>Bob Johnson</td><td>Marketing</td><td>2018-07-30</td><td>\$70,000</td></tr><tr><td>E005</td><td>Emily Davis</td><td>Finance</td><td>2022-02-19</td><td>\$60,000</td></tr><tr><td>E006</td><td>Michael Wilson</td><td>IT</td><td>2017-11-11</td><td>\$80,000</td></tr><tr><td>E007</td><td>Sarah Taylor</td><td>Operations</td><td>2023-04-05</td><td>\$68,000</td></tr></table> Table 3: Employee Income Details a. Create a new Maven project and choose <code>maven-archetype-webapp</code> as the archetype. Configure project details such as entering Group Id and Artifact Id, uncheck the run archetype generation interactively option, and click Finish. b. Add necessary dependencies for Jakarta EE, Servlet, and JSP. c. Create a class named <code>Employee.java</code> under <code>src/main/java/com/example/model/</code> which is the entity class for Employee. This class represents the Employee entity with properties such as ID, name, department, joining date, and salary. d. Create a Servlet named <code>EmployeeServlet.java</code> under <code>src/main/java/com/example/controller/</code> which is the controller servlet for the entity Employee. e. Create a JSP file named <code>employeeDetails.jsp</code> under the <code>src/main/webapp</code> directory. This JSP file presents the employee details.	Employee ID	Name	Department	Joining Date	Salary	E001	John Doe	Engineering	2021-01-15	\$75,000	E002	Jane Smith	HR	2019-03-22	\$65,000	E003	Alice Brown	Sales	2020-05-10	\$55,000	E004	Bob Johnson	Marketing	2018-07-30	\$70,000	E005	Emily Davis	Finance	2022-02-19	\$60,000	E006	Michael Wilson	IT	2017-11-11	\$80,000	E007	Sarah Taylor	Operations	2023-04-05	\$68,000
Employee ID	Name	Department	Joining Date	Salary																																					
E001	John Doe	Engineering	2021-01-15	\$75,000																																					
E002	Jane Smith	HR	2019-03-22	\$65,000																																					
E003	Alice Brown	Sales	2020-05-10	\$55,000																																					
E004	Bob Johnson	Marketing	2018-07-30	\$70,000																																					
E005	Emily Davis	Finance	2022-02-19	\$60,000																																					
E006	Michael Wilson	IT	2017-11-11	\$80,000																																					
E007	Sarah Taylor	Operations	2023-04-05	\$68,000																																					

Sr. No.	Case Studies
	f. Create a JSP file named <code>index.jsp</code> under the src/main/webapp directory. This JSP file serves as the entry point for the Web application, providing a link to access employee details.